نظام التنبؤ المالي القابل للتفسير

دليل دراسة تقني لرسالة الماجستير

محمد صالح — Python XAI Thesis System

نسخة مراجعة؛ الاعتماد العلمي والتحريري لم يكتمل.

تُعرض مخططات المعمارية كصور مدمجة؛ تخضع الصفحات والأسئلة والمراجع للمراجعة العلمية والمرئية.

فهرس المحتويات

1. لمحة عن النظام
2. مخططات بنية النظام ومساراته
3. البيانات والنوافذ الزمنية
4. نماذج التنبؤ وLSTM: ملاحظات أولية
5. شرح LSTM في المشروع
6. SHAP وLIME: التنفيذ المباشر
7. المقاييس والنتائج: عينة موثقة
8. لوحة Streamlit: العرض المبني على الأدلة المحفوظة
9. الاختبارات والنشر المحلي على CPU
10. حدود الاستنتاج العلمي
11. أسئلة الدفاع وإجاباتها: دفعة أولى
12. فهرس آلي أولي للشيفرة
13. سجل المراجعة العلمية: دفعة أولى
14. تقرير التغطية المؤقت
15. ملحق الشروح السطرية

لمحة عن النظام

يعيد المشروع بناء إطار بحثي للتنبؤ بالسلاسل المالية وتفسير مخرجاته؛ وهو لا يثبت أنه أعاد إنتاج الجدول الأصلي للرسالة. يوضح README أن البيانات والوحدات والتهيئة الأصلية غير مكتملة وأن ملحق الرسالة يستخدم نماذج مختلفة. لذلك ينبغي وصف المخرجات المحفوظة بأنها نتائج محسوبة حديثاً، لا نتائج أصلية مستعادة.

يستخدم التنفيذ أربع فئات تنبؤ: LinearRegression وRandomForest وGradientBoosting وLSTM، مع مرجع Persistence. يجري بناء السمات والنوافذ وتقسيمها زمنياً ثم اختيار النماذج باستخدام التحقق، وتقييمها على الاختبار. تحفظ الشيفرة التنبؤات والمقاييس والتفسيرات ويعرضها Streamlit. ينص README على أن العرض الافتراضي يعيد عرض نتائج محفوظة وليس تنبؤاً حياً بسوق اليوم.

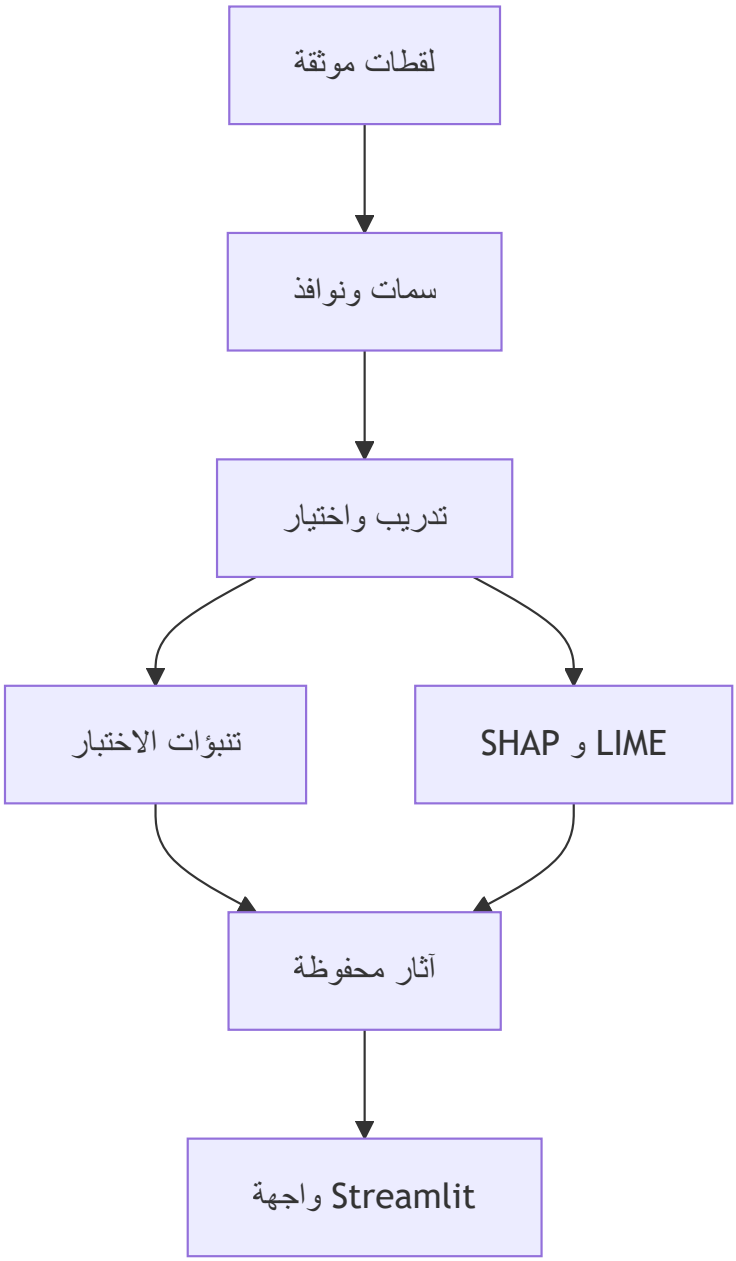
انظر الشكل (1) في فصل المعمارية.

هذا الرسم يلخص المسار العام الذي يظهر في `src/xai_finance/pipelines/train.py` وملفات الشرح والواجهة؛ ليس وصفاً لكل فروع التحقق العلمي أو التنزيل. يلزم تدقيق الرسوم التفصيلية المطلوبة لاحقاً.

مخططات بنية النظام ومساراته

هذه المخططات مستخلصة من الشيفرة المشار إليها أسفل كل رسم. نجح Mermaid CLI 11.4.2، المثبت في مجلد معزول للتوثيق، في تحليل الرسوم العشرة وتصديرها إلى SVG وPNG في `mermaid_rendered/`. فُحص الرسمان 01 و07 بصرياً فقط؛ لم تكتمل مراجعة بقية الرسوم أو اتجاه النص المختلط. نجاح التحليل لا يثبت صحة المخططات العلمية. الأسهم تلخص الاعتماد أو التسلسل ولا تعني تنفيذ كل خطوة عند عرض لوحة Streamlit.

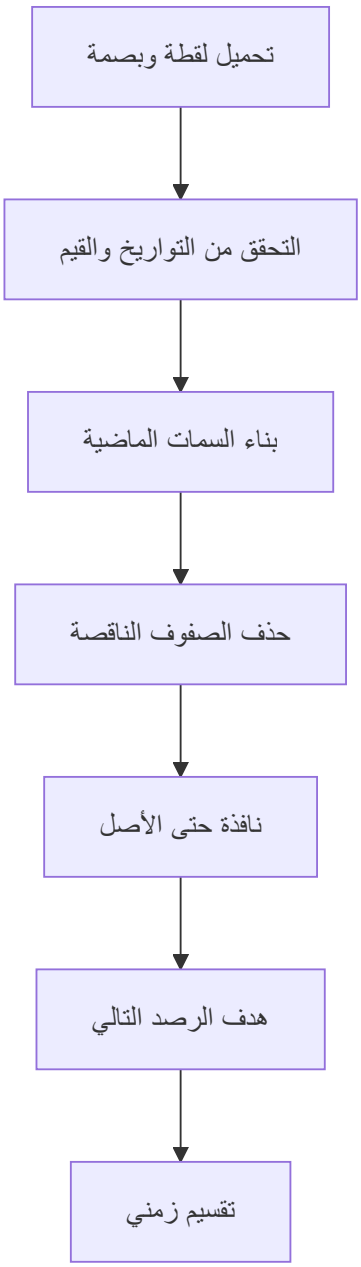
1. البنية العامة



الشكل (1): البنية العامة [FIG-01]

المصدر: `pipelines/train.py`، `pipelines/explain.py`، `dashboard/app.py`.

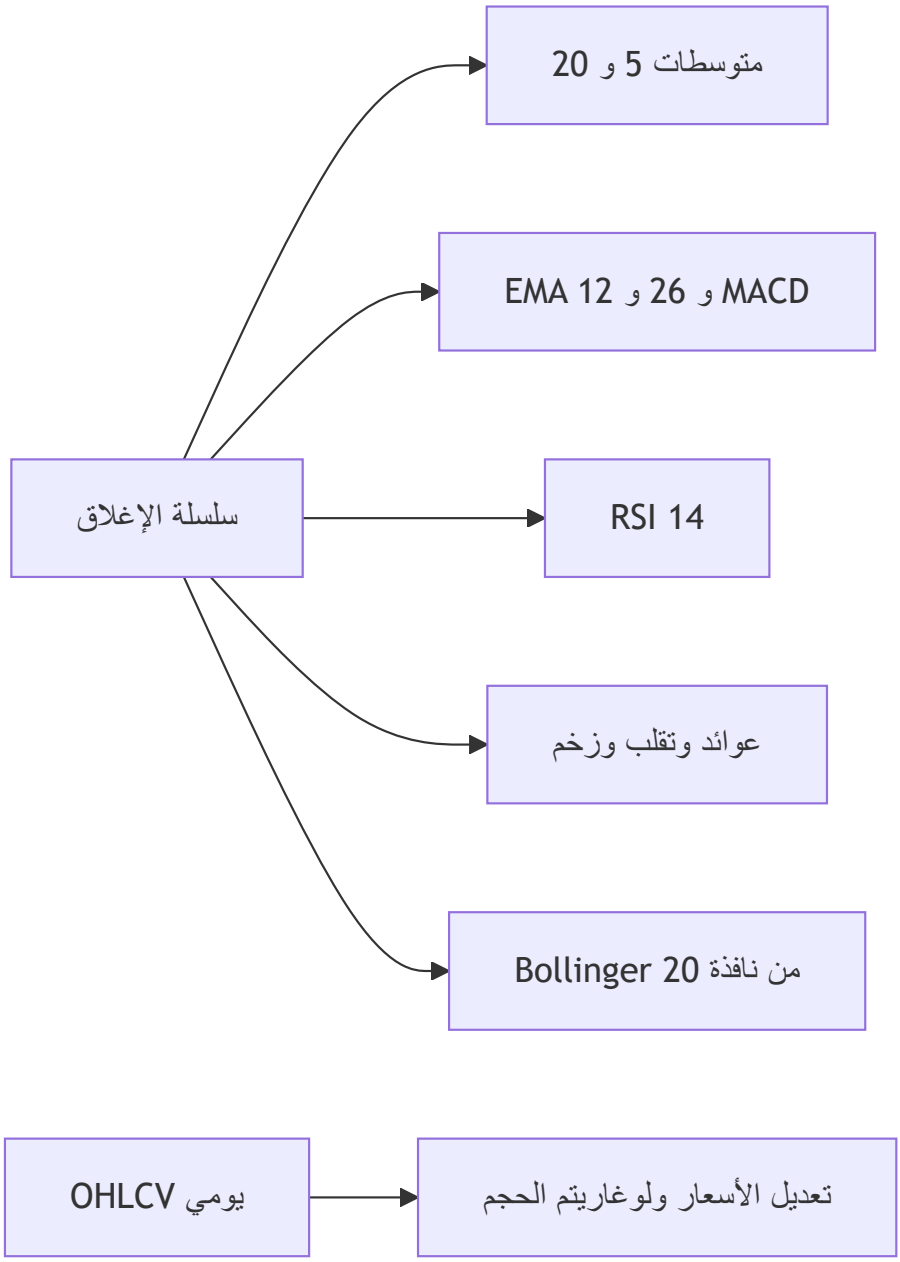
2. معالجة البيانات



الشكل (2): معالجة البيانات [FIG-02]

المصدر: `io/data_sources.py`, `data/validate.py`, `data/features.py`, `data/windows.py`.

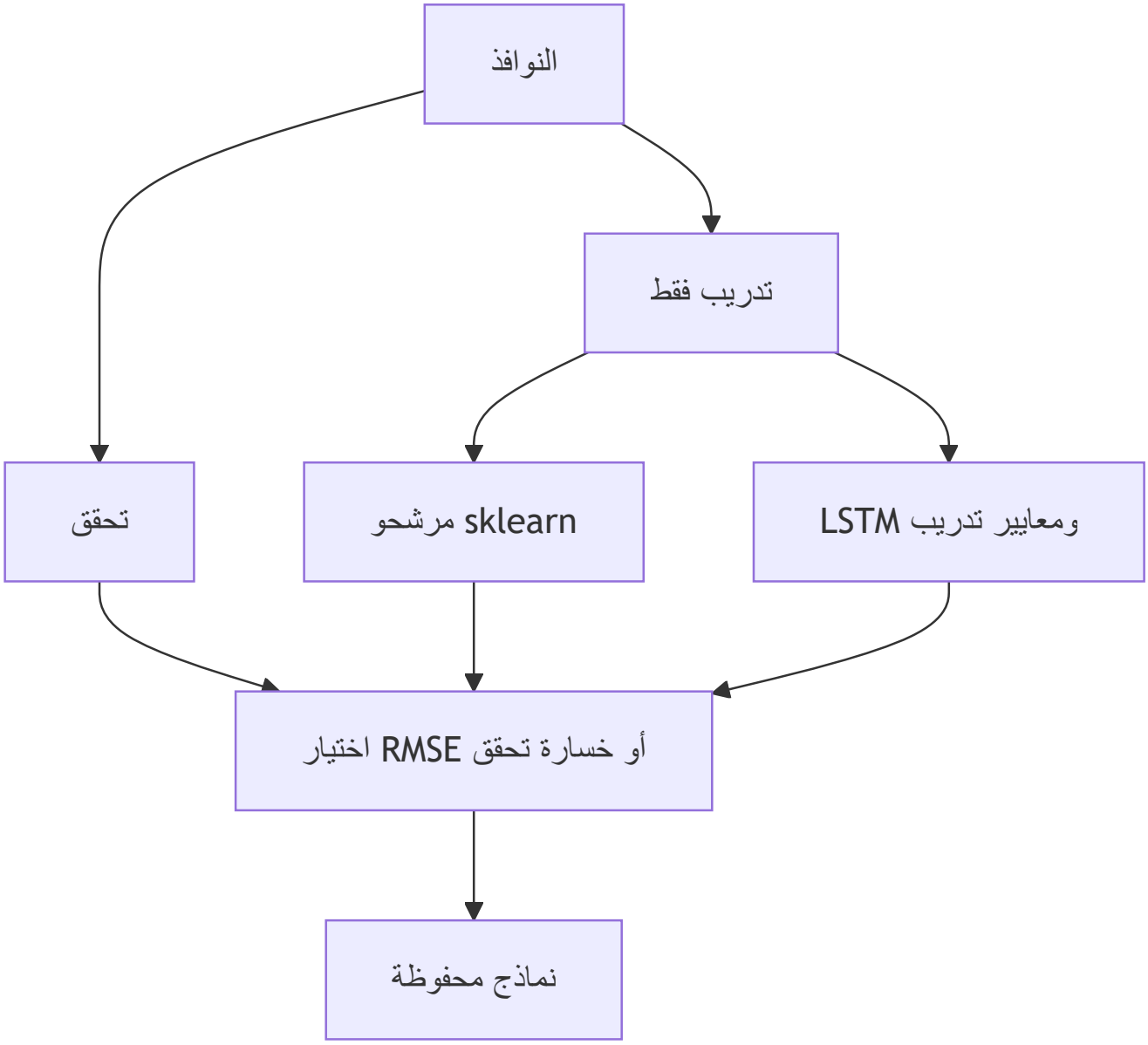
3. هندسة السمات



الشكل (3): هندسة السمات [FIG-03]

المصدر: `data/features.py`. تُطبق سمات OHLCV على التردد اليومي فقط.

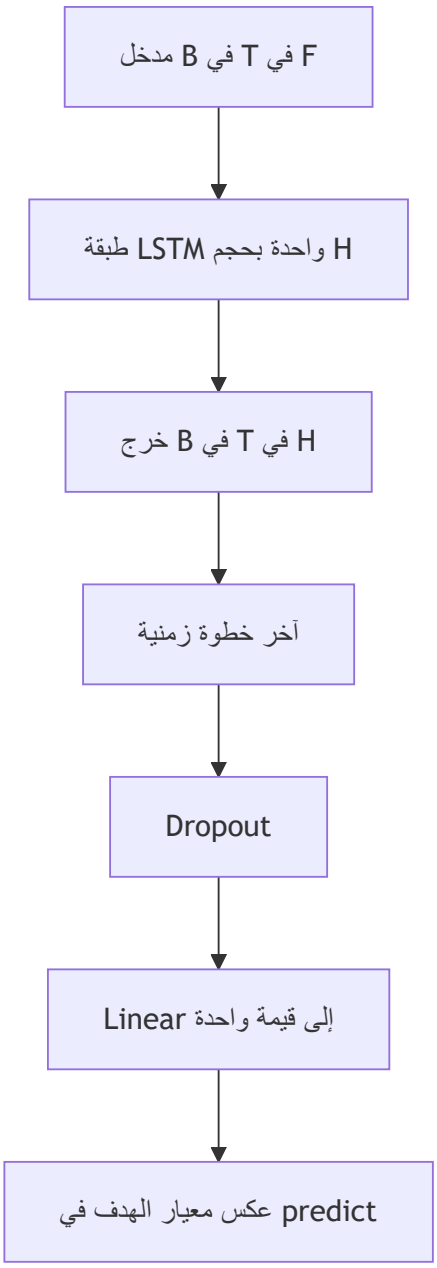
4. التدريب



الشكل (4): التدريب [FIG-04]

المصدر: `models/sklearn_models.py`، `models/lstm_model.py`، `pipelines/train.py`.

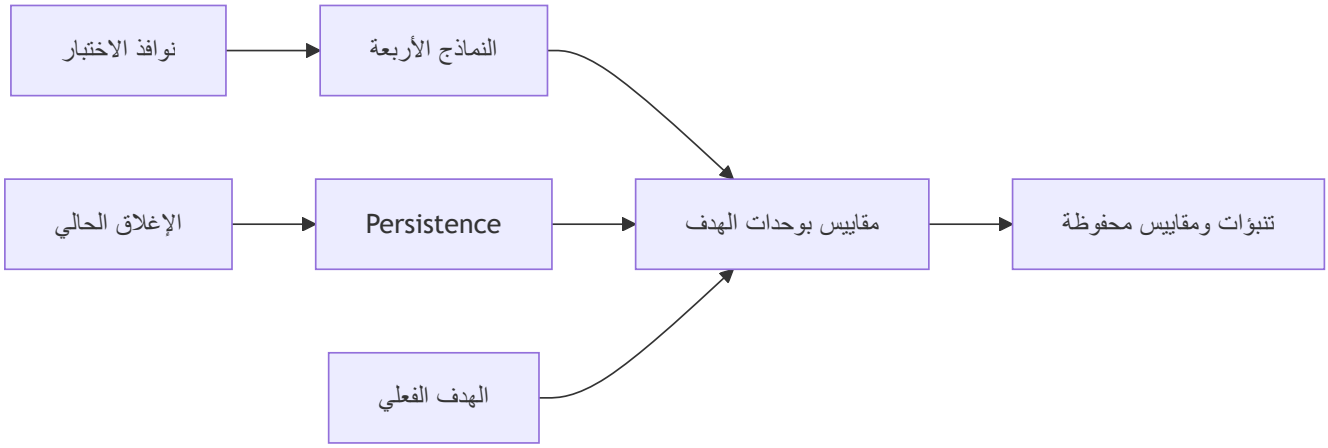
5. LSTM المنفذ



الشكل (5): LSTM المنفذ [FIG-05]

المصدر: `models/lstm_model.py:16-25,36-45`. لا تظهر الخلايا الداخلية كطبقات إضافية.

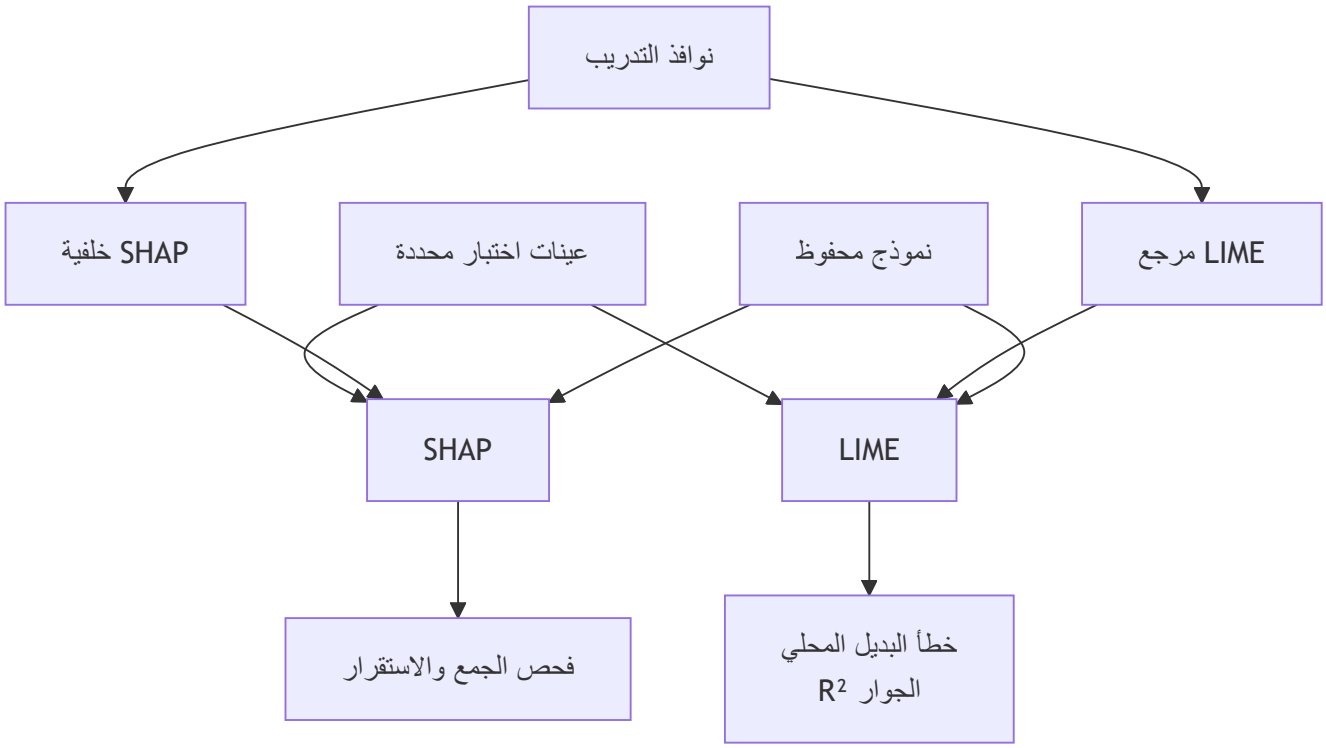
6. التنبؤ والتقييم



الشكل (6): التنبؤ والتقييم [FIG-06]

المصدر: `pipelines/train.py:89–107`, `evaluate/metrics.py`.

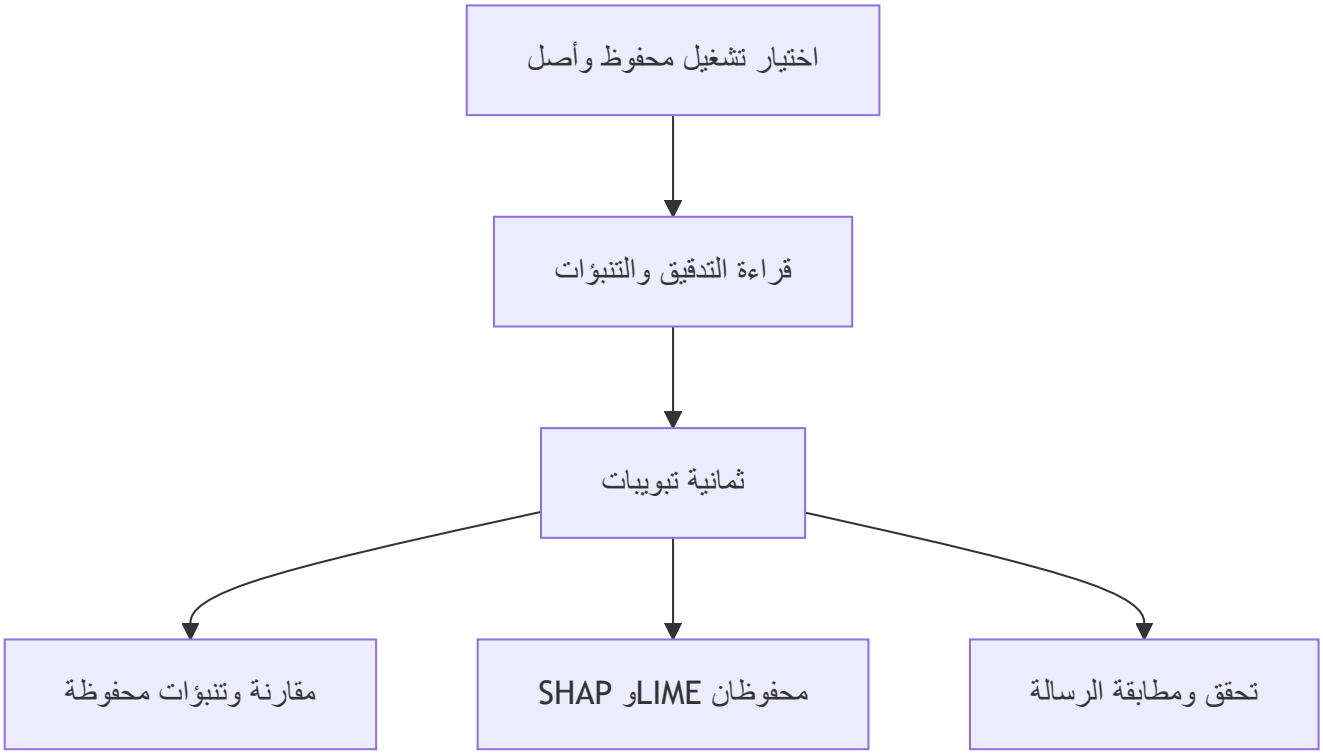
7. SHAP وLIME



الشكل (7): SHAP وLIME [FIG-07]

المصدر: `pipelines/explain.py`, `explain/shap_explain.py`, `explain/lime_explain.py`.

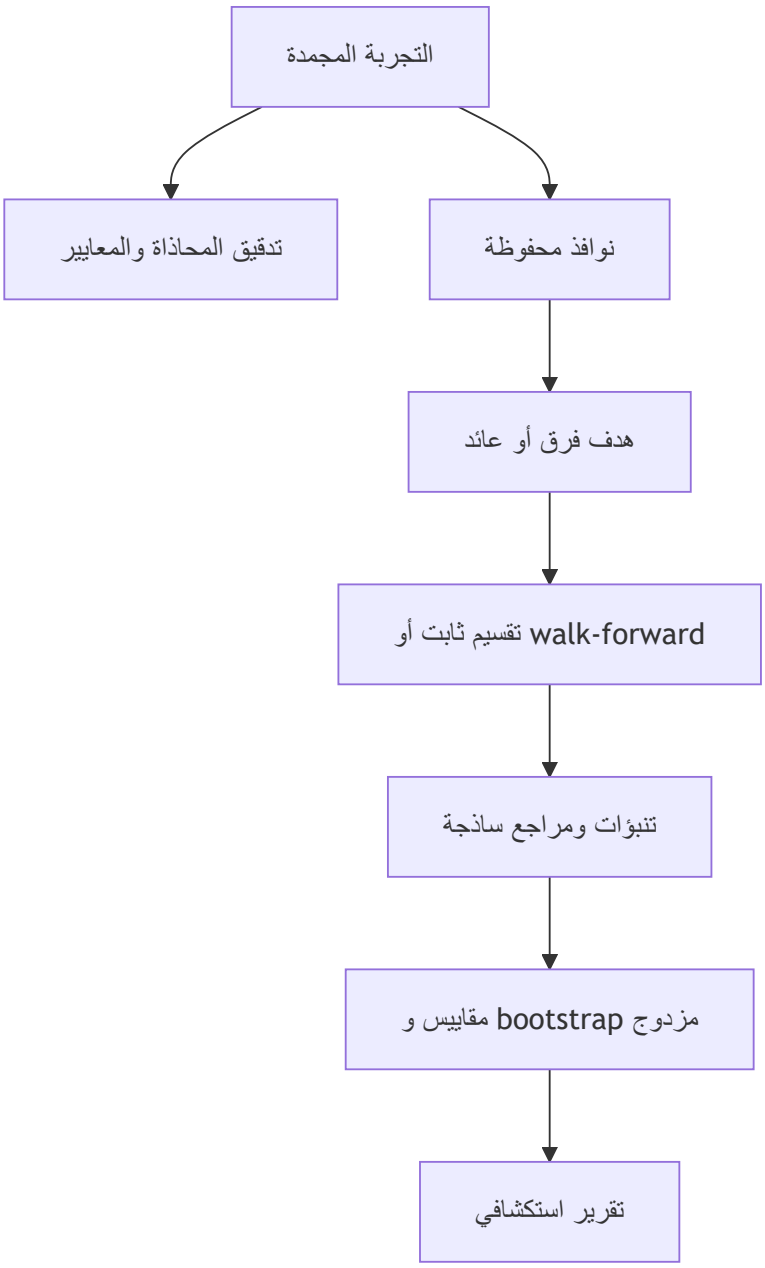
8. Streamlit



الشكل (8): Streamlit [FIG-08]

المصدر: `dashboard/app.py:179–205`. لا يستدعي الرسم تدريباً أو inference حياً.

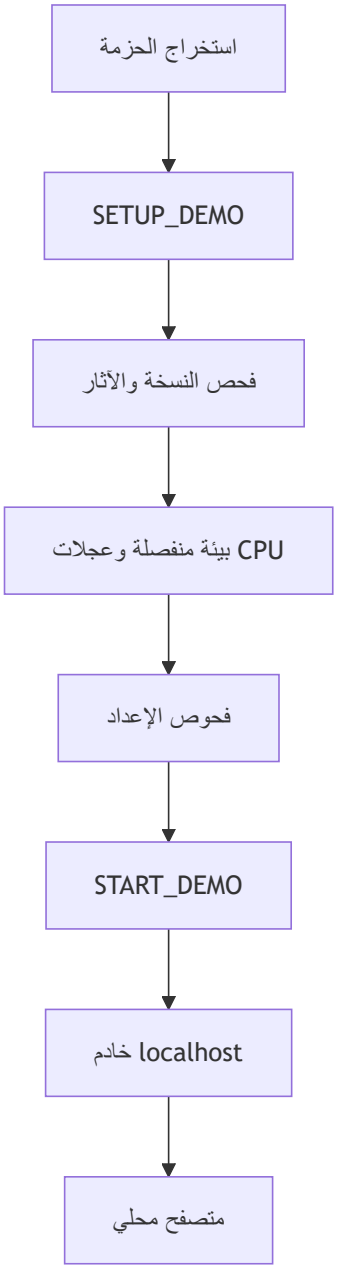
9. التحقق العلمي



الشكل (9): التحقق العلمي [FIG-09]

المصدر: `validation/audit.py`, `validation/design.py`, `validation/experiments.py`, `validation/statistics.py`. هذه التجارب السابقة لا تُعاد في إعداد التوثيق.

10. النشر المحلي CPU



الشكل (10): النشر المحلي CPU [FIG-10]

المصدر: `SETUP_DEMO.bat`, `START_DEMO.bat`, `scripts/demo_wizard.py`. التثبيت يعمل دون شبكة فقط عند توفر حزمة العجلات اللازمة؛ Python نفسه غير مرفق بحسب README.

البيانات والنوافذ الزمنية

في `data/features.py` يستعمل `features` سعر `adj_close` للبيانات اليومية و`close` للشهرية. للبيانات اليومية يضرب open وhigh وlow بنسبة `adj_close / close` ويحوّل الحجم باستخدام `log1p`. يُحسب متوسطان SMA لنافذتين من 5 و20 رصداً، وEMA بفترتي 12 و26، وRSI بفترة 14، وMACD وإشارته، وحدّي Bollinger بمقدار انحرافين معياريين من نافذة 20. كما يحسب العوائد والإزاحات والتقلب والزخم. تستبدل القيم اللانهائية بقيم مفقودة ثم تحذف الصفوف الناقصة؛ هذا يشمل نقص بداية نوافذ المؤشرات، وقد يشمل أسباباً أخرى.

في `build_windows` تكون نافذة الإدخال من الموضع `position-lookback+1` إلى `position` شاملاً، والهدف إغلاق الرصد التالي `position+1`. مثال توضيحي غير مأخوذ من AAPL: إذا كانت الإغلاقات 10 و11 و12 و13 و`lookback=3`، فإن النافذة الأولى تنتهي عند 12 وهدفها 13. لا تُستخدم قيمة الهدف بوصفها مدخلاً في هذه النافذة. العدد الفعلي في `configs/full_experiment.yaml` هو 30 للرصد اليومي و12 للشهري.

يقسّم `chronological_indices` العينات المتحاذية إلى أول `int(0.8*n)` للتدريب، ثم `int(0.1*n)` للتحقق والباقي للاختبار، ويشترط 30 عينة على الأقل. قد تتداخل نوافذ السياق المتجاورة عبر الحدود؛ لا يعني ذلك وحده استعمال أهداف مستقبلية في التدريب. لتقييم التسرب يجب فحص محاذاة الهدف والسمات والمصدر، ولا تكفي هذه النسبة وحدها.

في `fit_training_scalers` يقدّر `StandardScaler` للمدخلات من نوافذ التدريب وحدها وللأهداف من أهداف التدريب وحدها؛ تتكرر الصفوف عند تداخل نوافذ التدريب وبالتالي يختلف وزنها عن تقدير scaler مرة لكل تاريخ فريد. الصيغة لكل عمود هي $z=(x-\mu_{train})/\sigma_{train}$. يعكس `inverse_target` التحويل لتصبح التنبؤات بوحدات الهدف. يتبع ذلك مثال AAPL من آثار محفوظة؛ أما تواريخ الإتاحة التاريخية فلم تُثبت.

مثال AAPL من ملفات محفوظة، لا مثال افتراضي

من التشغيل `artifacts/runs/full_20261021T040901084647Z/AAPL`: طابقت بصمة SHA-256 للملف `data/raw/AAPL.csv` القيمة
`87e5bef4aec197b26bd182f3ee796c9c19b91ec8f50ceb70ee9d0fb89ab64755` المسجلة في `data_audit.json`. في اللقطة عند 29-11-2024 كان `close=237.330001831` و`adj_close=235.620101929`؛ الإغلاق المعدل هو `close` في جدول السمات، ويحسب فتح معدل

بضرب 234.809997559 في نسبة الإغلاقين لينتج 233.11825362137233 كما في features.csv. من سمات ذلك التاريخ SMA5 تساوي 232.32600097660003 وRSI14 تساوي 66.66982564972335.

في windows.npz أول فهرس اختبار هو 2432 وشكل جميع النوافذ (2703, 30, 20). يبدأ عمود الإغلاق لتلك النافذة بالقيمة 233.050476074 وينتهي عند 235.620101929 بتاريخ أصل 29-11-2024. الهدف المحفوظ هو 237.863830566 بتاريخ 02-12-2024 ويطابق adj_close في صف اللقطة التالي. تاريخا الرصد لا يفترضان أن الفترة يوم تقويمي واحد. يسجل data_audit.json 2162 نافذة تدريب و270 تحقق و271 اختبار، مع حذف 33 صف سمات في بداية السلسلة.

في LinearRegression/predictions.csv التنبؤ المحفوظ لهذا الهدف هو 236.67855707949116، والخطأ الفعلي ناقص المتوقع نحو 1.18527 دولار للسهم المعدل. هذه مطابقة قراءة ملفات وليست إعادة تشغيل للنموذج؛ لم نتحقق هنا من القيم المعيارية الداخلية أو مساهمات SHAP وLIME للرصد. اللقطة معدلة بأثر رجعي، فلا تثبت ما كان متاحاً زمنياً بصيغته التاريخية آنذاك.

نماذج التنبؤ وLSTM: ملاحظات أولية

يحوّل `WindowDataset.flat` النافذة إلى متجه ثنائي الأبعاد لنماذج sklearn. يكوّن `candidates` مسار LinearRegression من StandardScaler ثم IndependentColumns ثم الانحدار. يتنبأ الانحدار الخطي بالشكل $\hat{y}=\beta_0+\Sigma\beta_j x_j$؛ تزيل IndependentColumns الاعتماد العددي قبل الملاءمة، ويجب مراجعة ملفها لتفاصيل المعيار. لا يعني الوزن سببية.

تجمع RandomForest تنبؤات أشجار متعددة، بينما GradientBoosting يبني أشجاراً متتابعة لتحسين دالة الخسارة. في التهيئة الكاملة: `trees=100` و`search=true`؛ المرشحان للغابة بعمق 5 أو 10 و`min_samples_leaf=3`، وللتعزيز بعمق 2 ومعدل 0.03 أو عمق 3 ومعدل 0.1. يختار `fit_candidates` أقل RMSE على التحقق دون إعادة الملاءمة على التدريب والتحقق معاً. الأشجار ذات الأوراق الثابتة محدودة في استقراء مستويات خارج مجال التدريب.

تستعمل `PriceLSTM` طبقة `nn.LSTM` واحدة مع `batch_first=True` ثم Dropout وطبقة خطية ذات خرج واحد. يختار `forward` خرج آخر خطوة زمنية `sequence[:, -1, :]`؛ ليست Dropout طبقة LSTM ثانية. تتعلم الشبكة من نوافذ معيارية، ثم تعكس `LSTMPredictor.predict` معيار الهدف. القيم الافتراضية للتجربة الكاملة: Adam، dropout=0.1، hidden_size=32 بمعدل 0.001، حد 60 epoch وصبر 8. يُحفظ أفضل checkpoint وفق خسارة التحقق ويعاد تحميل حالته بعد التدريب. الخسارة MSE على الهدف المعياري ويطبق قص معيار التدرج عند 1.

في CUDA يفحص `probe_precision` BF16 إن كان مدعوماً وإلا FP16؛ يرفض الدقة المختلطة إذا كانت الخسارة غير منتهية أو اختلفت بأكثر من 5% عن FP32 على عينة الاختبار الداخلية للتدريب. CPU يستخدم FP32 ويختار دفعة لا تتجاوز 32. تحميل `best.pt` يستخدم `map_location` على الجهاز المختار. حساب البوابات والتفاضل عبر الزمن تنفذه مكتبة PyTorch داخل `nn.LSTM` وautograd؛ ليس مكتوباً يدوياً في المشروع. لا تكفي هذه الملاحظات لشرح LSTM أو كل أسطره بالتفصيل المطلوب.

شرح LSTM في المشروع

دور النموذج ومدخلاته

يتعلم النموذج علاقة غير خطية بين نافذة سمات تاريخية والهدف التالي في السلسلة. لا يضمن استخدام الذاكرة تحسناً على Persistence أو استقراء ناجحاً خارج مستويات التدريب. ينشئ `build_windows` مصفوفة (عدد العينات، طول النافذة، عدد السمات)، ويقيّس `fit_training_scalers` المدخلات والهدف من التدريب فقط. في آثار AAPL المحفوظة الشكل `(2703, 30, 20)`؛ مع دفعة B يكون دخل `PriceLSTM` بالشكل `(B, 30, 20)`. إعداد التجربة الكاملة `hidden_size=32`، ومن ثم سلسلة الخرج `(B, 30, 32)`؛ تأخذ الشيفرة الخطوة الأخيرة `(B, 32)`، ويعيد الرأس الخطي `(B, 1)` ثم `squeeze(-1)` الشكل `(B,)` حتى مع B=1. السلسلة التاريخية كلها مستخدمة في حساب الخرج الأخير؛ لا يجمع الرأس مخرجات الخطوات جميعها مباشرة.

معادلات الخلية وعلاقتها بالمكتبة

لم يكتب المشروع هذه البوابات يدوياً: يستدعي `PriceLSTM.__init__` في `src/xai_finance/models/lstm_model.py:19` طبقة `torch.nn.LSTM`. المعادلات التالية تصف خلية LSTM القياسية داخل PyTorch، حيث x_t متجه السمات في الزمن t، وh_t الحالة المخفية، وc_t حالة الخلية، وsigmoid σ، و⊙ ضرب عنصري. أوزان W وU والتحيزات b معاملات تتعلمها الطبقة؛ ليست أرقاماً استخرجت من checkpoint:

- بوابة النسيان: $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$ تحدد نسبة الاحتفاظ من حالة الخلية السابقة.
- بوابة الإدخال: $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$ تتحكم في إضافة المعلومة.
- الحالة المرشحة: $g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g)$ تمثل المحتوى المرشح للإضافة.
- تحديث الخلية: $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ يجمع الاحتفاظ والتحديث.
- بوابة الإخراج: $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$؛ الحالة المخفية $h_t = o_t \odot \tanh(c_t)$ هي خرج الخطوة.

التحيز في المعادلات اختزال لمجموع تحيزي الإدخال والتكرار في PyTorch. لكل عينة بعد h وc يساوي hidden_size=32 في التكوين الكامل. لا تمرر `forward` حالة ابتدائية صريحة؛ تستعمل المكتبة حالاتها الافتراضية الصفرية لكل استدعاء، ولا تحمل حالة من دفعة لأخرى بهذه الشيفرة. بوابات sigmoid محدودة بين صفر وواحد، مما يتيح تدفقاً انتقائياً للمعلومة والتدرج، لكنه لا يزيل كل مشكلات التدرج.

البنية المنفذة

انظر الشكل (5) في فصل المعمارية.

الرسم يميز مخرج الطبقة sequence عن الحالة النهائية التي تتجاهلها الشيفرة. في `eval()` تتوقف إسقاطات Dropout العشوائية. لا توجد طبقة LSTM ثانية أو attention في `PriceLSTM`. الرسم مبني على فحص المصدر، ولم يُفحص بمترجم Mermaid في هذه الجلسة.

التدريب والاختيار والاستدلال

`fit_lstm` يختار الجهاز ويهيئ معايير التدريب والموترات والشبكة؛ `scaled_loss` تحسب متوسط `(ŷ_scaled−y_scaled)²`. في `train_epoch` تحسب `backward()` تفاضل الرسم عبر الخطوات الزمنية (BPTT) وفق مشتقات PyTorch؛ ثم يفك GradScaler المقياس قبل قص معيار التدرج عند 1، وينفذ Adam تحديث المعاملات. دقة AMP تختبر على fixture من التدريب؛ تقبل فقط إذا كانت الخسارة منتهية والفرق النسبي عن FP32 لا يتجاوز 5%، وهذا ليس برهاناً على جودة كل التدرجات. على CPU الدقة FP32 وحجم الدفعة محدود بـ32؛ CUDA يجرب الأحجام ويختار أكبر حجم مقبول وفق قياس الذاكرة.

`optimize_epochs` يقيم خسارة التحقق دون تدرج في وضع eval بعد كل epoch، ويحفظ last وbest ويطبق early stopping حسب patience، ثم يعيد أوزان أدنى خسارة تحقق. لا يستخدم الاختبار للاختيار في هذا المسار. في الاستدلال يعيد `LSTMPredictor.predict` بناء النوافذ وتقييسها، ويستعمل no_grad ودفعات حتى 256، ثم يعكس تقييس الهدف. `load_lstm` يحمل أفضل checkpoint إلى الجهاز المطلوب عبر map_location مع scalers محملة منفصلاً.

مثال نتيجة محفوظة وحدودها

يسجل `artifacts/runs/full_20261021T040901084647Z/AAPL/training.json` تدريب CUDA بدقة BF16، ودفعة 64، وأفضل epoch رقم 2 من 10 وخسارة تحقق معيارية `0.34953898191452026`. هذا وصف لذلك السجل، لا إعداد مضمون لكل تشغيل. ويسجل `AAPL/LSTM/metrics.json` لاختبار 271 رصداً RMSE قدره `61.414866257066755` وR^2 قدره `-4.637234640332287`. لم نعد حساب هذه النتائج هنا. ترتبط القيم بمجموعة وفترة ونموذج مدربين محددين؛ لا تُثبت سبباً وحيداً لتفوق أو ضعف النموذج.

راجع الشرح السطري لـ LSTM ومسار التدريب للمواقع المحددة ومخارج الدوال وآثارها.

SHAP وLIME: التنفيذ المباشر

في `explain/shap_explain.py` يستخدم LinearRegression مفسّر `LinearExplainer` بعد تطبيق scaler وselector؛ تعاد المساهمات إلى مساحة السمات الأصلية بملء السمات المستبعدة بأصفار. يستخدم نموذجا الأشجار `TreeExplainer` مع background و`feature_perturbation="interventional"` وفحص الإضافة. لبقية النماذج يستخدم `PermutationExplainer` ودالة `model.predict` وقناع Independent، مع ميزانية `cycles*(2*p+1)` لتابع ذي p مدخلاً.

تمثل قيمة Shapley إسهاماً موزوناً للميزة عبر تحالفات الميزات بالنسبة إلى مرجع وتعريف للإخفاء؛ ليست تأثيراً سببياً. يفحص `shap_reconstruction` تقريباً أن base value زائد مجموع الإسهامات يساوي التنبؤ بتسامح نسبي 1e-4 ومطلق 1e-3. تحقق الجمع لا يدل على جودة التنبؤ أو أن توزيع السمات المصطنع واقعي.

يستخدم `explain_lime` مفسراً جدولياً للانحدار مع تقطيع المتغيرات المستمرة وبذرة `highest_weights`، ويطلب عشرة متغيرات وعدد عينات من الإعداد. يسجل تنبؤ النموذج والبديل المحلي والفارق المطلق وR^2 الموزون للحي. يمكن أن يختلف LIME وSHAP بسبب المرجع وتوليد الجوار وتقريب نموذج محلي مقابل توزيع الإسهامات؛ والارتباط قد يولد حالات غير واقعية. يرد أدناه مثال كمي محفوظ لمشاهدة واحدة؛ لم تُراجع جميع الأمثلة المحفوظة.

مثال محفوظ للتفسير وحدود صلاحيته

يسجل `s/full_20261021T040901084647Z/AAPL/LinearRegression/explanations/quality.json` عشرة تفسيرات SHAP وخلفية من ثماني نوافذ تدريب، وأقصى خطأ إعادة بناء محفوظ `3.694822225952521e-13`. هذا مؤشر على جمع المكونات للتنبؤ في المجموعة المحسوبة، وليس اختباراً لسببية أو جودة التنبؤ.

للرصد ذي هدف 02-12-2024 ونهاية معلومات 29-11-2024 يسجل الملف تنبؤ النموذج `236.6785570794911` وتنبؤ LIME المحلي `-266.1841807481178` وفارقاً مطلقاً `502.8627378276089` وR^2 للجوار `0.44369535545682715`، مع 1000 عينة وبذرة 42. الفارق الكبير يجعل الاعتماد على البديل المحلي لهذه الحالة غير مبرر دون تحليل إضافي. هذه قيم محفوظة قُرئت من التقرير وليست حسابات جديدة؛ لا يصح تعميمها على كل النماذج أو المشاهدات.

تحقيق الخطأ المحلي المحفوظ لـ AAPL

في explain/lime_explain.py تُحسب القيمة هكذا: absolute_local_error = abs(model_prediction - local_surrogate_prediction)، حيث model_prediction = local_surrogate_prediction = predict(observation[None])[0] و explanation.local_pred[0]. هذه **قيمة مطلقة لفارق تنبؤين عند الرصد الأصلي**، وليست MSE أو RMSE، وليست متوسط أخطاء العينات المضطربة. قيم الملف المحفوظ للرصد ذي هدف 02-12-2024 هي 236.6785570794911 للنموذج و266.1841807481178 للبديل، والفارق المسجل 502.8627378276089. وحدة الثلاثة دولار لكل سهم معدل، مثل مخرج نموذج السعر؛ ليس رقماً معيارياً أو نسبة مئوية.

يفحص التنفيذ المحلي المثبت في .demo_venv/Lib/site-packages/lime/lime_tabular.py السطور 340–355 و452–453: يولد LIME جواراً مضطرباً، ويستعمل تمثيل scaled_data للمسافات والبديل، ويمرر inverse إلى دالة تنبؤ النموذج. في lime_base.py:194–198 يمثل local_pred تنبؤ البديل على الصف الأول، وهو الرصد الأصلي؛ أما score فهو R^2 موزون لملاءمة البديل عبر الجوار. مع discretize_continuous=True يستعمل LIME تمثيلاً تفسيرياً قائماً على الفئات/الشرائح للبديل؛ لا يعني ذلك أن قيم النموذج نفسها تغذى معيرة بهذا التمثيل. يأخذ مسار المشروع windows.flat الأصلي للسمات والخلفية؛ يطبق Pipeline الخاص بالانحدار StandardScaler وIndependentColumns داخلياً قبل التنبؤ. مخرجات الانحدار وأهدافه بوحدات السعر الأصلية. هذا الوصف يعتمد النسخة المحلية المثبتة؛ لم نتحقق هنا من تطابق بصمة مكتبة بيئة التجربة التاريخية معها.

weighted_neighborhood_r2=0.44369535545682715 هو مقياس ملاءمة مختلف على الجوار الموزون، وليس مربع الخطأ المذكور ولا دقة تنبؤ الأسعار. الفرق الضخم عند العينة دليل مباشر على ضعف مطابقة البديل لتنبؤ النموذج **لهذا التفسير المحفوظ**، فلا يصلح الاعتماد على أوزانه وحدها لشرح هذه الحالة بثقة. الارتباط بين مؤشرات فنية متعددة وإمكانية أن ينتج الاضطراب حالات غير واقعية تفسير محتمل للضعف، لكنه غير مثبت سبباً وحيداً بهذا السجل. لا تثبت الحالة فشل جميع تفسيرات LIME أو أن SHAP صحيح سببياً.

تعرض dashboard/app.py:113–124 تنبؤ النموذج والبديل والخطأ المطلق وR^2 وتحذيراً عاماً من الارتباط والجوار غير الواقعي. الأرقام تتيح اكتشاف المشكلة، لكن الواجهة لا تضع عتبة خاصة أو حكماً صريحاً بضعف هذا التفسير عند كبر الخطأ؛ لذلك ينبغي للطالب شرح ذلك شفهياً.

إجابة دفاع مقترحة: «القيمة 502.86 هي القيمة المطلقة للفرق بين مخرج النموذج والبديل المحلي عند مشاهدة AAPL الأصلية، بوحدة الدولار المعدل، وليست RMSE. دُرب البديل على جوار مضطرب موزون

وكانت ملاءمته محدودة؛ لذلك لا أقدّم أوزانه دليلاً موثوقاً لهذه المشاهدة. أعرض الخطأ وR^2 وأحتفظ بالنتيجة كما هي، ولا أستنتج سببية أو فشلاً عاماً لـ LIME من حالة واحدة».

المقاييس والنتائج: عينة موثقة

في `evaluate/metrics.py` مع الخطأ $e_i=y_i-\hat{y}_i$: $MAE=mean(|e_i|)$، و$RMSE=sqrt(mean(e_i^2))$، و$MAPE=100\ mean(|e_i/y_i|)$ بعد استبعاد الأهداف التي لا تتجاوز قيمتها المطلقة 1e-12. يعيد R^2 القيمة $1-\Sigma e_i^2/\Sigma(y_i-\bar{y})^2$ عندما يكون عدد المشاهدات أكبر من واحد والهدف غير ثابت؛ وإلا يعيد None.

الدقة الاتجاهية في الشيفرة هي $mean(sign(\hat{y}-current)==sign(y-current))$. تميّز الصفر عن الصعود والهبوط، بينما مصفوفة الالتباس تستخدم شرط `current` < ثنائياً وتجمع التعادل مع غير الصعود. لذا قد تبدو دقة Persistence الاتجاهية صفراً حتى مع انخفاض خطئه: يتنبأ بثبات السعر، ولا يطابق إلا تغيّراً فعلياً صفرياً.

يوثق `docs/RECONSTRUCTED_RESULTS.md` لاختبار 271 AAPL، هدفاً بين 02-12-2024 و31-12-2025، RMSE قدره 4.18937 لـ Persistence و4.57158 للانحدار الخطي و61.4149 لـ LSTM بالدولار للسهم المعدّل. يذكر `docs/RESULTS_AUDIT.md` أن Persistence حقق أدنى RMSE في الأصول اليومية الخمسة. هذه أرقام من تقرير محفوظ ولم يجر هنا إعادة حسابها من التنبؤات الخام.

لم يُستعد ترتيب الرسالة الأصلي لأن سياق جدولها الأصلي من أصول وفترات ووحدات وإعدادات غير محدد بصورة كافية، والملحق يختلف عن السرد؛ لا يمكن الجزم بسبب واحد للفرق أو استنتاج أن النتيجة الأصلية خاطئة. تغير المستويات، وحدود الاستقراء، والتهيئة أمثلة لتفسيرات محتملة لا إثباتات سببية. البيانات المعدلة أو المنقحة بأثر رجعي والاختبار الثابت يحدان من التعميم.

التحقق الإحصائي الإضافي

يعرف `validation/design.py` هدف التغير $\Delta P=P_{t+1}-P_t$ والعائد البسيط $r=P_{t+1}/P_t-1$، ويعيد تحويل التنبؤ إلى السعر بـ $P_t+\Delta\hat{P}$ أو $P_t(1+\hat{r})$. يتحقق بروتوكول العائد من موجبية الأسعار ولا يعالج الملاحظة السالبة بقصها. مرجع Zero يعطي سعر Persistence بعد العكس. هذه تحليلات استكشافية لمجموعة اختبار عُرفت نتائجها سابقاً، فلا يصح وصفها بتأكيد مستقل نهائي.

يقسم `walk_forward_partitions` الاختبار إلى كتل متتابعة؛ لكل كتلة تدريب تاريخي وتحقق أخير ثابت الحجم قبلها، مع تأكيد توفر آخر هدف للتحقق عند أصل الاختبار الأول. في التجارب المسجلة يعاد تدريب النماذج بين الكتل، لا عند كل مشاهدة، ويستعمل السياق الفعلي المتاح عند كل أصل. لا يثبت فحص الترتيب توفر النسخة المعدلة من الأسعار وقتها.

يقارن `paired_bootstrap` لكل تاريخ مربع خطأ النموذج بمربع خطأ Persistence على الرصد نفسه، ويأخذ كتل دائرية بطول 10 و2000 سحب ببذرة 42؛ متوسط الفرق السالب يفضل النموذج. حدود 2.5% و97.5% تقريبية وتتأثر بملاءمة تمثيل الاعتماد الزمني والاستقرار، ولا تشمل إعادة تدريب النماذج أو تصحيح تعدد المقارنات. لا تخلط فرق MSE بوحدة سعر مربعة مع فرق RMSE بوحدة السعر.

يحسب `diagnostics.lstm_output_bound` حداً محافظاً للرأس الخطي: بما أن كل إحداثي خرج مخفي بين −1 و1، فإن الخرج المعياري في $[b-\Sigma|w_i|,\ b+\Sigma|w_i|]$؛ يعكس معيار الهدف هذا المجال إلى وحدات السعر. الحدود قد لا تكون قابلة للتحقيق معاً، وهي لهذا النموذج وبنيته ومعاييره المحفوظة. يقارن تدقيق الانزياح أيضاً مدخلات الاختبار بمجالات التدريب؛ التشخيص لا يثبت بمفرده السبب الكامل للأخطاء.

لوحة Streamlit: العرض المبني على الأدلة المحفوظة

نقطة الدخول `main.py` تشغّل `dashboard/app.py`. يختار المستخدم من الشريط الجانبي تجربة محفوظة وأصلاً؛ تقرأ `main` سجل التشغيل و`data_audit.json` و`predictions.csv`، وتحذر عند اختيار بيانات شهرية من المراجعات وأزمنة النشر غير المعروفة. لا تشغّل الواجهة النموذج في هذه المسارات؛ حتى تنبؤ الرصد التالي يقرأ من `next_forecast.json` المحفوظ ويعرض كاستقراء غير متحقق.

التبويب	ما يعرضه والآثار	التحكم وتفسير الدفاع
Overview	الهدف والوحدات والحالة، وصورة التدفق إن وجدت؛ `overview`.	يشرح الطالب أن نتائج إعادة البناء منفصلة عن الادعاءات التاريخية وأن التفسير غير سببي.
Data & preprocessing	لقطة CSV محلية إن وجدت، تقسيم زمني وصورة وسمات وبيانات المصدر؛ `preprocessing`.	يؤكد الطالب اختلاف الشكل المسطح لـ sklearn وتسلسل LSTM وأن الرسم محفوظ من اللقطة.
Forecasting	خطوط القيم الفعلية والمتوقعة وجدول الاختبار وتوقع لاحق محفوظ؛ `forecasting`.	يختار نموذجاً؛ لا يعني تغيير الاختيار إجراء inference جديد.
Compare models	`metrics.json` و`training.json`، جدول ورسم مقياس؛ `comparison`.	يختار MAE أو RMSE أو R^2 أو الدقة الاتجاهية ويقارن مع Persistence دون ادعاء ربح.
Explain predictions	`quality.json` و`shap.npz` وملف LIME وصور؛ `explanations`.	يختار النموذج وتاريخ هدف من مجموعة مسبقة؛ تعرض الطريقتان الرصد نفسه وتظهر جودة التقريب.
Regimes	`regimes.json` وصورة، وملف محركات LSTM إن توفر؛ `regimes`.	يصف المجموعات وأحجامها؛ نص الواجهة يستعمل عتبة 5% ويجب ربطها بإعداد التشغيل الفعلي.
Reproducibility	metadata وتقارير اختيارية واختبارات ونتائج تحقق محفوظة؛ `reproducibility`.	يختار بروتوكول تحقق، ويمكنه تنزيل تقارير؛ الأوامر المعروضة نصوص لا تنفذ تلقائياً.
Thesis alignment	مرجع تاريخي ومقاييس محسوبة وملف تتبع؛ `thesis_alignment`.	يوضح أن نقص وحدات ومجموعة بيانات الجدول الأصلي يمنع مقارنة رقمية عادلة.

تُعرض رسالة إذا غاب `quality.json` وصورة بديلة عند غياب صورة اختيارية، ويوقف `main` العرض برسالة إذا لم تتوفر تجارب. بالمقابل لا تحيط الشيفرة بكل قراءة بكتلة `try/except`؛ غياب بعض ملفات JSON وCSV المطلوبة أو فسادها قد يؤدي إلى استثناء. تبني الشيفرة عارضاً لكل تبويب وتستدعيه في حلقة، ولذلك لا يُفترض أن التبويب غير المختار يعفي من قراءة ملفاته.

في مناقشة الرسالة: «تختار الواجهة أثراً محفوظاً من تجربة محددة وتكشف المصدر والوحدات والقيود؛ ليست اتصالاً آنياً بالسوق، كما لا تجعل SHAP أو LIME العلاقة سببية». تفاصيل كل سطر في شرح الواجهة ونقطة الدخول. لم تُشغّل الواجهة في دفعة التوثيق هذه.

الاختبارات والنشر المحلي على CPU

الغرض والنطاق

تتحقق اختبارات `tests/` من خصائص زمنية وحسابية وسلوك الواجهة، بينما تفحص أدوات النشر بيئة العرض والملفات المحفوظة. قراءة الاختبارات توضح المقصود منها؛ لا تعني أن هذه الجلسة شغّلتها أو أن اجتيازها يثبت صلاحية النموذج في كل سوق.

الاختبارات

في `tests/test_temporal_integrity.py` يغير الاختبار أسعار المستقبل ويتأكد من بقاء السمات السابقة، ويفحص محاذاة الهدف والتقسيم والتقييس. في `test_metrics_and_models.py` أمثلة يمكن حسابها يدوياً ومرجع Persistence وحدود الحالات؛ وفي `test_explain_consistency.py` اختبار جمع SHAP وصيغة خطأ LIME، وليس شرطاً أن تكون جودة LIME عالية. يستخدم `conftest.py` بيانات اصطناعية؛ `test_provider_integration.py` يستخدم لقطة AAPL المحلية ويُدرّب نموذج اختبار عند تشغيله. يختبر `test_lstm_training.py` تدريباً قصيراً وcheckpoint، ويُتخطى اختبار CUDA عند غياب العتاد. تفاصيل جميع الاختبارات في الفهرس الرئيسي.

الإعداد والتشغيل

تختار `SETUP_DEMO.bat` مفسراً وتشغّل المعالج؛ يفحص `scripts/demo_wizard.py` نظام Windows وPython 3.12 وx64 والآثار، وينشئ `.demo_venv` منفصلة، ويثبت الاعتمادات ويجري فحوصاً. عند توفر عجلات CPU يستخدم `--no-index`؛ وعند فرض offline مع غياب عجلة torch المطلوبة يفشل. لا تتضمن الحزمة مثبّت Python بحسب README. يستخدم `START_DEMO.bat` البيئة لتشغيل Streamlit محلياً أو إيقاف عملية العرض المتعقبة. يتحقق المعالج من PID وتاريخ إنشاء العملية قبل محاولة إيقاف شجرتها، ويبحث عن منفذ متاح.

تتحقق حزمة CPU من ملفات وبصمات مدرجة؛ SHA-256 يكشف اختلاف البايتات مقارنة بمرجع موثوق ولا يثبت وحده مصدر المرجع. أرشيف على القرص نفسه ليس نسخة احتياطية مستقلة. تصف هذه الفقرة الشيفرة الموجودة ولا تشغّل التثبيت أو إعادة التشغيل؛ انظر SETUP_DEMO.bat والمعالج وتقرير CPU السابق.

ما ينبغي قوله في الدفاع

«أختبر الترتيب الزمني وعكس المقياس وسلامة إعادة التحميل، ثم أعرض النتائج المحفوظة في بيئة CPU محلية. الاختبارات لا تثبت جودة كل تفسير أو صلاحية البيانات المعدلة بأثر رجعي، والنسخة الاحتياطية المستقلة تحتاج تحققاً منفصلاً».

حدود الاستنتاج العلمي

إعادة البناء والمرجع التاريخي

نتائج المشروع الموسومة NEWLY_COMPUTED نتائج تجربة جديدة وليست استعادة دقيقة للجدول التاريخي. وفق `docs/RESULTS_AUDIT.md` و`docs/ASSUMPTIONS_AND_DEVIATIONS.md` لم تتوفر مواصفات كافية للأصول والوحدات والفترات والتهيئة الأصلية؛ ويعرض الملحق نماذج مختلفة. لا يبرر ذلك الجزم بكيفية إنتاج الجدول الأصلي أو اتهامه بتسرب لم تتوفر أدلته.

الزمن والبيانات

التقسيم والتحقق من توافر التسمية يعتمدان على ترتيب تواريخ الرصد وافتراض التنبؤ بعد إغلاق الأصل. لقطات الأسعار المعدلة وسلسلة الإسكان المنقحة استعادية، ولا تضمن نسخة البيانات التي كانت معروفة عند القرار. بيانات النفط قد تتضمن سعراً سالباً؛ يستبعد بروتوكول العائد البسيط النفط بدلاً من إزالة هذه الملاحظة من تجربة فرق السعر. راجع `validation/design.py` وبروتوكول التحقق المحفوظ.

النموذج والإحصاء

فوز Persistence في RMSE للأصول اليومية الخمسة كما يسجله `docs/RESULTS_AUDIT.md` نتيجة تخص التشغيل والتقسيم المحددين. قد يؤدي انجراف المستويات وثبات أوراق الأشجار وحد خرج شبكة LSTM المعيّنة إلى صعوبة الاستقراء، لكن لا يجوز نسبة كامل الفرق إلى سبب واحد دون اختبار مناسب. فواصل bootstrap المزدوجة استكشافية وغير معدلة للمقارنات المتعددة ولا تشمل عدم يقين إعادة التدريب؛ النتائج من اختبار معلوم سابقاً ليست تأكيداً مستقلاً.

التفسير والتطبيق

SHAP وLIME يصفان سلوك نموذج بالنسبة إلى خلفية أو جوار؛ لا يثبتان السببية أو ربحية التداول أو زيادة ثقة المستخدم. خطأ LIME المطلق المحفوظ لـ AAPL نحو 502.8627 عند مشاهدة واحدة، وليس RMSE؛ لا يعمم على التفسيرات الأخرى. لا تتوفر دراسة جديدة لتكلفة التداول أو ردود المشاركين في هذا التنفيذ. تتطلب دعاوى الأداء العام فترات مستقلة محددة مسبقاً وتقييم بذور متعددة ومصادر بيانات بإصدارات زمنية ملائمة.

ما ينبغي قوله في الدفاع

«أفصل بين ما ذكرته الرسالة وما قاسه التنفيذ الجديد، وأعرض Persistence والنتائج السلبية والقيود الزمنية. نتائج التفسير تعكس النموذج والمرجع المختار ولا تثبت سبباً أو منفعة استثمارية».

أسئلة الدفاع وإجاباتها: دفعة أولى

الحالة: 100 سؤال؛ تحتاج المجموعة إلى مراجعة علمية وتحريرية نهائية شاملة. الأجوبة المرتبطة بالشيفرة تصف التنفيذ المحفوظ، أما تفسيرات الخلية والمفاهيم فهي نظرية مرتبطة به.

1. ما المقصود بإعادة بناء الرسالة؟

الإجابة المختصرة: تنفيذ تجربة جديدة وفق معلومات متاحة وقرارات معلنة، لا استعادة تجربة مجهولة التفاصيل.

التفسير: تنفيذ تجربة جديدة وفق معلومات متاحة وقرارات معلنة، لا استعادة تجربة مجهولة التفاصيل. يفتقد الجدول التاريخي بيانات كافية لتطابق الأداة والفترة والوحدة؛ لذا تحفظ النتائج التاريخية منفصلة عن NEWLY_COMPUTED.

لماذا يهم؟ يمنع نسبة نتائج جديدة خطأً إلى التجربة الأصلية.

الدليل: ../ASSUMPTIONS_AND_DEVIATIONS.md.

سؤال متابعة: ما الأدلة المطلوبة لإعادة إنتاج الجدول الأصلي؟

2. ما هدف التنبؤ في البيانات اليومية؟

الإجابة المختصرة: الإغلاق المعدل للرصد التالي بعد آخر صف في نافذة السمات، وليس سعر جلسة حالية.

التفسير: الإغلاق المعدل للرصد التالي بعد آخر صف في نافذة السمات، وليس سعر جلسة حالية. يبني build_windows الهدف عند origins+1 ويحفظ تاريخ الأصل والهدف.

لماذا يهم؟ تحديد الأفق يمنع الخلط بين التنبؤ والشرح المتزامن.

الدليل: ../../src/xai_finance/data/windows.py.

سؤال متابعة: لماذا لا يلزم أن يكون الفرق يوماً تقويمياً؟

3. لماذا تُراجع بيانات الإسكان على نحو مختلف؟

الإجابة المختصرة: لأن السلسلة شهرية ومنقحة، وتاريخ الرصد لا يثبت تاريخ نشر القيمة أو نسختها المتاحة حينها.

التفسير: لأن السلسلة شهرية ومنقحة، وتاريخ الرصد لا يثبت تاريخ نشر القيمة أو نسختها المتاحة حينها. التجربة اللاحقة توصيف استعادي لا دليل توقع آني عند الإصدار.

لماذا يهم؟ يحمي الاستنتاج الزمني من ادعاء معلومات غير متاحة.

الدليل: ../RESULTS_AUDIT.md.

سؤال متابعة: ما البيانات التي يلزم جمعها لتقييم إصدار آني؟

4. كيف يحسب المتوسط البسيط في المشروع؟

الإجابة المختصرة: يحسب متوسط الإغلاق على نوافذ 5 و20 رصداً باستخدام rolling، ولا ينتج قيمة كاملة قبل توافر النافذة.

التفسير: يحسب متوسط الإغلاق على نوافذ 5 و20 رصداً باستخدام rolling، ولا ينتج قيمة كاملة قبل توافر النافذة. تُحذف الصفوف ذات السمات الناقصة لاحقاً.

لماذا يهم؟ يضمن تطابق التفسير مع السمات الفعلية.

الدليل: ../../src/xai_finance/data/features.py.

سؤال متابعة: ما أثر يوم عطلة على عدد الرصود؟

5. كيف تصنع النوافذ وما أبعاد AAPL؟

الإجابة المختصرة: تحتوي نافذة AAPL على 30 رصداً و20 سمة؛

التفسير: تحتوي نافذة AAPL على 30 رصداً و20 سمة؛ الشكل المحفوظ لجميع العينات هو 2703×30×20. آخر خطوة مدخل معلومة عند الأصل والهدف إغلاق الرصد التالي.

لماذا يهم؟ يحدد معلومات النموذج ويمنع إدراج الهدف داخل نافذته.

الدليل: DATA_PIPELINE_AR.md_03.

سؤال متابعة: كيف تتحول النافذة إلى مصفوفة لنموذج sklearn؟

6. هل تداخل النوافذ بين الأقسام تسرب تلقائياً؟

الإجابة المختصرة: لا؛

التفسير: لا؛ يمكن أن تشترك النوافذ في تاريخ كان معروفاً عند أصل التنبؤ اللاحق. المهم أن تكون التسمية والسمات متاحة في وقت القرار، وأن تلائم التحويلات على التدريب وحده؛ النسخ التاريخية المنقحة تبقى قيداً مستقلاً.

لماذا يهم؟ يفرق السياق التاريخي المشروع عن الوصول إلى معلومة مستقبلية.

الدليل: src/xai_finance/validation/audit.py/../..

سؤال متابعة: كيف تتحقق من توقيت آخر تسمية تدريب؟

7. كيف يحدد التقسيم الزمني؟

الإجابة المختصرة: يأخذ int(0.8n) من العينات للتدريب، ثم int(0.1n) للتحقق، والباقي للاختبار دون خلط.

التفسير: يأخذ int(0.8n) من العينات للتدريب، ثم int(0.1n) للتحقق، والباقي للاختبار دون خلط. يرفض أقل من 30 عينة؛ والتقريب يترك الباقي للاختبار.

لماذا يهم؟ يحاكي الفصل الزمني ويحدد مجال الاختيار والتقييم.

الدليل: src/xai_finance/data/split.py/../..

سؤال متابعة: هل يمنع التقسيم وحده تعديل البيانات بأثر رجعي؟

8. كيف يمنع LSTM تسرب التقييس؟

الإجابة المختصرة: تلائم fit_training_scalers معياري المدخل والهدف على نوافذ وأهداف التدريب فقط ثم تستعملهما ثابتين للتحقق والاختبار.

التفسير: تلائم fit_training_scalers معياري المدخل والهدف على نوافذ وأهداف التدريب فقط ثم تستعملهما ثابتين للتحقق والاختبار. تتكرر الصفوف عند تداخل نوافذ التدريب، فيختلف وزنها عن تقييس كل تاريخ مرة.

لماذا يهم؟ يمنع استعمال إحصاءات الاختبار في الملاءمة.

الدليل: src/xai_finance/data/preprocess.py/../..

سؤال متابعة: هل تستخدم نماذج الأشجار نفس المعايير؟

9. ما دور IndependentColumns قبل OLS؟

الإجابة المختصرة: يستخدم QR مع تبديل الأعمدة لتقدير رتبة عددية واختيار أعمدة تدريب مستقلة ضمن سماح نسبي؛

التفسير: يستخدم QR مع تبديل الأعمدة لتقدير رتبة عددية واختيار أعمدة تدريب مستقلة ضمن سماح نسبي؛ ثم يطبق الاختيار نفسه على البيانات اللاحقة. هذا يعالج اعتماداً عددياً ولا يثبت استقلالاً إحصائياً أو سببياً.

لماذا يهم؟ يقلل مشكلات التكييف في تصميم عالي الارتباط.

الدليل: src/xai_finance/models/linear_preprocess.py/../..

سؤال متابعة: هل يعني معامل عمود مستبعد صفراً أنه غير مفيد اقتصادياً؟

10. كيف يختار النظام الغابة والتعزيز؟

الإجابة المختصرة: يلائم المرشحين على التدريب، ويختار أقل RMSE على التحقق دون إعادة ملاءمة على التدريب والتحقق معاً.

التفسير: يلائم المرشحين على التدريب، ويختار أقل RMSE على التحقق دون إعادة ملاءمة على التدريب والتحقق معاً. يحفظ المرشح المختار ثم يقيّمه على الاختبار.

لماذا يهم؟ يفصل اختيار المعاملات عن نتيجة الاختبار.

الدليل: src/xai_finance/models/sklearn_models.py/../..

سؤال متابعة: ما كلفة عدم إعادة الملاءمة على كامل البيانات المتاحة؟

11. لماذا تعجز الغابة عادة عن استقراء مستوى خارج أهداف التدريب؟

الإجابة المختصرة: في غابة انحدار متوسطية تكون قيمة الورقة متوسط أهداف تدريب وتنبؤ الغابة متوسط الأشجار، فيبقى ضمن مجال أهداف التدريب.

التفسير: في غابة انحدار متوسطية تكون قيمة الورقة متوسط أهداف تدريب وتنبؤ الغابة متوسط الأشجار، فيبقى ضمن مجال أهداف التدريب. لا ينطبق هذا الحد نفسه بالضرورة على مجموع أشجار GradientBoosting.

لماذا يهم؟ يفسر أحد القيود المحتملة عند ارتفاع مستويات الاختبار.

الدليل: ../../src/xai_finance/validation/diagnostics.py.

سؤال متابعة: لماذا لا يجب تعميم حد الغابة على GBM؟

12. ما بنية LSTM الفعلية؟

الإجابة المختصرة: طبقة nn.LSTM واحدة ثم اختيار آخر خرج زمني وDropout ورأس Linear بخرج واحد.

التفسير: طبقة nn.LSTM واحدة ثم اختيار آخر خرج زمني وDropout ورأس Linear بخرج واحد. في التهيئة الكاملة 20 سمة يومية و32 وحدة مخفية؛ لا توجد طبقة attention في PriceLSTM.

لماذا يهم؟ يمنع وصف بنية لم تنفذ في البحث.

الدليل: ../../src/xai_finance/models/lstm_model.py.

سؤال متابعة: ما شكل موتر الخرج قبل squeeze؟

13. ما دور بوابة النسيان؟

الإجابة المختصرة: في خلية LSTM تضرب بوابة sigmoid الحالة السابقة عنصرياً قبل إضافة المحتوى الجديد، وتسمح بمرور جزء من المعلومة والتدرج.

التفسير: في خلية LSTM تضرب بوابة sigmoid الحالة السابقة عنصرياً قبل إضافة المحتوى الجديد، وتسمح بمرور جزء من المعلومة والتدرج. ينفذ PyTorch البوابات داخل nn.LSTM وليس المشروع يدوياً.

لماذا يهم؟ يربط المعادلة النظرية بالاستدعاء الفعلي.

الدليل: 05_LSTM_DEEP_EXPLANATION_AR.md.

سؤال متابعة: ما الفرق بين حالة الخلية والحالة المخفية؟

14. كيف تُختار epoch الشبكة؟

الإجابة المختصرة: تحسب optimize_epochs خسارة تحقق معيارية بعد كل epoch في وضع eval وبدون تدرج، وتحفظ أفضل حالة وتوقف عند بلوغ patience.

التفسير: تحسب optimize_epochs خسارة تحقق معيارية بعد كل epoch في وضع eval وبدون تدرج، وتحفظ أفضل حالة وتوقف عند بلوغ patience. لا تدخل أهداف الاختبار في معيار الاختيار هذا.

لماذا يهم؟ يوضح فصل الاختيار عن التقييم النهائي.

الدليل: ../../src/xai_finance/models/lstm_model.py.

سؤال متابعة: لماذا لا تعد أفضل خسارة تحقق تقديراً مستقلاً للاختبار؟

15. كيف يختبر النظام الدقة المختلطة؟

الإجابة المختصرة: على CUDA يقارن خسارة عينة تدريب بدقة FP32 ودقة مختلطة ويقبل الأخيرة إذا كانت منتهية والفرق النسبي لا يتجاوز 5%.

التفسير: على CUDA يقارن خسارة عينة تدريب بدقة FP32 ودقة مختلطة ويقبل الأخيرة إذا كانت منتهية والفرق النسبي لا يتجاوز 5%. الاختبار على fixture محدود ولا يضمن جميع التدرجات أو النتائج.

لماذا يهم؟ يفسر اختلاف مسار CUDA وCPU دون افتراض BF16 دائماً.

الدليل: ../../src/xai_finance/models/lstm_model.py.

سؤال متابعة: متى يختار FP16 بدلاً من BF16؟

16. ما معنى Persistence ولماذا يصعب تجاوزه؟

الإجابة المختصرة: يتنبأ بالإغلاق الحالي للقيمة التالية دون تعلم؛

التفسير: يتنبأ بالإغلاق الحالي للقيمة التالية دون تعلم؛ لأن مستويات الأسعار المتجاورة قد تكون متقاربة يمكن أن يكون RMSE له منخفضاً. لا يعني ذلك قدرة على توقع تغير غير صفري أو تحقيق ربح.

لماذا يهم؟ يوفر خط أساس قوي وعادل للمقارنة.

الدليل: ../../src/xai_finance/models/baselines.py.

سؤال متابعة: لماذا قد تكون دقته الاتجاهية صفراً؟

17. لماذا لم يستعد LSTM ترتيب الرسالة؟

الإجابة المختصرة: سجلات إعادة البناء تظهر النتائج الفعلية، ولكن تفاصيل مجموعة بيانات ووحدات وإعدادات الجدول التاريخي غير كافية للمطابقة.

التفسير: سجلات إعادة البناء تظهر النتائج الفعلية، ولكن تفاصيل مجموعة بيانات ووحدات وإعدادات الجدول التاريخي غير كافية للمطابقة. انزياح المستويات وقيد النموذج والتدريب عوامل يحققها التقرير، لا سبب وحيد مثبت لاختلاف التجربتين.

لماذا يهم؟ يمنع تعديل النتائج لإرضاء ترتيب مفترض.

الدليل: ../RESULTS_AUDIT.md.

سؤال متابعة: ما تجربة مستقلة تساعد على اختبار فرضية الانزياح؟

18. كيف تحسب MAE وRMSE؟

الإجابة المختصرة: MAE هو متوسط القيمة المطلقة للفروق، وRMSE جذر متوسط مربعاتها؛

التفسير: MAE هو متوسط القيمة المطلقة للفروق، وRMSE جذر متوسط مربعاتها؛ يعاقب الثاني الفروق الكبيرة بدرجة أكبر. كلاهما بوحدة الهدف الأصلية بعد عكس التقييس.

لماذا يهم؟ يسمح بمقارنة دقيقة ضمن الأصل والوحدة نفسيهما.

الدليل: ../../src/xai_finance/evaluate/metrics.py.

سؤال متابعة: لماذا لا يجوز جمع أخطاء أصول بوحدات مختلفة؟

19. لماذا يمكن أن تكون MAPE مضللة؟

الإجابة المختصرة: تقسم على القيمة الفعلية فتتضخم قرب الصفر؛

التفسير: تقسم على القيمة الفعلية فتتضخم قرب الصفر؛ يستبعد الكود الأهداف التي لا تتجاوز قيمتها المطلقة 1e-12 ويسجل عددها، ويعيد None عند غياب مقام صالح.

لماذا يهم؟ يمنع قراءة نسبة غير مستقرة كمقياس موثوق.

الدليل: ../../src/xai_finance/evaluate/metrics.py.

سؤال متابعة: ماذا يحدث لعوائد صغيرة غير صفرية؟

20. ما الفرق بين الدقة الاتجاهية ومصفوفة الصعود؟

الإجابة المختصرة: الدقة تستعمل sign بثلاث قيم سالب وصفر وموجب، بينما المصفوفة تستعمل شرط أكبر من الإغلاق الحالي وتضم التعادل إلى غير الصعود.

التفسير: الدقة تستعمل sign بثلاث قيم سالب وصفر وموجب، بينما المصفوفة تستعمل شرط أكبر من الإغلاق الحالي وتضم التعادل إلى غير الصعود. لذلك قد يختلف تفسير النتائج عند الثبات.

لماذا يهم؟ يوضح قيمة Persistence دون وصفها خطأً كمصنف هابط.

الدليل: ../../src/xai_finance/evaluate/metrics.py.

سؤال متابعة: ما معنى التغطية الاتجاهية عندما يكون التنبؤ صفراً؟

21. ماذا تعني قيمة SHAP المحلية؟

الإجابة المختصرة: هي إسهام منسوب إلى سمة بالنسبة إلى خلفية وتعريف إخفاء؛

التفسير: هي إسهام منسوب إلى سمة بالنسبة إلى خلفية وتعريف إخفاء؛ يتحقق المشروع من أن القيمة الأساسية ومجموع الإسهامات يعيدان التنبؤ تقريباً. هذا لا يثبت سببية أو صحة التنبؤ.

لماذا يهم؟ يفصل التفسير الرياضي عن الاستدلال السببي.

الدليل: ../../src/xai_finance/explain/shap_explain.py.

سؤال متابعة: كيف تؤثر سمات مترابطة في الخلفية؟

22. كيف يعمل LIME في هذا التنفيذ؟

الإجابة المختصرة: ينشئ LimeTabularExplainer جواراً مضطرباً ونموذجاً محلياً مبسطاً مع تقطيع مستمر وبذرة، ويخزن الأوزان وتنبؤه وR^2 الموزون للجوار والفارق عن تنبؤ النموذج على الأصل.

التفسير: ينشئ LimeTabularExplainer جواراً مضطرباً ونموذجاً محلياً مبسطاً مع تقطيع مستمر وبذرة، ويخزن الأوزان وتنبؤه وR^2 الموزون للجوار والفارق عن تنبؤ النموذج على الأصل.

لماذا يهم؟ يجعل جودة التقريب قابلة للفحص بدلاً من افتراضها.

الدليل: ../../src/xai_finance/explain/lime_explain.py.

سؤال متابعة: لماذا قد يكون جوار السمات المالية غير واقعي؟

23. لماذا بلغ خطأ LIME المحفوظ لـ AAPL نحو 502.8627؟

الإجابة المختصرة: لأنه القيمة المطلقة للفارق عند المشاهدة الأصلية بين 236.678557 تقريباً و−266.184181 تقريباً، بوحدة الدولار المعدل للسهم.

التفسير: لأنه القيمة المطلقة للفارق عند المشاهدة الأصلية بين 236.678557 تقريباً و−266.184181 تقريباً، بوحدة الدولار المعدل للسهم. هو خطأ محلي مطلق وليس MSE أو RMSE؛ يدل على ضعف تقريب هذه الحالة ولا يكفي لتحديد سبب وحيد أو تعميم الفشل.

لماذا يهم؟ يتيح الدفاع عن نتيجة صعبة دون إخفائها أو إساءة تفسيرها.

الدليل: SHAP_LIME_EXPLANATION_AR.md_06.

سؤال متابعة: ما الفرق بين هذا الخطأ وR^2 الموزون للجوار؟

24. لماذا قد يختلف SHAP وLIME؟

الإجابة المختصرة: يعتمد SHAP على خلفية وتعريف مساهمات عبر إخفاء الميزات، بينما يقدّر LIME بديلاً على جوار مضطرب موزون وبعدد سمات محدود.

التفسير: يعتمد SHAP على خلفية وتعريف مساهمات عبر إخفاء الميزات، بينما يقدّر LIME بديلاً على جوار مضطرب موزون وبعدد سمات محدود. الارتباط والمرجع والميزانية تغير الإسناد ولا تجعل أياً منهما سبباً.

لماذا يهم؟ يساعد على قراءة التفسيرات المتعارضة دون افتراض وجود حقيقة سببية واحدة.

الدليل: SHAP_LIME_EXPLANATION_AR.md_06.

سؤال متابعة: كيف تختبر حساسية النتيجة للخلفية والبذرة؟

25. ما الذي يقيسه bootstrap المزدوج؟

الإجابة المختصرة: يأخذ الفرق بين مربعي خطأي النموذج وPersistence في التواريخ نفسها ويعيد أخذ كتل دائرية بطول 10، 2000 مرة، لتقدير فاصل لمتوسط الفرق.

التفسير: يأخذ الفرق بين مربعي خطأي النموذج وPersistence في التواريخ نفسها ويعيد أخذ كتل دائرية بطول 10، 2000 مرة، لتقدير فاصل لمتوسط الفرق. الفاصل استكشافي ولا يضم عدم يقين إعادة التدريب أو تصحيح المقارنات المتعددة.

لماذا يهم؟ يعرض عدم اليقين المقترن زمنياً بدلاً من الاكتفاء بفوز نقطي.

الدليل: src/xai_finance/validation/statistics.py/../..

سؤال متابعة: ماذا يعني عبور الفاصل للصفر؟

26. ما دور main في نقطة الدخول؟

الإجابة المختصرة: يشغّل الملف الهدف عند التنفيذ المباشر.

التفسير: يستخدم runpy.run_path main.py مع run_name يساوي **main**؛ لذا يعمل حارس الدخول داخل dashboard/app.py. الاستيراد العادي لا يشغل هذا الحارس.

الدليل: ../../main.py.

لماذا يهم؟ يفسر كيف يبدأ العرض عبر Streamlit.

سؤال متابعة: هل يغيّر runpy مجلد العمل؟

27. لماذا يستخدم المشروع pathlib؟

الإجابة المختصرة: لبناء مسارات قابلة للتركيب.

التفسير: تحدد ملفات مثل config.py وmain.py المسار انطلاقاً من موقع الملف أو مجلد العمل، وتستعمل العامل / لضم المكونات. اختيار الجذر يخضع لشروط وجود ملفات الإعداد.

الدليل: ../../src/xai_finance/config.py.

لماذا يهم؟ يساعد في تتبع الملفات على جهاز الدفاع.

سؤال متابعة: ماذا يحدث إذا وُجد مشروع مطابق في مجلد العمل؟

28. ما أثر dataclass decorator على WindowDataset؟

الإجابة المختصرة: يولد منشئاً لحقول البيانات.

التفسير: تحتفظ الحاوية بالنوافذ والأهداف والتواريخ وأسماء السمات والتقسيمات، لكن تلميحات الأنواع لا تفحص صحة الأبعاد تلقائياً.

الدليل: ../../src/xai_finance/data/windows.py.

لماذا يهم؟ يوضح كيف تمر البيانات بين مراحل النظام.

سؤال متابعة: ما الفرق بين dataclass وDataFrame؟

29. ما معنى property في flat؟

الإجابة المختصرة: تعرض عملية التسطيح كخاصية محسوبة.

التفسير: يعيد reshape صفاً لكل عينة مع دمج بعدي الزمن والسمات، وتحدد flat_names ترتيب أسماء التأخيرات والسمات. لا يعني ذلك تغيير المعلومات إلى بيانات مستقبلية.

الدليل: ../../src/xai_finance/data/windows.py.

لماذا يهم؟ يربط مدخلات sklearn بنوافذ LSTM.

سؤال متابعة: كيف يرتبط lag_0 بآخر خطوة؟

30. لماذا يحافظ تحميل snapshot على SHA-256؟

الإجابة المختصرة: لكشف تغير بايتات الملف عن الفهرس المحفوظ.

التفسير: تقارن load_snapshot بصمة CSV بالبيان ثم تفحص التواريخ والقيم والعدد. يتوقف التحقق على أن قيمة البصمة المرجعية موثوقة.

الدليل: ../../src/xai_finance/io/data_sources.py.

لماذا يهم؟ يسند تتبع البيانات دون الادعاء بأن الهاش يثبت مصدرها وحده.

سؤال متابعة: ما الفرق بين السلامة وإثبات المنشأ؟

31. كيف يتعامل التنزيل مع لقطة موجودة؟

الإجابة المختصرة: يتحقق منها ويعيد استخدامها.

التفسير: يفحص download_asset وجود CSV ويستدعي load_snapshot ثم يعود؛ لا يستبدلها بصمت. عند غيابها يطلب من المزود ويحفظ لقطة وبيان منشأ.

الدليل: ../../src/xai_finance/io/data_sources.py.

لماذا يهم؟ يمنع تغيير بيانات تجربة محفوظة بلا سجل.

سؤال متابعة: ماذا يحدث إذا كان بيان اللقطة مفقوداً؟

32. كيف يعالج المشروع القيم المفقودة؟

الإجابة المختصرة: يحذف الصفوف غير المكتملة ولا يملأ الهدف.

التفسير: في التنزيل يتحول غير الرقمي إلى NaN وتُحذف الصفوف، وعند بناء السمات تستبدل اللانهاية بقيم ناقصة ثم تحذف الصفوف. يتطلب تفسير عدد المحذوف مراجعة السجل.

الدليل: ../../src/xai_finance/io/data_sources.py.

لماذا يهم؟ يمنع افتراض أن الاستيفاء استخدم فعلاً.

سؤال متابعة: هل حذف الصف يضمن انتظام جميع الجلسات؟

33. لماذا يعدل open وhigh وlow؟

الإجابة المختصرة: لتكون على مقياس الإغلاق المعدل.

التفسير: للبيانات اليومية تضرب الأعمدة بنسبة adj_close إلى close، بينما يبقى الحجم الأصلي قبل تحويل log1p. البيانات المعدلة بأثر رجعي ليست نسخة point-in-time.

الدليل: src/xai_finance/data/features.py/../..

لماذا يهم؟ يوحد المقياس المستخدم في المؤشرات اليومية.

سؤال متابعة: ما حدود هذا التعديل عند تجزئة الأسهم؟

34. كيف يعالج RSI القسمة على صفر؟

الإجابة المختصرة: يخصص قيمتين للحالات الحدية.

التفسير: تستبدل خسائر الصفر مؤقتاً بـ NaN، ثم يضع 100 عند وجود مكاسب بلا خسائر و50 عندما يكون الاثنان صفراً. يستعمل متوسطاً أُسياً بمعدل 1/14.

الدليل: src/xai_finance/data/features.py/../..

لماذا يهم؟ يتجنب تفسير قيمة غير معرفة على أنها ناتجة بلا معالجة.

سؤال متابعة: هل تتطابق تهيئة RSI مع كل مكتبة أخرى؟

35. ما معنى MACD في الكود؟

الإجابة المختصرة: فرق متوسطين أسيين للإغلاق.

التفسير: يطرح EMA26 من EMA12 ثم يحسب إشارته بمتوسط أسي span 9 مع حد أدنى للرصد. المؤشر مشتق من الأسعار الماضية حتى تاريخ الأصل.

الدليل: src/xai_finance/data/features.py/../..

لماذا يهم؟ يوضح أن السمات تحولات وليست معلومات خارجية مستقلة.

سؤال متابعة: ما أثر ترابط MACD مع أسعار الإدخال؟

36. ما معنى ddof=0 في Bollinger؟

الإجابة المختصرة: يستخدم انحرافاً معيارياً بقاسم حجم النافذة.

التفسير: تحسب الشيفرة انحراف 20 رصداً ثم تضيف وتطرح ضعفيه من SMA20؛ لا تستخدم تصحيح العينة $n-1$ في هذا الاستدعاء.

الدليل: src/xai_finance/data/features.py/../..

لماذا يهم؟ يضبط الصيغة العددية للمؤشر.

سؤال متابعة: كيف يختلف الحد عند ddof=1؟

37. ما ترتيب السمات اليومية في النافذة المسطحة؟

الإجابة المختصرة: تتجاور سمات كل خطوة زمنية بالترتيب.

التفسير: تسطح reshape أبعاد الزمن والسمات، وتولد flat_names أسماء من lag الأقدم حتى lag_0 مع ترتيب feature_names لكل خطوة.

الدليل: src/xai_finance/data/windows.py/../..

لماذا يهم؟ يتيح تفسير أسماء SHAP دون قلب الزمن.

سؤال متابعة: ما معنى lag_0؟

38. لماذا هدف اليوم التالي ليس بالضرورة غداً تقويمياً؟

الإجابة المختصرة: لأنه الرصد التالي في السلسلة المحفوظة.

التفسير: تبني build_windows الهدف من الصف التالي بعد تنظيف السمات، بينما يدقق alignment_check مقابل تاريخ الرصد الخام التالي في اللقطات المستخدمة.

الدليل: src/xai_finance/validation/audit.py/../..

لماذا يهم؟ يفصل أيام الإغلاق والعطلات عن أفق التنبؤ.

سؤال متابعة: ماذا يحدث إن حذف صف داخلي بسبب نقص؟

39. ما مثال AAPL موثق على المحاذاة؟

الإجابة المختصرة: أصل 2024-11-29 وهدف 2024-12-02.

التفسير: يسجل CSV إغلاقاً معدلاً 235.620101929 عند الأصل وهدفاً 237.863830566؛ هذه قراءة آثار محفوظة وليست إعادة تشغيل للنموذج.

الدليل: DATA_PIPELINE_AR.md_03.

لماذا يهم؟ يحول مفهوم المحاذاة إلى حالة قابلة للتتبع.

سؤال متابعة: ما تنبؤ OLS المحفوظ لذلك الهدف؟

40. كيف تتحقق من عدم استعمال هدف اختبار لتقدير scaler؟

الإجابة المختصرة: بفحص فهارس fit وباختبار الطفرة.

التفسير: تختار fit_training_scalers قسم train فقط، ويغير اختبار heldout نوافذ وأهداف الأقسام الأخرى ويتحقق من بقاء متوسطات التدريب.

الدليل: ../../tests/test_temporal_integrity.py.

لماذا يهم؟ يوفر دليلاً برمجياً على حد زمني محدد.

سؤال متابعة: هل يكشف الاختبار انحياز النسخ المنقحة؟

41. ما وظيفة validation مقارنة بالاختبار؟

الإجابة المختصرة: يستخدم لاختيار النموذج أو التوقف المبكر.

التفسير: يختار sklearn أقل RMSE تحقق وتختار LSTM أفضل خسارة تحقق. الاختبار النهائي المنفصل يصف الأداء بعد هذه القرارات، لكنه ليس مستقلاً إذا تكرر اتخاذ القرارات بناء عليه.

الدليل: ../../src/xai_finance/pipelines/train.py.

لماذا يهم؟ يمنع وصف بيانات الاختيار بأنها اختبار لم يُمس.

سؤال متابعة: هل أُعيد تدريب المرشح على التدريب والتحقق؟

42. كيف يعالج النموذج بيانات HOUSING؟

الإجابة المختصرة: ينفذ تجربة شهرية مستقلة.

التفسير: يستعمل close وطول نافذة 12 في الإعداد الكامل، وتبقى البيانات المنقحة وتاريخ إصدارها غير المؤكد قيداً على التفسير الآني.

الدليل: ../../configs/full_experiment.yaml.

لماذا يهم؟ يمنع مقارنة تردد شهري مع يومي بلا سياق.

سؤال متابعة: ما الفرق بين شهر الرصد وتاريخ النشر؟

43. ما وظيفة LinearRegression داخل Pipeline؟

الإجابة المختصرة: يلائم انحداراً خطياً على المدخلات المختارة.

التفسير: يطبق StandardScaler ثم IndependentColumns قبل الانحدار؛ التنبؤ مجموع معاملات مضروبة في المدخلات المتحولة مع ثابت. اختيار الأعمدة يتم على التدريب.

الدليل: ../../src/xai_finance/models/sklearn_models.py.

لماذا يهم؟ يربط الصيغة الخطية بآلية التنفيذ.

سؤال متابعة: كيف تعاد المعاملات إلى الوحدات الأصلية؟

44. ما سبب تحويل معاملات الانحدار عند التصدير؟

الإجابة المختصرة: لجعلها مرتبطة بوحدات السمات الأصلية.

التفسير: يقسم export_coefficients معاملات الأعمدة المحفوظة على scale ويعدل الثابت باستخدام المتوسط، ويضع صفراً للأعمدة المستبعدة عددياً.

الدليل: ../../src/xai_finance/pipelines/report.py.

لماذا يهم؟ يسمح بقراءة المعاملات خارج فضاء التقييس.

سؤال متابعة: لماذا لا يعني الصفر غياب أثر اقتصادي؟

45. ما فكرة RandomForest في المشروع؟

الإجابة المختصرة: متوسط تنبؤات أشجار انحدار متعددة.

التفسير: يبني candidates الغابة بعدد أشجار من الإعداد وعمق مرشح وmin_samples_leaf=3؛ يختار المرشح على التحقق ثم يجعل n_jobs=1 في التنبؤ.

الدليل: ../../src/xai_finance/models/sklearn_models.py.

لماذا يهم؟ يفسر المقارنة مع نموذج خطي وشبكة.

سؤال متابعة: لماذا لا يكفي متوسط الأشجار لتجاوز اتجاه خارج مجالها؟

46. ما فكرة GradientBoosting هنا؟

الإجابة المختصرة: يجمع متعلمين ضعفاء بالتتابع.

التفسير: تستخدم GradientBoostingRegressor إعدادات الأشجار والعمق ومعدل التعلم المحددة في candidates وتختار بالتقييم على التحقق. لا تفترض أنها غابة متوازية.

الدليل: ../../src/xai_finance/models/sklearn_models.py.

لماذا يهم؟ يوضح اختلاف التجميع المتتابع عن الغابة.

سؤال متابعة: كيف يغير معدل التعلم أثر كل شجرة؟

47. ما الفرق بين خطأ السعر وخطأ الهدف المحول؟

الإجابة المختصرة: وحداتهما وسياقهما مختلفان.

التفسير: في تجارب الفرق والعائد يعيد implied_prices التنبؤ إلى سعر، ثم تسجل evaluation_metrics مقاييس السعر ومقاييس الهدف منفصلة. خطأ العائد بلا وحدة بينما خطأ السعر بوحدة الأصل.

الدليل: ../../src/xai_finance/validation/statistics.py.

لماذا يهم؟ يمنع خلط مقاييس غير متجانسة.

سؤال متابعة: لماذا قد يختار تحقق العائد مرشحاً مختلفاً عن تحقق السعر؟

48. ما أثر غياب إعادة الملاءمة النهائية؟

الإجابة المختصرة: تبقى المعاملات ملائمة على التدريب فقط.

التفسير: تختار fit_candidates نموذجاً ملائماً بالفعل ولا تضيف التحقق إلى ملاءمته، بينما تضبط LSTM التوقف على التحقق. قد تزداد الفجوة الزمنية بين آخر هدف تدريب والاختبار.

الدليل: src/xai_finance/models/sklearn_models.py/../..

لماذا يهم؟ يشرح أحد قيود الاستقرار الزمني للتجربة المجمدة.

سؤال متابعة: كيف يصمم تقييم منصف لسياسة إعادة ملاءمة بديلة؟

49. كيف تُبنى نافذة walk-forward؟

الإجابة المختصرة: تستخدم تاريخاً متوسعاً وكتلة تحقق قبل الاختبار.

التفسير: تقسم الشيفرة فهارس الاختبار إلى كتل، وتضع آخر validation_size فهارس قبل كل كتلة للتحقق وما قبلها للتدريب، وتتحقق من إتاحة آخر هدف.

الدليل: src/xai_finance/validation/design.py/../..

لماذا يهم؟ يوضح توقيت إعادة الملاءمة وتوفر التسميات.

سؤال متابعة: هل تحدث إعادة التدريب عند كل يوم داخل الكتلة؟

50. ما الفرق بين هدف تغير السعر والعائد؟

الإجابة المختصرة: الأول فرق بوحدة السعر والثاني نسبة.

التفسير: target_values تحسب y−current أو y/current−1؛ تعيد implied_prices تحويل كل تنبؤ إلى سعر للمقارنة مع Persistence.

الدليل: src/xai_finance/validation/design.py/../..

لماذا يهم؟ يفسر لماذا لا تقارن المقادير الرقمية مباشرة.

سؤال متابعة: ما الشرط الإضافي لبروتوكول العائد؟

51. لماذا يستبعد النفط من بروتوكول العائد؟

الإجابة المختصرة: لوجود سعر سلبي تاريخي في اللقطة.

التفسير: يرفض target_values العائد إذا لم تكن الأسعار موجبة، بينما تبقي تجربة فرق السعر الملاحظة بدلاً من قصها أو حذفها وفق البروتوكول.

الدليل: scripts/start_scientific_validation.py/../..

لماذا يهم؟ يكشف أثر اختيار تعريف الهدف على نطاق الأصول.

سؤال متابعة: هل يمنع السعر السلبي حساب فرق السعر؟

52. كيف يستخدم آخر تغير معلوم كمرجع؟

الإجابة المختصرة: يعيد الهدف السابق عند أصل التنبؤ.

التفسير: تستعمل naive_predictions الفهرس test−1 وتؤكد أن تاريخ الهدف السابق لا يتجاوز أصل الاختبار الحالي؛ يمكن استخدام تسميات اختبار أقدم بعد تحققها.

الدليل: src/xai_finance/validation/design.py/../..

لماذا يهم؟ يميز المعلومات المتاحة عن معرفة الهدف الحالي مستقبلاً.

سؤال متابعة: كيف يفحص الاختبار هذا الافتراض؟

53. ما وظيفة direction_coverage؟

الإجابة المختصرة: تقيس نسبة التوقعات ذات الإشارة غير الصفرية.

التفسير: في directional_statistics يكون التنبؤ نشطاً عندما sign لا تساوي صفر، وتحسب دقة منفصلة لهذه الحالات إن وجدت؛ التوقع الصفري يمتنع عن إشارة غير صفرية.

الدليل: src/xai_finance/validation/statistics.py/../..

لماذا يهم؟ يمنع مقارنة الامتناع بمصنف نشط دون بيان التغطية.

سؤال متابعة: متى تكون الدقة المشروطة غير معرّفة؟

54. ما دلالة R^2 السالب؟

الإجابة المختصرة: أن الخطأ المربع أكبر من مرجع متوسط الهدف في ذلك الاختبار.

التفسير: يحسب sklearn SSE/SST−1 عندما يكون الهدف غير ثابت وعدد العينات كافياً؛ قيمة سالبة لا تعني وجود خلل حسابي تلقائياً.

الدليل: ../../src/xai_finance/evaluate/metrics.py.

لماذا يهم؟ يسمح بعرض نتائج الشبكة السلبية بصدق.

سؤال متابعة: لماذا لا يكفي R^2 المرتفع لإثبات ربحية التداول؟

55. كيف يتغير تفسير الخطأ عند أسعار مختلفة الوحدات؟

الإجابة المختصرة: تتبع MAE وRMSE وحدة الهدف.

التفسير: تقرير النتائج يفصل كل أصل ووحدته، فالدولار للسهم ونقاط المؤشر ليست مقادير قابلة للجمع دون تعريف تطبيع مناسب.

الدليل: ../RECONSTRUCTED_RESULTS.md.

لماذا يهم؟ يمنع متوسطاً مضللاً عبر أصول غير متجانسة.

سؤال متابعة: هل النسبة إلى Persistence تلغي جميع اختلافات المخاطر؟

56. ما معنى اختبار additivity في SHAP؟

الإجابة المختصرة: أن مجموع المساهمات والمرجع يعيد التنبؤ ضمن سماح.

التفسير: shap_reconstruction تقارن باستخدام rtol=1e-4 وatol=1e-3، وتبلغ أكبر ومتوسط فرق. نجاح الجمع لا يتحقق من واقعية الخلفية أو السببية.

الدليل: ../../src/xai_finance/explain/shap_explain.py.

لماذا يهم؟ يفصل فحص الاتساق العددي عن جودة التفسير.

سؤال متابعة: ما الذي يحدث إن فشل assert_allclose؟

57. كيف يتعامل SHAP الخطي مع الأعمدة المحذوفة؟

الإجابة المختصرة: يعيدها إلى متجه كامل بأصفار.

التفسير: يطبق scaler وselector على الخلفية والمشاهدات قبل LinearExplainer، ثم يضع مساهمات الأعمدة المحتفظ بها في مواضعها ويترك المستبعدة صفراً.

الدليل: src/xai_finance/explain/shap_explain.py/../..

لماذا يهم؟ يسمح بمطابقة أسماء التفسيرات مع نافذة المدخل.

سؤال متابعة: هل الصفر دليل على عدم فائدة العمود في نموذج آخر؟

58. لماذا لا تختبر خلفية SHAP على بيانات الاختبار؟

الإجابة المختصرة: لتكون الخلفية مستمدة من معلومات التدريب.

التفسير: يختار explain_asset فهارس متباعدة من قسم التدريب ويستخدم مجموعة اختبار محددة للتفسير؛ الخلفية القليلة قد لا تمثل التوزيع كاملاً.

الدليل: src/xai_finance/pipelines/explain.py/../..

لماذا يهم؟ يبيّن المرجع وحدوده في تفسير القيم.

سؤال متابعة: ما أثر تغيير حجم الخلفية أو اختيارها؟

59. ما حدود فحص ثبات بذرة SHAP؟

الإجابة المختصرة: يقارن إعادة تفسير محدودة للمشاهدة نفسها.

التفسير: يكرر explain_model أول مشاهدة ببذرة مختلفة ويحسب تداخل أهم عشرة وارتباط رتب المساهمات المطلقة. لا يثبت ثبات الأسواق أو ثقة المستخدم.

الدليل: src/xai_finance/explain/quality.py/../..

لماذا يهم؟ يمنع تحويل اختبار عددي ضيق إلى ادعاء عام.

سؤال متابعة: ماذا يحدث إذا كانت كل الإسهامات متساوية؟

60. كيف يعرض النظام أهمية السمات عبر التأخيرات؟

الإجابة المختصرة: يجمع مطلق SHAP عبر مواضع النافذة ثم يوسّط العينات.

التفسير: تعيد aggregate_lag_charts تشكيل المصفوفة إلى عينات وزمن وسمات وتحسب مجموع المطلقات، لذلك لا يبين الترتيب الاتجاه الموقع أو السببية.

الدليل: ../../src/xai_finance/reports/plots.py.

لماذا يهم؟ يمنع تفسير أهمية عالمية محدودة كأثر سببي.

سؤال متابعة: ما الفرق بين هذه الأهمية وخريطة عينة واحدة؟

61. هل تولد لوحة Streamlit تفسير LIME جديداً؟

الإجابة المختصرة: لا، تقرأ ملفات التفسير المحفوظة.

التفسير: تختار explanations نموذجاً وتاريخاً من quality.json، وتقرأ shap.npz وlime_index.json وتعرض مقاييس الجودة للعينات المحسوبة مسبقاً.

الدليل: ../../dashboard/app.py.

لماذا يهم؟ يميز العرض المحلي عن تشغيل نموذج مكلف مباشرة.

سؤال متابعة: ماذا تعرض إذا غاب quality.json؟

62. ما المدخلات التي يغيرها المستخدم في الواجهة؟

الإجابة المختصرة: التشغيل والأصل والنموذج والمقياس وتاريخ التفسير.

التفسير: تستخدم main ومقدمو التبويبات selectbox لتصفية الآثار المحلية، لكن وجود زر اختيار لا يعني استدعاء model.predict وقت العرض.

الدليل: ../../dashboard/app.py.

لماذا يهم؟ يساعد على شرح سلوك العرض أمام اللجنة.

سؤال متابعة: كيف تختار الواجهة التشغيل المفضل؟

63. لماذا تعرض الواجهة تحذيراً للإسكان؟

الإجابة المختصرة: لأن تجربته شهرية ومنقحة استعادياً.

التفسير: تتحقق main من التردد الشهري وتعرض أن إصدارات البيانات وتواريخ نشرها غير معروفة، فلا تدعم دعوى توقع عند وقت النشر.

الدليل: dashboard/app.py/../..

لماذا يهم؟ يمنع إسقاط تفسير التداول اليومي على مؤشر شهري.

سؤال متابعة: ما الذي يجب حفظه لدراسة point-in-time؟

64. ما وظيفة إعداد CUBLAS_WORKSPACE_CONFIG؟

الإجابة المختصرة: يهيئ خياراً داعماً للحتمية قبل عمليات CUDA.

التفسير: تضعه cli.py وseed.py باستخدام setdefault فلا يستبدلان قيمة موجودة، وتضبط seed_experiment بذوراً وخيارات deterministic إضافية.

الدليل: src/xai_finance/seed.py/../..

لماذا يهم؟ يوضح أن قابلية التكرار تعتمد على البيئة أيضاً.

سؤال متابعة: لماذا قد تختلف النتائج بين CPU وCUDA رغم تثبيت البذرة؟

65. ماذا تحفظ checkpoints بجانب الأوزان؟

الإجابة المختصرة: حالة المحسن ومقياس AMP وحالات RNG والتقدم.

التفسير: تجمع checkpoint_state هذه القيم مع architecture وconfiguration عبر progress؛ يستخدم التحميل للاستدلال state_dict والمعايير المحفوظة منفصلة.

الدليل: src/xai_finance/models/lstm_model.py/../..

لماذا يهم؟ يفرق ملف استئناف التدريب عن مجرد توقع محفوظ.

سؤال متابعة: أين تحفظ scalers الخاصة بـ LSTM؟

66. كيف تُحمل شبكة LSTM على CPU؟

الإجابة المختصرة: يستخدم التحميل map_location على الجهاز المختار.

التفسير: تقرأ load_lstm best.pt بخيار weights_only وتبني PriceLSTM من architecture، ثم تحمل state_dict وتطبق eval مع scalers الممررة.

الدليل: ../../src/xai_finance/models/lstm_model.py.

لماذا يهم؟ يسمح بنقل النموذج من تدريب CUDA إلى عرض CPU.

سؤال متابعة: هل map_location وحده يوفر ملف scaler؟

67. ما الفرق بين best.pt وlast.pt؟

الإجابة المختصرة: الأول أفضل تحقق والآخر أحدث epoch محفوظ.

التفسير: تكتب last optimize_epochs بعد كل epoch وbest عند تحسن خسارة التحقق؛ بعد التدريب تعيد حالة أفضل تحقق إلى الشبكة.

الدليل: ../../src/xai_finance/models/lstm_model.py.

لماذا يهم؟ يمنع الخلط بين آخر تدريب والنموذج المختار.

سؤال متابعة: ما سبب حفظ حالة المحسن أيضاً؟

68. ما أثر dropout عند العرض؟

الإجابة المختصرة: يعطّل وضع eval الإسقاط العشوائي.

التفسير: تستدعي LSTMPredictor.predict network.eval قبل حساب النتيجة، وno_grad لتعطيل بناء تدرجات الاستدلال. لا يضمن ذلك تطابقاً بتياً عبر أجهزة مختلفة.

الدليل: ../../src/xai_finance/models/lstm_model.py.

لماذا يهم؟ يوضح الاختلاف بين تدريب وتنبؤ.

سؤال متابعة: هل no_grad تعادل eval؟

69. لماذا تقص تدرجات LSTM؟

الإجابة المختصرة: لتقييد معيارها عند 1.

التفسير: بعد backward وفك مقياس AMP تطبق train_epoch _clip_grad_norm مع error_if_nonfinite، قبل خطوة المحسن. هذا يخفف انفجار التدرج ولا يضمن نجاح التعلم.

الدليل: ../../src/xai_finance/models/lstm_model.py.

لماذا يهم؟ يربط الاستقرار العددي بتسلسل التدريب.

سؤال متابعة: لماذا يفك مقياس FP16 قبل القص؟

70. ما طبيعة FP32 وBF16 في التجربة؟

الإجابة المختصرة: هما صيغ عددية وقد تختلف نتيجة عملياتهما.

التفسير: يفحص probe_precision الخسارة على عينة تدريبية قبل قبول AMP؛ في السجل المحفوظ لتجربة AAPL استعمل BF16، لكن الاختيار ليس مضموناً لكل جهاز.

الدليل: ../../artifacts/runs/full_20261021T040901084647Z/AAPL/training.json.

لماذا يهم؟ يفصل السجل الفعلي عن الإعداد المحتمل.

سؤال متابعة: ما معيار الرفض المسجل في الشيفرة؟

71. كيف تختار الشيفرة حجم دفعة CPU؟

الإجابة المختصرة: الأصغر بين 32 وعدد مدخلات fixture.

التفسير: تعيد batch_probe هذه القيمة في CPU، أما CUDA فيقيس المرشحين ويختار أكبر المقبولين بقيود الذاكرة. هذا الإجراء قد يختلف حسب الجهاز.

الدليل: ../../src/xai_finance/models/lstm_model.py.

لماذا يهم؟ يبين أن حجم الدفعة ليس ثابتاً عبر المنصات.

سؤال متابعة: هل تختار CUDA المرشح الأسرع بالضرورة؟

72. ما أثر عدم خلط دفعات التدريب؟

الإجابة المختصرة: تعالج الحلقة العينات بترتيبها المخزن.

التفسير: train_epoch تمر بالتتابع على slices من tensors ولا تستدعي shuffle؛ يساعد ذلك على فهم ترتيب التحديثات لكنه ليس دليلاً وحده على جودة التنبؤ.

الدليل: src/xai_finance/models/lstm_model.py/../..

لماذا يهم؟ يدعم إعادة وصف البروتوكول الفعلي.

سؤال متابعة: هل يحتفظ LSTM بحالته بين الدفعات؟

73. ما المقصود بحد خرج LSTM المحفوظ؟

الإجابة المختصرة: للرأس الخطي حد محافظ مشتق من الأوزان.

التفسير: عند eval كل إحداثي h بين −1 و1، فيكون خرج الرأس بين b−sum(abs(w)) وb+sum(abs(w)) قبل عكس تقييس الهدف. الأطراف قد لا تكون قابلة للتحقيق معاً.

الدليل: src/xai_finance/validation/diagnostics.py/../..

لماذا يهم؟ يفسر قيد الشبكة المعينة تحت انزياح الأسعار.

سؤال متابعة: هل هذا يعني أن جميع بنى LSTM لها نفس الحد الرقمي؟

74. ما أثر تغيير هدف LSTM إلى فرق السعر؟

الإجابة المختصرة: يتغير مقدار الهدف المراد تعلمه وإعادة تركيبه.

التفسير: تستعمل تجارب التحقق النوافذ نفسها مع هدف y−current ثم تضيف التوقع للإغلاق الحالي. لا يخلق تغيير التمثيل معلومات مستقبلية إضافية.

الدليل: src/xai_finance/validation/experiments.py/../..

لماذا يهم؟ يفصل تغيير التمثيل عن إثبات قابلية التنبؤ.

سؤال متابعة: لماذا قد يؤدي توقع فرق صفر إلى خطأ سعر منخفض؟

75. كيف يختبر المشروع حساسية دقة إعادة تركيب السعر؟

الإجابة المختصرة: يقارن ملف حساسية FP64 منفصلاً بالنتائج كما نُفذت.

التفسير: يحول audit_reconstruction_precision الهدف المحفوظ إلى float64 قبل إعادة التركيب ويكتب نتائج منفصلة؛ لا يستبدل النتائج الأصلية أو يعيد التدريب.

الدليل: scripts/audit_reconstruction_precision.py/../..

لماذا يهم؟ يمنع الخلط بين أثر التقريب وتغيير النموذج.

سؤال متابعة: لماذا قد يختلف r+1 بين FP32 وFP64؟

76. لماذا تقارن أخطاء النموذج وPersistence على التواريخ نفسها؟

الإجابة المختصرة: للمحافظة على الاقتران بين التوقعين عند كل رصد.

التفسير: تحسب `paired_bootstrap` الفرق بين مربعي الخطأين لكل تاريخ قبل إعادة أخذ الكتل، فيبقى أثر التاريخ مشتركاً في المقارنة. لا يجعل ذلك أصولاً مختلفة أو طيات متعددة مستقلة إحصائياً، ولا يشمل عدم يقين إعادة التدريب.

الدليل: validation/statistics.py.

لماذا يهم؟ يوضح وحدة المقارنة والاعتماد الزمني دون ادعاء استقلال البيانات.

سؤال متابعة: ماذا يفقد التحليل إذا أعاد أخذ أخطاء النموذج والمرجع بصورة مستقلة؟

77. هل فاصل bootstrap تصحيح لتعدد الاختبارات؟

الإجابة المختصرة: لا يوجد تصحيح للتعدد في هذا التنفيذ.

التفسير: التقرير يصف الفواصل بأنها استكشافية وغير معدلة، ويجب تجنب تحويل العديد من المقارنات إلى دعوى دلالة مؤكدة.

الدليل: src/xai_finance/validation/report.py/../..

لماذا يهم؟ يحد من الاستنتاج المفرط من كثرة النتائج.

سؤال متابعة: هل يشمل الفاصل عدم يقين إعادة التدريب؟

78. لماذا ليس الاختبار المعروف تأكيداً مستقلاً؟

الإجابة المختصرة: لأن النتائج الأصلية كانت معروفة قبل التحليل الإضافي.

التفسير: يصنف البروتوكول التجارب الجديدة بأنها استكشافية حتى لو حُددت إعداداتها قبل تشغيلها، بسبب اطلاع الباحث على فترة الاختبار الأصلية.

الدليل: scripts/start_scientific_validation.py/../..

لماذا يهم؟ يفصل التحليل الاستكشافي عن التحقق التأكيدي.

سؤال متابعة: ما تصميم holdout جديد مناسب؟

79. ما معنى Zero في تجارب التغير؟

الإجابة المختصرة: يتنبأ بتغير أو عائد صفري.

التفسير: تعيد implied_prices عند إضافة الصفر الإغلاق الحالي، فيطابق السعر المعاد بناؤه مرجع Persistence في الأفق التالي.

الدليل: src/xai_finance/validation/design.py/../..

لماذا يهم؟ يشرح العلاقة بين المراجع في فضائي الهدف والسعر.

سؤال متابعة: هل يمتلك Zero تغطية اتجاهية نشطة؟

80. كيف يعرف النظام حالة السوق؟

الإجابة المختصرة: من عائد متأخر وعتبة محددة.

التفسير: تضع regime_labels bull عند تجاوز العتبة وbear دون سالبها وsideways خلاف ذلك؛ قيم الحدود تقع في sideways.

الدليل: src/xai_finance/evaluate/regimes.py/../..

لماذا يهم؟ يمنع تفسير فئات مستقبلية بأنها معلومات عند الأصل.

سؤال متابعة: ما حجم النافذة التي أنتجت trailing_return_20؟

81. لماذا لا تثبت نتائج الأنظمة السوقية فاعلية تداول؟

الإجابة المختصرة: لأنها تلخص أخطاء تنبؤ لا عوائد استراتيجية.

التفسير: regime_metrics تجمع المشاهدات حسب النموذج والحالة وتستخدم forecast_metrics؛ لا تحسب تكاليف تداول أو انزلاقاً أو ربحاً.

الدليل: src/xai_finance/evaluate/regimes.py/../..

لماذا يهم؟ يفصل قياس التوقع عن التوصية الاستثمارية.

سؤال متابعة: ماذا يلزم لاختبار استراتيجية قابلة للتنفيذ؟

82. كيف تحفظ آثار التفسير؟

الإجابة المختصرة: في مجلد التفسير لكل أصل ونموذج.

التفسير: يحفظ explain_model ملف SHAP وقيم الجودة، وتكتب explain_local_dates ملفات LIME مرتبطة بفهرس الرصد وتواريخه.

الدليل: src/xai_finance/pipelines/explain.py/../..

لماذا يهم؟ يسمح بتتبع توافق التفسيرين على نفس الرصد.

سؤال متابعة: كيف تحدد الواجهة التاريخ المقابل؟

83. ماذا يعني weighted_neighborhood_r2 في LIME؟

الإجابة المختصرة: مقياس ملاءمة البديل على الجوار الموزون.

التفسير: يسجل explain_lime قيمة explanation.score إلى جانب الخطأ المطلق عند الأصل؛ يقيسان خاصيتين مختلفتين ولا يثبت أي منهما جودة التنبؤ المالي وحده.

الدليل: src/xai_finance/explain/lime_explain.py/../..

لماذا يهم؟ يساعد في تقييم وفاء البديل المحلي.

سؤال متابعة: هل R^2 جيد يضمن خطأ صغيراً عند الأصل؟

84. ما فضاء السمات الذي يستخدمه LIME عند شرح OLS؟

الإجابة المختصرة: يستقبل النوافذ الأصلية المسطحة ويستخدم تمثيلاً داخلياً للجوار، بينما يطبق النموذج تقييمه داخلياً.

التفسير: يمرر `explain_asset` بيانات التدريب والمشاهدة من `windows.flat` إلى LIME. تستخدم النسخة المحلية من LIME تمثيلاً تفسيرياً للجوار، وتستدعي دالة التنبؤ بقيم أعيدت إلى فضاء المدخل؛ يحتوي Pipeline الخطي على `StandardScaler` و`IndependentColumns`. مخرج النموذج والبديل في مثال السعر بوحدة الهدف الأصلية. لم يُثبت أن نسخة المكتبة المحلية هي نسخة بيئة التجربة التاريخية نفسها.

الدليل: مسار التفسير، النموذج الخطي، تحقيق LIME.

لماذا يهم؟ يفصل تقييس المدخلات الداخلي عن وحدة خطأ الوفاء المحلي.

سؤال متابعة: لماذا لا تعني سمات الجوار الداخلية أن خطأ البديل بوحدات معيارية؟

85. ما الذي يعرضه تبويب مقارنة النماذج؟

الإجابة المختصرة: جدول مقاييس ورسم للمقياس المختار.

التفسير: تقرأ comparison metrics.json وtraining.json وتتيح MAE وRMSE وR^2 والدقة الاتجاهية، وتنبّه إلى أن الترتيب لا يثبت قيمة استثمارية.

الدليل: dashboard/app.py/../..

لماذا يهم؟ يربط الرسم بمصدره المحفوظ.

سؤال متابعة: كيف يعالج العرض غياب زمن Persistence؟

86. ما الذي يحدث عند غياب تفسير محفوظ؟

الإجابة المختصرة: تعرض الواجهة رسالة وتعود من الدالة.

التفسير: تفحص explanations وجود quality.json، وتطلب تشغيل أمر التفسير إذا غاب بدلاً من توليد تفسير تلقائياً أثناء العرض.

الدليل: dashboard/app.py/../..

لماذا يهم؟ يوضح حدود العرض دون اتصال.

سؤال متابعة: هل تعالج كل ملفات الواجهة الناقصة بالطريقة نفسها؟

87. ما استخدام AppTest في الاختبارات؟

الإجابة المختصرة: يفحص واجهة Streamlit برمجياً.

التفسير: ينشئ اختبار غياب الآثار نسخة مؤقتة من dashboard ويؤكد ظهور إرشادات الإعداد دون استثناء؛ لا يمثل لقطة بصرية من متصفح حقيقي.

الدليل: tests/test_dashboard.py/../..

لماذا يهم؟ يفرق اختبار المكونات عن تجربة المستخدم الكاملة.

سؤال متابعة: ما السكربت الذي يستعمل متصفح Edge؟

88. كيف يتحقق اختبار SHAP من عدم تغيير التنبؤ؟

الإجابة المختصرة: يقارن التنبؤ قبل وبعد التفسير.

التفسير: يلائم اختبار اصطناعي نماذج جدولية صغيرة ثم يفحص المساواة وتجميع SHAP. نطاقه النماذج والعينات المختبرة وليس جميع الآثار.

الدليل: tests/test_explain_consistency.py/../..

لماذا يهم؟ يؤكد أن التفسير اللاحق لا يعدل مخرجات النموذج في الاختبار.

سؤال متابعة: هل يختبر ذلك صحة سببية المساهمات؟

89. لماذا لا يكفي SHA-256 منفرداً لإثبات صحة بحث؟

الإجابة المختصرة: لأنه يثبت تطابق البايتات مع مرجع مفترض الثقة.

التفسير: sha256 تحسب بصمة ملف، ويقارنها التحقق بفهرس محفوظ؛ الهاش لا يضمن صحة البيانات الأصلية أو منهج جمعها.

الدليل: src/xai_finance/storage.py/../..

لماذا يهم؟ يفصل سلامة الملف عن سلامة الاستنتاج العلمي.

سؤال متابعة: كيف تحمي الفهرس المرجعي نفسه؟

90. كيف يعمل تحقق إعادة تحميل CPU؟

الإجابة المختصرة: يقارن تنبؤات النموذج المعاد تحميله بقيم محفوظة ضمن سماح.

التفسير: تستخدم verify_asset نماذج CPU ونوافذ الاختبار وتتحقق من المقاييس والتواريخ والتفسيرات؛ الإجراء عند تشغيله حساب تدقيق وليس تدريباً.

الدليل: scripts/verify_artifacts.py/../..

لماذا يهم؟ يتحقق من قابلية استخدام النموذج بعد نقله.

سؤال متابعة: هل يثبت ذلك تطابق كل جهاز بتاً بت؟

91. لماذا تختلف سماحات LSTM بين CPU وCUDA؟

الإجابة المختصرة: بسبب اختلاف تراكم العمليات العددية.

التفسير: تستخدم replay_job سماحاً مطلقاً قدره ثمانية أضعاف epsilon لـ float32 مضروباً في معيار هدف التدريب لتوقع LSTM؛ هذا معيار مقارنة محدد لا تغيير للنتائج.

الدليل: scripts/verify_scientific_validation.py/../..

لماذا يهم؟ يوضح كيفية قبول فرق عددي صغير دون الادعاء بالتطابق البتي.

سؤال متابعة: لماذا يستخدم باقي النماذج سماحاً مختلفاً؟

92. هل يعني تشغيل verify_cpu_replay عدم كتابة أي ملف؟

الإجابة المختصرة: لا، يكتب إيصالاً في وجهة محددة.

التفسير: يقرأ الأدلة الأصلية ويجري probes تفسير محدودة ويكتب تقريره الافتراضي تحت artifacts/deployment؛ هذا لا يعادل إعادة تدريب لكنه ليس صفري الكتابة.

الدليل: scripts/verify_cpu_replay.py/../..

لماذا يهم؟ يميز حماية الأدلة من غياب الآثار الجانبية.

سؤال متابعة: كيف تختار وجهة إيصال منفصلة؟

93. ما دور SETUP_DEMO.bat؟

الإجابة المختصرة: يختار مفسراً ويطلق معالج الإعداد.

التفسير: يفضل البيئة المحلية ثم خيارات Python المتاحة، ويعيد رمز نتيجة الإعداد؛ تفاصيل تثبيت الحزم وفحصها في demo_wizard.py.

الدليل: ../../SETUP_DEMO.bat.

لماذا يهم؟ يساعد الطالب على شرح خطوات الدفاع العملي.

سؤال متابعة: هل تتضمن الحزمة مثبت Python؟

94. كيف يمنع المعالج إيقاف عملية أخرى؟

الإجابة المختصرة: يتحقق من جذر التشغيل وPID وطابع إنشاء العملية.

التفسير: تقرأ owned_state الحالة المسجلة وتطابق process_stamp، ثم يعيد terminate_owned_tree فحص الهوية قبل إرسال الإنهاء للأبناء؛ لا يوقف خدمة لمجرد شغل منفذ.

الدليل: ../../scripts/demo_wizard.py.

لماذا يهم؟ يحمي الخدمات المحلية المجاورة أثناء العرض.

سؤال متابعة: لماذا لا يكفي PID وحده؟

95. هل العرض المحلي يحتاج CUDA؟

الإجابة المختصرة: لا، حزمة الدفاع معدة لـ PyTorch CPU.

التفسير: يطلب runtime_check نسخة torch 2.5.1+cpu وغياب CUDA في build، وتعرض لوحة Streamlit الآثار المحفوظة. تدريب البحث السابق قد يكون على CUDA.

الدليل: ../../scripts/demo_wizard.py.

لماذا يهم؟ يفصل جهاز التدريب عن متطلبات العرض.

سؤال متابعة: هل توفر الحزمة وحدها مثبت Windows Python؟

96. ما قيود اختبار الاتصال دون شبكة؟

الإجابة المختصرة: يحجب واجهات محددة وليس كل مسارات الشبكة.

التفسير: يستبدل offline_guard بعض دوال socket عند ضبط المتغير، بينما يستخدم اختبار المتصفح وكيلاً مغلقاً للاتصال الخارجي؛ لكل فحص نطاقه ولا يمثل جدار حماية شاملاً.

الدليل: ../../scripts/offline_guard/sitecustomize.py.

لماذا يهم؟ يمنع المبالغة في استنتاج اختبار القبول.

سؤال متابعة: ما الفرق بين فحص Python وفحص المتصفح؟

97. ما الفرق بين CRC وSHA-256 في الحزمة؟

الإجابة المختصرة: كلاهما يفحص السلامة بخصائص مختلفة.

التفسير: يستخدم بناء testzip CPU للتحقق من CRC ثم يحسب SHA-256 لكل عضو مقابل manifest ويحسب بصمة الأرشيف؛ الثقة بالفهرس تبقى ضرورية.

الدليل: ../../scripts/package_cpu_defense.py.

لماذا يهم؟ يوضح كيف تُراجع الحزمة بعد النسخ.

سؤال متابعة: هل النسخة على القرص نفسه تحمي من فشله؟

98. لماذا توثق نسخة المصدر وبصمتها؟

الإجابة المختصرة: لتحديد الشيفرة المرتبطة بتجربة محددة.

التفسير: يسجل generate_report بصمات ملفات src والقفل، وتحتفظ الشروح ببصمات مقتطفاتها. تغير الملف قد يجعل أرقام الأسطر أو الشرح قديماً.

الدليل: ../../src/xai_finance/pipelines/report.py.

لماذا يهم؟ يجعل من الممكن فحص صلاحية الإحالات.

سؤال متابعة: هل توافق البصمة يثبت صحة الاستنتاج الإحصائي؟

99. ما حدود استقرار إعادة التدريب؟

الإجابة المختصرة: فحص التكرار المسجل محدود بمهمة وبيئة.

التفسير: replicate_validation_lstm يعيد ملاءمة تجربة AAPL واحدة ويقارن الخسائر والتنبؤات بتياً عند تشغيله؛ لا يدرس توزيعات بذور أو أجهزة متعددة.

الدليل: ../../scripts/replicate_validation_lstm.py.

لماذا يهم؟ يمنع تعميم قابلية التكرار المحلية.

سؤال متابعة: ما الذي تضيفه تجارب متعددة البذور؟

100. ما أهم نتيجة يجب عرضها أمام اللجنة؟

الإجابة المختصرة: عرض النتائج المحفوظة مع Persistence والقيود.

التفسير: يذكر تدقيق النتائج أن Persistence حقق أقل RMSE على الأصول اليومية الخمسة في التشغيل المجمد، بينما المرجع التاريخي منفصل وغير معاد إنتاجه؛ لا يفرض هذا فائزاً عاماً.

الدليل: ../RESULTS_AUDIT.md.

لماذا يهم؟ يضمن عرض دليل مخالف للتوقع بوضوح.

سؤال متابعة: ما تجربة مستقبلية مستقلة يمكنها اختبار التعميم؟

فهرس آلي أولي للشيفرة

يشمل main.py ومجلدات dashboard وsrc وscripts وtests. تستبعد نسخ artifacts المحفوظة ومخرجات البناء والبيئات. هذا جرد نحوي آلي وليس دليلاً على اكتمال الشرح. عدد الأسطر القابلة للتنفيذ تقريبي من أرقام أسطر bytecode وقد يتغير حسب إصدار Python. الأسطر ذات الرموز تحتسب بداية الرمز فقط ولا تكفي وحدها لإثبات التغطية.

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
main.py	11	5	—	pathlib, runpy
dashboard/app.py	209	173	read_json:22, load_table:27, csv_table:31, saved_image:35, overview:42, preprocessing:54, forecasting:70, comparison:85, explanations:95, regimes:127, reproducibility:137, thesis_alignment:162, main:179	json, numpy, pandas, pathlib, plotly.express, plotly.graph_objects, streamlit
scripts/audit_reconstruction_precision.py	37	31	main:13	argparse, numpy, pandas, pathlib, xai_finance.storage, xai_finance.validation.design, xai_finance.validation.statistics
scripts/bootstrap_cpu.py	23	21	—	os, pathlib, subprocess, sys
scripts/browser_smoke_cpu.py	118	101	CDP:19, **init**:20, call:25, evaluate:38, main:45	argparse, base64, json, pathlib, subprocess, time, urllib.request, websockets.sync.client

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
scripts/build_validation_report.py	28	23	main:10	argparse, base64, html, pathlib, xai_finance.storage, xai_finance.validation.report
scripts/check_dashboard_cpu.py	64	53	main:13, check:20, connect:24, connect_ex:28	argparse, json, pathlib, socket, streamlit.testing.v1
scripts/demo_wizard.py	350	298	say:30, environment:37, run:42, sha256:51, verify_manifest:59, verify_artifacts:74, runtime_check:96, process_stamp:101, owned_state:124, terminate_owned_tree:134, Entry:139, healthy:175, launch_lock:184, start:202, stop:253, setup:275, main:315	argparse, contextlib, ctypes, datetime, hashlib, json, msvcrt, os, pathlib, platform, signal, socket, struct, subprocess, sys, time, urllib.request, venv, webbrowser
scripts/diagnose_validation_replay.py	38	32	main:15	argparse, joblib, numpy, pandas, pathlib, xai_finance.models.lstm_model, xai_finance.pipelines.train, xai_finance.seed, xai_finance.storage
scripts/download_data.py	4	3	—	xai_finance.config, xai_finance.io.data_sources
scripts/generate_defense_package.py	54	45	copy_project:11, archive_package:24, main:39	pathlib, shutil, xai_finance.config, xai_finance.pipelines.train, xai_finance.storage, zipfile

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
`scripts/launch_demo.py`	8	7	—	os, pathlib, subprocess, sys
`scripts/offline_demo.py`	9	8	—	pathlib, subprocess, sys
`scripts/offline_guard/sitecustomize.py`	32	25	check:12, connect:16, connect_ex:21, getaddrinfo:26_	os, socket
`scripts/offline_smoke.py`	38	31	loopback_only:13	json, pathlib, socket, streamlit.testing.v1
`scripts/package_cpu_defense.py`	97	87	digest:14, main:19	argparse, datetime, hashlib, json, pathlib, shutil, zipfile
`scripts/package_scientific_validation.py`	79	64	EmbeddedImages:16, **init**:17, handle_starttag:21, verify_offline_page:26, snapshot_source:39, package_addendum:50, main:69	PIL, argparse, base64, hashlib, html.parser, pathlib, shutil, xai_finance.config, xai_finance.storage, zipfile

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
scripts/replicate_validation_lstm.py	51	45	main:17	argparse, joblib, numpy, pathlib, threadpoolctl, xai_finance.config, xai_finance.models.lstm_model, xai_finance.pipelines.train, xai_finance.seed, xai_finance.storage, xai_finance.validation.design
scripts/run_full.py	9	8	—	os, pathlib, subprocess, sys
scripts/run_quick.py	9	8	—	os, pathlib, subprocess, sys
scripts/run_scientific_validation.py	29	23	main:11	argparse, pathlib, xai_finance.config, xai_finance.storage, xai_finance.validation.audit, xai_finance.validation.experiments
scripts/setup_offline.py	8	6	—	pathlib, subprocess, sys
scripts/start_scientific_validation.py	44	36	protected_files:9, main:14	datetime, pathlib, xai_finance.config, xai_finance.storage

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
`scripts/test_deployment_cpu.py`	82	72	main:15, batch:26	argparse, importlib.util, json, os, pathlib, socket, subprocess, urllib.request
`scripts/verify_artifacts.py`	75	63	verify_asset:17, verify_explanation:42, main:59	numpy, pandas, pathlib, xai_finance.config, xai_finance.io.data_sources, xai_finance.pipelines.report, xai_finance.pipelines.train, xai_finance.storage
`scripts/verify_cpu_replay.py`	90	81	main:18, connect:24	argparse, importlib, json, numpy, os, pathlib, pkgutil, socket, sys, threadpoolctl, torch, verify_artifacts, verify_scientific_validation, xai_finance, xai_finance.explain.lime_explain, xai_finance.explain.shap_explain, xai_finance.pipelines.train, xai_finance.seed, xai_finance.storage
`scripts/verify_environment.py`	3	2	—	xai_finance.environment
`scripts/verify_preservation_cpu.py`	31	23	digest:11, main:16	argparse, hashlib, json, pathlib
`scripts/verify_scientific_validation.py`	93	78	replay_job:18, verify_aggregate:49, preservation_receipt:66, main:77	argparse, dataclasses, datetime, numpy, pandas, pathlib, xai_finance.config, xai_finance.pipelines.train, xai_finance.seed, xai_finance.storage, xai_finance.validation.design, xai_finance.validation.statistics

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/__init__.py	3	2	—	—
src/xai_finance/cli.py	42	33	main:9, execute:18	argparse, os, xai_finance.config, xai_finance.environment, xai_finance.io.data_sources, xai_finance.pipelines.explain, xai_finance.pipelines.report, xai_finance.pipelines.train
src/xai_finance/config.py	21	15	project_root:5, load_config:16	pathlib, yaml
src/xai_finance/data/__init__.py	0	0	—	—
src/xai_finance/data/features.py	43	37	relative_strength:5, features:14, return_features:35	numpy, pandas
src/xai_finance/data/preprocess.py	28	20	TrainingScalers:10, transform_windows:16, inverse_target:20, fit_training_scalers:24	dataclasses, numpy, sklearn.preprocessing, xai_finance.data.windows

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/data/split.py	9	7	chronological_indices:4	numpy
src/xai_finance/data/validate.py	28	24	validate_prices:5, validate_ohlc:22	numpy, pandas
src/xai_finance/data/windows.py	37	28	WindowDataset:10, flat:21, flat_names:25, build_windows:29	dataclasses, numpy, pandas, xai_finance.data.split
src/xai_finance/environment.py	24	20	verify_environment:11	json, torch, xai_finance.config, xai_finance.pipelines.train, xai_finance.seed, xai_finance.storage
src/xai_finance/evaluate/__init__.py	0	0	—	—
src/xai_finance/evaluate/metrics.py	14	12	forecast_metrics:5	numpy, sklearn.metrics

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/evaluate/regimes.py	15	10	regime_labels:7, regime_metrics:11	numpy, pandas, xai_finance.evaluate.metrics
src/xai_finance/explain/__init__.py	0	0	—	—
src/xai_finance/explain/lime_explain.py	11	9	explain_lime:5	lime.lime_tabular, numpy
src/xai_finance/explain/quality.py	17	13	attribution_stability:5, lime_weight_vector:13	numpy, scipy.stats
src/xai_finance/explain/shap_explain.py	28	24	explain_shap:5, shap_reconstruction:24	numpy, shap

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/io/__init__.py	0	0	—	—
src/xai_finance/io/data_sources.py	66	54	yahoo_prices:13, fred_prices:23, download_asset:34, load_snapshot:52, download_all:64	datetime, io, pandas, requests, xai_finance.config, xai_finance.data.validate, xai_finance.storage, yfinance
src/xai_finance/models/__init__.py	0	0	—	—
src/xai_finance/models/baselines.py	5	3	persistence:4	numpy
src/xai_finance/models/linear_preprocess.py	22	16	IndependentColumns:6, **init**:9, fit:12, transform:21	numpy, scipy.linalg, sklearn.base

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/models/lstm_model.py	205	176	PriceLSTM:16, **init**:17, forward:23, LSTMPredictor:29, predict:36, probe_precision:48, scaled_loss:68, batch_probe:75, measure_batch:100, train_epoch:117, checkpoint_state:136, optimize_epochs:140, fit_lstm:167, training_tensors:189, load_lstm:199	copy, dataclasses, numpy, pathlib, time, torch, xai_finance.data.preprocess, xai_finance.data.windows, xai_finance.seed, xai_finance.storage
src/xai_finance/models/sklearn_models.py	38	32	candidates:14, fit_candidates:24	numpy, sklearn.ensemble, sklearn.linear_model, sklearn.metrics, sklearn.pipeline, sklearn.preprocessing, time, xai_finance.data.windows, xai_finance.models.linear_preprocess
src/xai_finance/pipelines/__init__.py	0	0	—	—
src/xai_finance/pipelines/explain.py	108	93	fixed_subset:18, explain_experiment:22, explain_asset:35, explain_model:50, explain_local_dates:76, save_regime_drivers:98	numpy, pandas, pathlib, threadpoolctl, time, torch, xai_finance.evaluate.regimes, xai_finance.explain.lime_explain, xai_finance.explain.quality, xai_finance.explain.shap_explain, xai_finance.pipelines.train, xai_finance.reports.plots, xai_finance.seed, xai_finance.storage
src/xai_finance/pipelines/report.py	119	101	generate_report:17, asset_report:33, verify_saved_metrics:52, extra_charts:61, export_coefficients:87, research_limitations:99, write_research_markdown:103, write_inventory:117	datetime, matplotlib.pyplot, numpy, pandas, pathlib, plotly.express, shutil, xai_finance.config, xai_finance.evaluate.metrics, xai_finance.pipelines.train, xai_finance.reports.plots, xai_finance.storage

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/pipelines/train.py	134	110	software_environment:28, train_experiment:34, save_windows:52, load_windows:57, split_manifest:62, train_asset:66, evaluate_asset:89, load_models:110, verify_reload:117, save_next_forecasts:126, resolve_run:132	datetime, importlib.metadata, joblib, numpy, os, pandas, pathlib, platform, shutil, subprocess, sys, threadpoolctl, xai_finance.config, xai_finance.data.features, xai_finance.data.windows, xai_finance.evaluate.metrics, xai_finance.evaluate.regimes, xai_finance.io.data_sources, xai_finance.models.baselines, xai_finance.models.lstm_model, xai_finance.models.sklearn_models, xai_finance.seed, xai_finance.storage
src/xai_finance/reports/__init__.py	0	0	—	—
src/xai_finance/reports/plots.py	108	91	save_figure:16, forecast_charts:24, metric_charts:39, loss_chart:49, research_flow:60, explanation_charts:75, aggregate_lag_charts:92	matplotlib, matplotlib.pyplot, numpy, pandas, pathlib, shap
src/xai_finance/seed.py	37	31	seed_experiment:10, select_device:22	numpy, os, random, torch
src/xai_finance/storage.py	29	20	sha256:8, json_default:13, write_json:23, read_json:28	hashlib, json, numpy, pathlib

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/validation/__init__.py	1	1	—	—
src/xai_finance/validation/audit.py	92	79	causal_feature_check:14, alignment_check:22, scaler_check:33, distribution_shift:45, frozen_metric_check:64, audit_original:78	joblib, numpy, pandas, xai_finance.data.features, xai_finance.data.preprocess, xai_finance.data.windows, xai_finance.evaluate.metrics, xai_finance.io.data_sources, xai_finance.pipelines.train, xai_finance.storage
src/xai_finance/validation/design.py	54	42	target_values:8, implied_prices:18, transformed_windows:26, walk_forward_partitions:30, naive_predictions:44	dataclasses, numpy, xai_finance.data.windows
src/xai_finance/validation/diagnostics.py	60	51	target_distribution:11, lstm_output_bound:22, supplemental_audit:32, prediction_amplitudes:54	joblib, numpy, pandas, torch, xai_finance.pipelines.train, xai_finance.storage, xai_finance.validation.statistics
src/xai_finance/validation/experiments.py	102	88	fit_models:20, predictions_table:31, job_configuration:42, experiment_job:49, run_experiments:78, aggregate_experiments:92	dataclasses, gc, joblib, numpy, pandas, pathlib, threadpoolctl, torch, xai_finance.models.lstm_model, xai_finance.models.sklearn_models, xai_finance.pipelines.train, xai_finance.seed, xai_finance.storage, xai_finance.validation.design, xai_finance.validation.statistics

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
src/xai_finance/validation/figures.py	127	108	save_figure:18, frozen_forecasts:24, rmse_comparison:42, bootstrap_intervals:65, walk_forward_errors:82, directional_accuracy:100, generate_figures:116	matplotlib, matplotlib.colors, matplotlib.pyplot, numpy, pandas, pathlib, xai_finance.storage
src/xai_finance/validation/report.py	242	115	markdown_table:12, rmse_table:21, dataset_table:28, execution_summary:36, evidence_tables:45, principal_findings:62, audit_narrative:72, experiment_narrative:100, write_report:163	pandas, pathlib, xai_finance.config, xai_finance.storage, xai_finance.validation.diagnostics, xai_finance.validation.figures, xml.etree.ElementTree
src/xai_finance/validation/statistics.py	31	24	directional_statistics:6, paired_bootstrap:13, evaluation_metrics:23	numpy, xai_finance.evaluate.metrics
tests/conftest.py	19	13	analytic_prices:10, window_fixture:18	numpy, pandas, pytest, xai_finance.data.features, xai_finance.data.windows
tests/test_dashboard.py	15	12	test_missing_artifacts_shows_actionable_setup_commands:7	pathlib, shutil, streamlit.testing.v1
tests/test_explain_consistency.py	30	25	test_shap_additivity_and_posthoc_explanation_preserve_forecast:10, test_lime_accepts_batch_prediction_and_returns_measured_surrogate_error:22	numpy, pytest, xai_finance.explain.lime_explain, xai_finance.explain.shap_explain, xai_finance.models.sklearn_models

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
`tests/test_lstm_training.py`	28	22	test_cuda_batch_probe_supports_cudnn_backward:10, test_cpu_lstm_training_checkpoint_and_inverse_scale_roundtrip:20	numpy, pytest, torch, xai_finance.models.lstm_model, xai_finance.seed
`tests/test_metrics_and_models.py`	54	40	test_metrics_match_hand_computable_original_unit_errors:11, test_zero_target_mape_is_explicitly_undefined:20, test_persistence_uses_current_close_without_future_labels:27, test_repeated_forest_inference_is_bitwise_stable:32, test_regime_boundary_uses_declared_trailing_return_threshold:41, test_saved_tabular_model_preserves_predictions_without_refitting:46	joblib, numpy, pytest, xai_finance.evaluate.metrics, xai_finance.evaluate.regimes, xai_finance.models.baselines, xai_finance.models.sklearn_models
`tests/test_provider_integration.py`	22	18	test_real_provider_ols_explanation_reconstructs_stably:12	numpy, pytest, xai_finance.data.features, xai_finance.data.windows, xai_finance.explain.shap_explain, xai_finance.io.data_sources, xai_finance.models.sklearn_models
`tests/test_scientific_validation.py`	60	48	test_target_reconstruction_and_scaling_restore_next_price:12, test_return_protocol_rejects_negative_price_but_changes_preserve_it:22, test_walk_forward_blocks_are_disjoint_and_fit_only_available_labels:31, test_naive_predictions_do_not_use_unknown_test_labels:42, test_zero_prediction_is_directional_abstention:55	dataclasses, numpy, pytest, xai_finance.data.preprocess, xai_finance.validation.design, xai_finance.validation.statistics

المسار	الأسطر الفعلية	تنفيذية تقريبية	الأصناف والدوال (الاسم:السطر)	الواردات
tests/test_temporal_integrity.py	78	62	test_future_price_mutation_preserves_past_features:13, test_targets_are_exactly_next_observed_close_and_windows_end_at_origin:23, test_split_targets_never_cross_partition_boundary:34, test_heldout_mutation_cannot_change_fitted_scalers:45, test_target_inverse_transform_restores_original_units:56, test_invalid_provider_snapshot_is_rejected:63, test_insufficient_history_fails_instead_of_empty_test_set:76	numpy, pandas, pytest, xai_finance.data.features, xai_finance.data.preprocess, xai_finance.data.split, xai_finance.data.validate, xai_finance.data.windows

المجموع: 77 ملفاً؛ 3929 سطراً فعلياً؛ 3181 سطراً تنفيذياً تقريبياً.

لا يستخرج هذا الجرد آلياً المسؤوليات أو نقاط الدخول أو الملفات المقروءة والمكتوبة؛ تحتاج إلى فحص يدوي. أرقام الدوال المتداخلة تظهر أيضاً، ولا يثبت الاستيراد تنفيذ المسار وقت التشغيل.

سجل المراجعة العلمية: دفعة أولى

هذه مراجعة مؤلف مستقلة عن فحوص مطابقة الاقتباس، وليست مراجعة خبير خارجي. نطاقها في هذه الدفعة الفصول 01 و03 و04 و05 و06 و07 و08 والمواضع المرتبطة بها من `features.py` و`windows.py` و`preprocess.py` و`sklearn_models.py` و`lstm_model.py` وملفات التفسير والمقاييس و`validation/statistics.py`، مع قراءة سجلات AAPL المذكورة سابقاً. لم تُراجع بعد جميع الصفحات السطرية أو كل النتائج والاقتباسات العلمية؛ لم تُشغّل تجربة.

التصنيف	المسألة	الإجراء أو حالة الدليل
صياغة قديمة غير صحيحة	فصل 06 قال إن المقارنة الكمية لم تُراجع، رغم وجود مثال AAPL محفوظ أدناه.	صُحح إلى أن مثالاً واحداً روجع، مع بقاء بقية الأمثلة دون مراجعة شاملة.
ملتبس لغوياً	عبارة فترتي SMA في فصل 03 قد تلتبس.	أصبحت نافذتين من 5 و20 رصداً، كما في `features.py:22-23`.
صحيح ويحتاج نطاقاً	فحص حذف المستقبل والتقسيم الزمني لا يثبت توفر النسخ المعدلة في الوقت التاريخي.	يبقى القيد منصوصاً في فصلي البيانات والتقييم؛ `validation/audit.py:14-30` يقارن الرصد ومصدر اللقطة وليس تواريخ إصدارها.
صحيح ويحتاج تفسيراً	تقييس LSTM يكرر صفوف التدريب في النوافذ المتداخلة؛ OLS يملك مسار تقييس مختلفاً.	مبين في فصل البيانات و`data/preprocess.py:24-27` و`models/sklearn_models.py:21`.
غير مثبت	سبب فشل ترتيب LSTM التاريخي بدقة.	غياب إعدادات الأصل يمنع تفسيراً سببياً حاسماً؛ يوضح `docs/ASSUMPTIONS_AND_DEVIATIONS.md` و`RESULTS_AUDIT.md` الفرق بين المرجع وإعادة البناء.
صحيح ويحتاج تفسيراً	استقرار إعادة بناء SHAP لا يثبت صدقاً سببياً؛ خطأ LIME للرصد المحفوظ مطلق عند الرصد، لا RMSE.	موثق بمقادير ومصدر `lime_explain.py:8-10` وملف `quality.json` في فصل 06. لا يعمم على كل التفسيرات.
صحيح ويحتاج تفسيراً	فواصل bootstrap لا تشمل عدم يقين تدريب النماذج ولا تصحيح التعدد.	`validation/statistics.py:13-20` والسرد في `validation/report.py`؛ فاصل MSE بوحدة السعر المربعة.
غير متحقق	قابلية تفسير Mermaid من محلل مستقل وعرض العربية في PDF.	لم يُثبت بعد؛ لا تدّع التحقق ولا تنتج PDF غير مفحوص.

مراجعة الصيغ: `PriceLSTM.forward` يأخذ آخر خرج من طبقة واحدة؛ تقدير التدرجات وتحديث Adam وقص المعيار في `lstm_model.py`. دقة الاتجاه في `forecast_metrics` ثلاثية الإشارة، بينما مصفوفة الالتباس ثنائية وتعد التعادل غير صعود؛ هذا يفسر اختلاف تفسير Persistence. هذه الملاحظات لا تحل محل تدقيق كل فصل وصفحة ونتيجة.

متابعة المراجعة: فحص الأسئلة ومخطط النشر

موضع الادعاء	الادعاء والدليل	النتيجة والإجراء	الحالة
سؤالا 23 و84 في النسخة الأولية	كلاهما كان يسأل عن تعريف خطأ AAPL؛ `lime_explain.py:8–10` يحدد الخطأ المطلق.	تداخل موضوعي؛ استُبدل 84 بسؤال مستقل عن فضاء السمات وتقييس OLS مع إبقاء مثال 23.	صُحح؛ بقيت مراجعة التداخل الدلالي لبقية المجموعة.
سؤالا 25 و76	كلاهما ركّز على bootstrap المزدوج؛ `validation/statistics.py:13–20` يحسب الفرق لكل تاريخ.	أُعيد تخصيص 76 لسبب الاقتران بين أخطاء التاريخ نفسه، بينما يبقى 25 عن الإجراء الكلي.	صُحح؛ لا يدعي الفحص الآلي إثبات عدم وجود تداخلات أخرى.
فصل 02	عشرة مخططات مرتبطة بمسارات المصدر؛ `npm.cmd cache ls mermaid` لم يعثر على حزمة، و`require.resolve` لم يجدها.	لا يوجد محلل محلي مثبت؛ لم يُنزل اعتماد أو يتغير إعداد التطبيق.	التحقق التركيبي غير منجز.
مسودة HTML	`audit_print_draft.py` يتحقق من RTL وCSS للشيفرة و14 فصلاً و81 صفحة مصدر؛ Edge فشل في عملية GPU قبل إنتاج معاينة.	فحص بنيوي فقط؛ أرقام الصفحات وتشكيل العربية وفواصل الطباعة والرسوم غير مثبتة.	غير صالح للاعتماد النهائي أو PDF مفحوص.
كل الفصول والنتائج	فُحصت موضوعات محددة في الدفعة الأولى ومثال LIME؛ لم تُطابق جميع الادعاءات والسجلات سطراً بسطر.	تبقى مراجعة شاملة ومراجع سطرية أدق وقراءة مقارنة مستقلة.	غير مكتمل؛ لا توجد مراجعة خبير خارجي.

الفحص البنيوي للأسئلة يؤكد 100 عنوان متسلسل وحقولاً ووجود ملفات الأدلة، ولا يحكم على صحة إجابة أو كفاية دليل. أول 25 سؤالاً أضيفت لها إجابة مختصرة مأخوذة من تفسيرها الموجود؛ تحتاج إعادة صياغة تحريرية لتجنب التكرار داخل الإجابة.

تحديث فحص النشر

نجح Mermaid CLI 11.4.2 في تحليل المخططات العشرة الرئيسية وتصديرها، ضمن تثبيت مستقل عن بيئة Python. فُحص مخططان بصرياً فقط؛ صحة جميع الرسوم العلمية واتجاه النص المختلط لم تُراجع بالكامل. نجح Edge headless عند تعطيل GPU وcompositing في إنتاج نسخة PDF إثبات من مسودة HTML السابقة. أظهرت صور الصفحات 1 و138 و274 اتصال الحروف العربية واتجاه الشيفرة وأرقام الصفحات في تلك العينات فقط؛ لا يثبت ذلك سلامة بقية الصفحات أو الفهرس أو المعادلات. نسخة الإثبات لا تحتوي الرسوم المصدرة وهي غير نهائية. لم تُستكمل المراجعة العلمية لكل الأسئلة والفصول أو مراجعة خبير خارجي.

تقرير التغطية المؤقت

النطاق: main.py ومجلدات dashboard وsrc وscripts وtests. تستبعد البيئات ونسخ المصدر المولّدة تحت artifacts؛ يلزم البت في سكربتات artifacts/deployment المستقلة. ملفات batch والإعدادات ليست في المقام.

- الجرد: 77 ملفاً، 3929 سطراً فعلياً؛ الأسطر التنفيذية التقريبية: 3181.
- مقياس بدايات رموز Python غير التعليقات: 3210 من 3210 (100.00%). لا يحسب كل أسطر السلاسل متعددة الأسطر، ولا يقيس جودة الشرح الأكاديمي.
- صفحات يدوية تغطي جميع بدايات الرموز في الجرد: 77؛ ملفات فيها فجوات: 0.
- يتحقق هذا التدقيق من بصمات المصدر واقتباسات جميع دفعات manifest المشمولة آلياً؛ تدقق stamp_source_versions.py بصمات صفحات الشرح مقابل الجرد وتدرجها في الصفحات. تدقق أداة verify_legacy_quotes.py اقتباسات الصفحتين الأقدم وبصمتيهما؛ شغّلها مع هذا التدقيق.
- المراجعة الأكاديمية: تفسيرات الصفحات روجعت يدوياً مقابل المصدر؛ لم يختبر تشغيل الواجهة، ولم تُراجع الشروح مراجعة مستقلة. لا يدل التطابق النصي على صحة التفسير.

الملفات المغطاة بمقياس بدايات الرموز

- `main.py`
- `dashboard/app.py`
- `scripts/audit_reconstruction_precision.py`
- `scripts/bootstrap_cpu.py`
- `scripts/browser_smoke_cpu.py`
- `scripts/build_validation_report.py`
- `scripts/check_dashboard_cpu.py`
- `scripts/demo_wizard.py`
- `scripts/diagnose_validation_replay.py`
- `scripts/download_data.py`
- `scripts/generate_defense_package.py`
- `scripts/launch_demo.py`
- `scripts/offline_demo.py`
- `scripts/offline_guard/sitecustomize.py`

- scripts/offline_smoke.py
- scripts/package_cpu_defense.py
- scripts/package_scientific_validation.py
- scripts/replicate_validation_lstm.py
- scripts/run_full.py
- scripts/run_quick.py
- scripts/run_scientific_validation.py
- scripts/setup_offline.py
- scripts/start_scientific_validation.py
- scripts/test_deployment_cpu.py
- scripts/verify_artifacts.py
- scripts/verify_cpu_replay.py
- scripts/verify_environment.py
- scripts/verify_preservation_cpu.py
- scripts/verify_scientific_validation.py
- src/xai_finance/__init__.py
- src/xai_finance/cli.py
- src/xai_finance/config.py
- src/xai_finance/data/__init__.py
- src/xai_finance/data/features.py
- src/xai_finance/data/preprocess.py
- src/xai_finance/data/split.py
- src/xai_finance/data/validate.py
- src/xai_finance/data/windows.py
- src/xai_finance/environment.py
- src/xai_finance/evaluate/__init__.py
- src/xai_finance/evaluate/metrics.py
- src/xai_finance/evaluate/regimes.py
- src/xai_finance/explain/__init__.py
- src/xai_finance/explain/lime_explain.py
- src/xai_finance/explain/quality.py
- src/xai_finance/explain/shap_explain.py
- src/xai_finance/io/__init__.py

- src/xai_finance/io/data_sources.py
- src/xai_finance/models/__init__.py
- src/xai_finance/models/baselines.py
- src/xai_finance/models/linear_preprocess.py
- src/xai_finance/models/lstm_model.py
- src/xai_finance/models/sklearn_models.py
- src/xai_finance/pipelines/__init__.py
- src/xai_finance/pipelines/explain.py
- src/xai_finance/pipelines/report.py
- src/xai_finance/pipelines/train.py
- src/xai_finance/reports/__init__.py
- src/xai_finance/reports/plots.py
- src/xai_finance/seed.py
- src/xai_finance/storage.py
- src/xai_finance/validation/__init__.py
- src/xai_finance/validation/audit.py
- src/xai_finance/validation/design.py
- src/xai_finance/validation/diagnostics.py
- src/xai_finance/validation/experiments.py
- src/xai_finance/validation/figures.py
- src/xai_finance/validation/report.py
- src/xai_finance/validation/statistics.py
- tests/conftest.py
- tests/test_dashboard.py
- tests/test_explain_consistency.py
- tests/test_lstm_training.py
- tests/test_metrics_and_models.py
- tests/test_provider_integration.py
- tests/test_scientific_validation.py
- tests/test_temporal_integrity.py

الفجوات: أرقام أسطر بدايات الرموز غير المشروحة

النطاقات تجمع الأسطر المتتالية فقط؛ التعليقات والأسطر الفارغة وأسطر استمرار السلاسل لا تدخل هذا المقياس.

المطلوب أيضاً: تدقيق الاقتباسات والتفسير لجميع الصفحات، وفصل تغطية الأكواد عن الجودة، وإكمال الفصول والرسوم والأمثلة الفعلية والاختبارات والنشر وأسئلة الدفاع والتقرير القابل للطباعة. لا يجوز ادعاء اكتمال 100%.

نطاق موسع مستقل

وُجد سكربتان من شيفرة المشروع تحت artifacts/deployment خارج جرد 77: initial_check.py وverify_final_release.py. تدقق verify_deployment_extra.py بصمتيهما واقتباساتهما و57 بداية رمز فيهما؛ معهما يصبح النطاق الموسع 79 ملفاً و3267 بداية رمز عند نجاح جميع التدقيقات. لا يشمل ذلك نسخ المصدر المحفوظة أو سكربتات توليد وتدقيق هذا التوثيق. هذه الأرقام لا تثبت اكتمال الشرح الأكاديمي أو دقة النتائج.

ملحق الشروح السطرية

كل قسم يحتفظ بمسار المصدر وبصمته وأرقام الأسطر الموثقة؛ يعتمد اكتمال المعنى على المراجعة العلمية المنفصلة.

شرح artifacts/deployment/initial_check.py

بصمة المصدر SHA-256:

.8b17f8938e1f14907271816a38016de97415cf858c3cb36ab5f31f3778d989bc

سكربت نشر خارج جرد التطبيق الأصلي؛ لم يُنفذ هنا.

الأسطر 1–5

```
import hashlib,json
from pathlib import Path
root=Path.cwd()
files=[p for folder in ['artifacts/runs','artifacts/scientific_validation','data/raw','data/manifests'] for p in
(root/folder).rglob('*') if p.is_file()]
manifest={p.relative_to(root).as_posix():hashlib.sha256(p.read_bytes()).hexdigest() for p in files}
```

يستورد hashlib وjson وPath، ويعتبر مجلد العمل الجذر ويجمع ملفات النتائج والبيانات ويحفظ بصماتها في قاموس؛ يفترض التشغيل من جذر المشروع ويقرأ كل ملف كاملاً إلى الذاكرة.

الأسطر 6–9

```
output=root/'artifacts/deployment/original_evidence_sha256.json'
if output.exists(): raise RuntimeError('Preservation baseline already exists')
output.write_text(json.dumps(manifest,indent=2))
print('Preservation baseline:',len(files),'files;',sum(p.stat().st_size for p in
files)/1024**2,'MiB',flush=True)
```

يحدد ملف baseline ويرفض الكتابة إن وجد ثم يكتب البصمات ويطبع العدد والحجم؛ له أثر كتابة ولا يجوز تشغيله لاستبدال الأدلة.

الأسطر 10–15

```
from streamlit.testing.v1 import AppTest
app=AppTest.from_file(str(root/'main.py'),default_timeout=90).run()
print('Exceptions:',[e.message for e in app.exception])
print('Title:',[x.value for x in app.title]);print('Tabs:',[x.label for x in
app.tabs]);print('Datasets:',app.selectbox(key='asset').options)
assert not app.exception
assert len(app.tabs)==8
```

يشغل Streamlit AppTest على main.py ويطبع الاستثناءات والعنوان والتبويبات والأصول ويفرض عدم وجود استثناء وثمانية تبويبات؛ قد يفشل قبل assertions إذا لم يوجد عنصر الاختيار. لم ينفذ هنا.

شرح
ıcts/deployment/verify_final_release.py

بصمة المصدر SHA-256:

`69b3222f1d90e10e8cb48a987a7c6e227dccc72ad9efa8084f7d623f9f2a51cc`.

سكربت نشر خارج جرد التطبيق الأصلي؛ لم يُنفذ هنا.

الأسطر 1–8

```
"""Final distribution acceptance, independent of the archive builder."""

import argparse
import hashlib
import json
from pathlib import Path
import zipfile
```

سلسلة توثيق واستيرادات قياسية لـ CLI والبصمات وJSON والمسارات وZIP.

الأسطر 9–17

```
root = Path(__file__).resolve().parents[2]
archive = root / "Mohammad_XAI_Defense_CPU.zip"
candidate = root / "dist/CPU_Candidate.zip"
parser = argparse.ArgumentParser()
parser.add_argument("--destination", type=Path, default=root / "dist/final_release_acceptance")
args = parser.parse_args()
destination = args.destination.resolve()
assert destination.is_relative_to(root.resolve())
assert not destination.exists(), "Do not overwrite a previous extracted verification"
```

يحدد الجذر والأرشيف النهائي والمرشح ووجهة الاستخراج، ويتحقق من بقاء الوجهة في الجذر وعدم وجودها؛ يعتمد على assert الذي يمكن تعطيله في Python المحسن.

الأسطر 18–25

```
with archive.open("rb") as stream:
    archive_hash = hashlib.file_digest(stream, "sha256").hexdigest()
assert archive_hash == archive.with_suffix(".zip.sha256").read_text().split()[0]
prefix = "Mohammad_XAI_Defense_CPU/"
with zipfile.ZipFile(archive) as final, zipfile.ZipFile(candidate) as tested:
    assert final.testzip() is None
    manifest = json.loads(final.read(prefix + "CPU_PACKAGE_MANIFEST.json"))
    prior = json.loads(tested.read(prefix + "CPU_PACKAGE_MANIFEST.json"))
```

يحسب بصمة أرشيف الإصدار ويقارن ملف sha256، ثم يفتح الإصدار والمرشح ويفحص CRC ويقرأ manifest لكل منهما.

الأسطر 26–35

```
    changed = []
    for relative, old in prior["files"].items():
        assert relative in manifest["files"], f"Tested file omitted: {relative}"
        if manifest["files"][relative]["sha256"] != old["sha256"]:
            changed.append(relative)
    allowed = {"docs/CPU_DEPLOYMENT_REPORT.md", "scripts/package_cpu_defense.py"}
    assert all(p in allowed or p.startswith("artifacts/deployment/") for p in changed), changed
    assert all("/.demo_venv/" not in n and "/.venv/" not in n and "/provider_cache/" not in n for n in
final.namelist())
    assert all((destination / name).resolve().is_relative_to(destination.resolve()) for name in
final.namelist())
    final.extractall(destination)
```

يتحقق من احتواء ملفات المرشح ويجمع الملفات التي تغيرت، ويسمح بتغييرات محددة أو تحت artifacts/deployment، ويفحص استبعاد بعض البيئات ومسارات الاستخراج، ثم يستخرج الأرشيف. المسار والروابط داخل ZIP تحتاج سياق نظام الملفات، ولا يدعي الفحص صحة كل استخدام محتمل.

الأسطر 36–41

```
package = destination / "Mohammad_XAI_Defense_CPU"
for relative, expected in manifest["files"].items():
    path = package / relative
    assert path.stat().st_size == expected["bytes"], relative
    with path.open("rb") as stream:
        assert hashlib.file_digest(stream, "sha256").hexdigest() == expected["sha256"], relative
```

لكل ملف بالفهرس يفحص الحجم وبصمة SHA-256 بعد الاستخراج؛ يفترض سلامة الفهرس ووجهته ويتطلب قراءة الملفات.

الأسطر 42–44

```
receipt = {"status": "PASSED", "archive": str(archive), "bytes": archive.stat().st_size, "sha256": archive_hash,
"manifest_files": len(manifest["files"]), "crc": "PASSED", "all_extracted_member_sizes_and_sha256": "PASSED",
"tested_candidate_runtime_wheels_and_research_evidence": "BYTE_IDENTICAL", "changed_tested_members": changed,
"fresh_extraction": str(package), "bundled_virtual_environments": False, "original_evidence_files":
manifest["original_evidence_files_included_unchanged"], "dependency_wheels": manifest["dependency_wheels"]}
(root / "artifacts/deployment/final_release.json").write_text(json.dumps(receipt, indent=2), encoding="utf-8")
print(json.dumps(receipt, indent=2), flush=True)
```

يبني إيصال نجاح ويكتبه في artifacts/deployment/final_release.json ويطبعه؛ التنفيذ يكتب ويستخرج ملفات ولم يحدث هنا.

شرح dashboard/app.py

بصمة المصدر SHA-256:

.e0bc35adc2b56a65578fbd8c18c38f4168f8c6df24a48e8c68001637575286a7

واجهة Streamlit لقراءة الآثار المحفوظة. تنشئ دوال العرض عناصر واجهة وتقرأ CSV وJSON وNPZ وصوراً وتقارير محلية؛ لا تُدرّب نموذجاً أو تُجري استدلالاً حياً. يربط main الدوال الثماني بتبويباتها. أخطاء الملفات المطلوبة غير ملتقطة عموماً، باستثناء فحوص الوجود الموضحة أدناه.

كل مقطع مقتبس آلياً من الملف المحدد؛ الأسطر الفارغة للفصل والتنسيق. التفسير من مراجعة المصدر ولا يمثل اختبار تشغيل للواجهة.

السطر 1

```
"""Saved-artifact research dashboard; viewing never downloads data or refits models."""
```

توثق السلسلة أن الواجهة تعرض الآثار المحفوظة ولا تجري تنزيل البيانات أو إعادة ملاءمة النماذج عند المشاهدة.

الأسطر 3–4

```
from pathlib import Path
import json
```

Path وjson من المكتبة القياسية: الأول للمسارات والثاني لفك JSON.

الأسطر 6–10

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st
```

مكتبات خارجية: NumPy للمصفوفات وملف NPZ؛ pandas لجداول CSV؛ plotly.express وgraph_objects للرسوم؛ Streamlit لبناء الواجهة وتخزين البيانات مؤقتاً.

الأسطر 12–13

```
ROOT = Path(__file__).resolve().parents[1]
```

```
st.set_page_config(page_title="Saleh | Financial XAI research", page_icon="◈", layout="wide")
```

يحسب ROOT من أصل مجلد dashboard ويهيئ عنوان الصفحة والرمز والتخطيط العريض؛ لهذا استدعاء واجهة عند تحميل الملف.

الأسطر 14–19

```
st.markdown("""<style>
 .block-container {max-width:1480px;padding-top:2rem}
 h1 {font-family:Georgia,serif;font-weight:500;letter-spacing:-.035em}
 [data-testid='stMetric'] {border-top:3px solid #22577a;padding-top:.7rem}
 .research-strip {border-left:5px solid #b15b24;padding:.8rem 1rem;background:#e9eff5;margin-bottom:1rem}
 </style>""", unsafe_allow_html=True)
```

يمرر CSS ثابتاً إلى st.markdown مع السماح بـ HTML لضبط العرض والخط والبطاقات والشريط؛ ليست هذه الحسابات نموذجاً أو بيانات بحثية.

الأسطر 22–23

```
def read_json(path: Path):
    return json.loads(path.read_text(encoding="utf-8"))
```

read_json يستقبل Path، يقرأ النص UTF-8 ويفك JSON ويعيد كائناً بحسب محتواه. أخطاء غياب الملف أو فساد JSON غير ملتقطة هنا. يستعمله معظم دوال العرض.

الأسطر 26–28

```
@st.cache_data
def load_table(path: str, modification_time: float) -> pd.DataFrame:
    return pd.read_csv(path)
```

يخزن st.cache_data decorator ناتج load_table؛ تقرأ الدالة CSV وتعيد DataFrame. مع أن modification_time لا يستخدم داخل الجسم فهو جزء من مدخلات التخزين المؤقت لتغيير المفتاح عند تغير وقت الملف.

الأسطر 31–32

```
def csv_table(path: Path) -> pd.DataFrame:
    return load_table(str(path), path.stat().st_mtime)
```

csv_table تستقبل Path وتمرير اسمه ووقت تعديله إلى load_table؛ stat قد يرفع استثناء إذا لم يوجد الملف. يستخدمها العرض والتقييم المحفوظ.

الأسطر 35–39

```
def saved_image(path: Path, caption: str) -> None:
    if path.exists():
        st.image(str(path), caption=caption, use_container_width=True)
    else:
        st.info("This artifact has not been generated. Run the explain/report command shown in
Reproducibility.")
```

saved_image تعرض صورة محلية مع تعليق إذا وُجدت وإلا تعرض رسالة معلومات. المدخل مسار وتعليق؛ أثرها عرض Streamlit وليس إنشاء صورة، ولا تعالج ملف صورة فاسداً.

الأسطر 42–47

```
def overview(metadata: dict, audit: dict) -> None:
    st.subheader("A forecast with an evidence trail")
    st.write("Predict the next observed close from information through the preceding close. Compare four models
with persistence, then inspect SHAP and LIME explanations of the same saved predictions.")
    st.markdown("**Observed history → trailing features → chronological 80/10/10 → fitted models → untouched
test → explanations**")
    st.write("Researcher: Mohammad Saleh · Damghan University · Reconstruction for defense")
    st.caption("New computations and historical thesis claims are stored separately. Explanations describe model
associations; they do not establish causality, trading profitability or improved human trust.")
```

overview تقبل بيانات التشغيل والتدقيق وتعرض الغرض والتسلسل والباحث والتحذير من استنتاج السببية أو الربح أو الثقة البشرية؛ هي عرض نصي وليس حساباً.

الأسطر 48–51

```
    flow = ROOT / "artifacts/runs" / metadata["run_id"] / "plots/research_flow.png"
    if flow.exists():
        saved_image(flow, "Implemented chronological forecasting and explanation protocol.")
    st.json({"run_id": metadata["run_id"], "status": metadata["status"], "target": audit["target"], "units":
audit["snapshot"]["units"]}, expanded=True)
```

يبني مسار صورة تدفق البحث باستخدام run_id ويعرضها إن وجدت، ثم يعرض معرّف التشغيل وحالته والهدف والوحدات من القواميس. مفاتيح ناقصة قد ترفع KeyError.

الأسطر 54–61

```
def preprocessing(folder: Path, audit: dict) -> None:
    st.subheader("What the model could know")
    snapshot = ROOT / "data/raw" / f"{folder.name}.csv"
    if snapshot.exists():
        history = csv_table(snapshot)
        price = "adj_close" if audit["snapshot"]["frequency"] == "daily" else "close"
        st.plotly_chart(px.line(history, x="date", y=price, title="Historical dataset snapshot"),
use_container_width=True)
        st.caption("Saved provider snapshot in original units; no live data request.")
```

preprocessing تقبل مجلد الأصل وبيانات تدقيقه. تبني مسار CSV محلي باسم المجلد، وإذا وُجد تقرأه وتختار adj_close لليومي وclose لغيره ثم ترسم التاريخ؛ لا تقدم طلباً حياً.

الأسطر 62–67

```
    st.dataframe(pd.DataFrame(audit["split"]).T, use_container_width=True)
    saved_image(folder / "plots/temporal_split.png", "Target dates remain chronological; validation and test
transformations are frozen.")
    st.write("Every model receives the same complete historical window. Tabular models flatten it; LSTM
preserves its time axis.")
    st.write("Features:", ", ".join(audit["features"]))
    st.json(audit["snapshot"], expanded=False)
    st.caption(f"Lookback {audit['lookback']}; feature warm-up rows removed:
{audit['feature_rows_dropped_warmup']}. Missing records are dropped without interpolation or target filling.")
```

تعرض تقسيم التواريخ بصورة جدول، وصورة التقسيم إن وُجدت، ووصف شكل مدخلات النماذج وأسماء السمات ومصدر اللقطة وطول النافذة وعدد الصفوف المحذوفة. هذه عبارات واجهة؛ تدقيق صحتها يستلزم شيفرة التحضير.

الأسطر 70–73

```
def forecasting(predictions: pd.DataFrame, folder: Path) -> None:
    st.info("Saved prediction replay: these forecasts were produced by the recorded experiment. Viewing this
chart does not run live inference.")
    model = st.selectbox("Forecast model", sorted(predictions.model.unique()), key="forecast_model")
    selected = predictions[predictions.model == model]
```

forecasting تستقبل DataFrame التنبؤات ومجلد الأصل. تحذر من إعادة العرض وتتيح اختيار نموذج من القيم الموجودة ثم ترشح الصفوف؛ لا تستدعي predict.

الأسطر 74–79

```
    figure = go.Figure()
    figure.add_scatter(x=selected.target_date, y=selected.actual, name="Actual", line={"color": "#22374a"})
    figure.add_scatter(x=selected.target_date, y=selected.predicted, name=model, line={"color": "#b15b24"})
    figure.update_layout(height=420, xaxis_title="Held-out target date", yaxis_title="Original target units")
    st.plotly_chart(figure, use_container_width=True)
    st.dataframe(selected[["feature_cutoff", "target_date", "actual", "predicted", "current_close"]],
use_container_width=True)
```

تنشئ Figure وخطي القيم الفعلية والمتوقعة وفق target_date وتضبط المحاور ثم تعرض الرسم وجدول تواريخ المدخل والهدف والقيم؛ الوحدات أصلية بحسب الأثر المحفوظ.

الأسطر 80–82

```
    with st.expander("Next observation after the saved snapshot"):
        st.warning("Unverified extrapolation after the snapshot cutoff. This is not a current live-market
forecast.")
        st.json(read_json(folder / "next_forecast.json"))
```

داخل expander تعرض تحذيراً ثم تقرأ next_forecast.json: توقع محفوظ غير متحقق يتجاوز نهاية اللقطة، وليس توقعاً للسوق الحالي. لا يوجد فحص وجود للملف هنا.

الأسطر 85–88

```
def comparison(folder: Path) -> None:
    metrics = pd.DataFrame(read_json(folder / "metrics.json"))
    training = read_json(folder / "training.json")
    metrics["training_seconds"] = metrics.model.map({name: audit["training_seconds"] for name, audit in
training.items()}).fillna(0)
```

comparison تقرأ metrics.json وtraining.json؛ تربط اسم النموذج بزمن تدريبه وتمثل القيمة غير الموجودة بصفر. هذا الصفر بديل للعرض ولا يثبت زمن تدريب مقاساً للمرجع.

الأسطر 89–92

```
    st.dataframe(metrics[["model", "mae", "rmse", "mape_percent", "r2", "direction_accuracy", "n",
"training_seconds"]], use_container_width=True, hide_index=True)
    metric = st.selectbox("Comparison measure", ["mae", "rmse", "r2", "direction_accuracy"],
key="comparison_metric")
    st.plotly_chart(px.bar(metrics, x="model", y=metric, color="model"), use_container_width=True)
    st.caption("Price-level R2 can be high for persistence. Model rankings here describe this dataset and split;
they do not establish investment value.")
```

تعرض المقاييس وعدد العينات والزمن وتتيح اختيار مقياس لرسم أعمدة؛ تنبه إلى أن R^2 المرتفع لا يثبت قيمة استثمارية وأن الترتيب مرتبط بالبيانات والتقسيم.

الأسطر 95–100

```
def explanations(folder: Path) -> None:
    model = st.selectbox("Explain model", ["LinearRegression", "RandomForest", "GradientBoosting", "LSTM"],
key="explain_model")
    explanation_folder = folder / model / "explanations"
    if not (explanation_folder / "quality.json").exists():
        st.info("No saved explanations yet. Run python -m xai_finance.cli explain --config
configs/full_experiment.yaml")
        return
```

explanations تتيح اختيار أحد النماذج الأربعة وتحدد مجلد تفسيره. إذا غاب quality.json تعرض إرشاداً وتعود مبكراً، فلا تحاول قراءة بقية الآثار.

الأسطر 101–108

```
    quality = read_json(explanation_folder / "quality.json")
    date = st.selectbox("Held-out target date (precomputed subset)", quality["target_dates"],
key="explanation_date")
    position = quality["target_dates"].index(date)
    with np.load(explanation_folder / "shap.npz", allow_pickle=False) as saved:
        signed = saved["values"][position]
        names = saved["feature_names"].astype(str)
```

```
        baseline = float(np.asarray(saved["base_values"]).ravel()[position])
        index = int(saved["indices"][position])
```

تقرأ جودة التفسير وتختار تاريخ هدف من المجموعة المحسوبة مسبقاً وتجد موضعه. تفتح shap.npz مع allow_pickle=False ثم تستخرج الإسهامات وأسماء السمات والقيمة الأساسية وفهرس العينة؛ تفترض توافق ترتيب المصفوفات مع التواريخ.

الأسطر 109–112

```
    order = np.argsort(np.abs(signed))[-12:]
    st.caption(f"Feature cutoff: {quality['feature_cutoffs'][position]} · SHAP baseline: {baseline:.5g} ·
forecast: {baseline + signed.sum():.5g} (original units)")
    local = pd.DataFrame({"feature": names[order], "SHAP contribution": signed[order]})
    st.plotly_chart(px.bar(local, x="SHAP contribution", y="feature", orientation="h", color="SHAP
contribution", color_continuous_scale="RdBu"), use_container_width=True)
```

ترتب الإسهامات بالقيمة المطلقة وتختار آخر 12 فهرساً، وتعرض تاريخ نهاية المعلومات والقيمة الأساسية ومجموعها مع جميع الإسهامات، ثم ترسم السمات الاثنتي عشرة. المعروض في الرسم مختزل لكن الجمع يستخدم signed كاملاً.

الأسطر 113–116

```
    lime = read_json(explanation_folder / f"lime_{index}.json")
    st.write("LIME local surrogate for the same observation")
    st.dataframe(pd.DataFrame(lime["features"], columns=["Feature condition", "Local weight"]), hide_index=True,
use_container_width=True)
    st.caption(f"Model {lime['model_prediction']:.5g}; local approximation
{lime['local_surrogate_prediction']:.5g}; absolute error {lime['absolute_local_error']:.5g}; neighborhood R2
{lime['weighted_neighborhood_r2']:.4g}.")
```

تقرأ lime_<index>.json للعينة نفسها، وتعرض شروط السمات وأوزان النموذج البديل وتنبؤ النموذج والبديل والخطأ المطلق وR^2 الجوار المحفوظ. هي لا تولد اضطرابات جديدة.

الأسطر 117–124

```
    first, second = st.columns(2)
    with first:
        saved_image(explanation_folder / "global_importance.png", "Mean sum of absolute lag contributions on the
declared subset.")
    with second:
        saved_image(explanation_folder / "feature_time_heatmap.png", "First declared observation only; signed
contributions by feature and window position.")
    with st.expander("Fidelity, seed stability, overhead and limitations"):
        st.json(quality)
    st.warning("Correlated-feature masking and local sampling may create implausible combinations. These are
model explanations, not causes or investment advice.")
```

تنشئ عمودين لصور الأهمية العالمية وخريطة الزمن المحفوظتين وتعرض quality في expander ثم تحذر من التركيبات غير الواقعية والخلط بين التفسير والسببية. saved_image تعالج غياب الصورتين فقط.

الأسطر 127–131

```
def regimes(folder: Path) -> None:
    st.write("A priori regime: trailing 20 observations return >+5% bull, <-5% bear, otherwise sideways. Monthly
means 20 months. Only information through the feature cutoff is used.")
    table = pd.DataFrame(read_json(folder / "regimes.json"))
    st.dataframe(table[["model", "regime", "n", "mae", "rmse", "direction_accuracy"]], hide_index=True,
use_container_width=True)
    saved_image(folder / "plots/regime_errors.png", "Small regime groups are descriptive; no unobserved regimes
or winners are invented.")
```

regimes تعرض تعريف الحالات النصي ثم تقرأ regimes.json وتعرض عدد وأخطاء كل مجموعة وصورة محفوظة. النص يصف عتبة 5% ثابتة في الواجهة؛ يجب مقارنة إعداد التشغيل الفعلي قبل تعميمه.

الأسطر 132–134

```
    path = folder / "LSTM/explanations/regime_drivers.csv"
    if path.exists():
        st.dataframe(csv_table(path), use_container_width=True)
```

إذا وُجد regime_drivers.csv تحت تفسير LSTM، تقرأه وتعرضه؛ عدم وجوده لا يوقف الواجهة.

الأسطر 137–140

```
def reproducibility(metadata: dict, folder: Path) -> None:
    st.write("This dashboard reads local artifacts and never trains, downloads or recalculates model
selection.")
    st.code("python -m xai_finance.cli doctor\npython -m xai_finance.cli validate --config
configs/full_experiment.yaml\npython -m pytest -q\npython -m xai_finance.cli explain --config
configs/full_experiment.yaml\npython -m xai_finance.cli report --config configs/full_experiment.yaml")
    st.json(metadata, expanded=False)
```

reproducibility تعرض وصف إعادة التشغيل وأوامراً كنص عبر st.code وتعرض metadata؛ عرض النص لا ينفذ الأوامر أو يعيد التدريب.

الأسطر 141–147

```
    if (folder / "RESULTS.md").exists():
        st.download_button("Download computed research report", (folder / "RESULTS.md").read_bytes(),
"RESULTS.md")
    if (folder / "inventory.json").exists():
        st.download_button("Download artifact hash inventory", (folder / "inventory.json").read_bytes(),
"inventory.json")
    tests = ROOT / "artifacts/tests/summary.json"
    if tests.exists():
        st.json(read_json(tests))
```

تضيف أزرار تنزيل للتقرير وفهرس الهاش إذا وجدا وتعرض ملخص الاختبارات إن وُجد. read_bytes يقرأ الملفات ولا يكتبها؛ التنزيل استجابة لتفاعل المستخدم.

الأسطر 148–155

```
    validations = sorted((ROOT / "artifacts/scientific_validation").glob("*/protocol.json"))
    if validations:
        with st.expander("Saved scientific validation results"):
            selected = st.selectbox("Scientific validation", validations, format_func=lambda path:
path.parent.name, key="scientific_validation")
            validation = selected.parent
            protocol = read_json(selected)
            st.write(protocol["status"])
            st.caption(f"Original experiment: {protocol['original_run']}. Results below retain the recorded
model rankings and limitations.")
```

تبحث عن protocol.json في مجلدات التحقق العلمي وترتبها؛ إن وجدت تنشئ expander واختياراً بصيغة اسم المجلد ثم تقرأ البروتوكول وتعرض حالته ومعرّف التجربة الأصلية.

الأسطر 156–159

```
            st.dataframe(csv_table(validation / "new_metrics.csv"), use_container_width=True, hide_index=True)
            st.download_button("Download scientific validation report", (validation /
"SCIENTIFIC_VALIDATION_REPORT.md").read_bytes(), "SCIENTIFIC_VALIDATION_REPORT.md")
            st.download_button("Download standalone offline validation page", (validation /
"offline_validation.html").read_bytes(), "offline_validation.html", mime="text/html")
    st.caption("Trusted locally generated joblib/checkpoint files only. Viewing this dashboard does not
deserialize model files.")
```

تعرض new_metrics.csv وأزرار تنزيل تقرير وصفحة تحقق محفوظين، ثم توضح أن الواجهة لا تفك تسلسل ملفات النماذج. الملفات الثلاثة داخل هذا الفرع مطلوبة دون فحص وجود منفصل.

الأسطر 162–168

```
def thesis_alignment(folder: Path) -> None:
    historical = read_json(ROOT / "docs/thesis_reported_reference.json")
    first, second = st.columns(2)
    with first:
        st.write("THESIS_REPORTED_NOT_REPRODUCED")
        st.dataframe(pd.DataFrame(historical["models"]), hide_index=True)
        st.caption("Table 4, Chapter 4 section 4.2, P1026-P1049; TOC p101. Historical dataset and units
unspecified.")
```

thesis_alignment تقرأ المرجع التاريخي، تنشئ عمودين، وتعرض في الأول نماذج الرسالة مع تنبيه عدم إعادة الإنتاج ونقص الوحدات والبيانات.

الأسطر 169–176

```
    with second:
        st.write("NEWLY_COMPUTED")
        st.dataframe(pd.DataFrame(read_json(folder / "metrics.json"))[["model", "mae", "rmse", "r2"]],
hide_index=True)
    st.warning("The tables have different/unspecified historical units and datasets; numerical differences are
not replication errors or evidence of improvement.")
    st.write("Historical survey percentages 78%, 85%, 70% are verified in the thesis text, but participant
counts and raw responses are unavailable. No new survey was conducted.")
    st.write("The appendix screenshots train Decision Tree/XGBoost on missing ether.csv, use an 80/20 split and
```

```
full-data mean imputation, and show no next-day target shift. This reconstruction follows the narrative's four
models and corrects temporal leakage risks.")
    with st.expander("Verified source traceability"):
        st.markdown((ROOT / "docs/THESIS_TRACEABILITY.md").read_text(encoding="utf-8"))
```

في العمود الثاني تعرض مقاييس التجربة الحالية، وتحذر من المقارنة العددية غير المتجانسة، وتعرض وصف الاستبيان والملحق وتتبع المصدر من ملف Markdown. هذه ادعاءات من نص الواجهة تحتاج مطابقتها بالمصدر العلمي مستقلاً.

الأسطر 179–183

```
def main() -> None:
    st.title("Financial forecasts, explained")
    st.caption("MOHAMMAD SALEH · MASTER'S THESIS RECONSTRUCTION · SAVED RESEARCH EVIDENCE")
    runs = sorted((ROOT / "artifacts/runs").glob("*/run.json"), reverse=True)
    available = [path.parent for path in runs if read_json(path).get("assets_completed") and read_json(path)
["status"] != "SUPERSEDED_DIAGNOSTIC_NOT_FINAL"]
```

main نقطة تجميع الواجهة؛ تعرض العنوان ثم تجمع run.json تحت artifacts/runs وتحتفظ بالتشغيلات ذات assets_completed غير الموسومة superseded. قد ترفع قراءة JSON استثناء عند ملف تالف؛ تعاد قراءته مرتين لبعض المرشحين.

الأسطر 184–187

```
    preferred = ROOT / "artifacts/latest_full.json"
    if preferred.exists():
        run_id = read_json(preferred)["run_id"]
        available.sort(key=lambda path: (path.name == run_id, path.name), reverse=True)
```

إذا وجد latest_full.json يستخرج run_id ويرتب التشغيل المفضل أولاً ثم الاسم تنازلياً. وجود المؤشر لا يضمن أن التشغيل المشار إليه من ضمن المتاح.

الأسطر 188–191

```
    if not available:
        st.info("No saved experiment is ready. Prepare it with the commands below.")
        st.code("python -m xai_finance.cli download --config configs/quick_demo.yaml\npython -m xai_finance.cli
train --config configs/quick_demo.yaml\npython -m xai_finance.cli explain --config
configs/quick_demo.yaml\npython -m xai_finance.cli report --config configs/quick_demo.yaml")
        return
```

إذا لم توجد تشغيلات متاحة يعرض رسالة وأوامراً نصية لإعداد تجربة ثم يرجع مبكراً؛ الأوامر لا تنفذ تلقائياً.

الأسطر 192–197

```
    run_folder = st.sidebar.selectbox("Saved experiment", available, format_func=lambda path: path.name,
key="run")
    metadata = read_json(run_folder / "run.json")
    asset = st.sidebar.selectbox("Dataset", metadata["assets_completed"], key="asset")
```

```
    folder = run_folder / asset
    audit = read_json(folder / "data_audit.json")
    st.markdown(f"<div class='research-strip'><b>{asset}</b> · {audit['snapshot']['units']} · TEST
{audit['split']['test']['target_start']} – {audit['split']['test']['target_end']}</div>",
unsafe_allow_html=True)
```

يختار المستخدم التشغيل والأصل من الشريط الجانبي؛ تقرأ metadata وdata_audit وتحدد المجلد وتعرض الأصل ووحدة القياس وحدود أهداف الاختبار. يستخدم HTML في شريط نصي مركب من بيانات محفوظة.

الأسطر 198–200

```
    if audit["snapshot"]["frequency"] == "monthly":
        st.warning("Separate monthly retrospective index experiment. Revised data and unknown release vintages
prevent a publication-time forecasting claim.")
    predictions = csv_table(folder / "predictions.csv")
```

إذا كان التردد شهرياً يعرض تحذير البيانات المنقحة وأزمنة النشر؛ يقرأ predictions.csv المحفوظ في جميع الأحوال قبل بناء التبويبات.

الأسطر 201–205

```
    tabs = st.tabs(["Overview", "Data & preprocessing", "Forecasting", "Compare models", "Explain predictions",
"Regimes", "Reproducibility", "Thesis alignment"])
    renderers = [lambda: overview(metadata, audit), lambda: preprocessing(folder, audit), lambda:
forecasting(predictions, folder), lambda: comparison(folder), lambda: explanations(folder), lambda:
regimes(folder), lambda: reproducibility(metadata, run_folder), lambda: thesis_alignment(folder)]
    for tab, render in zip(tabs, renderers):
        with tab:
            render()
```

ينشئ ثمانية تبويبات وقائمة دوال lambda تؤخر نداء كل عارض إلى الحلقة، ثم يربط كل تبويب بعارضه عبر zip ويدخله بـ with وينفذه. لا تتضمن الشيفرة شرطاً يمنع تنفيذ عارض التبويب غير المحدد؛ تتطلب ملفات جميع العارضات ذاتها عند إعادة تشغيل الصفحة.

الأسطر 208–209

```
if __name__ == "__main__":
    main()
```

عند التنفيذ باسم **main** يستدعي main. يستعمل main.py قيمة run_name هذه، وكذلك تشغيل app.py مباشرة؛ الاستيراد باسم آخر لا يستدعي main وإن كانت تهيئة المستوى الأعلى تنفذ.

شرح `main.py`

بصمة المصدر SHA-256:

`bd298e28fe24d362920cf792a09e9efc052c72d110f2e9b5dd17c85bd6abd061`.

نقطة الدخول إلى العرض المحلي. لا تتلقى معاملات دوال ولا تقرأ بيانات بحثية مباشرة؛ تمرر التنفيذ إلى dashboard/app.py. الاستدعاء المتوقع من Streamlit موثق في رأس الملف.

كل مقطع مقتبس آلياً من الملف المحدد؛ الأسطر الفارغة للفصل والتنسيق. التفسير من مراجعة المصدر ولا يمثل اختبار تشغيل للواجهة.

الأسطر 1–4

```
"""Stable Streamlit entry point for the original saved-evidence dashboard.

Run with ``python -m streamlit run main.py`` or START_DEMO.bat.
"""
```

سلسلة التوثيق تعرّف هذا الملف نقطة دخول ثابتة وتذكر أمر Streamlit وملف التشغيل؛ لا تنفذ الأمر بنفسها.

الأسطر 6–7

```
from pathlib import Path
import runpy
```

يستورد Path من المكتبة القياسية لبناء مسار مستقل عن مجلد العمل، وrunpy لتشغيل ملف Python آخر.

الأسطر 10–11

```
if __name__ == "__main__":
    runpy.run_path(str(Path(__file__).resolve().parent / "dashboard" / "app.py"), run_name="__main__")
```

يفحص شرط **main** ما إذا كان الملف منفذاً كنقطة دخول. إذا تحقق، يبني المسار المطلق من مكان الملف إلى dashboard/app.py ويشغله عبر run_path مع run_name="**main**"، فيُفعّل شرط الدخول في الملف الهدف. لا يعيد قيمة؛ أخطاء قراءة الملف أو تنفيذه تنتشر إلى المستدعي.

شرح
ipts/audit_reconstruction_precision.py

بصمة المصدر SHA-256:

.de19a61fb8a087653e0a96164a00d338734e7a9f5ebededcce00b58ca97dde810

لم يُشغّل السكربت في جلسة التوثيق.

الأسطر 1–10

```
"""Retain as-executed forecasts and save a separate FP64 reconstruction sensitivity."""
import argparse
from pathlib import Path

import numpy as np
import pandas as pd

from xai_finance.storage import sha256, write_json
from xai_finance.validation.design import implied_prices
from xai_finance.validation.statistics import evaluation_metrics
```

توثيق ومكتبات قياسية وخارجية لتدقيق حساسية FP64، ووظائف التحويل والمقاييس والبصمة الداخلية.

الأسطر 13–18

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("folder", type=Path)
    destination = parser.parse_args().folder.resolve()
    predictions = pd.read_csv(destination / "new_predictions.csv", float_precision="round_trip")
    comparisons, corrected_tables, metric_rows = [], [], []
```

main تقرأ مجلد التحقق ثم تنبؤاته المحفوظة بدقة round_trip وتبدأ قوائم المقارنة والجداول.

الأسطر 19–24

```
    for keys, group in predictions.groupby(["design", "target", "asset", "model"], sort=False):
        group = group.sort_values("target_date").copy()
        corrected = group.copy()
        corrected["as_executed_predicted_price"] = group.predicted_price
        corrected["predicted_price"] = implied_prices(group.predicted_target.to_numpy(dtype=np.float64),
group.current_price.to_numpy(dtype=np.float64), keys[1])
        corrected_tables.append(corrected)
```

لكل تصميم وهدف وأصل ونموذج ترتب التواريخ وتنسخ الجدول وتحفظ التنبؤ الأصلي وتعيد تركيب السعر بعد تحويل الهدف والسعر الحالي إلى float64. هذه حساسية حسابية منفصلة لا تعيد تدريباً.

الأسطر 25–28

```
        before, after = evaluation_metrics(group), evaluation_metrics(corrected)
        identity = dict(zip(["design", "target", "asset", "model"], keys))
        metric_rows.append({**identity, **after})
        comparisons.append({**identity, "max_price_difference": float(np.max(np.abs(corrected.predicted_price -
group.predicted_price))), "as_executed_rmse": before["rmse"], "float64_rmse": after["rmse"], "rmse_difference":
after["rmse"] - before["rmse"], "direction_predictions_changed": int(np.sum(np.sign(group.predicted_price -
group.current_price) != np.sign(corrected.predicted_price - corrected.current_price)))})
```

تحسب مقاييس قبل وبعد وتجمع الهوية وأعظم فرق سعر وفرق RMSE وعدد تغيرات إشارة التوقع.
الحساب يقرأ التنبؤ المحفوظ ولا يثبت جودة النموذج.

الأسطر 29–33

```
    pd.DataFrame(comparisons).to_csv(destination / "reconstruction_precision_comparison.csv", index=False)
    pd.DataFrame(metric_rows).to_csv(destination / "float64_reconstructed_metrics.csv", index=False)
    pd.concat(corrected_tables, ignore_index=True).to_csv(destination / "float64_reconstructed_predictions.csv",
index=False)
    write_json(destination / "reconstruction_precision_provenance.json", {"original_predictions_sha256":
sha256(destination / "new_predictions.csv"), "target_values_unchanged": True, "models_retrained": False,
"algorithm": "Cast saved target prediction to float64 before 1+r and price reconstruction; as-executed files
untouched", "as_executed_lstm_target_dtype": "float32", "reason": "Native float32 1+r rounds before
multiplication by float64 current price; separate numerical sensitivity, not a replacement experiment"})
    print(pd.DataFrame(comparisons).query("max_price_difference > 1e-8").to_string(index=False))
```

تكتب ملفات مقارنة وتنبؤات ومقاييس FP64 منفصلة مع بصمة الأصل والمنشأ وتطبع الفروق؛ التنفيذ يعدل آثار التحقق ولم يُشغّل هنا.

الأسطر 36–37

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ يستدعي main.

شرح scripts/bootstrap_cpu.py

بصمة المصدر SHA-256:

.a568f19f82e8f9e133d83b6bbf27b52b1982f98147d4d1b78436ce6e3d83fb9a

وصف ثابت؛ لم تُشغّل السكربتات.

الأسطر 1–6

```
"""Find an already-installed x64 Python 3.12; never download a runtime implicitly."""

import os
from pathlib import Path
import subprocess
import sys
```

توثيق البحث المحلي عن Python 3.12 x64، واستيرادات البيئة والمسارات والعمليات القياسية.

الأسطر 8–11

```
ROOT = Path(__file__).resolve().parents[1]
check = "import sys,struct; assert sys.version_info[:2]==(3,12) and struct.calcsize('P')==8"
candidates = [Path(os.environ["XAI_PYTHON"])] if os.environ.get("XAI_PYTHON") else []
candidates.append(Path(sys.executable))
```

يحدد الجذر وسلسلة تحقق لإصدار 3.12 وحجم مؤشر 8 بايت، ويبدأ قائمة المرشحين من XAI_PYTHON إن وجد ثم Python الحالي.

الأسطر 12–17

```
for variable in ["APPDATA", "LOCALAPPDATA"]:
    if os.environ.get(variable):
        folder = Path(os.environ[variable]) / "uv/python"
        candidates.extend(sorted(folder.glob("cpython-3.12*-windows-x86_64-none/python.exe"), reverse=True))
if os.environ.get("UV_PYTHON_INSTALL_DIR"):
    candidates.extend(sorted(Path(os.environ["UV_PYTHON_INSTALL_DIR"]).glob("cpython-3.12*-windows-x86_64-
none/python.exe"), reverse=True))
```

يبحث في مجلدات uv تحت APPDATA وLOCALAPPDATA وفي UV_PYTHON_INSTALL_DIR عند وجودها عن Python Windows x86_64، ويرتب النتائج تنازلياً.

الأسطر 18–23

```
for python in dict.fromkeys(candidates):
    if python.is_file() and subprocess.run([str(python), "-c", check], stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL).returncode == 0:
```

```
        raise SystemExit(subprocess.call([str(python), str(ROOT / "scripts/demo_wizard.py"), "setup",
*sys.argv[1:]], cwd=ROOT))
print("Windows x64 Python 3.12 was not found. Install it for your user, then rerun SETUP_DEMO.bat.")
print("If it is installed in a custom location, set XAI_PYTHON to its full python.exe path.")
raise SystemExit(1)
```

يزيل تكرار المرشحين بترتيب الإدراج ويشغل تحقق الإصدار على كل ملف؛ عند النجاح يشغل wizard setup ويعيد رمزه. إن لم يوجد يطبع إرشادات ويخرج 1؛ لا ينزل Python تلقائياً.

شرح scripts/browser_smoke_cpu.py

بصمة المصدر SHA-256:

.feb83a0e6fd44b841d381d7b8d6142612cf60dc6db4345819fadf560ab71bdba

لم يُنفذ اختبار النشر أو المتصفح في هذه الجلسة.

الأسطر 1–16

```
"""Real Edge browser acceptance via CDP; no Selenium/Playwright dependency.

Browser external HTTP(S) is sent to a closed loopback proxy; local app bypasses it.
"""

import argparse
import base64
import json
from pathlib import Path
import subprocess
import time
from urllib.request import urlopen

from websockets.sync.client import connect

ROOT = Path(__file__).resolve().parents[1]
```

توثيق تجربة Edge ووكيل loopback، واستيرادات الأدوات القياسية وwebsockets الخارجية وتعريف الجذر.

الأسطر 19–23

```
class CDP:
    def __init__(self, url):
        self.ws = connect(url, max_size=32 * 1024 * 1024, proxy=None, legacy=True)
        self.number = 0
        self.errors = []
```

CDP ينشئ اتصال WebSocket إلى بروتوكول تصحيح المتصفح ويبدأ عداد الرسائل وقائمة الأخطاء.

الأسطر 25–36

```
    def call(self, method, params=None):
        self.number += 1
        number = self.number
        self.ws.send(json.dumps({"id": number, "method": method, "params": params or {}}))
        while True:
            message = json.loads(self.ws.recv(timeout=60))
            if message.get("method") == "Runtime.exceptionThrown":
                self.errors.append(message["params"])
            if message.get("id") == number:
                if "error" in message:
                    raise RuntimeError(message["error"])
                return message.get("result", {})
```

call يزيد المعرف ويرسل طلب JSON ويستقبل حتى يجد الاستجابة المطابقة، ويسجل استثناءات Runtime ويرفع خطأ البروتوكول؛ timeout الاستقبال 60 ثانية.

الأسطر 38–42

```
    def evaluate(self, expression):
        result = self.call("Runtime.evaluate", {"expression": expression, "returnByValue": True, "awaitPromise":
True})
        if "exceptionDetails" in result:
            raise RuntimeError(result["exceptionDetails"])
        return result.get("result", {}).get("value")
```

evaluate تنفذ تعبير JavaScript عبر CDP وتعيد القيمة أو ترفع تفاصيل الاستثناء.

الأسطر 45–58

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--url", default="http://127.0.0.1:8511")
    parser.add_argument("--debug-port", type=int, default=9228)
    parser.add_argument("--output", type=Path, default=ROOT / "artifacts/deployment/browser")
    args = parser.parse_args()
    args.output = args.output.resolve()
    args.output.mkdir(parents=True, exist_ok=True)
    edge = Path("C:/Program Files (x86)/Microsoft/Edge/Application/msedge.exe")
    if not edge.exists():
        raise RuntimeError("Microsoft Edge is required for this developer browser test.")
    profile = args.output / "profile"
    with (args.output / "edge.log").open("w") as log:
        process = subprocess.Popen([str(edge), "--headless=new", "--no-sandbox", "--disable-gpu", "--no-first-
run", "--disable-background-networking", "--disable-quic", "--proxy-server=http://127.0.0.1:9", "--proxy-bypass-
list=localhost;127.0.0.1;[::1]", f"--remote-debugging-port={args.debug_port}", f"--user-data-dir={profile}",
"about:blank"], creationflags=0x08000000, stdout=log, stderr=log)
```

main تقرأ رابط التطبيق ومنفذ التصحيح والوجهة، وتنشئ الوجهة وتطلب Edge في المسار الثابت، ثم تطلقه headless بملف شخصي ووكيل خارجي إلى منفذ loopback مغلق واستثناء localhost؛ لا يثبت ذلك منع كل بروتوكول شبكة.

الأسطر 59–77

```
        browser = None
        try:
            for _ in range(60):
                try:
                    with urlopen(f"http://127.0.0.1:{args.debug_port}/json/version", timeout=2) as response:
                        version = json.load(response)
                    break
                except OSError:
                    time.sleep(.5)
            else:
                raise RuntimeError("Edge did not expose its developer port; see edge.log")
            browser = CDP(version["webSocketDebuggerUrl"])
            browser.call("Target.createTarget", {"url": args.url})
            with urlopen(f"http://127.0.0.1:{args.debug_port}/json/list") as response:
                target = next(t for t in json.load(response) if t.get("url", "").startswith(args.url))
            page = CDP(target["webSocketDebuggerUrl"])
            page.call("Runtime.enable")
```

```
            page.call("Page.enable")
            page.call("Emulation.setDeviceMetricsOverride", {"width": 1440, "height": 1050, "deviceScaleFactor":
1, "mobile": False})
```

تنتظر منفذ المطور وتفتح اتصال CDP وهدف التطبيق وتحدد الصفحة وتمكن Runtime وPage وتحاكي نافذة 1440×1050.

الأسطر 78–92

```
            for _ in range(120):
                if page.evaluate("document.querySelectorAll('[role=tab]').length") == 8:
                    break
                time.sleep(.5)
            else:
                raise AssertionError(page.evaluate("document.body.innerText"))
            time.sleep(2)
            tabs = page.evaluate("Array.from(document.querySelectorAll('[role=tab]')).map(x=>x.innerText)")
            expected = ["A forecast with an evidence trail", "What the model could know", "Actual", "mae", "LIME
local surrogate", "A priori regime", "local artifacts", "THESIS_REPORTED_NOT_REPRODUCED"]
            for index, label in enumerate(tabs):
                page.evaluate(f"document.querySelectorAll('[role=tab]')[{index}].click()")
                time.sleep(.3)
                visible =
page.evaluate("Array.from(document.querySelectorAll('[role=tabpanel]')).filter(x=>!x.hidden&&x.getBoundingClient
Rect().height>0).map(x=>x.innerText).join('\\n')")
                assert expected[index] in visible, (label, visible)
                assert not page.evaluate("document.querySelector('[data-testid=stException]') !== null")
```

تنتظر ثمانية تبويبات وتضغط كلاً منها وتتحقق من نص ظاهر وعدم وجود stException؛ انتظار الزمن الثابت قد يتأثر بسرعة الجهاز.

الأسطر 93–105

```
                if index == 6:

page.evaluate("Array.from(document.querySelectorAll('summary')).find(x=>x.innerText.includes('Saved scientific
validation results')).click()")
                    time.sleep(.3)
                    assert page.evaluate("document.body.innerText.includes('EXPLORATORY: original test results
already observed')")
                    assert page.evaluate("document.body.innerText.includes('Download standalone offline
validation page')")
                shot = page.call("Page.captureScreenshot", {"format": "png"})
                (args.output / f"tab_{index+1}.png").write_bytes(base64.b64decode(shot["data"]))
                print(f"BROWSER PASS: {label}", flush=True)
            text = page.evaluate("document.body.innerText")
            assert "Financial forecasts, explained" in text
            assert not page.errors, page.errors
            receipt = {"status": "PASSED", "browser": version["Browser"], "url": args.url,
"tabs_clicked_and_content_checked": tabs, "scientific_validation_expanded": True, "javascript_exceptions":
page.errors, "external_browser_http": "closed loopback proxy; localhost bypass only", "screenshots": 8}
            (args.output / "receipt.json").write_text(json.dumps(receipt, indent=2), encoding="utf-8")
```

في تبويب إعادة الإنتاج توسع التحقق العلمي وتتحقق من النص وأزرار التنزيل، وتلتقط PNG لكل تبويب ثم تفحص العنوان واستثناءات JS وتكتب إيصالاً.

الأسطر 106–114

```
            page.ws.close()
        finally:
            if browser:
                try:
                    browser.call("Browser.close")
                except Exception:
                    pass
            if process.poll() is None:
                process.terminate()
```

تغلق قناة الصفحة وتحاول إغلاق المتصفح في finally ثم تنهي العملية إن بقيت؛ تترك صوراً وسجلات وملف profile في الوجهة.

الأسطر 117–118

```
if __name__ == "__main__":
    main()
```

حارس التشغيل. لم يشغل المتصفح هنا.

شرح scripts/build_validation_report.py

بصمة المصدر SHA-256:

.6359790f490bf1d13a23bfe3df70dff0c33b589ab3a1f483255f8d4249e98d32

الاقتباسات من الملف؛ لم تُشغّل السكربتات أو تجاربها.

الأسطر 1–7

```
import argparse
import base64
import html
from pathlib import Path

from xai_finance.storage import read_json
from xai_finance.validation.report import write_report
```

argparse وbase64 وhtml وPath قياسية، وread_json وwrite_report من المشروع.

الأسطر 10–18

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("folder", type=Path)
    destination = parser.parse_args().folder.resolve()
    write_report(destination)
    figures = []
    for path in sorted((destination / "figures").glob("*.png")):
        encoded = base64.b64encode(path.read_bytes()).decode("ascii")
        figures.append(f'<figure><figcaption>{html.escape(path.stem.replace("_", " "))}</figcaption><img alt="
{html.escape(path.stem)}" src="{encoded}"></figure>')
```

main تقرأ مجلد التحقق وتستدعي كتابة التقرير، ثم تجمع صور PNG وترمز بايتاتها base64 في figure وتقوم بتهريب العناوين. يستلزم write_report آثاراً سابقة ويولد تقارير ورسومات.

الأسطر 19–24

```
    report = html.escape((destination / "SCIENTIFIC_VALIDATION_REPORT.md").read_text(encoding="utf-8"))
    page = '<!doctype html><html lang="en"><meta charset="utf-8"><title>XAI scientific validation</title>
<style>body{max-width:1200px;margin:40px auto;padding:20px;font:16px system-
ui;background:#f7f8fa;color:#17212b}img{max-width:100%}pre{white-space:pre-wrap;font:14px monospace;overflow-
wrap:anywhere}figure{background:white;padding:20px;margin:30px 0}figcaption{font-size:22px}</style>
<h1>Scientific validation: evidence and limitations</h1><p>Offline supplement; original defense package
preserved unchanged.</p>' + ''.join(figures) + '<h2>Full report</h2><pre>' + report + '</pre></html>'
    (destination / "offline_validation.html").write_text(page, encoding="utf-8")
    assert read_json(destination / "verification.json")["preservation"]["mismatches"] == []
    print(destination / "SCIENTIFIC_VALIDATION_REPORT.md")
    print(destination / "offline_validation.html")
```

تهرب نص Markdown وتدرجه كما هو داخل pre في صفحة HTML إنجليزية ذات صور مدمجة offline، وتكتب الصفحة ثم تؤكد خلو mismatches في verification.json وتطبع المسارات. فحص المحافظة يأتي بعد الكتابة وليس قبلها؛ الصفحة لا تحقق متطلبات التقرير العربي RTL.

الأسطر 27–28

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ يستدعي main.

شرح scripts/check_dashboard_cpu.py

بصمة المصدر SHA-256:

.afa1fe0e9083bf26367b03833f8ce2b16ca09a3e5d6174d2fea7520257e9997b

لم يُنفذ اختبار النشر أو المتصفح في هذه الجلسة.

الأسطر 1–10

```
"""Exercise the original eight tabs with external Python connections denied."""

import argparse
import json
from pathlib import Path
import socket

from streamlit.testing.v1 import AppTest

ROOT = Path(__file__).resolve().parents[1]
```

توثيق واستيرادات argparse وJSON والمسار وsocket وAppTest، ثم تعيين ROOT.

الأسطر 13–22

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--output", type=Path, default=ROOT / "artifacts/deployment/dashboard.json")
    args = parser.parse_args()
    original_connect = socket.socket.connect
    original_connect_ex = socket.socket.connect_ex

    def check(address):
        if isinstance(address, tuple) and address[0] not in {"localhost", "127.0.0.1", "::1"}:
            raise RuntimeError(f"External connection denied during offline test: {address}")
```

main تحلل وجهة الإيصال وتحتفظ بدالتي الاتصال الأصليتين وتعرف check التي ترفض عناوين tuple الخارجية؛ ليست جدار حماية عاماً.

الأسطر 24–33

```
    def connect(connection, address):
        check(address)
        return original_connect(connection, address)

    def connect_ex(connection, address):
        check(address)
        return original_connect_ex(connection, address)

    socket.socket.connect = connect
    socket.socket.connect_ex = connect_ex
```

connect وconnect_ex تغلفان الدالتين بالفحص، ثم يستبدلهما السكربت داخل العملية للاختبار.

الأسطر 34–43

```
    app = AppTest.from_file(str(ROOT / "main.py"), default_timeout=90).run()
    assert not app.exception, app.exception
    assert app.title[0].value == "Financial forecasts, explained"
    assert len(app.tabs) == 8
    receipt = {"status": "PASSED", "external_python_connections": "denied", "tabs": [t.label for t in app.tabs],
"experiments": []}
    runs = list(app.selectbox(key="run").options)
    for run in runs:
        selected_run = next(p for p in app.selectbox(key="run").value.parent.iterdir() if p.name == run)
        app.selectbox(key="run").set_value(selected_run).run()
        assert not app.exception, app.exception
```

يشغل AppTest من main.py ويتحقق من عدم الاستثناء والعنوان وثمانية تبويبات، وينشئ إيصالاً ويمر على التجارب ويحدد مجلد التجربة من الاسم.

الأسطر 44–58

```
        assets = list(app.selectbox(key="asset").options)
        for asset in assets:
            app.selectbox(key="asset").set_value(asset).run()
            assert not app.exception, app.exception
            for model in ["LinearRegression", "RandomForest", "GradientBoosting", "LSTM"]:
                app.selectbox(key="forecast_model").set_value(model)
                app.selectbox(key="explain_model").set_value(model).run()
                assert not app.exception, app.exception
                dates = app.selectbox(key="explanation_date").options
                for date in {dates[0], dates[-1]}:
                    app.selectbox(key="explanation_date").set_value(date).run()
                    assert not app.exception, app.exception
                assert len(app.dataframe) >= 6
            print(f"PASS {run}/{asset}: four forecasts, paired SHAP/LIME, eight tabs", flush=True)
        receipt["experiments"].append({"run": run, "datasets": assets, "models_per_dataset": 4,
"explanation_dates": "first and last for each model"})
```

لكل أصل ونموذج يختار توقعه وتفسيره ويفحص أول وآخر تاريخ وغياب الاستثناء وعدداً أدنى من الجداول، ويسجل النتيجة. الفحص لا يقيس جودة التفسيرات أو الصور.

الأسطر 59–64

```
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(receipt, indent=2), encoding="utf-8")


if __name__ == "__main__":
    main()
```

ينشئ مجلد الإخراج ويكتب JSON؛ حارس main يشغله عند التنفيذ. لم يشغل هنا.

شرح scripts/demo_wizard.py

بصمة المصدر SHA-256:

.75054772a52bc6795f77b6dffd4413ead4ca27d092b985fb3fcdda62ffe1ba89

مراجعة ثابتة للكود؛ لم يشغل الإعداد أو الخادم أو الاختبارات.

الأسطر 1–1

```
"""Windows CPU setup/launch wizard. Uses a new venv, never the research environment."""
```

سلسلة التوثيق تصف معالج Windows CPU وبيئة مستقلة؛ لا تثبت وحدها ما يحدث داخل الدوال.

الأسطر 3–19

```
import argparse
from contextlib import contextmanager
import ctypes
from datetime import datetime, timezone
import hashlib
import json
import os
from pathlib import Path
import platform
import socket
import struct
import subprocess
import sys
import time
from urllib.request import urlopen
import venv
import webbrowser
```

استيرادات قياسية: argparse للخيارات، contextmanager للقفل، ctypes لواجهات Windows، الوقت، hashlib وos، json وPath وplatform وsocket وsubprocess، struct وsys وtime، urlopen لفحص HTTP المحلي، venv وwebbrowser.

الأسطر 21–27

```
ROOT = Path(__file__).resolve().parents[1]
RUNTIME = ROOT / ".demo_runtime"
PYTHON = ROOT / ".demo_venv/Scripts/python.exe"
STATE = RUNTIME / "server.json"
RUN_ID = "full_20261021T040901084647Z"
VALIDATION_ID = "validation_20261021T055002439194Z"
LOG = None
```

ثوابت المسارات للجذر وحالة التشغيل والبيئة، ومعرفا التشغيل والتحقق المحفوظين، وLOG المبدئي None؛ المعرفات تربط العرض بأدلة محددة وليست بحثاً تلقائياً عن أحدث تجربة.

الأسطر 30–34

```
def say(message):
    print(message, flush=True)
    if LOG:
        LOG.write(message + "\n")
        LOG.flush()
```

say تطبع الرسالة فوراً وتكتبها وتفرغها في السجل المفتوح إن وجد؛ لا تعيد قيمة. تعتمد على LOG العامة.

الأسطر 37–39

```
def environment():
    search_path = os.pathsep.join(filter(None, [str(ROOT / "src"), os.environ.get("PYTHONPATH")]))
    return {**os.environ, "PYTHONUTF8": "1", "PYTHONPATH": search_path, "MPLCONFIGDIR": str(RUNTIME /
"matplotlib"), "STREAMLIT_BROWSER_GATHER_USAGE_STATS": "false", "PYTHONHASHSEED": "42"}
```

environment تنشئ بيئة عملية ابن بنسخ البيئة الحالية وإضافة src إلى PYTHONPATH وتعريف UTF-8 ومجلد matplotlib وإلغاء إحصاء Streamlit وبذرة الهاش. لا تحذف باقي متغيرات البيئة.

الأسطر 42–48

```
def run(command):
    say("Running: " + subprocess.list2cmdline([str(x) for x in command]))
    with subprocess.Popen([str(x) for x in command], cwd=ROOT, env=environment(), stdout=subprocess.PIPE,
stderr=subprocess.STDOUT, text=True, encoding="utf-8", errors="replace") as process:
        for line in process.stdout:
            say(line.rstrip())
        if process.wait():
            raise RuntimeError("The command above failed. Check its error and the diagnostic log, then run setup
again.")
```

run تعرض أمر العملية، وتشغله في جذر المشروع بالبيئة المحددة مع جمع stdout وstderr وتدفق السجل؛ تنتظر نهايته وترفع RuntimeError عند رمز غير صفري. تنفيذ الأمر قد يكتب أو يثبت حزم.

الأسطر 51–56

```
def sha256(path):
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()
```

sha256 تقرأ ملفاً على دفعات MiB وتحدث digest حتى bytes فارغة، ثم تعيد البصمة الست عشرية؛ لا يثبت الهاش مصدر الملف وحده.

الأسطر 59–64

```
def verify_manifest(root=ROOT):
    manifest_path = root / "CPU_PACKAGE_MANIFEST.json"
    if not manifest_path.exists():
        say("Source checkout: package manifest absent; checking required research inventories instead.")
        return
    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
```

verify_manifest تقبل root افتراضياً وتبحث عن CPU_PACKAGE_MANIFEST.json؛ عند غيابه تعرض رسالة نسخة مصدر وتعود بلا تحقق لحزمة. دوال أخرى تتحقق من آثار البحث.

الأسطر 65–71

```
    for relative, expected in manifest["files"].items():
        path = (root / relative).resolve()
        if not path.is_relative_to(root.resolve()):
            raise RuntimeError(f"Unsafe manifest path: {relative}")
        if not path.is_file() or path.stat().st_size != expected["bytes"] or sha256(path) != expected["sha256"]:
            raise RuntimeError(f"Package file missing or damaged: {relative}. Extract the verified ZIP again.")
    say(f"Integrity verified: {len(manifest['files'])} packaged files.")
```

لكل ملف بالفهرس تحل المسار وتمنع خروجه من root، ثم تتحقق من الملف والحجم والبصمة وتفشل عند الاختلاف؛ تطبع العدد عند النجاح. لا تفحص غياب ملفات إضافية.

الأسطر 74–80

```
def verify_artifacts():
    run_folder = ROOT / "artifacts/runs" / RUN_ID
    validation = ROOT / "artifacts/scientific_validation" / VALIDATION_ID
    required = [ROOT / "main.py", ROOT / "dashboard/app.py", ROOT / "artifacts/latest_full.json", run_folder /
"run.json", run_folder / "inventory.json", validation / "protocol.json", validation / "new_metrics.csv",
validation / "new_predictions.csv", validation / "offline_validation.html", ROOT /
"docs/thesis_reported_reference.json", ROOT / "docs/THESIS_TRACEABILITY.md"]
    if not (run_folder / "run.json").exists():
        raise RuntimeError(f"Required saved experiment missing: {run_folder / 'run.json'}. Restore it from the
original research backup; do not retrain to replace evidence.")
    metadata = json.loads((run_folder / "run.json").read_text(encoding="utf-8"))
```

verify_artifacts تحدد تشغيل البحث والتحقق الثابتين وتبني قائمة الأدلة الواجبة؛ تتحقق أولاً من run.json وتقرأ metadata أو تطلب استعادة النسخة بدلاً من إعادة التدريب.

الأسطر 81–89

```
    for asset in metadata["assets_completed"]:
        folder = run_folder / asset
        required.extend([ROOT / "data/raw" / f"{asset}.csv", ROOT / "data/manifests" / f"{asset}.json"])
        required.extend(folder / name for name in ["windows.npz", "data_audit.json", "training.json",
"metrics.json", "predictions.csv", "next_forecast.json", "regimes.json", "LinearRegression.joblib",
"RandomForest.joblib", "GradientBoosting.joblib", "LSTM/best.pt", "LSTM/last.pt", "LSTM/scalers.joblib"])
        for model in ["LinearRegression", "RandomForest", "GradientBoosting", "LSTM"]:
            required.extend(folder / model / "explanations" / name for name in ["quality.json", "shap.npz",
"lime_local.png"])
    missing = [str(path.relative_to(ROOT)) for path in required if not path.is_file() or not
```

```
path.stat().st_size]
    if missing:
        raise RuntimeError("Required artifacts missing or empty:\n" + "\n".join(missing))
```

لكل أصل مكتمل تضيف لقطاته ونوافذه ونماذجه ونقاط LSTM والتفسيرات المطلوبة، ثم ترفض الملفات الناقصة أو الفارغة. فحص الوجود لا يثبت وحده صحة النتائج.

الأسطر 90–93

```
    for relative, expected in json.loads((run_folder / "inventory.json").read_text()).items():
        if sha256(run_folder / relative) != expected:
            raise RuntimeError(f"Saved research inventory mismatch: {run_folder / relative}")
    say(f"Required datasets, checkpoints, predictions and explanations found for
{len(metadata['assets_completed'])} datasets.")
```

تقرأ inventory.json للتشغيل وتتحقق من SHA-256 لكل مسار مدرج ثم تطبع عدد الأصول. الافتراض أن الفهرس نفسه موثوق والمسارات صحيحة.

الأسطر 96–98

```
def runtime_check():
    code = "import json,sys,torch,streamlit,shap,lime,numpy,pandas,sklearn,plotly; assert sys.version_info[:2]==
(3,12); assert torch.__version__=='2.5.1+cpu'; assert torch.version.cuda is None;
print(json.dumps({'python':sys.version,'torch':torch.__version__,'cuda_available':torch.cuda.is_available()}))"
    run([PYTHON, "-c", code])
```

runtime_check يبني برنامجاً قصيراً يفحص استيراد المكتبات وPython 3.12 وtorch 2.5.1+cpu وغياب CUDA في build، ثم يشغله باستخدام Python البيئة عبر run.

الأسطر 101–109

```
def process_stamp(pid):
    """Windows creation timestamp prevents reuse or stopping an unrelated recycled PID."""
    kernel = ctypes.WinDLL("kernel32", use_last_error=True)
    kernel.OpenProcess.argtypes = [ctypes.c_ulong, ctypes.c_int, ctypes.c_ulong]
    kernel.OpenProcess.restype = ctypes.c_void_p
    kernel.GetProcessTimes.argtypes = [ctypes.c_void_p] + [ctypes.c_void_p] * 4
    kernel.GetExitCodeProcess.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_ulong)]
    kernel.CloseHandle.argtypes = [ctypes.c_void_p]
    handle = kernel.OpenProcess(0x1000, False, pid)
```

process_stamp تقبل PID وتجهز دوال kernel32 وتواقيع ctypes ثم تفتح العملية بصلاحية الاستعلام المحدودة 0x1000؛ الهدف التمييز بين PID الأصلي وPID معاد الاستخدام.

الأسطر 110–121

```
    if not handle:
        return None
    try:
        exit_code = ctypes.c_ulong()
```

```
        if not kernel.GetExitCodeProcess(handle, ctypes.byref(exit_code)) or exit_code.value != 259:
            return None
        values = [ctypes.c_ulonglong() for _ in range(4)]
        if not kernel.GetProcessTimes(handle, *[ctypes.byref(value) for value in values]):
            return None
        return values[0].value
    finally:
        kernel.CloseHandle(handle)
```

إن تعذر فتح المقبض تعيد None؛ وإلا تتحقق من رمز STILL_ACTIVE=259 ثم تجلب أزمنة العملية وتعيد وقت الإنشاء، وتغلق المقبض في finally حتى عند العودة. لا يتعرف الفحص وحده على هوية التطبيق من الاسم.

الأسطر 124–131

```
def owned_state():
    try:
        state = json.loads(STATE.read_text())
        if state["root"] == str(ROOT.resolve()) and state.get("creation_stamp") is not None and
state["creation_stamp"] == process_stamp(state["pid"]):
            return state
    except (OSError, ValueError, KeyError):
        pass
    return None
```

owned_state تقرأ JSON للحالة وتقبلها فقط عند تطابق جذر المشروع وبصمة الإنشاء الحالية لـ PID؛ تعالج أخطاء القراءة أو JSON أو المفاتيح بإرجاع None.

الأسطر 134–140

```
def terminate_owned_tree(state):
    """Find this exact process's descendants without WMI/taskkill privileges."""
    import signal
    from ctypes import wintypes

    class Entry(ctypes.Structure):
        _fields_ = [("dwSize", wintypes.DWORD), ("cntUsage", wintypes.DWORD), ("th32ProcessID", wintypes.DWORD),
("th32DefaultHeapID", ctypes.c_size_t), ("th32ModuleID", wintypes.DWORD), ("cntThreads", wintypes.DWORD),
("th32ParentProcessID", wintypes.DWORD), ("pcPriClassBase", wintypes.LONG), ("dwFlags", wintypes.DWORD),
("szExeFile", wintypes.WCHAR * 260)]
```

terminate_owned_tree تستورد signal وwintypes وتعرف بنية Entry لحصر عمليات Windows؛ تستخدم حجم المؤشرات المناسب للحقول المحددة.

الأسطر 142–150

```
    kernel = ctypes.WinDLL("kernel32", use_last_error=True)
    kernel.CreateToolhelp32Snapshot.argtypes = [wintypes.DWORD, wintypes.DWORD]
    kernel.CreateToolhelp32Snapshot.restype = wintypes.HANDLE
    kernel.Process32FirstW.argtypes = [wintypes.HANDLE, ctypes.POINTER(Entry)]
    kernel.Process32NextW.argtypes = [wintypes.HANDLE, ctypes.POINTER(Entry)]
    kernel.CloseHandle.argtypes = [wintypes.HANDLE]
    snapshot = kernel.CreateToolhelp32Snapshot(2, 0)
    if snapshot == ctypes.c_void_p(-1).value:
        raise ctypes.WinError(ctypes.get_last_error())
```

تحدد تواقيع Toolhelp API وتنشئ لقطة عمليات، وترفع WinError إن أعادت قيمة فشل المقبض؛ هذا مسار خاص بـ Windows.

الأسطر 151–160

```
    entries = []
    try:
        entry = Entry()
        entry.dwSize = ctypes.sizeof(entry)
        valid = kernel.Process32FirstW(snapshot, ctypes.byref(entry))
        while valid:
            entries.append((entry.th32ProcessID, entry.th32ParentProcessID))
            valid = kernel.Process32NextW(snapshot, ctypes.byref(entry))
    finally:
        kernel.CloseHandle(snapshot)
```

تحضر Entry وحجمه وتجتاز snapshot لتجمع أزواج PID والأب، وتغلق المقبض في finally. snapshot لقطة لحظة معينة وقد تتغير العمليات بعدها.

الأسطر 161–172

```
    selected = [(state["pid"], state["creation_stamp"])]
    for parent, _ in selected:
        for pid, parent_pid in entries:
            if parent_pid == parent and pid not in {p for p, _ in selected}:
                stamp = process_stamp(pid)
                if stamp is not None and stamp >= state["creation_stamp"]:
                    selected.append((pid, stamp))
    if process_stamp(state["pid"]) != state["creation_stamp"]:
        raise RuntimeError("Tracked server identity changed; refusing to stop any process.")
    for pid, stamp in reversed(selected):
        if process_stamp(pid) == stamp:
            os.kill(pid, signal.SIGTERM)
```

تبدأ بالشجرة المسجلة وتضيف أبناء مطابقين لأب مختار مع وقت إنشاء لا يسبق الأصل، وتعيد فحص هوية الأصل قبل الإنهاء، ثم تفحص طابع كل عملية وترسل SIGTERM بترتيب عكسي. هذا لتجنب إيقاف عملية أخرى ذات PID معاد؛ العمليات قد تتغير بين الفحص والإشارة.

الأسطر 175–180

```
def healthy(port):
    try:
        with urlopen(f"http://127.0.0.1:{port}/_stcore/health", timeout=2) as response:
            return response.status == 200 and response.read().strip() == b"ok"
    except OSError:
        return False
```

healthy تطلب endpoint الصحة المحلي 127.0.0.1 وتعيد True فقط لـ HTTP 200 ومحتوى ok؛ تعيد False على OSError. صحة المنفذ وحدها ليست إثبات هوية العملية.

الأسطر 183–190

```
@contextmanager
def launch_lock(name="launcher.lock"):
    import msvcrt
    with (RUNTIME / name).open("a+b") as lock:
        lock.seek(0)
        lock.write(b"0")
        lock.flush()
        lock.seek(0)
```

launch_lock contextmanager يستورد msvcrt ويفتح ملف قفل في RUNTIME ويضمن وجود بايت وينقل المؤشر لبدايته؛ يفترض أن مجلد runtime أُنشئ مسبقاً.

الأسطر 191–199

```
        try:
            msvcrt.locking(lock.fileno(), msvcrt.LK_NBLCK, 1)
        except OSError as error:
            raise RuntimeError("Another setup or launcher is running. Wait for it to finish, then retry.") from
error
        try:
            yield
        finally:
            lock.seek(0)
            msvcrt.locking(lock.fileno(), msvcrt.LK_UNLCK, 1)
```

يقفل البايت دون انتظار ويرفع رسالة إذا تعذر، ثم yield للعمل داخل القفل ويفكه في finally؛ التنسيق يحمي المتعاونين الذين يستخدمون نفس القفل.

الأسطر 202–213

```
def start(port=8501, no_browser=False):
    if not PYTHON.is_file():
        raise RuntimeError("Run SETUP_DEMO.bat first. The isolated CPU environment is missing.")
    with launch_lock():
        state = owned_state()
        if state:
            if not healthy(state["port"]):
                raise RuntimeError("The existing demo process is not healthy. Run START_DEMO.bat stop, then
start again. See its server log.")
            say(f"Dashboard already running; reusing {state['url']}")
            if not no_browser:
                webbrowser.open(state["url"])
            return state, True
```

start تستقبل منفذاً وخيار المتصفح وتتحقق من Python، ثم تحت قفل تفحص الخادم المملوك؛ إن وجد وغير صحي تفشل، وإلا تعيد استخدامه وتفتح المتصفح اختيارياً وتعيد الحالة مع True.

الأسطر 214–225

```
        runtime_check()
        chosen = None
        for candidate in range(port, min(port + 20, 65536)):
```

```
            with socket.socket() as probe:
                try:
                    probe.bind(("127.0.0.1", candidate))
                    chosen = candidate
                    break
                except OSError:
                    say(f"Port {candidate} is occupied; leaving that service alone.")
        if chosen is None:
            raise RuntimeError(f"No free port between {port} and {min(port+19,65535)}. Close an unused service
or pass --port NUMBER.")
```

عند عدم وجود خادم تفحص runtime، ثم تجرب الربط على localhost لعشرين منفذاً كحد أعلى دون إيقاف خدمة مشغولة، وتفشل إن لم تجد منفذاً. تحرير probe قبل إطلاق Streamlit يترك احتمال تنافس على المنفذ.

الأسطر 226–230

```
        server_log = RUNTIME / "logs" / f"server_{datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')}.log"
        command = [str(PYTHON), "-m", "streamlit", "run", str(ROOT / "main.py"), "--server.address",
"127.0.0.1", "--server.port", str(chosen), "--server.headless", "true", "--browser.gatherUsageStats", "false",
"--server.fileWatcherType", "none"]
        say(f"Starting the original eight-tab dashboard on CPU at http://127.0.0.1:{chosen}")
        with server_log.open("w", encoding="utf-8") as output:
            process = subprocess.Popen(command, cwd=ROOT, env=environment(), stdin=subprocess.DEVNULL,
stdout=output, stderr=subprocess.STDOUT, creationflags=subprocess.CREATE_NO_WINDOW |
subprocess.CREATE_NEW_PROCESS_GROUP)
```

تبني مسار سجل UTC وأمر Streamlit على العنوان المحلي بلا browser telemetry أو file watcher، وتشغل العملية دون نافذة مع إعادة توجيه خرجها إلى ملف. هذا أثر تشغيل خادم وكتابة سجل.

الأسطر 231–240

```
        stamp = None
        for _ in range(20):
            stamp = process_stamp(process.pid)
            if stamp:
                break
            if process.poll() is not None:
                break
            time.sleep(.1)
        state = {"root": str(ROOT.resolve()), "pid": process.pid, "creation_stamp": stamp, "port": chosen,
"url": f"http://127.0.0.1:{chosen}", "log": str(server_log)}
        STATE.write_text(json.dumps(state, indent=2), encoding="utf-8")
```

تحاول الحصول على بصمة إنشاء PID عشرين مرة أو تتوقف إن انتهت العملية، ثم تكتب server.json بالجذر والهوية والمنفذ والرابط والسجل. قد تبقى stamp فارغة عند فشل قراءتها.

الأسطر 241–250

```
        for _ in range(90):
            if process.poll() is not None:
                raise RuntimeError(f"Streamlit exited during startup. Read {server_log}")
            if healthy(chosen):
                say(f"Dashboard ready: {state['url']}\nServer log: {server_log}")
                if not no_browser:
```

```
                webbrowser.open(state["url"])
            return state, False
        time.sleep(.5)
    raise RuntimeError(f"Streamlit did not become healthy in 45 seconds. Read {server_log}")
```

تنتظر حتى 90 محاولة بفاصل نصف ثانية تقريباً؛ تفشل إن انتهت العملية أو لم تصبح صحية، وإلا تفتح المتصفح اختيارياً وتعيد الحالة مع False للدلالة على إنشاء جديد. الفحص صحيّ وليس تحققاً من جميع محتويات الواجهة.

الأسطر 253–260

```
def stop():
    with launch_lock():
        state = owned_state()
        if state:
            # Terminate only this launcher's exact PID + creation time under this root.
            # Windows venv's python.exe can be a redirector with a child interpreter.
            # Stop the owned process tree so that the child cannot keep the port open.
            terminate_owned_tree(state)
```

stop تحت القفل تسترجع الحالة المملوكة وتطلب إنهاء شجرة العملية المحددة، ويشرح التعليق احتمال وجود مفسر ابن من Windows venv.

الأسطر 261–272

```
            for _ in range(50):
                if not process_stamp(state["pid"]):
                    break
                time.sleep(.1)
            if process_stamp(state["pid"]):
                raise RuntimeError("The owned demo process did not stop; check its server log.")
            if healthy(state["port"]):
                raise RuntimeError("The old port still serves an application. Check the server log before
restarting.")
            say("Stopped this folder's demo server.")
        else:
            say("No owned demo server is running.")
        STATE.unlink(missing_ok=True)
```

تنتظر انتهاء الأصل حتى خمس ثوان تقريباً، وتفشل إذا بقي أو إذا استجاب المنفذ بعده، وإلا تبلغ النجاح؛ عند غياب حالة مملوكة تبلغ ذلك، ثم تحذف ملف الحالة. لا تقصد إيقاف خدمة أخرى على المنفذ.

الأسطر 275–281

```
def setup(args):
    say("[1/7] Checking Windows, Python 3.12 and x64 compatibility")
    if platform.system() != "Windows" or sys.version_info[:2] != (3, 12) or struct.calcsize("P") != 8:
        raise RuntimeError("Install Windows x64 CPython 3.12 for your user, then run SETUP_DEMO.bat again. No
CUDA or administrator account is required.")
    say("[2/7] Verifying package integrity and required research evidence")
    verify_manifest()
    verify_artifacts()
```

setup تتحقق من Windows وإصدار Python 3.12 ومعمارية 64 بت ثم تتحقق من الفهرس والآثار؛ أي فشل يوقف الإعداد قبل التثبيت.

الأسطر 282–288

```
    say("[3/7] Preparing a separate local CPU environment (.demo_venv)")
    if not PYTHON.exists():
        venv.EnvBuilder(with_pip=True).create(PYTHON.parents[1])
    wheelhouse = ROOT / "wheelhouse-cpu"
    bundled = wheelhouse.is_dir() and any(wheelhouse.glob("torch-2.5.1+cpu-*.whl"))
    if args.offline and not bundled:
        raise RuntimeError("Offline CPU wheel bundle missing: wheelhouse-cpu/torch-2.5.1+cpu-*.whl. Use the
complete CPU defense ZIP or run setup online.")
```

تنشئ .demo_venv مع pip إن غاب Python، وتتحقق من حزمة عجلات torch CPU في wheelhouse؛ عند فرض offline وغيابها ترفع خطأ. وجود عجلة torch وحده لا يثبت اكتمال جميع العجلات.

الأسطر 289–296

```
    say("[4/7] Detecting and installing missing CPU dependencies")
    install = [PYTHON, "-m", "pip", "install", "--disable-pip-version-check", "--no-compile", "--cache-dir",
RUNTIME / "pip-cache"]
    if bundled:
        say("Using bundled Windows CPU wheels; internet is not needed.")
        install.extend(["--no-index", "--find-links", wheelhouse, "--only-binary=:all:"])
    else:
        say("CPU wheels are not bundled here; first installation needs internet access.")
    run([*install, "-r", ROOT / "requirements-cpu.txt"])
```

تبني أمر pip مع cache محلي؛ إذا توفرت العجلات تقيد التثبيت إلى wheelhouse وbinary و--no-index، وإلا تشير إلى الحاجة للإنترنت، ثم تثبت requirements-cpu.txt. هذه عملية تثبيت عند التنفيذ.

الأسطر 297–302

```
    project_wheels = sorted(wheelhouse.glob("xai_finance_thesis-*.whl")) if bundled else []
    if project_wheels:
        run([*install, "--no-deps", "--force-reinstall", project_wheels[-1]])
    else:
        run([*install, "--no-deps", "--force-reinstall", "--no-build-isolation", ROOT])
    run([PYTHON, "-m", "pip", "check"])
```

تختار آخر عجلة مشروع مرتبة إن وجدت وتعيد تثبيتها بلا تبعيات، وإلا تثبت المشروع المحلي بلا عزل بناء، ثم تشغل pip check؛ اختيار آخر اسم ليس بالضرورة أحدث إصدار دلالياً.

الأسطر 303–312

```
    say("[5/7] Checking imports, CPU checkpoint replay, metrics, SHAP and LIME")
    runtime_check()
    run([PYTHON, ROOT / "scripts/verify_cpu_replay.py", "--output", RUNTIME / "cpu_replay.json"])
    say("[6/7] Checking ports, actual dashboard rendering and server health")
    run([PYTHON, "-c", "from streamlit.testing.v1 import AppTest;
```

```
app=AppTest.from_file('main.py',default_timeout=90).run(); assert not app.exception, app.exception; assert
len(app.tabs)==8; assert app.title[0].value=='Financial forecasts, explained'; print('Original dashboard
rendered: eight tabs, no exceptions')"])
    _, reused = start(args.port, no_browser=True)
    if not reused:
        stop()
    say("[7/7] Installation successful. Double-click START_DEMO.bat. Internet and CUDA are not required for the
saved demonstration.")
    (RUNTIME / "setup_success.json").write_text(json.dumps({"status": "PASSED", "python": str(PYTHON),
"offline_wheels": bundled, "completed_utc": datetime.now(timezone.utc).isoformat()}, indent=2), encoding="utf-
8")
```

تفحص البيئة وتستدعي CPU replay إلى .demo_runtime، ثم تشغل AppTest للواجهة وثمانية تبويبات وتبدأ الخادم للفحص الصحي وتوقفه إن أنشأته، ثم تكتب سجل النجاح. هذه الفحوص قد تستغرق وقتاً وتقرأ النماذج ولم تنفذ هنا.

الأسطر 315–324

```
def main():
    global LOG
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("action", choices=["setup", "start", "stop", "verify-package"])
    parser.add_argument("--offline", action="store_true")
    parser.add_argument("--no-browser", action="store_true")
    parser.add_argument("--port", type=int, default=8501)
    args = parser.parse_args()
    (RUNTIME / "logs").mkdir(parents=True, exist_ok=True)
    path = RUNTIME / "logs" / f"{args.action}_{datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')}.log"
```

main تعلن LOG عامة وتحدد أفعال setup/start/stop/verify-package وخيارات offline والمتصفح والمنفذ، ثم تنشئ مجلد السجل واسم ملف زمني.

الأسطر 325–331

```
    with path.open("x", encoding="utf-8") as log:
        LOG = log
        try:
            if platform.system() != "Windows":
                raise RuntimeError("This wizard requires Windows x64.")
            if not 1024 <= args.port <= 65535:
                raise RuntimeError("Choose a local port between 1024 and 65535.")
```

تفتح السجل حصرياً وتخزنه في LOG، وتتحقق داخل try من Windows ومجال المنفذ 1024–65535؛ الفشل يعالج لاحقاً.

الأسطر 332–340

```
            if args.action == "setup":
                with launch_lock("setup.lock"):
                    setup(args)
            elif args.action == "start":
                start(args.port, args.no_browser)
            elif args.action == "stop":
                stop()
```

```
            else:
                verify_manifest()
```

توجه الفعل إلى setup تحت قفل منفصل أو start أو stop أو verify_manifest؛ القفل والإجراءات تحدث آثارها فقط عند تنفيذ الأمر.

الأسطر 341–346

```
        except Exception as error:
            say(f"ERROR: {error}\nDiagnostic log: {path}")
            return 1
        finally:
            LOG = None
    return 0
```

تلتقط Exception وتكتب رسالة وتعيد 1، وتعيد LOG إلى None في finally ثم تعيد 0 عند النجاح؛ لا تلتقط كل BaseException مثل KeyboardInterrupt.

الأسطر 349–350

```
if __name__ == "__main__":
    raise SystemExit(main())
```

عند التشغيل مباشرة يستدعي main ويحول القيمة إلى رمز خروج النظام.

شرح scripts/diagnose_validation_replay.py

بصمة المصدر SHA-256:

.85b81348e6c1526984648cf03650f24589fca23b5af0d388fcbccf951bd50ce1

لم يُشغّل السكربت في جلسة التوثيق.

الأسطر 1–12

```
"""Measure cross-device error before choosing a precision-aware replay criterion."""
import argparse
from pathlib import Path

import joblib
import numpy as np
import pandas as pd

from xai_finance.models.lstm_model import load_lstm
from xai_finance.pipelines.train import load_windows
from xai_finance.seed import seed_experiment
from xai_finance.storage import read_json, write_json
```

توثيق واستيرادات قراءة ونماذج LSTM ونوافذ وبذور وتخزين؛ يتطلب torch وCUDA ضمن load_lstm.

الأسطر 15–20

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("folder", type=Path)
    destination = parser.parse_args().folder.resolve()
    seed_experiment(42)
    diagnostics = []
```

main تقرأ مجلد التحقق وتضبط البذرة وتبدأ نتائج التشخيص.

الأسطر 21–28

```
    for path in sorted((destination / "experiments").rglob("design.json")):
        design = read_json(path)
        windows = load_windows(Path(design["input_windows_source"]).parent)
        with np.load(path.parent / "target_and_partitions.npz") as saved:
            fixture = windows.flat[saved["test"]]
        table = pd.read_csv(path.parent / "predictions.csv", float_precision="round_trip")
        recorded = table.loc[table.model == "LSTM", "predicted_target"].to_numpy()
        scalers = joblib.load(path.parent / "LSTM/scalers.joblib")
```

تمر على design.json للمهام وتحمل نوافذ المصدر وفهارس الاختبار والتنبؤ المحفوظ والمعايير؛ joblib ينبغي تحميله من مصدر موثوق.

الأسطر 29–34

```
        cpu = load_lstm(path.parent / "LSTM", scalers, "cpu").predict(fixture)
        gpu = load_lstm(path.parent / "LSTM", scalers, "cuda").predict(fixture)
        np.testing.assert_array_equal(gpu, recorded)
        diagnostics.append({**{name: design[name] for name in ["asset", "target", "design", "fold"]},
"cuda_replay_bitwise_equal": bool(np.array_equal(gpu, recorded)), "cpu_max_abs_error_target_units":
float(np.max(np.abs(cpu - recorded))), "training_target_scale": float(scalers.target.scale_[0]),
"cpu_max_error_training_std_units": float(np.max(np.abs(cpu - recorded)) / scalers.target.scale_[0])})
    write_json(destination / "cross_device_diagnostics.json", diagnostics)
    print(pd.DataFrame(diagnostics).to_string(index=False))
```

تحسب التنبؤ ذاته بإعادة تحميل CPU وCUDA وتفرض تطابق GPU والمحفوظه بتآ بت، وتسجل أقصى خطأ CPU بالوحدة الأصلية ومعيار التدريب ثم تكتب JSON وتطبع. يتطلب CUDA ولا يثبت تطابق أجهزة أخرى؛ لم يُنفذ هنا.

الأسطر 37–38

```
if __name__ == "__main__":
    main()
```

حارس التشغيل يستدعي main.

شرح `scripts/download_data.py`

بصمة المصدر SHA-256:

`ba95c0c7f7ffc4743e4e23751218a222ea3ed59e79dcc658e2f838226bf9b3eb`.

الاقتباسات من الملف؛ لم تُشغّل السكربتات أو تجاربها.

الأسطر 1–2

```
from xai_finance.config import load_config
from xai_finance.io.data_sources import download_all
```

يستورد load_config وdownload_all داخليتين.

الأسطر 4–4

```
download_all(load_config("configs/full_experiment.yaml"))
```

عند تنفيذ الملف أو استيراده يستدعي تنزيل الأصول وفق full_experiment.yaml؛ لقطة موجودة تمر بالتحقق كما في download_all. قد يحدث طلب شبكة وكتابة ولم ينفذ هنا.

شرح scripts/generate_defense_package.py

بصمة المصدر SHA-256:

.4f8457037544d7d5e285652c9a8844a8b0921d836db77e8ea901252821b95b5e

لم تنفذ عمليات النسخ أو الحزم في جلسة التوثيق.

الأسطر 1–8

```
from pathlib import Path
import shutil
import zipfile

from xai_finance.config import ROOT
from xai_finance.pipelines.train import resolve_run
from xai_finance.config import load_config
from xai_finance.storage import sha256, write_json
```

Path وshutil وzipfile قياسية؛ ROOT وتحميل الإعداد والتشغيل والبصمات والتخزين داخلية.

الأسطر 11–21

```
def copy_project(destination: Path, run_folder: Path) -> None:
    for name in ["src", "dashboard", "configs", "docs", "scripts", "tests", ".streamlit", "wheelhouse"]:
        shutil.copytree(ROOT / name, destination / name, dirs_exist_ok=True,
ignore=shutil.ignore_patterns("__pycache__", "*.egg-info"))
    for name in ["pyproject.toml", "requirements-lock.txt", "requirements-gpu.txt", "README.md",
"CODEX_XAI_THESIS_IMPLEMENTATION_PLAN.md"]:
        shutil.copy2(ROOT / name, destination / name)
    for name in ["raw", "manifests"]:
        shutil.copytree(ROOT / "data" / name, destination / "data" / name, dirs_exist_ok=True)
    shutil.copytree(run_folder, destination / "artifacts/runs" / run_folder.name, dirs_exist_ok=True)
    for name in ["tests", "environment"]:
        shutil.copytree(ROOT / "artifacts" / name, destination / "artifacts" / name, dirs_exist_ok=True)
    shutil.copy2(ROOT / "artifacts/latest_full.json", destination / "artifacts/latest_full.json")
```

copy_project تنسخ الشيفرة والإعدادات والوثائق والعجلات واللقطات والتشغيل وآثار الاختبارات والبيئة والمؤشر إلى الوجهة؛ dirs_exist_ok يسمح بالدمج مع ملفات موجودة وقد يستبدل نسخاً، فلا يعني إنشاء حزمة معزولة تلقائياً.

الأسطر 24–31

```
def archive_package(destination: Path, run_id: str) -> Path:
    backup_folder = ROOT / "backups"
    backup_folder.mkdir(exist_ok=True)
    archive = backup_folder / f"defense_{run_id}.zip"
    with zipfile.ZipFile(archive, "w", allowZip64=True) as bundle:
```

```
        for path in sorted(destination.rglob("*")):
            if path.is_file():
                bundle.write(path, path.relative_to(destination), compress_type=zipfile.ZIP_STORED)
```

archive_package تنشئ backups وتفتح ZIP باسم التشغيل بوضع w وتضيف الملفات دون ضغط؛ وضع w يمكنه استبدال ملف موجود بالاسم نفسه، ولذلك هذا السكربت أقدم من ضمان عدم الاستبدال في حزمة CPU.

الأسطر 32–36

```
    with zipfile.ZipFile(archive) as bundle:
        if bundle.testzip() is not None:
            raise ValueError("Defense ZIP CRC verification failed")
    write_json(archive.with_suffix(".sha256.json"), {"archive": archive.name, "sha256": sha256(archive),
"backup_scope": "Local same-disk backup; second physical medium/cloud copy not verified"})
    return archive
```

تفحص CRC باستخدام testzip وتكتب JSON لبصمة ZIP ثم تعيد المسار؛ النسخة على القرص نفسه ولا تثبت نسخة خارجية.

الأسطر 39–50

```
def main() -> None:
    destination = (ROOT / "defense_package").resolve()
    if not destination.is_relative_to(ROOT.resolve()):
        raise ValueError("Defense package must stay inside project workspace")
    destination.mkdir(exist_ok=True)
    run_folder = resolve_run(load_config(ROOT / "configs/full_experiment.yaml"))
    copy_project(destination, run_folder)
    entries = {str(path.relative_to(destination)).replace("\\", "/"): sha256(path) for path in
sorted(destination.rglob("*")) if path.is_file() and path.name != "PACKAGE_MANIFEST.json"}
    write_json(destination / "PACKAGE_MANIFEST.json", {"run_id": run_folder.name, "files": entries, "platform":
"Windows CPython 3.12 AMD64 wheels; Python installer not bundled"})
    archive = archive_package(destination, run_folder.name)
    print("Defense package:", destination)
    print("CRC-checked backup:", archive)
```

main تحدد مجلد defense_package داخل الجذر وتتحقق من احتوائه وتنسخ تجربة full وتحسب بصمات الملفات عدا manifest ثم تكتب فهرساً وتحزم وتطبع؛ لها آثار نسخ وكتابة.

الأسطر 53–54

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ المباشر يستدعي main؛ لم يشغل هنا.

شرح scripts/launch_demo.py

بصمة المصدر SHA-256:

.dd79873ce8b441b06505e590f18152d3b5e7ccf54e2960d9d0ce59ed9dfc35b2

وصف ثابت؛ لم تُشغّل السكربتات.

الأسطر 1–4

```
import os
from pathlib import Path
import subprocess
import sys
```

مكتبات قياسية للبيئة والمسارات والعملية الفرعية والوسائط.

الأسطر 6–8

```
ROOT = Path(__file__).resolve().parents[1]
os.environ["STREAMLIT_BROWSER_GATHER_USAGE_STATS"] = "false"
raise SystemExit(subprocess.call([sys.executable, str(ROOT / "scripts/demo_wizard.py"), "start", *sys.argv[1:]],
cwd=ROOT))
```

يحدد جذر المشروع ويعطل جمع إحصاء استخدام المتصفح في Streamlit، ويشغل demo_wizard.py start بوسائط المستخدم ويعيد رمز العملية عبر SystemExit. يتغير متغير البيئة وتبدأ عملية عند التنفيذ أو الاستيراد.

شرح scripts/offline_demo.py

بصمة المصدر SHA-256:

80d8c4d43d91a613b4209ea650894135e947a9330b45a6b0788783044c138595.

وصف ثابت؛ لم تُشغّل السكربتات.

الأسطر 1–3

```
from pathlib import Path
import subprocess
import sys
```

Path وsubprocess وsys قياسية لتحديد الملفات وتشغيل العمليات.

الأسطر 5–9

```
ROOT = Path(__file__).resolve().parents[1]
python = ROOT / ".demo_venv/Scripts/python.exe"
if not python.exists():
    subprocess.run([sys.executable, str(ROOT / "scripts/setup_offline.py")], cwd=ROOT, check=True)
subprocess.run([str(python), str(ROOT / "scripts/launch_demo.py"), *sys.argv[1:]], cwd=ROOT, check=True)
```

يحدد Python في .demo_venv؛ إذا غاب ينفذ setup_offline باستخدام Python الحالي مع check=True، ثم ينفذ launch_demo بالبيئة والوسائط. قد ينشئ بيئة ويشغل خادماً؛ لا يفحص وجود كل اعتماد بمجرد وجود المفسر.

شرح scripts/offline_guard/sitecustomize.py

بصمة المصدر SHA-256:

.a3896b513d9353603084ef86386666a57ad1f35d1a32b37c2414af26d7001b5a

وصف ثابت؛ لم تُشغّل السكربتات.

الأسطر 1–4

```
"""Test-only network guard, enabled explicitly by XAI_TEST_OFFLINE=1."""

import os
import socket
```

سلسلة توثق أنه حارس اختبار اختياري؛ os وsocket قياسيتان.

الأسطر 6–10

```
if os.environ.get("XAI_TEST_OFFLINE") == "1":
    _connect = socket.socket.connect
    _connect_ex = socket.socket.connect_ex
    _getaddrinfo = socket.getaddrinfo
    _local = {"localhost", "127.0.0.1", "::1", None}
```

فقط عند XAI_TEST_OFFLINE=1 يحفظ دوال الاتصال وحل الأسماء الأصلية ويحدد أسماء loopback المقبولة. يحتاج تحميل sitecustomize من مسار Python المناسب.

الأسطر 12–14

```
    def _check(host):
        if host not in _local:
            raise OSError(f"Offline acceptance test denied external network: {host}")
```

_check ترفض host خارج المجموعة بخطأ OSError؛ السماح بالأسماء المحددة لا يثبت حجب كل وسائل الشبكة الممكنة.

الأسطر 16–24

```
    def connect(connection, address):
        if isinstance(address, tuple):
            _check(address[0])
        return _connect(connection, address)
```

```
    def connect_ex(connection, address):
        if isinstance(address, tuple):
            _check(address[0])
        return _connect_ex(connection, address)
```

connect وconnect_ex تفحصان عنوان tuple ثم تفوضان الاتصال الأصلي؛ عناوين غير tuple لا يفحصها هذا الشرط.

الأسطر 26–32

```
    def getaddrinfo(host, *args, **kwargs):
        _check(host)
        return _getaddrinfo(host, *args, **kwargs)

    socket.socket.connect = connect
    socket.socket.connect_ex = connect_ex
    socket.getaddrinfo = getaddrinfo
```

getaddrinfo يفحص المضيف ويفوض الأصل ثم يستبدل ثلاث دوال socket داخل العملية. هذا حارس اختبار لواجهات محددة لا جدار حماية شامل.

شرح scripts/offline_smoke.py

بصمة المصدر SHA-256:

`b8fc7ee98de68c6d372cf9ca5c55ebbb3eb8df494be57f8fc7f1120f54a348ce`.

لم يُشغّل السكربت في جلسة التوثيق.

الأسطر 1–10

```
"""Block external Python sockets while exercising the saved-artifact dashboard."""

from pathlib import Path
import json
import socket

from streamlit.testing.v1 import AppTest

ROOT = Path(__file__).resolve().parents[1]
original_connect = socket.socket.connect
```

توثيق واستيرادات Path وjson وsocket وAppTest، وتحديد الجذر وحفظ دالة اتصال socket الأصلية.

الأسطر 13–16

```
def loopback_only(connection, address):
    if isinstance(address, tuple) and address[0] not in {"127.0.0.1", "localhost", "::1"}:
        raise RuntimeError(f"Offline rehearsal blocked external network: {address[0]}")
    return original_connect(connection, address)
```

loopback_only ترفض عنوان tuple غير localhost ثم تفوض الاتصال الأصلي. لا تغطي وحدها جميع طرق الشبكة أو كل أشكال العنوان.

الأسطر 19–26

```
socket.socket.connect = loopback_only
app = AppTest.from_file(str(ROOT / "dashboard/app.py"), default_timeout=90).run()
assert not app.exception, app.exception
assert len(app.tabs) == 8
assets = app.selectbox(key="asset").options
pointer = json.loads((ROOT / "artifacts/latest_full.json").read_text())
metadata = json.loads((ROOT / "artifacts/runs" / pointer["run_id"] / "run.json").read_text())
assert set(assets) == set(metadata["configuration"]["assets"])
```

تستبدل socket.connect في العملية وتشغل AppTest للواجهة، وتفحص غياب الاستثناء وثمانية تبويبات وتطابق خيارات الأصول مع إعداد التشغيل المحفوظ.

الأسطر 27–35

```
for asset in assets:
    app.selectbox(key="asset").set_value(asset).run()
    assert not app.exception, app.exception
    for model in ["LinearRegression", "RandomForest", "GradientBoosting", "LSTM"]:
        app.selectbox(key="explain_model").set_value(model).run()
        assert not app.exception, app.exception
    dates = app.selectbox(key="explanation_date").options
    app.selectbox(key="explanation_date").set_value(dates[-1]).run()
    assert not app.exception, app.exception
```

تختار كل أصل وكل نموذج تفسير وتتحقق من غياب الاستثناء، ثم تختار آخر تاريخ محسوب مسبقاً. حالة أول تاريخ تفترض الاختيار الافتراضي ولا تتحقق صراحة من جميع التواريخ.

الأسطر 36–38

```
receipt = {"status": "PASSED", "run_id": pointer["run_id"], "assets": assets, "models_per_asset": 4, "tabs":
len(app.tabs), "date_check": "first and last precomputed dates", "external_socket_connections": "blocked",
"scope": "Saved-artifact viewing only; not retraining or a browser pixel screenshot"}
(ROOT / "artifacts/tests/offline_dashboard.json").write_text(json.dumps(receipt, indent=2), encoding="utf-8")
print(f"OFFLINE DASHBOARD PASS: {len(assets)} datasets, four explanation models, first/last precomputed dates,
eight tabs. External sockets blocked.")
```

تكتب إيصالاً في artifacts/tests وتطبع النجاح؛ يصف العرض المحفوظ دون تدريب أو اختبار صور متصفح. لم يشغل هنا.

شرح scripts/package_cpu_defense.py

بصمة المصدر SHA-256:

.631f1680abb55b54d04c551386248d9e7dc89e93c90eac85a6d8f55394b2025b

لم تنفذ عمليات النسخ أو الحزم في جلسة التوثيق.

الأسطر 1–16

```
"""Build an exclusive, hash-verified CPU package without replacing older archives."""

import argparse
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path
import shutil
import zipfile

ROOT = Path(__file__).resolve().parents[1]


def digest(path):
    with path.open("rb") as stream:
        return hashlib.file_digest(stream, "sha256").hexdigest()
```

توثيق واستيرادات قياسية، وROOT، وdigest لحساب SHA-256 لملف ثنائي.

الأسطر 19–28

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--archive", type=Path, default=ROOT / "Mohammad_XAI_Defense_CPU.zip")
    args = parser.parse_args()
    archive = args.archive.resolve()
    if not archive.is_relative_to(ROOT.resolve()) or archive.suffix != ".zip":
        raise ValueError("The archive must be a ZIP inside this workspace.")
    archive.parent.mkdir(parents=True, exist_ok=True)
    if archive.exists():
        raise FileExistsError(f"Refusing to overwrite an existing defense archive: {archive}")
```

main تقبل archive وتفرض امتداد ZIP ووجوده داخل الجذر، وتنشئ الأب وترفض أرشيفاً موجوداً؛ لا تنفذ الحزم أثناء التوثيق.

الأسطر 29–37

```
    timestamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S%fZ")
    destination = ROOT / "dist" / f"cpu_build_{timestamp}" / "Mohammad_XAI_Defense_CPU"
    destination.mkdir(parents=True, exist_ok=False)
    receipt = json.loads((ROOT / "artifacts/deployment/windows_deployment.json").read_text())
    assert receipt["status"] == "PASSED"
    wheels = sorted((ROOT / "wheelhouse-cpu").glob("*.whl"))
```

```
    assert len(wheels) == 90, f"Expected 89 pinned dependencies plus project wheel, found {len(wheels)}"
    assert any(p.name.startswith("torch-2.5.1+cpu-") for p in wheels)
    assert not any("nvidia" in p.name or "+cu121" in p.name for p in wheels)
```

تحدد مجلد بناء زمني جديداً، وتقرأ إيصال النشر وتؤكد نجاحه ووجود 90 عجلة وtorch CPU وعدم وجود عجلات CUDA/NVIDIA؛ assertions قد تُعطل في Python المحسن.

الأسطر 38–50

```
    for name in ["src", "dashboard", "configs", "docs", "scripts", "tests", ".streamlit", "wheelhouse-cpu"]:
        print(f"Copying {name}", flush=True)
        shutil.copytree(ROOT / name, destination / name, ignore=shutil.ignore_patterns("__pycache__", "*.egg-
info"))
    for name in ["main.py", "SETUP_DEMO.bat", "START_DEMO.bat", "README.md", "pyproject.toml", "requirements-
cpu.txt", "requirements-gpu.txt", "requirements-lock.txt", "CODEX_XAI_THESIS_IMPLEMENTATION_PLAN.md",
".env.example"]:
        shutil.copy2(ROOT / name, destination / name)
    for name in ["raw", "manifests"]:
        shutil.copytree(ROOT / "data" / name, destination / "data" / name)
    # Include every research run, including retained superseded diagnostic runs.
    for name in ["runs", "scientific_validation", "environment", "tests"]:
        print(f"Copying all artifacts/{name}", flush=True)
        shutil.copytree(ROOT / "artifacts" / name, destination / "artifacts" / name)
    for pointer in (ROOT / "artifacts").glob("latest*.json"):
        shutil.copy2(pointer, destination / "artifacts" / pointer.name)
```

تنسخ المصدر والوثائق والإعدادات والعجلات واللقطات وجميع تشغيلات البحث والتحقق وآثار البيئة والاختبارات ومؤشرات latest، مع استبعاد ذاكرات Python من المصدر.

الأسطر 51–63

```
    evidence = destination / "artifacts/deployment"
    evidence.mkdir(parents=True)
    external_release_receipts = {"package.json", "final_release.json", "final_release_check.log",
"release_build.log"}
    for path in (ROOT / "artifacts/deployment").iterdir():
        if path.is_file() and path.suffix in {".json", ".xml", ".log"} and path.name not in
external_release_receipts:
            shutil.copy2(path, evidence / path.name)
    for folder in (ROOT / "artifacts/deployment").glob("browser*"):
        if folder.is_dir() and (folder / "receipt.json").exists():
            target = evidence / folder.name
            target.mkdir()
            for path in folder.iterdir():
                if path.suffix == ".png" or path.name == "receipt.json":
                    shutil.copy2(path, target / path.name)
```

تنسخ إيصالات نشر مختارة عدا إيصالات الإصدار الخارجي، وصور وإيصالات متصفح فقط من المجلدات المطابقة.

الأسطر 64–71

```
    baseline = json.loads((evidence / "original_evidence_sha256.json").read_text())
    print(f"Verifying preservation of all {len(baseline)} original evidence files", flush=True)
    for relative, expected in baseline.items():
        assert (destination / relative).is_file(), f"Research evidence omitted: {relative}"
```

```
        assert digest(destination / relative) == expected, f"Research evidence changed: {relative}"
    entries = {p.relative_to(destination).as_posix(): {"bytes": p.stat().st_size, "sha256": digest(p)} for p in
sorted(destination.rglob("*")) if p.is_file()}
    manifest = {"format": 1, "created_utc": datetime.now(timezone.utc).isoformat(), "platform": "Windows x64
CPython 3.12", "torch": "2.5.1+cpu", "dependency_wheels": len(wheels), "python_installer_bundled": False,
"original_evidence_files_included_unchanged": len(baseline), "excluded_nonresearch_content": ["virtual
environments", "pip caches", "provider cookies", "browser profiles", "Git metadata", "bytecode caches in active
source", "older distribution archives (preserved in place)"], "files": entries}
    (destination / "CPU_PACKAGE_MANIFEST.json").write_text(json.dumps(manifest, indent=2), encoding="utf-8")
```

تقرأ بصمات الأدلة الأصلية وتتحقق من وجودها وتطابقها في staging، ثم تحصر الملفات وأحجامها وبصماتها وتكتب CPU_PACKAGE_MANIFEST. هذه سلامة مقابل baseline المسجل.

الأسطر 72–80

```
    pending = archive.with_suffix(".zip.pending")
    if pending.exists():
        raise FileExistsError(pending)
    print(f"Writing {len(entries)+1} files; all {len(baseline)} original evidence files included unchanged.",
flush=True)
    with zipfile.ZipFile(pending, "x", allowZip64=True) as bundle:
        for path in sorted(destination.rglob("*")):
            if path.is_file():
                compression = zipfile.ZIP_STORED if path.suffix in {".whl", ".png", ".npz", ".docx", ".jpg"}
else zipfile.ZIP_DEFLATED
                bundle.write(path, (Path(destination.name) / path.relative_to(destination)).as_posix(),
compress_type=compression, compresslevel=6)
```

ترفض ملف pending موجوداً وتفتح ZIP حصرياً وتضيف الملفات باختيار ضغط بحسب الامتداد؛ لا تضغط عجلات وصوراً وأنواعاً مضغوطة مسبقاً.

الأسطر 81–86

```
    with zipfile.ZipFile(pending) as bundle:
        assert bundle.testzip() is None, "ZIP CRC failure"
        packaged = {name.removeprefix(destination.name + "/") for name in bundle.namelist()}
        assert packaged == set(entries) | {"CPU_PACKAGE_MANIFEST.json"}
        for relative, expected in entries.items():
            assert hashlib.sha256(bundle.read(destination.name + "/" + relative)).hexdigest() ==
expected["sha256"], relative
```

تفحص CRC وقائمة الأعضاء وبصمة كل عضو مقابل الفهرس؛ قد تستهلك الذاكرة عند قراءة عضو كبير كاملاً.

الأسطر 87–93

```
    # rename() refuses an existing destination on Windows, preserving prior packages.
    pending.rename(archive)
    archive_hash = digest(archive)
    archive.with_suffix(".zip.sha256").write_text(f"{archive_hash}  {archive.name}\n", encoding="ascii")
    verification = {"status": "PASSED", "archive": str(archive), "sha256": archive_hash, "bytes":
archive.stat().st_size, "manifest_files": len(entries), "original_evidence_files": len(baseline),
"dependency_wheels": len(wheels), "crc_and_all_member_sha256": "PASSED", "staging_folder": str(destination)}
    (ROOT / "artifacts/deployment/package.json").write_text(json.dumps(verification, indent=2), encoding="utf-
8")
    print(json.dumps(verification, indent=2), flush=True)
```

تعيد تسمية pending إلى الأرشيف، وتحسب بصمته وتكتب ملف sha256 وإيصال package.json وتطبع التقرير؛ يلاحظ التعليق أن rename في Windows يرفض وجهة موجودة.

الأسطر 96–97

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ يستدعي main.

شرح :ripts/package_scientific_validation.py

بصمة المصدر SHA-256:

.cdb47baf5c2eb739f2706f87e58840ca2cd36c58aff9cf870d92d0e893220e39

لم تنفذ عمليات النسخ أو الحزم في جلسة التوثيق.

الأسطر 1–13

```
"""Package the executed addendum separately from the untouched defense bundle."""
import argparse
import base64
import hashlib
from html.parser import HTMLParser
from pathlib import Path
import shutil
from zipfile import ZipFile, ZIP_DEFLATED

from PIL import Image

from xai_finance.config import ROOT
from xai_finance.storage import sha256, write_json
```

توثيق وأدوات قياسية للوسائط والبيانات المضمنة والبصمات وHTML وZIP، وPillow خارجية، ووظائف المشروع.

الأسطر 16–23

```
class EmbeddedImages(HTMLParser):
    def __init__(self):
        super().__init__()
        self.sources = []

    def handle_starttag(self, tag, attributes):
        if tag == "img":
            self.sources.append(dict(attributes)["src"])
```

EmbeddedImages يرث HTMLParser ويجمع قيم src من كل عنصر img؛ يفترض وجود السمة.

الأسطر 26–36

```
def verify_offline_page(destination: Path) -> None:
    parser = EmbeddedImages()
    parser.feed((destination / "offline_validation.html").read_text(encoding="utf-8"))
    figures = sorted((destination / "figures").glob("*.png"))
    assert len(parser.sources) == len(figures) == 5
    for source, path in zip(parser.sources, figures):
        assert source.startswith("")
        assert base64.b64decode(source.split(",", 1)[1]) == path.read_bytes()
```

```
        with Image.open(path) as chart:
            chart.verify()
    write_json(destination / "offline_page_verification.json", {"embedded_pngs": len(figures),
"all_image_bytes_match_exports": True, "png_decoding_verified": True, "requires_network": False})
```

verify_offline_page تحلل HTML المحفوظ وتقارن الصور الخمس المضمنة بايتاً بملفات PNG المفروزة وتطلب Pillow التحقق من صيغتها، ثم تكتب تقريراً. فحص الصور المحددة لا يمسح كل أشكال الاتصالات أو يثبت بمفرده أن الصفحة بأكملها لا تحتاج شبكة.

الأسطر 39–47

```
def snapshot_source(destination: Path) -> None:
    snapshot = destination / "source_snapshot"
    snapshot.mkdir(exist_ok=False)
    paths = [path for category in ["src", "scripts", "tests", "configs"] for path in (ROOT /
category).rglob("*") if path.is_file() and "__pycache__" not in path.parts and path.suffix in {".py", ".yaml"}]
    paths.extend(ROOT / name for name in ["pyproject.toml", "requirements-lock.txt", "requirements-gpu.txt",
"docs/SCIENTIFIC_VALIDATION_REPRODUCIBILITY.md", "docs/SCIENTIFIC_VALIDATION_REPORT.md"])
    for path in paths:
        target = snapshot / path.relative_to(ROOT)
        target.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(path, target)
```

snapshot_source تنشئ source_snapshot جديداً وتنسخ ملفات Python وYAML من مجلدات محددة مع إعدادات وتقارير مرجعية، محافظة على المسارات؛ لا تستخدم نسخة المصدر هذه كتطبيق نشط.

الأسطر 50–57

```
def package_addendum(destination: Path) -> None:
    paths = sorted(path for path in destination.rglob("*") if path.is_file())
    manifest = {str(path.relative_to(destination)): {"bytes": path.stat().st_size, "sha256": sha256(path)} for
path in paths}
    write_json(destination / "addendum_manifest.json", manifest)
    archive = ROOT / "backups" / f"{destination.name}_addendum.zip"
    with ZipFile(archive, "x", compression=ZIP_DEFLATED, compresslevel=6) as bundle:
        for path in paths + [destination / "addendum_manifest.json"]:
            bundle.write(path, path.relative_to(destination).as_posix())
```

package_addendum تحصر الملفات وتكتب أحجامها وبصماتها ثم تفتح ZIP إضافياً حصرياً وتضيف الملفات والفهرس؛ تفترض وجود مجلد backups.

الأسطر 58–66

```
    with ZipFile(archive) as bundle:
        assert bundle.testzip() is None
        assert len(bundle.infolist()) == len(paths) + 1
        for relative, record in manifest.items():
            archived = bundle.read(relative.replace("\\", "/"))
            assert len(archived) == record["bytes"]
            assert hashlib.sha256(archived).hexdigest() == record["sha256"]
    write_json(destination / "addendum_archive.json", {"path": str(archive.relative_to(ROOT)), "bytes":
archive.stat().st_size, "sha256": sha256(archive), "manifest_files": len(manifest), "archive_crc_verified":
True, "archive_member_sha256_verified": True})
    print(f"Separate addendum: {archive}; SHA256 {sha256(archive)}")
```

تفحص CRC وعدد الأعضاء وطول وبصمة كل ملف ثم تكتب إيصال الأرشيف وتطبع بصمته؛ هذه آثار كتابة لا تنشئ نسخة خارج القرص.

الأسطر 69–75

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("folder", type=Path)
    destination = parser.parse_args().folder.resolve()
    verify_offline_page(destination)
    snapshot_source(destination)
    package_addendum(destination)
```

main تقرأ مجلد الوجهة وتتحقق من صفحة offline وتنسخ الشيفرة وتبني الملحق بهذا الترتيب؛ كلها آثار تنفيذ ولم تحدث هنا.

الأسطر 78–79

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ المباشر.

شرح scripts/replicate_validation_lstm.py

بصمة المصدر SHA-256:

.d95bd5ef5cb381409e38ba355d9822eff275869dcd1b989803a3cd7317eae2c3

لم يُشغّل السكربت في جلسة التوثيق.

الأسطر 1–14

```
"""Repeat one CUDA fit using the recorded configuration; never overwrite a fit."""
import argparse
from pathlib import Path

import joblib
import numpy as np
from threadpoolctl import threadpool_limits

from xai_finance.config import ROOT
from xai_finance.models.lstm_model import fit_lstm, load_lstm
from xai_finance.pipelines.train import load_windows
from xai_finance.seed import seed_experiment
from xai_finance.storage import read_json, write_json
from xai_finance.validation.design import transformed_windows
```

توثيق واستيرادات إعداد وإعادة تدريب LSTM وقراءة النوافذ والهدف والحفظ؛ هذا سكربت تجربة لا توثيق فقط.

الأسطر 17–28

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("folder", type=Path)
    arguments = parser.parse_args()
    destination = arguments.folder.resolve()
    folder = destination / "experiments/fixed_split/price_change/AAPL_fold0"
    design = read_json(folder / "design.json")
    protocol = read_json(destination / "protocol.json")
    windows = transformed_windows(load_windows(ROOT / "artifacts/runs" / protocol["original_run"] / "AAPL"),
"price_change")
    output = destination / "determinism_replication"
    output.mkdir(exist_ok=False)
    seed_experiment(protocol["seed"])
```

main تقرأ الوجهة والتصميم والبروتوكول وتحول نوافذ AAPL إلى فرق السعر وتخلق مجلد تكرار exist_ok=False وتضبط البذرة. يمنع وجود المجلد إعادة استخدامه.

الأسطر 29–35

```
    with threadpool_limits(limits=4):
        predictor, metadata = fit_lstm(windows, design["configuration"], output)
    joblib.dump(predictor.scalers, output / "scalers.joblib")
    reference_scalers = joblib.load(folder / "LSTM/scalers.joblib")
    for name in ["inputs", "target"]:
        for attribute in ["mean_", "scale_", "var_"]:
            np.testing.assert_array_equal(getattr(getattr(predictor.scalers, name), attribute),
getattr(getattr(reference_scalers, name), attribute))
```

تدرب LSTM مع حدود الخيوط وتحفظ scalers وتقارن متوسطاتها ومقاييسها وتباينها مع المرجع. إعادة التدريب تغير آثاراً ولم تُنفذ هنا.

الأسطر 36–45

```
    reference = load_lstm(folder / "LSTM", reference_scalers, "cuda" if predictor.device.type == "cuda" else
"cpu")
    fixture = windows.flat[windows.partitions["test"]]
    repeated, previous = predictor.predict(fixture), reference.predict(fixture)
    np.testing.assert_array_equal(repeated, previous)
    reference_history = read_json(folder / "LSTM/history.json")
    repeated_history = read_json(output / "history.json")
    assert len(reference_history) == len(repeated_history)
    for before, after in zip(reference_history, repeated_history):
        for key in ["epoch", "train_loss", "validation_loss"]:
            assert before[key] == after[key]
```

تحمل مرجعاً على نوع جهاز التكرار وتقارن تنبؤات الاختبار بتاً بت ثم تقارن طول التاريخ وكل epoch وخسارة تدريب وتحقق. لا تقارن الزمن وتفترض إمكانية إعادة إنتاج البيئة.

الأسطر 46–47

```
    write_json(output / "verification.json", {"training_metadata": metadata, "test_n": len(fixture),
"predictions_bitwise_identical": bool(np.array_equal(repeated, previous)), "max_abs_prediction_difference":
float(np.max(np.abs(repeated - previous))), "epoch_losses_identical": True, "scope": "One AAPL price-change LSTM
repeat on same hardware/software; not a multi-seed robustness study"})
    print("Repeated LSTM: all epoch losses and 271 predictions bitwise identical")
```

تكتب إيصال المقارنة وتطبع النتيجة؛ النطاق تجربة AAPL واحدة على بيئة محددة، لا برهان استقرار متعدد البذور.

الأسطر 50–51

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ يستدعي main.

شرح `scripts/run_full.py`

بصمة المصدر SHA-256:

`92057d2f60510454413260b8774ac73016f435ce3f25dfebdd3fd59c71460be6`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الأسطر 1–4

```
import os
from pathlib import Path
import subprocess
import sys
```

os وPath وsubprocess وsys قياسية لاختيار الجذر وتشغيل عملية فرعية.

الأسطر 6–9

```
ROOT = Path(__file__).resolve().parents[1]
os.environ["PYTHONHASHSEED"] = "42"
os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
subprocess.run([sys.executable, "-m", "xai_finance.cli", "all", "--config", "configs/full_experiment.yaml"],
cwd=ROOT, check=True)
```

يعين جذر المشروع وبذرة الهاش وإعداد CUBLAS ويشغل CLI all مع full_experiment.yaml؛ check=True يكشف فشل العملية. تنفيذ السكربت يجري سير التجربة الكامل ولم يحدث هنا.

شرح `scripts/run_quick.py`

بصمة المصدر SHA-256:

`363667f99ecbd1269c19f654f69fa87971cd41d6c4053cd7d17fa0a6fc30fdca`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الأسطر 1–4

```
import os
from pathlib import Path
import subprocess
import sys
```

os وPath وsubprocess وsys مكتبات قياسية للمسارات وبيئة وتشغيل عملية Python.

الأسطر 6–9

```
ROOT = Path(__file__).resolve().parents[1]
os.environ["PYTHONHASHSEED"] = "42"
os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
subprocess.run([sys.executable, "-m", "xai_finance.cli", "all", "--config", "configs/quick_demo.yaml"],
cwd=ROOT, check=True)
```

يستخرج الجذر من موقع السكربت ويعين PYTHONHASHSEED وCUBLAS لعملية الابن ثم يشغل CLI all بتهيئة quick_demo وفي الجذر؛ check=True يرفع خطأ عند فشل الأمر. هذه نقطة دخول ذات آثار تنزيل وتدريب وتفسير وتقارير ولم تشغل هنا.

شرح scripts/run_scientific_validation.py

بصمة المصدر SHA-256:

.9d8bd2280e9ea79bc236f77c96d0db1ef051b5c6959e8fa8ada72518f76e3741

الاقتباسات من الملف؛ لم تُشغّل السكربتات أو تجاربها.

الأسطر 1–8

```
"""Run additive validation without touching the original run or defense bundle."""
import argparse
from pathlib import Path

from xai_finance.config import ROOT
from xai_finance.storage import read_json
from xai_finance.validation.audit import audit_original
from xai_finance.validation.experiments import aggregate_experiments, run_experiments
```

التوثيق يصف تحققاً إضافياً؛ argparse وPath قياسيتان، والبقية وظائف داخلية للقراءة والتدقيق وتشغيل وتجميع التجارب.

الأسطر 11–18

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("folder", type=Path)
    parser.add_argument("action", choices=["audit", "train", "aggregate"])
    arguments = parser.parse_args()
    destination = arguments.folder.resolve()
    protocol = read_json(destination / "protocol.json")
    original = ROOT / "artifacts/runs" / protocol["original_run"]
```

main تحلل مجلد الوجهة والعمل audit أو train أو aggregate، ثم تقرأ البروتوكول وتحدد التشغيل الأصلي المشار إليه.

الأسطر 19–25

```
    if arguments.action == "audit":
        audit_original(original, destination)
    elif arguments.action == "train":
        run_experiments({"protocol": protocol, "destination": destination, "original": original})
        aggregate_experiments(destination)
    else:
        aggregate_experiments(destination)
```

audit يستدعي فحص الأصل؛ train يشغل التجارب ثم يجمعها؛ aggregate يجمع فقط. هذه الأوامر تحسب أو تدرب وتكتب في الوجهة؛ لم تشغل أثناء التوثيق.

الأسطر 28–29

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ المباشر يستدعي main.

شرح `scripts/setup_offline.py`

بصمة المصدر SHA-256:

`955a5ec1562e5e2e48d10c6cfbf9b126ea754cd4177b995c81c4e29e86156148`.

وصف ثابت؛ لم تُشغّل السكربتات.

الأسطر 1–5

```
"""Compatibility command for the CPU-only offline setup wizard."""

from pathlib import Path
import subprocess
import sys
```

سلسلة توثيق ومكتبات قياسية للمسار والعملية والوسائط.

الأسطر 7–8

```
ROOT = Path(__file__).resolve().parents[1]
raise SystemExit(subprocess.call([sys.executable, str(ROOT / "scripts/demo_wizard.py"), "setup", "--offline",
*sys.argv[1:]], cwd=ROOT))
```

يشغّل wizard بأمر setup و--offline ووسائط المستخدم من الجذر، ويمرر رمز الخروج. قد يثبت حزم محلية؛ لم يشغل هنا.

شرح scripts/start_scientific_validation.py

بصمة المصدر SHA-256:

.b9a5b7a82b8ba6c25aa1dabba2d18bfa24ec8491df5b19bd17d55bca21903924

الاقتباسات من الملف؛ لم تُشغّل السكربتات أو تجاربها.

الأسطر 1–6

```
"""Freeze an inventory and protocol before running additional experiments."""
from datetime import datetime, timezone
from pathlib import Path

from xai_finance.config import ROOT
from xai_finance.storage import sha256, write_json
```

يوثق تجميد البروتوكول، ويستورد الوقت والمسارات والبصمات والكتابة.

الأسطر 9–11

```
def protected_files() -> list[Path]:
    excluded = {".venv", ".offline_verify", ".idea", ".pytest_cache", ".ruff_cache", "__pycache__", "build",
"provider_cache", "scientific_validation"}
    return sorted(path for path in ROOT.rglob("*") if path.is_file() and not
excluded.intersection(path.relative_to(ROOT).parts))
```

protected_files تحصر ملفات الجذر باستثناء المسارات التي تحتوي أسماء المجلدات المستبعدة. القائمة الثابتة لا تضمن استبعاد كل بيئة أو ملف حساس في كل نشر؛ تتحقق من الأسماء ضمن مكونات المسار.

الأسطر 14–19

```
def main() -> None:
    inventory = {str(path.relative_to(ROOT)): {"bytes": path.stat().st_size, "sha256": sha256(path)} for path in
protected_files()}
    run_id = "validation_" + datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S%fZ")
    folder = ROOT / "artifacts/scientific_validation" / run_id
    folder.mkdir(parents=True, exist_ok=False)
    write_json(folder / "preservation_before.json", inventory)
```

main تحصر حجم وبصمة الملفات المحمية وتحدد مجلد تحقق جديداً باسم وقت UTC وتكتبه؛ exist_ok=False يمنع استعمال مجلد موجود.

الأسطر 20–37

```
    protocol = {
        "validation_id": run_id, "original_run": "full_20261021T0409010846472",
        "created_utc": datetime.now(timezone.utc).isoformat(), "seed": 42,
        "status": "EXPLORATORY: original test results already observed; no new independent holdout",
        "fixed_split": {"price_change": ["AAPL", "SP500", "NASDAQ", "GOLD", "OIL"], "simple_return": ["AAPL",
"SP500", "NASDAQ", "GOLD"]},
        "return_exclusion": "OIL has a negative historical price; price_change retains every oil observation; no
return filtering or clipping",
        "features": "Exact frozen 30x20 windows; target-only controlled comparison; no feature search",
        "targets": {"price_change": "P[t+1]-P[t]", "simple_return": "P[t+1]/P[t]-1"},
        "reconstruction": {"price_change": "P[t]+predicted_change", "simple_return": "P[t]*
(1+predicted_return)"},
        "models": ["LinearRegression", "RandomForest", "GradientBoosting", "LSTM"],
        "tabular_search": "Same two RF and GBM candidates as frozen run; validation target RMSE; training-only
fits; no final refit",
        "lstm": {"epochs": 60, "patience": 8, "hidden_size": 32, "dropout": 0.1, "learning_rate": 0.001,
"batch_candidates": [32, 64], "amp": True, "device": "auto", "effective_lookback": 30},
        "baselines": ["Zero", "TrainMean", "LastObservedChangeOrReturn",
"TrainMajorityDirectionMedianMagnitude"],
        "walk_forward": {"assets": ["AAPL", "SP500"], "target": "price_change", "folds": 3, "test": "Three
contiguous disjoint blocks covering original test period", "fit": "Expanding history; latest 270 pre-block
observations for validation; all earlier observations for training; no test fitted preprocessing",
"lstm_epochs": 30},
        "direction": "Three-way sign accuracy, nonzero prediction coverage and conditional hit rate; binary-up
accuracy also reported with zero treated as not-up",
        "uncertainty": "Paired circular block bootstrap of price squared-error difference versus Zero, block
length 10, 2000 draws, seeded; exploratory unadjusted 95% intervals",
        "restrictions": "No tuning after seeing new test results; no edits to original run, snapshots, source,
checkpoints or offline package",
    }
```

يبني قاموس البروتوكول الاستكشافي: الأصل المجمد والبذرة والأهداف والأصول ونوافذ 30×20 والنماذج والإعدادات والمراجع وثلاث طيات وحدود عدم اليقين والقيود. النفط مستبعد من العائد لا من فرق السعر لوجود سعر سالب؛ بيانات الاختبار الأصلية معروفة، فلا يمثل هذا اختباراً تأكيدياً مستقلاً.

الأسطر 38–44

```
    write_json(folder / "protocol.json", protocol)
    print(folder)
    print(f"Protected files: {len(inventory)}; protocol SHA256: {sha256(folder / 'protocol.json')}")


if __name__ == "__main__":
    main()
```

يكتب protocol.json ويطبع المسار والعدد وبصمة البروتوكول؛ حارس **main** يستدعي التنفيذ المباشر. لم يشغل هذا السكربت في التوثيق.

شرح scripts/test_deployment_cpu.py

بصمة المصدر SHA-256:

.8977f5c0b9b2b5fd3700e68e8241b96001df6f873845ead021085ff3c51c2693

لم يُنفذ اختبار النشر أو المتصفح في هذه الجلسة.

الأسطر 1–12

```
"""Execute Windows batch launch, duplicate/restart/occupied-port/offline checks."""

import argparse
import importlib.util
import json
import os
from pathlib import Path
import socket
import subprocess
from urllib.request import urlopen

ROOT = Path(__file__).resolve().parents[1]
```

توثيق واستخدام argparse وimportlib وJSON والبيئة والمسار وsocket والعمليات وHTTP المحلي.

الأسطر 15–24

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--root", type=Path, default=ROOT)
    parser.add_argument("--port", type=int, default=8561)
    parser.add_argument("--setup", action="store_true")
    args = parser.parse_args()
    root = args.root.resolve()
    env = {**os.environ, "XAI_DEMO_NO_PAUSE": "1", "XAI_TEST_OFFLINE": "1", "PYTHONPATH": str(root /
"scripts/offline_guard"), "PYTHONUTF8": "1"}
    output = root / ".demo_runtime/acceptance"
    output.mkdir(parents=True, exist_ok=True)
```

main تقرأ الجذر والمنفذ وخيار setup وتجهز بيئة اختبار دون pause وحارس شبكة Python، وتنشئ مجلد acceptance.

الأسطر 26–34

```
    def batch(name, *options, check=True):
        command = ["cmd.exe", "/d", "/c", str(root / name), *options]
        result = subprocess.run(command, cwd=root, env=env, text=True, encoding="utf-8", errors="replace",
stdout=subprocess.PIPE, stderr=subprocess.STDOUT, timeout=900)
        print(result.stdout, flush=True)
        with (output / "batch.log").open("a", encoding="utf-8") as stream:
            stream.write(result.stdout)
        if check:
```

```
            assert result.returncode == 0, result.stdout
        return result
```

الدالة الداخلية batch تشغل ملف Windows عبر cmd بمهلة 900 ثانية وتسجل خرجه وتؤكد نجاحه إن طلب check، وتعيد النتيجة.

الأسطر 36–41

```
    if args.setup:
        batch("SETUP_DEMO.bat", "--offline")
    python = root / ".demo_venv/Scripts/python.exe"
    guard = subprocess.run([str(python), "-c", "import socket;
socket.create_connection(('example.com',443),timeout=2)"], env=env, capture_output=True, text=True)
    assert guard.returncode != 0 and "Offline acceptance test denied" in guard.stderr
    receipt = {"status": "PASSED", "root": str(root), "offline_setup_executed": args.setup,
"external_python_network_guard_verified": True}
```

عند طلب setup تنفذ التثبيت offline؛ تختبر أن اتصال Python خارجي محظور وتبدأ الإيصال. يتطلب Python البيئة الموجودة.

الأسطر 42–54

```
    with socket.socket() as occupant:
        occupant.bind(("127.0.0.1", args.port))
        occupant.listen()
        first = batch("START_DEMO.bat", "--no-browser", "--port", str(args.port))
        assert "occupied" in first.stdout
        state = json.loads((root / ".demo_runtime/server.json").read_text())
        assert state["port"] == args.port + 1
        with urlopen(state["url"] + "/_stcore/health") as response:
            assert response.read() == b"ok"
        batch("START_DEMO.bat", "--no-browser", "--port", str(args.port))
        assert json.loads((root / ".demo_runtime/server.json").read_text())["pid"] == state["pid"]
        receipt["occupied_port"] = {"occupied": args.port, "selected": state["port"],
"existing_service_preserved": occupant.getsockname()[1] == args.port}
        receipt["duplicate_start_reused_pid"] = state["pid"]
```

تحجز المنفذ، وتطلق العرض في منفذ تالٍ وتفحص الصحة وإعادة استخدام PID عند البدء المكرر، وتوثق بقاء خدمة المنفذ المشغول.

الأسطر 55–66

```
        # Exercise the default browser-opening branch using a recorded callback.
        spec = importlib.util.spec_from_file_location("tested_wizard", root / "scripts/demo_wizard.py")
        wizard = importlib.util.module_from_spec(spec)
        spec.loader.exec_module(wizard)
        opened = []
        wizard.webbrowser.open = lambda url: opened.append(url)
        _, reused = wizard.start(args.port)
        assert reused and opened == [state["url"]]
        receipt["browser_open_callback"] = opened
        invalid = batch("START_DEMO.bat", "--no-browser", "--port", "80", check=False)
        assert invalid.returncode != 0 and "between 1024 and 65535" in invalid.stdout
        receipt["invalid_port_actionable_error"] = True
```

تحمل wizard ديناميكياً وتستبدل webbrowser.open برد مسجل لاختبار فرع فتح المتصفح ثم تفحص رفض منفذ 80.

الأسطر 67–78

```
        appcheck = subprocess.run([str(python), str(root / "scripts/check_dashboard_cpu.py"), "--output",
str(output / "dashboard.json")], cwd=root, env=env, text=True, encoding="utf-8", errors="replace",
stdout=subprocess.PIPE, stderr=subprocess.STDOUT, timeout=300)
        print(appcheck.stdout, flush=True)
        assert appcheck.returncode == 0
        batch("START_DEMO.bat", "stop")
        batch("START_DEMO.bat", "--no-browser", "--port", str(args.port))
        restarted = json.loads((root / ".demo_runtime/server.json").read_text())
        assert restarted["pid"] != state["pid"]
        receipt["restart_new_pid"] = restarted["pid"]
        # Leave the tested server running for the subsequent browser smoke.
        receipt["running_url"] = restarted["url"]
    (output / "deployment.json").write_text(json.dumps(receipt, indent=2), encoding="utf-8")
    print("DEPLOYMENT ACCEPTANCE PASSED:", receipt["running_url"], flush=True)
```

تشغل اختبار الواجهة وتوقف الخادم وتعيد تشغيله وتتحقق من تغير PID وتترك الخادم للاختبار التالي، ثم تكتب الإيصال. التنفيذ يبدأ ويوقف عمليات ويكتب آثاراً؛ لم ينفذ هنا.

الأسطر 81–82

```
if __name__ == "__main__":
    main()
```

حارس التشغيل يستدعي main.

شرح scripts/verify_artifacts.py

بصمة المصدر SHA-256:

.a5debefd13f188f1fcc126b3b69427a7e98b275242a004e18a6c781e624392ff

لم تُنفذ إعادة التشغيل أو تدقيق النتائج في هذه الجلسة.

الأسطر 1–14

```
"""Independent original-unit and source-date audit of the current full experiment."""

from pathlib import Path

import numpy as np
import pandas as pd

from xai_finance.config import ROOT, load_config
from xai_finance.io.data_sources import load_snapshot
from xai_finance.pipelines.report import verify_saved_metrics
from xai_finance.pipelines.train import load_models, load_windows, resolve_run
from xai_finance.storage import read_json, sha256, write_json

MODELS = ["LinearRegression", "RandomForest", "GradientBoosting", "LSTM", "Persistence"]
```

توثيق واستيرادات المسار وNumPy وpandas ودوال الإعداد واللقطات والتقارير وتحميل النماذج والتخزين؛ MODELS تشمل أربعة نماذج وPersistence.

الأسطر 17–24

```
def verify_asset(folder: Path) -> dict:
    prices, manifest = load_snapshot(folder.name)
    windows = load_windows(folder)
    test = windows.partitions["test"]
    predictions = pd.read_csv(folder / "predictions.csv", parse_dates=["feature_cutoff", "target_date"])
    assert set(predictions.model) == set(MODELS)
    verify_saved_metrics(predictions, read_json(folder / "metrics.json"))
    labels = prices["adj_close"] if manifest["frequency"] == "daily" else prices["close"]
```

verify_asset تحمل اللقطة المحققة والنوافذ والتنبؤات وتتحقق من أسماء النماذج ومقاييسها وتختار الإغلاق المعدل لليومي والإغلاق للشهري.

الأسطر 25–30

```
    for name in MODELS:
        subset = predictions[predictions.model == name]
        assert pd.DatetimeIndex(subset.target_date).equals(windows.targets[test])
        assert (subset.feature_cutoff < subset.target_date).all()
        np.testing.assert_allclose(subset.actual, labels.loc[subset.target_date], atol=1e-8)
        np.testing.assert_allclose(subset.current_close, labels.loc[subset.feature_cutoff], atol=1e-8)
```

لكل نموذج تفحص تطابق تواريخ الأهداف مع الاختبار وترتيب الأصل قبل الهدف ومطابقة القيم الحالية واللاحقة مع اللقطة؛ لا يثبت ذلك تواريخ إتاحة البيانات المنقحة.

الأسطر 31–39

```
    models = load_models(folder, "cpu")
    for name, model in models.items():
        expected = predictions.loc[predictions.model == name, "predicted"].to_numpy()
        np.testing.assert_allclose(model.predict(windows.flat[test]), expected, rtol=1e-5, atol=1e-4)
        verify_explanation(folder / name / "explanations", windows, model)
    cutoff = models["LSTM"].scalers.fitted_target_cutoff
    assert cutoff == str(windows.targets[windows.partitions["train"][-1]].date())
    assert cutoff < str(windows.targets[test[0]].date())
    return {"asset": folder.name, "rows": manifest["rows"], "test_samples_per_model": len(test), "models":
len(MODELS), "explanation_models": len(models), "status":
"VERIFIED_FROM_ORIGINAL_SNAPSHOT_AND_RELOADED_CPU_MODELS"}
```

تعيد تحميل أربعة نماذج على CPU وتقارن تنبؤاتها بالمحفوظ ضمن tolerances وتفحص تفسيراتها، ثم تتحقق من تاريخ آخر هدف لتقييس LSTM وتعيد ملخص الأصل.

الأسطر 42–50

```
def verify_explanation(folder: Path, windows, model) -> None:
    quality = read_json(folder / "quality.json")
    assert quality["prediction_unchanged_max_abs"] == 0
    assert max(quality["background_indices"]) <= windows.partitions["train"][-1]
    with np.load(folder / "shap.npz", allow_pickle=False) as saved:
        predictions = model.predict(saved["observations"])
        np.testing.assert_allclose(np.asarray(saved["base_values"]).ravel() + saved["values"].sum(axis=1),
predictions, rtol=1e-4, atol=1e-3)
        for index in saved["indices"]:
            assert index in windows.partitions["test"]
```

verify_explanation تقرأ جودة التفسير وتؤكد عدم تغير التنبؤ وأن الخلفية في التدريب، وتعيد بناء تنبؤ SHAP من base ومجموع القيم، وتتحقق من أن كل فهرس اختبار.

الأسطر 51–56

```
            local = read_json(folder / f"lime_{index}.json")
            assert local["target_date"] == str(windows.targets[index].date())
            assert local["feature_cutoff"] == str(windows.origins[index].date())
            assert np.isclose(local["absolute_local_error"], abs(local["model_prediction"] -
local["local_surrogate_prediction"]))
    for name in ["shap_summary.png", "shap_waterfall.svg", "shap_dependence.png", "shap_force.html",
"lime_local.png", "feature_time_heatmap.png", "global_importance.csv", "regime_drivers.csv"]:
        assert (folder / name).stat().st_size > 0
```

تقرأ LIME لنفس الفهرس وتتحقق من تواريخ الهدف والأصل وصيغة الخطأ المطلق، ثم من أحجام الرسوم؛ لا تختبر أن خطأ LIME صغير أو أن التفسير سببي.

الأسطر 59–71

```
def main() -> None:
    folder = resolve_run(load_config(ROOT / "configs/full_experiment.yaml"))
    metadata = read_json(folder / "run.json")
    assert metadata["source_sha256"] == sha256(ROOT / "docs/original_thesis.docx")
    assert metadata["provenance"] == "NEWLY_COMPUTED"
    assert read_json(ROOT / "docs/thesis_reported_reference.json")["provenance"] ==
"THESIS_REPORTED_NOT_REPRODUCED"
    for relative, expected in read_json(folder / "inventory.json").items():
        assert sha256(folder / relative) == expected, relative
    assets = [verify_asset(folder / asset) for asset in metadata["configuration"]["assets"]]
    write_json(folder / "verification.json", {"run_id": folder.name, "assets": assets, "status": "PASSED",
"checks": "snapshot hashes, original dates/targets, metrics recomputation, common rows, CPU model reload, scaler
cutoff, SHAP reconstruction/background, paired LIME dates, real chart files"})
    print("VERIFIED", folder.name)
    for audit in assets:
        print(audit)
```

main تحدد تشغيل full وتتحقق من بصمة الرسالة ومنشأ النتائج والمرجع التاريخي وبصمات الفهرس، ثم تدقق كل أصل وتكتب verification.json وتطبع. لم يُنفذ هنا؛ كتابته تغير ملف أثر.

الأسطر 74–75

```
if __name__ == "__main__":
    main()
```

حارس التشغيل المباشر.

شرح scripts/verify_cpu_replay.py

بصمة المصدر SHA-256:

.c6e074eff31470057e4f6fe73801aa4ceed9453b236686c9538a5dbd6e531578

لم تُنفذ إعادة التشغيل أو تدقيق النتائج في هذه الجلسة.

الأسطر 1–15

```
"""Read-only CPU replay and fresh explanation probes; never rewrite research evidence."""

import argparse
import importlib
import json
import os
from pathlib import Path
import pkgutil
import socket
import sys

ROOT = Path(__file__).resolve().parents[1]
os.environ.setdefault("MPLCONFIGDIR", str(ROOT / ".demo_runtime/matplotlib"))
sys.path.insert(0, str(ROOT / "src"))
sys.path.insert(0, str(ROOT / "scripts"))
```

توثيق واستيرادات قياسية وتعريف الجذر وMPLCONFIGDIR ومساري src وscripts على sys.path؛ قد ينشئ matplotlib cache عند الاستيراد.

الأسطر 18–27

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--output", type=Path, default=ROOT / "artifacts/deployment/cpu_replay.json")
    args = parser.parse_args()
    original_connect = socket.socket.connect

    def connect(connection, address):
        if isinstance(address, tuple) and address[0] not in {"127.0.0.1", "localhost", "::1"}:
            raise RuntimeError(f"External connection denied: {address}")
        return original_connect(connection, address)
```

main تقرأ مسار إيصال افتراضي وتحتفظ بـ socket.connect ثم تعرف غلافاً يرفض الاتصال الخارجي ذي عنوان tuple.

الأسطر 29–40

```
    socket.socket.connect = connect
    import numpy as np
    import torch
    import xai_finance
    from threadpoolctl import threadpool_limits
```

```
    from verify_artifacts import verify_asset
    from verify_scientific_validation import replay_job, verify_aggregate
    from xai_finance.explain.lime_explain import explain_lime
    from xai_finance.explain.shap_explain import explain_shap, shap_reconstruction
    from xai_finance.pipelines.train import load_models, load_windows
    from xai_finance.seed import seed_experiment
    from xai_finance.storage import read_json, sha256
```

تستبدل connect وتستورد NumPy وTorch والحزمة وأدوات التحقق والتفسير والتحميل والبذور. هذا الحجب يقتصر على المسار المستبدل ولا يثبت منع كل أشكال الشبكة.

الأسطر 42–53

```
    assert torch.__version__ == "2.5.1+cpu", torch.__version__
    assert torch.version.cuda is None
    assert not torch.cuda.is_available()
    seed_experiment(42)
    torch.set_num_threads(4)
    imports = []
    for module in pkgutil.walk_packages(xai_finance.__path__, "xai_finance."):
        importlib.import_module(module.name)
        imports.append(module.name)
    run = ROOT / "artifacts/runs/full_20261021T040901084647Z"
    for relative, digest in read_json(run / "inventory.json").items():
        assert sha256(run / relative) == digest, relative
```

تؤكد نسخة torch CPU وغياب CUDA وتضبط البذرة والخيوط، وتستورد وحدات الحزمة وتسجلها، ثم تتحقق من بصمات تشغيل full.

الأسطر 54–65

```
    assets = []
    probes = []
    with threadpool_limits(limits=4):
        for asset in read_json(run / "run.json")["assets_completed"]:
            assets.append(verify_asset(run / asset))
            print(f"CPU REPLAY PASS: {asset}: four models, metrics, SHAP/LIME", flush=True)
        windows = load_windows(run / "AAPL")
        models = load_models(run / "AAPL", "cpu")
        assert models["LSTM"].device.type == "cpu"
        training = windows.flat[windows.partitions["train"]]
        background = training[np.linspace(0, len(training)-1, 4, dtype=int)]
        observation = windows.flat[windows.partitions["test"][:1]]
```

تحت حد الخيوط تفحص كل أصل ثم تحمل نوافذ ونماذج AAPL على CPU، وتختار أربع خلفيات تدريبية وعينة اختبار واحدة لفحص توافق محدود.

الأسطر 66–74

```
        for name, model in models.items():
            before = model.predict(observation).copy()
            explanation = explain_shap(name, model, background, {"observations": observation, "names":
windows.flat_names, "seed": 42, "cycles": 1})
            shap_audit = shap_reconstruction(explanation, before)
            _, lime_audit = explain_lime(training, model.predict, observation[0], {"names": windows.flat_names,
"seed": 42, "samples": 100})
```

```
            np.testing.assert_array_equal(before, model.predict(observation))
            assert np.isfinite(lime_audit["weighted_neighborhood_r2"])
            probes.append({"asset": "AAPL", "model": name, "shap": shap_audit, "lime": lime_audit, "scope":
"fresh bounded compatibility probe on existing model; not replacement research evidence"})
            print(f"FRESH CPU SHAP/LIME PASS: {name}", flush=True)
```

لكل نموذج تحسب تنبؤاً وتفسير SHAP وLIME جديداً بميزانية صغيرة، وتتحقق من جمع SHAP وثبات التنبؤ ونهائية R^2 وتحفظ نتائج probe في الذاكرة؛ هذا حساب تفسير جديد لا تدريب ولا بديل عن الآثار الأصلية.

الأسطر 75–86

```
        validation = ROOT / "artifacts/scientific_validation/validation_20261021T055002439194Z"
        jobs = sorted((validation / "experiments").rglob("completion.json"))
        assert len(jobs) == 15
        replay = []
        for job in jobs:
            replay.append(replay_job(job.parent, run))
            print(f"VALIDATION CPU REPLAY PASS: {job.parent.relative_to(validation)}", flush=True)
        aggregate = verify_aggregate(validation)
    receipt = {"status": "PASSED", "torch": torch.__version__, "cuda_available": False,
"external_python_connections": "denied", "modules_imported": imports, "assets": assets,
"fresh_explanation_probes": probes, "scientific_validation_jobs": replay, "scientific_validation_aggregate":
aggregate}
    destination = args.output
    destination.parent.mkdir(parents=True, exist_ok=True)
    destination.write_text(json.dumps(receipt, indent=2), encoding="utf-8")
```

تجد 15 مهمة تحقق وتعيد تشغيل تنبؤاتها ومقاييسها، ثم تكتب إيصالاً في الوجهة؛ الافتراضي تحت artifacts/deployment، لذلك الوصف read-only يتعلق بالأدلة الأصلية لا بغياب أي كتابة.

الأسطر 89–90

```
if __name__ == "__main__":
    main()
```

حارس التشغيل؛ لم ينفذ في جلسة التوثيق.

شرح `scripts/verify_environment.py`

بصمة المصدر SHA-256:

`2af3643f81dccec4e566397fcb79b59f5f412c29fcd6c8dee05482bbef24b132`.

وصف ثابت؛ لم تُشغّل السكربتات.

الأسطر 1–1

```
from xai_finance.environment import verify_environment
```

يستورد تشخيص البيئة من الحزمة.

الأسطر 3–3

```
verify_environment()
```

يستدعي التشخيص حتى عند استيراد السكربت؛ يختبر LSTM عشوائياً ويكتب تقرير البيئة كما في الدالة، وليس نتائج مالية.

شرح scripts/verify_preservation_cpu.py

بصمة المصدر SHA-256:

.e5e9a23eec4d3bd30edd2548337ed4386f87d04483d598c9bb8366ede1ed9a51

وصف ثابت؛ لم تُشغّل السكربتات.

الأسطر 1–8

```
"""Confirm every original research evidence file matches the pre-repair baseline."""

import argparse
import hashlib
import json
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
```

التوثيق يصف مقارنة الأدلة بمرجع؛ argparse وhashlib وjson وPath قياسية، وROOT أصل المشروع.

الأسطر 11–13

```
def digest(path):
    with path.open("rb") as stream:
        return hashlib.file_digest(stream, "sha256").hexdigest()
```

digest تقرأ ملفاً ثنائياً وتحسب SHA-256؛ تعيد بصمة النص الست عشري.

الأسطر 16–23

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--output", type=Path, default=ROOT / "artifacts/deployment/preservation.json")
    args = parser.parse_args()
    baseline = ROOT / "artifacts/deployment/original_evidence_sha256.json"
    entries = json.loads(baseline.read_text(encoding="utf-8"))
    mismatches = [relative for relative, expected in entries.items() if not (ROOT / relative).is_file() or
digest(ROOT / relative) != expected]
    assert not mismatches, mismatches
```

main تقرأ خيار output الافتراضي وملف baseline ذي البصمات، وتجمع الملفات الناقصة أو المختلفة وتفرض عدم وجود مخالفات. يعتمد الاستنتاج على شمول وموثوقية baseline؛ assert يمكن تعطيله بوضع Python المحسن.

الأسطر 24–27

```
    receipt = {"status": "PASSED", "original_evidence_files": len(entries), "mismatches": mismatches,
"baseline_sha256": digest(baseline), "protected_run": "full_20261021T0409010846472", "protected_validation":
"validation_20261021T055002439194Z"}
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(receipt, indent=2), encoding="utf-8")
    print(f"PRESERVATION PASS: {len(entries)} original evidence files unchanged")
```

تكتب إيصال نجاح يتضمن عدد الأدلة وبصمة baseline وأسماء التشغيلات المحمية، وتنشئ مجلد الوجهة وتطبع النجاح؛ هذا الفحص يكتب ملفاً افتراضياً داخل artifacts إذا شُغّل ولم يشغل هنا.

الأسطر 30–31

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ يستدعي main.

شرح scripts/verify_scientific_validation.py

بصمة المصدر SHA-256:

`3e2f23a91b3db309dfaef515c7df378810965f9069d868c9e894abbd97eca36f`.

لم تُنفذ إعادة التشغيل أو تدقيق النتائج في هذه الجلسة.

الأسطر 1–15

```
"""Read-only replay of every additional prediction plus frozen-file preservation."""
import argparse
from dataclasses import replace
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd

from xai_finance.config import ROOT
from xai_finance.pipelines.train import load_models, load_windows
from xai_finance.seed import seed_experiment
from xai_finance.storage import read_json, sha256, write_json
from xai_finance.validation.design import implied_prices, naive_predictions, target_values
from xai_finance.validation.statistics import evaluation_metrics
```

توثيق واستيرادات argparse وreplace والوقت والمسارات وNumPy وpandas ووظائف المشروع للتحميل والتنبؤ والتقييم والهاش.

الأسطر 18–28

```
def replay_job(folder: Path, original: Path) -> dict:
    design = read_json(folder / "design.json")
    windows = load_windows(original / design["asset"])
    with np.load(folder / "target_and_partitions.npz", allow_pickle=False) as saved:
        windows = replace(windows, partitions={name: saved[name].copy() for name in ["train", "validation",
"test"]})
        np.testing.assert_array_equal(saved["y"], target_values(windows, design["target"]))
    table = pd.read_csv(folder / "predictions.csv", float_precision="round_trip")
    test = windows.partitions["test"]
    models = load_models(folder, "cpu")
    expected = {name: model.predict(windows.flat[test]) for name, model in models.items()}
    expected.update(naive_predictions(windows, design["target"]))
```

replay_job تحمل التصميم والنوافذ وتستبدل التقسيمات من NPZ دون pickle، وتفحص الهدف المحول وتقرأ الجدول وتحمل نماذج CPU وتحسب توقعات النماذج والمراجع.

الأسطر 29–39

```
    maximum_error = {}
    for name, group in table.groupby("model", sort=False):
        np.testing.assert_array_equal(pd.to_datetime(group.target_date), windows.targets[test])
        np.testing.assert_array_equal(pd.to_datetime(group.feature_cutoff), windows.origins[test])
        np.testing.assert_allclose(group.actual_price, windows.y[test], rtol=1e-12)
        if name == "LSTM":
            # CPU and CUDA accumulate FP32 recurrent operations differently; compare in training standard-
deviation units.
            absolute_tolerance = 8 * np.finfo(np.float32).eps * models[name].scalers.target.scale_[0]
            np.testing.assert_allclose(group.predicted_target, expected[name], rtol=0, atol=absolute_tolerance)
        else:
            np.testing.assert_allclose(group.predicted_target, expected[name], rtol=1e-9, atol=1e-9)
```

لكل نموذج تتحقق من تواريخ الهدف والأصل والقيم الفعلية، وتقارن الهدف المتوقع: LSTM بسماح مطلق يساوي ثمانية أضعاف epsilon لـ float32 مضروباً في مقياس هدف التدريب، والآخرون بسماح 1e-9. التسامح يعالج فروق الأجهزة لا يضمن تماثل كل البيئات.

الأسطر 40–46

```
        native_target = group.predicted_target.to_numpy().astype(expected[name].dtype)
        np.testing.assert_allclose(group.predicted_price, implied_prices(native_target,
group.current_price.to_numpy(), design["target"]), rtol=1e-12)
        maximum_error[name] = float(np.max(np.abs(group.predicted_target - expected[name])))
    completion = read_json(folder / "completion.json")
    for name, digest in completion["files"].items():
        assert sha256(folder / name) == digest
    return {"job": str(folder.relative_to(original.parent.parent.parent)), "rows_replayed": len(table),
"cpu_target_prediction_max_abs_error": maximum_error}
```

تحول التوقع المحفوظ إلى dtype التنبؤ الأصلي قبل إعادة تركيب السعر لتكرار دقة الحساب، وتحسب أقصى اختلاف وتتحقق من بصمات completion وتعيد ملخص المهمة.

الأسطر 49–63

```
def verify_aggregate(destination: Path) -> dict:
    table = pd.read_csv(destination / "new_predictions.csv")
    metrics = read_json(destination / "new_metrics.json")
    for row in metrics:
        subset = table
        for key in ["design", "target", "asset", "model"]:
            subset = subset[subset[key] == row[key]]
        assert not subset.target_date.duplicated().any()
        recomputed = evaluation_metrics(subset.sort_values("target_date"))
        for name, value in recomputed.items():
            if isinstance(value, float):
                np.testing.assert_allclose(value, row[name], rtol=1e-8, atol=1e-8)
            else:
                assert value == row[name]
    return {"aggregate_rows": len(table), "aggregate_metric_rows": len(metrics),
"all_metrics_recomputed_from_serialized_predictions": True}
```

verify_aggregate تقرأ التنبؤات والمقاييس المجمعة، وترشح لكل مجموعة وتمنع تكرار تاريخ الهدف ثم تعيد حساب evaluation_metrics وتقارن جميع النتائج، وتعيد عدد الصفوف. bootstrap ضمن

evaluation_metrics يعاد حسابه عند التشغيل.

الأسطر 66–74

```
def preservation_receipt(destination: Path) -> dict:
    inventory = read_json(destination / "preservation_before.json")
    mismatches = []
    for relative, previous in inventory.items():
        path = ROOT / relative
        if not path.exists() or path.stat().st_size != previous["bytes"] or sha256(path) != previous["sha256"]:
            mismatches.append(relative)
    assert not mismatches, f"Protected files changed: {mismatches}"
    return {"protected_files": len(inventory), "mismatches": mismatches, "before_inventory_sha256":
sha256(destination / "preservation_before.json"), "offline_zip_sha256": sha256(ROOT /
"backups/defense_full_20261021T040901084647Z.zip")}
```

preservation_receipt تقارن حجم وبصمة الملفات مع سجل ما قبل التحقق وتفرض عدم تغيرها، وتعيد العدد وبصمتي السجل وZIP؛ تعتمد على شمول السجل وصحته.

الأسطر 77–89

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("folder", type=Path)
    arguments = parser.parse_args()
    destination = arguments.folder.resolve()
    seed_experiment(42)
    original = ROOT / "artifacts/runs" / read_json(destination / "protocol.json")["original_run"]
    jobs = sorted((destination / "experiments").rglob("completion.json"))
    assert len(jobs) == 15, f"Expected 15 executed jobs; found {len(jobs)}"
    replay = [replay_job(path.parent, original) for path in jobs]
    receipt = {"verified_utc": datetime.now(timezone.utc).isoformat(), "jobs": replay,
"cpu_lstm_tolerance_training_std_units": float(8 * np.finfo(np.float32).eps), **verify_aggregate(destination),
"preservation": preservation_receipt(destination)}
    write_json(destination / "verification.json", receipt)
    print(f"Verified {len(replay)} jobs, {receipt['aggregate_rows']} serialized predictions and
{receipt['preservation']['protected_files']} unchanged protected files")
```

main تقرأ الوجهة وتضبط البذرة وتحدد الأصل وتؤكد وجود 15 مهمة، ثم تعيد تشغيلها وتدقق التجميع والمحافظة وتكتب verification.json وتطبع. ليست قراءة خالصة لأن الإيصال يكتب؛ لم يُنفذ هنا.

الأسطر 92–93

```
if __name__ == "__main__":
    main()
```

حارس التنفيذ المباشر.

شرح SETUP_DEMO.bat

بصمة المصدر SHA-256:

`19ef858e28246181b180df9b96ccb071f96e2133338f99266f67316c218813db`.

لا ينفذ هذا الشرح ملفات التشغيل.

الأسطر 1–4

```
@echo off
setlocal
cd /d "%~dp0"
title Mohammad XAI - CPU Setup Wizard
```

يخفي عرض الأوامر ويعزل متغيرات البيئة ويغير القرص والمجلد إلى موقع batch ويحدد عنوان النافذة.

الأسطر 5–8

```
if exist ".demo_venv\Scripts\python.exe" (
  ".demo_venv\Scripts\python.exe" scripts\demo_wizard.py setup %*
  goto finished
)
```

إذا وجد Python داخل .demo_venv يشغل wizard setup مع جميع الوسائط %* ثم ينتقل إلى finished.

الأسطر 9–12

```
if exist ".venv\Scripts\python.exe" (
  ".venv\Scripts\python.exe" scripts\demo_wizard.py setup %*
  goto finished
)
```

وإلا إن وجد Python في .venv يشغل الإعداد به وينتقل إلى النهاية؛ مجرد وجود الملف لا يضمن سلامة البيئة.

الأسطر 13–20

```
where py >nul 2>nul
if not errorlevel 1 (
  py -3.12 -c "import sys; assert sys.version_info[:2] == (3,12)" >nul 2>nul
  if not errorlevel 1 (
    py -3.12 scripts\demo_wizard.py setup %*
    goto finished
  )
)
```

يبحث عن py دون إظهار الخرج، ثم يتحقق من Python 3.12 ويشغل wizard عبره إن توفر. فحص الإصدار هنا لا يتحقق صراحة من معمارية 64 بت.

الأسطر 21–27

```
where python >nul 2>nul
if not errorlevel 1 (
  python scripts\bootstrap_cpu.py %*
  goto finished
)
echo Python 3.12 x64 was not found. Install it for your user, then run SETUP_DEMO.bat again.
exit /b 1
```

إن وجد python في PATH يستدعي bootstrap_cpu للبحث عن إصدار مناسب؛ وإلا يعرض تعليمات وينهي برمز 1.

الأسطر 28–32

```
:finished
set "demo_result=%errorlevel%"
if not "%demo_result%"=="0" echo Setup failed. See the message above and .demo_runtime\logs.
if /i not "%XAI_DEMO_NO_PAUSE%"=="1" pause
exit /b %demo_result%
```

label finished يحفظ errorlevel ويعرض رسالة عند الفشل، ويتوقف للقراءة ما لم يُضبط XAI_DEMO_NO_PAUSE=1، ثم يعيد رمز النتيجة.

شرح `src/xai_finance/__init__.py`

بصمة المصدر SHA-256:

`993af911465c69e69eebd4127a7592e8c6718b32d326755fd8b11b3322f815d5`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الأسطر 1–1

```
"""Reproducible, chronological financial forecasting and post-hoc explanations."""
```

سلسلة توثيق تصف الحزمة بأنها تنبؤ زمني وتفسيرات لاحقة؛ لا تنفذ تدريباً.

الأسطر 3–3

```
__version__ = "0.1.0"
```

يعين **version** إلى 0.1.0 كبيان إصدار الحزمة؛ لا يثبت إصدار البيئة أو البيانات.

شرح `src/xai_finance/cli.py`

بصمة المصدر SHA-256:

`1cbd6e8586f7f1830766661cd594ece259adb489650bc9b141e09724f13c2aed`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الأسطر 1–6

```
import argparse
import os

os.environ.setdefault("CUBLAS_WORKSPACE_CONFIG", ":4096:8")

from xai_finance.config import load_config
```

argparse وos من المكتبة القياسية. setdefault يضبط إعداد CUBLAS إن لم يكن موجوداً قبل استيراد بقية المسار؛ لا يستبدل قيمة موجودة. load_config داخلي لقراءة YAML.

الأسطر 9–15

```
def main() -> None:
    parser = argparse.ArgumentParser(description="Reproducible thesis reconstruction")
    parser.add_argument("command", choices=["doctor", "download", "validate", "train", "explain", "evaluate",
"report", "all"])
    parser.add_argument("--config", default="configs/default.yaml")
    arguments = parser.parse_args()
    configuration = load_config(arguments.config)
    execute(arguments.command, configuration)
```

main ينشئ محلل سطر الأوامر ويحدد command من الخيارات وconfig الافتراضي، ويقرأ الوسائط والإعداد ثم يستدعي execute؛ الوسيط غير الصالح يوقف argparse برسالة.

الأسطر 18–20

```
def execute(command: str, configuration: dict) -> None:
    from xai_finance.io.data_sources import download_all, load_snapshot
    from xai_finance.pipelines.train import resolve_run, train_experiment
```

execute تستقبل الأمر والقاموس وتستورد تحميل البيانات والتدريب وحل مسار التشغيل؛ الاستيراد مؤجل إلى الاستدعاء.

الأسطر 22–30

```
    if command == "doctor":
        from xai_finance.environment import verify_environment
        verify_environment()
    if command in {"download", "all"}:
        download_all(configuration)
    if command == "validate":
        for asset in configuration["assets"]:
            prices, manifest = load_snapshot(asset)
            print(asset, len(prices), manifest["sha256"])
```

doctor يستدعي فحص البيئة؛ download وall يستدعيان تنزيل أو تحقق اللقطات؛ validate يحمل كل أصل ويفحصه ويطبع العدد والبصمة. هذه فروع مستقلة وليست سلسلة elif.

الأسطر 31–38

```
    if command in {"train", "all"}:
        train_experiment(configuration)
    if command in {"explain", "all"}:
        from xai_finance.pipelines.explain import explain_experiment
        explain_experiment(resolve_run(configuration))
    if command in {"evaluate", "report", "all"}:
        from xai_finance.pipelines.report import generate_report
        generate_report(resolve_run(configuration))
```

train وall يدربان؛ explain وall يفسران آخر تشغيل؛ evaluate وreport وall تستدعي جميعاً generate_report، الذي يقرأ المقاييس ويعيد التحقق ويكتب تقارير. all يسير وفق ترتيب الفروع؛ تشغيل هذه الأوامر قد ينشئ آثاراً ولم ينفذ للتوثيق.

الأسطر 41–42

```
if __name__ == "__main__":
    main()
```

حارس **main** يشغّل main عند التنفيذ كوحدة أو ملف، وليس عند الاستيراد المعتاد.

شرح src/xai_finance/config.py

بصمة المصدر SHA-256:

.cd4bc1670b874ac8c6d41fb3552d077f5d6d6289e3e217c117c5a8d7742944df

الاقتباسات من المصدر؛ لم تنفذ أعمال التقرير أو تعدل البيانات.

الأسطر 1–3

```
from pathlib import Path

import yaml
```

Path قياسية وyaml خارجية لقراءة الإعدادات.

الأسطر 5–10

```
def project_root() -> Path:
    candidates = [Path.cwd(), *Path(__file__).resolve().parents]
    for candidate in candidates:
        if (candidate / "configs/datasets.yaml").exists() and (candidate / "pyproject.toml").exists():
            return candidate
    raise RuntimeError("Run from the research/defense project directory containing configs/datasets.yaml")
```

project_root تبحث أولاً في مجلد العمل ثم أسلاف ملف الوحدة عن configs/datasets.yaml
وpyproject.toml؛ تعيد أول مطابقة أو ترفع RuntimeError. قد يحدد مجلد عمل مطابق مشروعاً قبل موقع الوحدة.

الأسطر 13–13

```
ROOT = project_root()
```

ROOT يحسب عند استيراد الوحدة، لذا قد يفشل الاستيراد خارج شجرة مشروع مناسبة.

الأسطر 16–21

```
def load_config(path: str | Path) -> dict:
    with Path(path).open(encoding="utf-8") as stream:
        configuration = yaml.safe_load(stream)
    with (ROOT / "configs/datasets.yaml").open(encoding="utf-8") as stream:
        configuration["datasets"] = yaml.safe_load(stream)["datasets"]
    return configuration
```

load_config تقرأ YAML المدخل بترميز UTF-8 وsafe_load ثم تضيف datasets من إعداد المشروع وتعيد القاموس؛ لا تتحقق هنا من صحة المخطط وتستبدل المفتاح datasets إن كان موجوداً.

شرح `src/xai_finance/data/__init__.py`

بصمة المصدر SHA-256:

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الملف فارغ عند البصمة المذكورة؛ لا يحتوي أسطراً أو استيرادات أو دوال، ويحدد مجلد الحزمة عند الاستيراد.

شرح src/xai_finance/data/features.py

بصمة المصدر SHA-256:

.dca14d81cf7b8d7bf7702e5507934a37b6a92113c9ea4447829d9e5bcbd2110a

الاقتباسات من المصدر الفعلي؛ الأسطر الفارغة للتنسيق. لا ينفذ هذا الشرح التدريب أو يعدل البيانات.

الأسطر 1–2

```
import numpy as np
import pandas as pd
```

يستورد NumPy للعمليات العددية والقيم غير المنتهية، وpandas للسلاسل والجداول. كلاهما مكتبتان خارجيتان.

الأسطر 5–6

```
def relative_strength(close: pd.Series) -> pd.Series:
    changes = close.diff()
```

relative_strength تستقبل سلسلة إغلاق وتعيد سلسلة RSI؛ diff يحسب تغير كل رصد عن السابق، فتكون البداية ناقصة.

الأسطر 7–8

```
    gains = changes.clip(lower=0).ewm(alpha=1 / 14, adjust=False, min_periods=14).mean()
    losses = (-changes.clip(upper=0)).ewm(alpha=1 / 14, adjust=False, min_periods=14).mean()
```

يفصل التغيرات الموجبة والخسائر المطلقة بالقص ثم يحسب متوسطاً أُسّياً بمعامل 1/14 وadjust=False وبعد 14 قيمة على الأقل. هذا تهيئة EWM في pandas ولا يلزم افتراض تطابق كل تفاصيلها مع كل تعريف آخر لـ RSI.

الأسطر 9–11

```
    ratio = gains / losses.replace(0, np.nan)
    rsi = 100 - 100 / (1 + ratio)
    return rsi.mask((losses == 0) & (gains > 0), 100).mask((losses == 0) & (gains == 0), 50)
```

يقسم متوسط المكاسب على الخسائر بعد استبدال صفر المقام بـ NaN، ثم يحسب 100−100/(1+ratio). يعالج غياب الخسائر مع مكاسب بـ100 وغياب الاثنين بـ50؛ بيانات أخرى ناقصة تبقى ناقصة.

الأسطر 14–16

```
def features(prices: pd.DataFrame, frequency: str) -> pd.DataFrame:
    close = prices["close"] if frequency == "monthly" else prices["adj_close"]
    engineered = pd.DataFrame({"close": close}, index=prices.index)
```

features تستقبل جدول الأسعار والتردد وتنشئ جدولاً بمحاذاة فهرس المدخل؛ تستخدم close للشهري وadj_close لأي قيمة أخرى. مسار الاستدعاء المعتاد train_asset يمرر تردد بيان المصدر.

الأسطر 17–21

```
    if frequency == "daily":
        adjustment = prices["adj_close"] / prices["close"]
        for column in ["open", "high", "low"]:
            engineered[column] = prices[column] * adjustment
        engineered["log_volume"] = np.log1p(prices["volume"])
```

للبيانات اليومية يحسب معامل تعديل الإغلاق ثم يضرب open وhigh وlow فيه، ويضيف log1p(volume) أي log(1+volume). القسمة على صفر أو قيم غير صالحة ستعالج كقيم غير منتهية أو مفقودة لاحقاً.

الأسطر 22–23

```
    engineered["sma_5"] = close.rolling(5).mean()
    engineered["sma_20"] = close.rolling(20).mean()
```

يحسب المتوسطين المتحركين البسيطين لآخر 5 و20 رصداً: $SMA_n(t) = \Sigma close(t-k)/n$. بداية كل نافذة الكاملة ناقصة.

الأسطر 24–26

```
    engineered["ema_12"] = close.ewm(span=12, adjust=False, min_periods=12).mean()
    engineered["ema_26"] = close.ewm(span=26, adjust=False, min_periods=26).mean()
    engineered["rsi_14"] = relative_strength(close)
```

يحسب EMA لفترتي 12 و26 مع حد أدنى مطابق لكل فترة، ويستدعي relative_strength على الإغلاق. EMA وزن أسي للماضي لا متوسطاً بسيطاً.

الأسطر 27–28

```
    engineered["macd"] = engineered["ema_12"] - engineered["ema_26"]
    engineered["macd_signal"] = engineered["macd"].ewm(span=9, adjust=False, min_periods=9).mean()
```

MACD هو الفرق بين EMA12 وEMA26؛ signal متوسط أسي لـ MACD بفترة 9 ويتطلب تسع قيم صالحة.

الأسطر 29–31

```
    dispersion = close.rolling(20).std(ddof=0)
    engineered["bollinger_upper"] = engineered["sma_20"] + 2 * dispersion
    engineered["bollinger_lower"] = engineered["sma_20"] - 2 * dispersion
```

الانحراف المعياري لنافذة 20 يستخدم ddof=0، ثم الحد الأعلى SMA20+2σ والأدنى SMA20−2σ. لا يشير تجاوز الحد تلقائياً إلى علاقة سببية.

السطر 32

```
    return return_features(engineered, close).replace([np.inf, -np.inf], np.nan).dropna()
```

يستدعي return_features لإضافة السمات ثم يستبدل ±inf بـ NaN ويحذف كل صف يحوي نقصاً ويعيد الجدول. قد يحذف أكثر من فترة التهيئة؛ لا يملأ فجوات.

الأسطر 35–39

```
def return_features(engineered: pd.DataFrame, close: pd.Series) -> pd.DataFrame:
    returns = close.pct_change(fill_method=None)
    engineered["return_1"] = returns
    engineered["return_lag_1"] = returns.shift(1)
    engineered["return_lag_5"] = returns.shift(5)
```

return_features تعدل جدول engineered الممرر وتعيده. pct_change دون ملء تحسب $c_t/c_{(t-1)}-1$؛ return_1 القيمة الحالية، واللاحقتان تؤخرانها رصداً أو خمسة رصود.

الأسطر 40–43

```
    engineered["volatility_20"] = returns.rolling(20).std(ddof=0)
    engineered["momentum_10"] = close - close.shift(10)
    engineered["trailing_return_20"] = close.pct_change(20, fill_method=None)
    return engineered
```

تضيف تقلب العوائد على 20 رصداً بانحراف ddof=0، وزخم فرق الإغلاق عن عشرة رصود، والعائد النسبي عبر 20 رصداً؛ ثم تعيد الجدول المعدل. الفترات هي رصود، وليست بالضرورة أيام تقويمية.

شرح `src/xai_finance/data/preprocess.py`

بصمة المصدر SHA-256:

`795123aca48a8ee72b305571c46707aa19428e248927ddc162a2a28ca46ff2b0`.

الاقتباسات من المصدر الفعلي؛ الأسطر الفارغة للتنسيق. لا ينفذ هذا الشرح التدريب أو يعدل البيانات.

الأسطر 1–6

```
from dataclasses import dataclass

import numpy as np
from sklearn.preprocessing import StandardScaler

from xai_finance.data.windows import WindowDataset
```

dataclass من المكتبة القياسية، وNumPy وStandardScaler من مكتبتين خارجيتين، وWindowDataset نوع داخلي.

الأسطر 9–14

```
@dataclass
class TrainingScalers:
    inputs: StandardScaler
    target: StandardScaler
    fitted_origin_cutoff: str
    fitted_target_cutoff: str
```

TrainingScalers حاوية لمعياري المدخلات والهدف ونص تاريخ آخر أصل وآخر هدف استُعمل في الملاءمة. ينشئ dataclass منشئاً وفق ترتيب الحقول.

الأسطر 16–18

```
    def transform_windows(self, windows: np.ndarray) -> np.ndarray:
        shape = windows.shape
        return self.inputs.transform(windows.reshape(-1, shape[-1])).reshape(shape).astype(np.float32)
```

transform_windows تحفظ الشكل، وتسطح كل الخطوات إلى صفوف سمات، وتطبق معيار المدخلات المُلائم، ثم تعيد الشكل الأصلي وتحول إلى float32؛ لا تعيد ملاءمة المعيار.

الأسطر 20–21

```
    def inverse_target(self, predictions: np.ndarray) -> np.ndarray:
        return self.target.inverse_transform(np.asarray(predictions).reshape(-1, 1)).ravel()
```

inverse_target تحول التنبؤات إلى عمود، وتطبق inverse_transform لمعيار الهدف، ثم ravel لإرجاع متجه بالوحدات الأصلية. الصيغة x=zσ+μ وفق معلمات التدريب.

الأسطر 24–28

```
def fit_training_scalers(windows: WindowDataset) -> TrainingScalers:
    training = windows.partitions["train"]
    sequence = windows.x[training]
    scalers = TrainingScalers(StandardScaler().fit(sequence.reshape(-1, sequence.shape[-1])),
StandardScaler().fit(windows.y[training, None]), str(windows.origins[training[-1]].date()),
str(windows.targets[training[-1]].date()))
    return scalers
```

fit_training_scalers تختار فهارس التدريب والنوافذ الموافقة، وتلائم StandardScaler للمدخلات على جميع خطوات نوافذ التدريب وللهدف على أهداف التدريب فقط، وتوثق آخر تواريخ الأصل والهدف وتعيد الحاوية. النوافذ المتداخلة تكرر بعض الرصود في ملاءمة المدخلات؛ لا تستعمل أهداف الاختبار. يستدعيها fit_lstm.

src/xai_finance/data/split.py

بصمة المصدر SHA-256:
`8a6532b160cf68aacf146ab90fcb158f873c16361337192c8ddf918aff7fbc78`.

المسؤولية: إرجاع فهارس تقسيم العينات المتحاذية زمنياً. يستدعيه `build_windows` في `data/windows.py` بعد حساب مواضع النوافذ؛ لا يقرأ ملفات ولا يكتبها. الوارد الوحيد NumPy مكتبة خارجية، وتستخدم `np.ndarray` في التلميح و`np.arange` لإنشاء الفهارس. الأسطر الفارغة 2–3 للفصل البصري.

1 `import numpy as np` — يستورد NumPy بالاسم المختصر `np` لاستخدامه في تعريف نوع المصفوفات وإنشائها؛ المدخل بيئة Python، والنتيجة ربط الاسم بالوحدة.

4: `def chronological_indices(count: int) -> dict[str, np.ndarray]` — يعرّف دالة تأخذ عدد العينات الصحيح وتعيد قاموساً من أسماء التقسيمات إلى مصفوفات فهارس. تلميحات النوع توثيق ولا تتحقق وحدها وقت التنفيذ.

5–6: `if count < 30:` ثم `raise ValueError("Too few aligned samples for chronological train/validation/test")` — تتحقق من الحد الأدنى؛ إذا كان العدد أصغر توقف التنفيذ باستثناء بدلاً من إنتاج تقسيم غير كافٍ. لا تفحص أن العدد من النوع الصحيح صراحة.

7 `train_end = int(0.8 * count)` — يحسب نهاية التدريب غير المشمولة في المدى. للعدد الموجب يقطع `int` الكسر؛ فمثلاً عند 31 تكون النهاية 24.

8 `validation_end = train_end + int(0.1 * count)` — يضيف حجم التحقق المقطوع إلى نهاية التدريب؛ عند 31 تكون النهاية 27.

9 `return {"train": np.arange(train_end), "validation": np.arange(train_end, validation_end), "test": np.arange(validation_end, count)}` — ينشئ نطاقات صحيحة متتابعة غير متقاطعة ويعيدها. عند 31: التدريب 0–23 والتحقق 24–26 والاختبار 27–30. الباقي بعد التقريب يذهب للاختبار؛ هذه الفهارس تفترض أن ترتيب المدخلات زمني ولا تتحقق بنفسها من التواريخ.

شرح src/xai_finance/data/validate.py

بصمة المصدر SHA-256:

.454cd1ccd21e3d8437714df3ae9b95df7b2f8762935e8a13173a3058c1bc0488

اقتباسات من المصدر؛ الفراغات للتنسيق. الوصف لا يعني تنفيذ التنزيل.

الأسطر 1–2

```
import numpy as np
import pandas as pd
```

يستورد NumPy وpandas مكتبتين خارجيتين للتحقق من القيم والفهرس والجدول.

الأسطر 5–8

```
def validate_prices(prices: pd.DataFrame, frequency: str) -> pd.DataFrame:
    required = ["close"] if frequency == "monthly" else ["open", "high", "low", "close", "adj_close", "volume"]
    if not set(required).issubset(prices.columns):
        raise ValueError(f"Missing required columns: {set(required) - set(prices.columns)}")
```

validate_prices تستقبل جدولاً وترددآ وتحدد الأعمدة المطلوبة: close للشهري وإلا OHLC وadj_close وvolume؛ ترفع ValueError مع أسماء الأعمدة الناقصة إن لم توجد. لا تتحقق من أن قيمة frequency واحدة من قيم معروفة هنا.

الأسطر 9–12

```
    if not isinstance(prices.index, pd.DatetimeIndex) or prices.index.hasnans:
        raise ValueError("A non-null DatetimeIndex is required")
    if not prices.index.is_monotonic_increasing or prices.index.has_duplicates:
        raise ValueError("Dates must be unique and strictly chronological")
```

تتطلب DatetimeIndex دون تواريخ ناقصة، ثم ترتيباً تصاعدياً دون تكرار. بذلك يكون التسلسل صارماً مع اجتماع الشرطين؛ لا يفحص هذا وجود كل جلسة تداول متوقعة.

الأسطر 13–16

```
    if not np.isfinite(prices[required].to_numpy(dtype=float)).all():
        raise ValueError("Unresolved missing or infinite observations")
    if frequency == "daily":
        validate_ohlc(prices)
```

تحول الأعمدة المطلوبة إلى أعداد وتتحقق من انتهاء جميعها ثم تستدعي validate_ohlc لليومي. فشل التحويل أو وجود نقص أو لانهاية يوقف التنفيذ.

الأسطر 17–19

```
    if len(prices) < 100:
        raise ValueError("At least 100 genuine observations required")
    return prices
```

تطلب 100 مشاهدة على الأقل وتعيد جدول الأسعار نفسه؛ لا تنسخ أو تستكمل السجلات الناقصة.

الأسطر 22–26

```
def validate_ohlc(prices: pd.DataFrame) -> None:
    high_ok = prices["high"] + 1e-6 >= prices[["open", "close", "low"]].max(axis=1)
    low_ok = prices["low"] - 1e-6 <= prices[["open", "close", "high"]].min(axis=1)
    if not (high_ok & low_ok).all():
        raise ValueError("OHLC ordering is inconsistent")
```

validate_ohlc تتحقق لكل صف أن high لا يقل عن أعلى open وclose وlow وأن low لا يتجاوز أدناها، بتسامح 1e-6. إن فشل أي صف ترفع ValueError؛ الاختبار للاتساق ولا يثبت صحة السعر من المزود.

الأسطر 27–28

```
    if (prices["volume"] < 0).any() or (prices["close"] == 0).any():
        raise ValueError("Negative volume or zero close cannot be processed")
```

ترفض حجماً سالباً أو close يساوي صفراً لأن الحجم اللوغاريتمي ونسبة التعديل يحتاجان قيماً قابلة للمعالجة. لا ترفض كل سعر سالب بصورة عامة.

شرح `src/xai_finance/data/windows.py`

بصمة المصدر SHA-256:
`5a2dd67bb06590ee870e77dff8787faeb89027f2de566095b50b6295d5728743`.

الاقتباسات من المصدر الفعلي؛ الأسطر الفارغة للتنسيق. لا ينفذ هذا الشرح التدريب أو يعدل البيانات.

الأسطر 1–6

```
from dataclasses import dataclass

import numpy as np
import pandas as pd

from xai_finance.data.split import chronological_indices
```

dataclass من المكتبة القياسية لإنشاء حاوية البيانات؛ NumPy وpandas خارجيتان.
chronological_indices استيراد داخلي لتقسيم الفهارس زمنياً.

الأسطر 9–18

```
@dataclass
class WindowDataset:
    x: np.ndarray
    y: np.ndarray
    origins: pd.DatetimeIndex
    targets: pd.DatetimeIndex
    current_close: np.ndarray
    trailing_returns: np.ndarray
    feature_names: list[str]
    partitions: dict[str, np.ndarray]
```

WindowDataset صنف بيانات يولد dataclass منشئه؛ x نوافذ ثلاثية الأبعاد، y أهداف، origins تواريخ آخر مدخل، targets تواريخ الأهداف، current_close وtrailing_returns للقيم عند الأصل، وأسماء السمات وقواميس التقسيم. التلميحات لا تتحقق من الأشكال وحدها.

الأسطر 20–22

```
    @property
    def flat(self) -> np.ndarray:
        return self.x.reshape(len(self.x), -1)
```

خاصية flat تعيد تشكيل x إلى صف لكل نافذة وعمود لكل زوج زمن/سمة؛ -1 يستنتج عدد الأعمدة.
تستعملها النماذج الجدولية بينما تحتفظ x ببعد الزمن لـ LSTM.

الأسطر 24–26

```
    @property
    def flat_names(self) -> list[str]:
        return [f"lag_{lag}__{name}" for lag in range(self.x.shape[1] - 1, -1, -1) for name in
self.feature_names]
```

خاصية flat_names تبني أسماء lag تنازلياً من lookback−1 إلى 0، ثم السمات حسب ترتيبها لكل تأخر؛ توافق ترتيب التسطيح طالما ترتيب أبعاد x المعتاد لم يتغير.

الأسطر 29–31

```
def build_windows(engineered: pd.DataFrame, lookback: int) -> WindowDataset:
    if lookback < 1:
        raise ValueError("lookback must be positive")
```

build_windows تستقبل جدول سمات مرتباً زمنياً وطول نافذة، وترفع ValueError إن كان lookback أقل من واحد. لا تفحص ترتيب فهرس التاريخ هنا.

الأسطر 32–34

```
    # Boundary cases: first origin has exactly lookback rows; final row is context only.
    origins = np.arange(lookback - 1, len(engineered) - 1)
    partitions = chronological_indices(len(origins))
```

التعليق يشرح الحد؛ origins من lookback−1 حتى ما قبل آخر صف، وتستدعي chronological_indices لتقسيم عدد الأصول. آخر صف لا يمكن أن يكون أصلاً له هدف تالٍ، لكنه يصلح هدفاً للأصل السابق؛ عبارة التعليق final row is context only مضللة إذا فُهمت على أنها تنفي ذلك.

الأسطر 35–36

```
    matrix = engineered.to_numpy(dtype=np.float64)
    windows = np.stack([matrix[position - lookback + 1:position + 1] for position in origins])
```

يحوّل الجدول لمصفوفة float64 ويبني لكل أصل شريحة تضم lookback صفوفاً حتى الأصل شاملاً ثم يكدسها. عند عدم كفاية العينات يرفض التقسيم الأقل من 30 قبل الوصول إلى stack.

السطر 37

```
    return WindowDataset(windows, engineered.close.iloc[origins + 1].to_numpy(), engineered.index[origins],
engineered.index[origins + 1], engineered.close.iloc[origins].to_numpy(),
engineered.trailing_return_20.iloc[origins].to_numpy(), list(engineered.columns), partitions)
```

يعيد WindowDataset بالترتيب: النوافذ، إغلاق الصف التالي هدفاً، تاريخ الأصل والهدف، الإغلاق والعائد المتأخر عند الأصل، أسماء الأعمدة، والتقسيمات. يعتمد الوصول إلى close وtrailing_return_20 على وجودهما؛ لا يقرأ ملفات أو يكتبها.

شرح src/xai_finance/environment.py

بصمة المصدر SHA-256:

.e2a6e042ad20127ec074dc8c2d05d30daa074c2d0f0e4f1d809ea8ca156879ed

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الأسطر 1–8

```
import json

import torch

from xai_finance.config import ROOT
from xai_finance.pipelines.train import software_environment
from xai_finance.seed import seed_experiment, select_device
from xai_finance.storage import write_json
```

json قياسية وtorch خارجية؛ ROOT وsoftware_environment والبذرة واختيار الجهاز وwrite_json من المشروع.

الأسطر 11–19

```
def verify_environment() -> None:
    seed_experiment(42)
    device, reason = select_device("auto")
    network = torch.nn.LSTM(8, 16, batch_first=True).to(device)
    output, _ = network(torch.randn(8, 30, 8, device=device))
    output.square().mean().backward()
    if not torch.isfinite(output).all():
        raise FloatingPointError("Hardware LSTM diagnostic nonfinite")
    report = {**software_environment(), "device": str(device), "reason": reason, "torch_cuda":
torch.version.cuda, "cuda_available": torch.cuda.is_available(), "lstm_probe_finite": True, "scope": "random
tensor hardware diagnostic, not financial results"}
```

verify_environment تثبت بذرة 42 وتختار الجهاز، وتبني LSTM تشخيصية بثماني سمات و16 مخفية على مدخل عشوائي 8×30×8 وتجري backward على متوسط مربع الخرج، وتتحقق من finiteness للخرج ثم تجمع معلومات البيئة. هذا اختبار عتاد عشوائي لا نتيجة مالية ولا يتحقق صراحة من finiteness كل تدرج.

الأسطر 20–24

```
    if device.type == "cuda":
        report["gpu_name"] = torch.cuda.get_device_name(0)
        report["vram_bytes"] = torch.cuda.get_device_properties(0).total_memory
    write_json(ROOT / "artifacts/environment/current.json", report)
    print(json.dumps(report, indent=2))
```

تضيف اسم GPU وذاكرته عند CUDA وتكتب التقرير في artifacts/environment/current.json وتطبعه؛ لها أثر كتابة وتغير البذور ولم تشغل هنا.

شرح `src/xai_finance/evaluate/__init__.py`

بصمة المصدر SHA-256:

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الملف فارغ عند البصمة المذكورة؛ لا يحتوي أسطراً أو استيرادات أو دوال، ويحدد مجلد الحزمة عند الاستيراد.

شرح src/xai_finance/evaluate/metrics.py

بصمة المصدر SHA-256:

.2a89abe82731ef5e17cc351e0eaed243f08d66422ebe49985af9d2cbd00ff746

المقاطع مقتبسة من المصدر، والوصف مراجعة يدوية لا يشغل التجارب.

الأسطر 1–2

```
import numpy as np
from sklearn.metrics import confusion_matrix, mean_absolute_error, mean_squared_error, r2_score
```

NumPy وsklearn مكتبتان خارجيتان للحساب العددي والمقاييس ومصفوفة الالتباس.

الأسطر 5–10

```
def forecast_metrics(actual: np.ndarray, predicted: np.ndarray, current: np.ndarray) -> dict:
    actual, predicted, current = [np.asarray(series, dtype=float).ravel() for series in (actual, predicted,
current)]
    if len(actual) == 0 or actual.shape != predicted.shape or actual.shape != current.shape:
        raise ValueError("Aligned, nonempty actual/predicted/current arrays required")
    if not all(np.isfinite(series).all() for series in (actual, predicted, current)):
        raise ValueError("Metrics require finite values in original units")
```

forecast_metrics تستقبل actual وpredicted وcurrent وتسطحها كأعداد عشرية وتتحقق من تساوي الأشكال وغير الفراغ والقيم المنتهية؛ ترفع ValueError عند الإخلال. لا تتحقق بنفسها من الوحدات أو تواريخ المحاذاة.

الأسطر 11–13

```
    nonzero = np.abs(actual) > 1e-12
    up_true, up_pred = actual > current, predicted > current
    confusion = confusion_matrix(up_true, up_pred, labels=[False, True])
```

تحدد الأهداف غير القريبة من الصفر بـ |y|>1e-12، وتصنف الصعود بمقارنة القيمة مع current، وتبني مصفوفة ثنائية بترتيب False ثم True؛ التعادل يعامل كغير صعود فيها.

الأسطر 14–14

```
    return {"mae": float(mean_absolute_error(actual, predicted)), "rmse":
float(np.sqrt(mean_squared_error(actual, predicted))), "mape_percent": float(np.mean(np.abs((actual[nonzero] -
```

```
predicted[nonzero]) / actual[nonzero])) * 100) if nonzero.any() else None, "mape_excluded_zero_targets":
int((~nonzero).sum()), "r2": float(r2_score(actual, predicted)) if len(actual) > 1 and np.ptp(actual) > 0 else
None, "direction_accuracy": float(np.mean(np.sign(predicted - current) == np.sign(actual - current))),
"up_down_confusion": confusion.tolist(), "up_precision": float(confusion[1, 1] / confusion[:, 1].sum()) if
confusion[:, 1].sum() else None, "up_recall": float(confusion[1, 1] / confusion[1].sum()) if confusion[1].sum()
else None, "n": len(actual)}
```

تعيد $MAE=mean|y-\hat{y}|$ و$RMSE=sqrt(mean(y-\hat{y})^2)$ وMAPE بالنسبة المئوية للغايات غير الصفرية وعدد المستبعد، وR^2 إذا تعددت القيم ولم يكن الهدف ثابتاً، ودقة إشارة التغير ثلاثية الحالات ومصفوفة الالتباس وprecision/recall للصعود والحجم. عند غياب مقام ملائم تعيد None. دقة الإشارة تميز التعادل بخلاف التصنيف الثنائي؛ MAPE قرب الصفر غير مستقر وR^2 المرتفع لمستوى السعر لا يثبت مهارة تداول.

شرح `src/xai_finance/evaluate/regimes.py`

بصمة المصدر SHA-256:

`13f423ee03fd1f069ce5ce5129216e6fa6649873cf6bd48e0f16faceb305f152`.

المقاطع مقتبسة من المصدر، والوصف مراجعة يدوية لا يشغل التجارب.

الأسطر 1–4

```
import numpy as np
import pandas as pd

from xai_finance.evaluate.metrics import forecast_metrics
```

NumPy للتصنيف وpandas للجدول من مكتبات خارجية، وforecast_metrics دالة داخلية.

الأسطر 7–8

```
def regime_labels(trailing_returns: np.ndarray, threshold: float) -> np.ndarray:
    return np.select([trailing_returns > threshold, trailing_returns < -threshold], ["bull", "bear"],
default="sideways")
```

regime_labels تقبل عوائد متأخرة وعتبة، وتعيد bull إذا زاد العائد عن threshold وbear إن قل عن سالبها وإلا sideways؛ المساواة تدخل sideways. لا تتحقق من إيجابية العتبة أو انتهاء القيم.

الأسطر 11–15

```
def regime_metrics(predictions: pd.DataFrame) -> list[dict]:
    summaries = []
    for (model, regime), subset in predictions.groupby(["model", "regime"]):
        summaries.append({"model": model, "regime": regime, **forecast_metrics(subset.actual, subset.predicted,
subset.current_close)})
    return summaries
```

regime_metrics تقسم DataFrame حسب model وregime وتستدعي forecast_metrics لكل مجموعة وتعيد قائمة قواميس؛ لا تولد فئات غائبة. المجموعات الصغيرة وصفية وقد تكون تقديراتها غير مستقرة.

شرح `src/xai_finance/explain/__init__.py`

بصمة المصدر SHA-256:

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الملف فارغ عند البصمة المذكورة؛ لا يحتوي أسطراً أو استيرادات أو دوال، ويحدد مجلد الحزمة عند الاستيراد.

شرح src/xai_finance/explain/lime_explain.py

بصمة المصدر SHA-256:

.3ec646ba008e1fc255ebc9b8bb0eeaecefb9be679f3df4471c2ef658692f5bc3

الاقتباسات من المصدر؛ التفسير مراجعة يدوية لا إجراء حسابات جديدة.

الأسطر 1–2

```
import numpy as np
from lime.lime_tabular import LimeTabularExplainer
```

يستورد NumPy لتلميحات الأنواع وLimeTabularExplainer من مكتبة LIME الخارجية.

الأسطر 5–7

```
def explain_lime(training: np.ndarray, predict, observation: np.ndarray, request: dict):
    explainer = LimeTabularExplainer(training, feature_names=request["names"], mode="regression",
discretize_continuous=True, random_state=request["seed"], feature_selection="highest_weights")
    explanation = explainer.explain_instance(observation, predict, num_features=10,
num_samples=request["samples"])
```

explain_lime تستقبل بيانات تدريب ودالة predict وعينة وقاموس إعداد؛ تنشئ مفسر انحدار بأسماء السمات وتقطيع مستمر وبذرة واختيار أعلى الأوزان. تطلب تفسير العينة باستخدام عشر سمات وعدد perturbations من samples. يعاد استخدام بيانات التدريب كمرجع لتوليد الجوار لا كدليل سببي.

الأسطر 8–11

```
    local_prediction = float(explanation.local_pred[0])
    prediction = float(predict(observation[None])[0])
    audit = {"model_prediction": prediction, "local_surrogate_prediction": local_prediction,
"absolute_local_error": abs(prediction - local_prediction), "weighted_neighborhood_r2":
float(explanation.score), "seed": request["seed"], "num_samples": request["samples"], "features":
explanation.as_list()}
    return explanation, audit
```

تستخرج التنبؤ المحلي من البديل وتنبؤ النموذج على العينة ذات محور دفعة، وتبني قاموس الفرق المطلق وR^2 الجوار الموزون والبذرة وعدد العينات والأوزان، ثم تعيد كائن التفسير والتدقيق. قد يكون تقريب الجوار ضعيفاً خصوصاً في أبعاد مترابطة.

شرح src/xai_finance/explain/quality.py

بصمة المصدر SHA-256:

.e1f89e524a7473a6dfbc2a05c0cf41cf94302e77201d3c0f98a949b5b3349072

الاقتباسات من المصدر؛ التفسير مراجعة يدوية لا إجراء حسابات جديدة.

الأسطر 1–2

```
import numpy as np
from scipy.stats import spearmanr
```

NumPy للحسابات وspearmanr من SciPy لارتباط الرتب؛ مكتبتان خارجيتان.

الأسطر 5–8

```
def attribution_stability(first: np.ndarray, second: np.ndarray) -> dict:
    first, second = np.abs(first), np.abs(second)
    count = min(10, len(first))
    overlap = len(set(np.argsort(first)[-count:]) & set(np.argsort(second)[-count:])) / count
```

attribution_stability تستقبل متجهي إسهام وتقارن قيمهما المطلقة؛ تختار أصغر 10 وعدد السمات وتحسب نسبة تداخل فهارس أعلى الإسهامات. تساوي 1 يدل على تطابق المجموعتين وفق ترتيب argsort، لا على تطابق القيم.

الأسطر 9–10

```
    correlation = spearmanr(first, second).statistic if np.ptp(first) and np.ptp(second) else None
    return {"top_10_overlap": overlap, "spearman_abs_attribution_rank": float(correlation) if correlation is not
None and np.isfinite(correlation) else None, "scope": "Same observation rerun with a different explainer seed;
not human trust or robustness to all market shifts"}
```

تحسب ارتباط Spearman للرتب المطلقة عند تغير كلا المتجهين، وتعطي None للقيمة غير المتاحة أو غير المنتهية؛ تعيد أيضاً توصيف نطاق اختبار بذرتين للعينة نفسها. المتجه الفارغ قد يسبب قسمة على صفر، وتفترض الدالة تطابق أطوال الإدخال.

الأسطر 13–17

```
def lime_weight_vector(explanation, count: int) -> np.ndarray:
    weights = np.zeros(count)
    for feature, weight in explanation.as_map()[1]:
        weights[feature] = weight
    return weights
```

lime_weight_vector تنشئ متجه أصفار بطول count وتملؤه بأوزان feature من explanation.as_map() [1]، ثم تعيده. السمات غير الظاهرة تبقى صفراً؛ هذا لا يثبت انعدام أهميتها خارج البديل المحدود. تفترض وجود المفتاح 1 وفهارس ضمن المجال.

شرح src/xai_finance/explain/shap_explain.py

بصمة المصدر SHA-256:

.3bf26cc8a78f1c0b257e489739d5712edfa200af961aaebe10003bad5ded3e6b

الاقتباسات من المصدر؛ التفسير مراجعة يدوية لا إجراء حسابات جديدة.

الأسطر 1–2

```
import numpy as np
import shap
```

يستورد NumPy للمصفوفات وSHAP للمفسرات وكائن Explanation؛ مكتبتان خارجيتان.

الأسطر 5–6

```
def explain_shap(model_name: str, model, background: np.ndarray, request: dict) -> shap.Explanation:
    observations, names = request["observations"], request["names"]
```

explain_shap تقبل اسم النموذج وكائنه وbackground وطلباً يحوي observations وnames، وتعيد Explanation. يستدعيها مسار توليد التفسيرات؛ تفترض توافق الأشكال والأسماء.

الأسطر 7–10

```
    if model_name == "LinearRegression":
        scaler, selector, regressor = [step[1] for step in model.steps]
        explainer = shap.LinearExplainer(regressor, selector.transform(scaler.transform(background)))
        explanation = explainer(selector.transform(scaler.transform(observations)))
```

للخطي يفك الخطوات الثلاث scaler وselector وregressor من Pipeline، ويحول الخلفية والمشاهدات باستخدام المعاملات الملائمة ثم يستعمل LinearExplainer. تغيير ترتيب الخطوات يفشل هذا الافتراض.

الأسطر 11–13

```
        attributions = np.zeros_like(observations)
        attributions[:, selector.retained_] = explanation.values
        return shap.Explanation(values=attributions, base_values=explanation.base_values, data=observations,
feature_names=names)
```

ينشئ مصفوفة أصفار كشكل المشاهدات ويضع قيم السمات المختارة في retained_ ثم يعيد Explanation بمرجع التفسير وبيانات وأسماء المجال الأصلي؛ الأصفار للسمات التي حذفها selector لا تثبت عدم أهميتها في نموذج بديل.

الأسطر 14–18

```
    if model_name in {"RandomForest", "GradientBoosting"}:
        explainer = shap.TreeExplainer(model, data=background, feature_perturbation="interventional")
        explanation = explainer(observations, check_additivity=True)
        explanation.feature_names = names
        return explanation
```

للغابة والتعزيز ينشئ TreeExplainer مع خلفية وfeature_perturbation="interventional"، ويفسر المشاهدات مع check_additivity ثم يضيف الأسماء ويعيد التفسير. آلية الإخفاء والخلفية تؤثران في الإسناد.

الأسطر 19–21

```
    masker = shap.maskers.Independent(background, max_samples=len(background))
    explainer = shap.PermutationExplainer(model.predict, masker, feature_names=names, seed=request["seed"])
    return explainer(observations, max_evals=request["cycles"] * (2 * observations.shape[1] + 1),
batch_size=256, silent=True)
```

لأي اسم آخر ينشئ Independent masker من الخلفية وPermutationExplainer يستدعي model.predict ببذرة وأسماء؛ عدد تقييمات الطلب cycles×(2p+1) حيث p عدد الأعمدة، ودفعات 256. هذا تقريب يعتمد ميزانية التقييم وليس قيماً سببية.

الأسطر 24–28

```
def shap_reconstruction(explanation: shap.Explanation, predicted: np.ndarray) -> dict:
    reconstructed = np.asarray(explanation.base_values).reshape(-1) + explanation.values.sum(axis=1)
    errors = np.abs(reconstructed - predicted)
    np.testing.assert_allclose(reconstructed, predicted, rtol=1e-4, atol=1e-3)
    return {"max_absolute_error": float(errors.max()), "mean_absolute_error": float(errors.mean()), "rtol": 1e-
4, "atol": 1e-3}
```

shap_reconstruction تجمع base_values ومساهمات الصف وتقارنها بتنبؤ النموذج ضمن rtol=1e-4 وatol=1e-3؛ تعيد أعظم ومتوسط فرق مطلق، وترفع AssertionError عند فشل allclose. الجمع لا يثبت أداء النموذج أو سلامة تفسير سببي.

شرح `src/xai_finance/io/__init__.py`

بصمة المصدر SHA-256:

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الملف فارغ عند البصمة المذكورة؛ لا يحتوي أسطراً أو استيرادات أو دوال، ويحدد مجلد الحزمة عند الاستيراد.

شرح src/xai_finance/io/data_sources.py

بصمة المصدر SHA-256:

.c2092e3c5dd9173fccedf3879ad4743640e7c905b4a85e8fb075c9ffadb7a042

اقتباسات من المصدر؛ الفراغات للتنسيق. الوصف لا يعني تنفيذ التنزيل.

الأسطر 1–2

```
from datetime import datetime, timezone
from io import StringIO
```

datetime وtimezone لتوثيق الاسترجاع UTC وStringIO لقراءة نص CSV؛ جميعها من المكتبة القياسية.

الأسطر 4–6

```
import pandas as pd
import requests
import yfinance as yf
```

pandas لمعالجة الجداول وrequests لتنزيل HTTP وyfinance لمزود Yahoo؛ مكتبات خارجية.

الأسطر 8–10

```
from xai_finance.config import ROOT
from xai_finance.data.validate import validate_prices
from xai_finance.storage import read_json, sha256, write_json
```

يستورد ROOT وإجراء التحقق وعمليات JSON وSHA-256 الداخلية. تُستخدم للقراءة والكتابة والتحقق من اللقطات.

الأسطر 13–17

```
def yahoo_prices(specification: dict) -> tuple[pd.DataFrame, dict]:
    yf.set_tz_cache_location(str(ROOT / "data/provider_cache"))
    prices = yf.download(specification["symbol"], start=specification["start"], end=specification["end"],
auto_adjust=False, actions=False, progress=False, threads=False, multi_level_index=False, timeout=30)
    if prices.empty:
        raise RuntimeError(f"Yahoo returned no genuine data for {specification['symbol']}; no synthetic
fallback")
```

yahoo_prices تضبط مجلد ذاكرة المنطقة الزمنية وتطلب symbol والفترة من yfinance دون تعديل تلقائي أو أحداث، وبمهلة 30 ثانية وخيط تنزيل معطل. تعيد خطأ عند نتيجة فارغة بدلاً من اختلاق بيانات.

الطلب شبكة فعلية عند استدعاء التنزيل.

الأسطر 18–20

```
    prices = prices.rename(columns=lambda column: column.lower().replace(" ", "_"))
    prices.index = pd.DatetimeIndex(prices.index).tz_localize(None).normalize()
    return prices, {"source_url": f"https://finance.yahoo.com/quote/{specification['symbol']}/history/",
"timezone": "America/New_York exchange session date", "adjustment": "Yahoo Adj Close; OHLC multiplied by Adj
Close/Close during feature extraction; volume unadjusted", "license": "Yahoo personal/research use;
redistribution subject to Yahoo terms", "revision_caution": "Retrospectively adjusted snapshot, not point-in-
time corporate-action vintages; continuous futures roll conventions provider-defined"}
```

توحد أسماء الأعمدة بأحرف صغيرة وشرطات سفلية، وتجرد فهرس الزمن من المنطقة الزمنية وتطبعه إلى اليوم، ثم تعيد الجدول وقاموس المصدر والوحدات الزمنية والتحفظ على التعديلات التاريخية. تطبيع اليوم ليس تحققاً من تاريخ الإتاحة.

الأسطر 23–28

```
def fred_prices(specification: dict) -> tuple[pd.DataFrame, dict]:
    url = f"https://fred.stlouisfed.org/graph/?g=0&id={specification['symbol']}&cosd=
{specification['start']}&coed={specification['end']}"
    csv_url = f"https://fred.stlouisfed.org/graph/fredgraph.csv?id={specification['symbol']}"
    response = requests.get(csv_url, timeout=45)
    response.raise_for_status()
    prices = pd.read_csv(StringIO(response.text), index_col=0, parse_dates=True, na_values=".")
```

fred_prices تبني رابط وصف ورابط CSV للسلسلة، تطلب الثاني بمهلة 45 ثانية وتستدعي raise_for_status، ثم تقرأ النص بعمود تاريخ وتتعامل مع النقطة كقيمة ناقصة. أخطاء الشبكة أو HTTP تنتشر.

الأسطر 29–31

```
    prices.columns = ["close"]
    prices = prices.loc[(prices.index >= specification["start"]) & (prices.index < specification["end"])]
    return prices, {"source_url": url, "download_url": csv_url, "timezone": "monthly observation period",
"adjustment": "Latest revised non-seasonally-adjusted index; no OHLCV invented", "license": "S&P Dow Jones
Indices copyright; FRED series notes/terms apply", "revision_caution": "Exploratory retrospective next-
observation experiment, NOT a real-time publication-date forecast. Release vintages and availability dates
absent."}
```

تسمي عمود السلسلة close وتقتطع المدة start شاملة وend مستبعدة وتعيدها مع مصدر وترخيص وتنبيه البيانات المنقحة وعدم توافر تواريخ الإصدارات. تفترض أن CSV يحوي عمود قيمة واحداً.

الأسطر 34–39

```
def download_asset(asset: str, specification: dict) -> None:
    csv_path = ROOT / f"data/raw/{asset}.csv"
    if csv_path.exists():
        load_snapshot(asset)
```

```
        print(f"{asset}: verified existing snapshot", flush=True)
        return
```

download_asset تحدد مسار CSV؛ إذا وجد تستدعي load_snapshot للتحقق ثم تطبع وتعود، فلا تستبدله تلقائياً. وجود CSV دون manifest صالح يسبب خطأ بدلاً من تنزيل جديد.

الأسطر 40–44

```
    downloader = {"yahoo": yahoo_prices, "fred": fred_prices}[specification["provider"]]
    prices, provenance = downloader(specification)
    before = len(prices)
    prices = prices.apply(pd.to_numeric, errors="coerce").dropna()
    validate_prices(prices, specification["frequency"])
```

تختار دالة المزود حسب specification وتحمّل البيانات، تحفظ العدد الأولي، تحول القيم رقمياً مع تحويل غير الصالح إلى نقص ثم تحذف الصفوف الناقصة وتتحقق من الأسعار. مزود غير معروف يسبب KeyError.

الأسطر 45–49

```
    csv_path.parent.mkdir(parents=True, exist_ok=True)
    prices.to_csv(csv_path, index_label="date", float_format="%.12g")
    manifest = {**specification, **provenance, "asset": asset, "status": "AUTHENTIC_PROVIDER_SNAPSHOT",
"retrieved_utc": datetime.now(timezone.utc).isoformat(), "sha256": sha256(csv_path), "rows": len(prices),
"rows_dropped_missing": before - len(prices), "first_date": str(prices.index.min().date()), "last_date":
str(prices.index.max().date()), "max_calendar_gap_days": int(prices.index.to_series().diff().dt.days.max()),
"negative_close_count": int((prices.close < 0).sum())}
    write_json(ROOT / f"data/manifests/{asset}.json", manifest)
    print(f"{asset}: saved {len(prices)} observations, sha256={manifest['sha256']}", flush=True)
```

تنشئ مجلد البيانات وتكتب CSV بدقة تنسيق 12 رقماً مع فهرس date، ثم تبني manifest يتضمن مواصفات المصدر والوقت والهاش والعدد والصفوف المحذوفة والمدى وأكبر فجوة وعدد الإغلاقات السالبة، وتكتب JSON وتطبع النتيجة. هذه آثار كتابة عند تشغيل التنزيل، ولم تنفذ في هذا التوثيق.

الأسطر 52–56

```
def load_snapshot(asset: str) -> tuple[pd.DataFrame, dict]:
    csv_path = ROOT / f"data/raw/{asset}.csv"
    manifest = read_json(ROOT / f"data/manifests/{asset}.json")
    if sha256(csv_path) != manifest["sha256"]:
        raise ValueError(f"Snapshot hash mismatch: {asset}")
```

load_snapshot تبني مسار اللقطة وتقرأ manifest وتقارن SHA-256 قبل قراءة الجدول؛ اختلاف البصمة يرفع ValueError. الهاش يكشف اختلاف البايتات عن القيمة المسجلة ولا يثبت وحده مصدر المزود.

الأسطر 57–61

```
    prices = pd.read_csv(csv_path, index_col="date", parse_dates=True)
    validate_prices(prices, manifest["frequency"])
    if len(prices) != manifest["rows"]:
```

```
        raise ValueError("Snapshot observation count mismatch")
    return prices, manifest
```

تقرأ CSV مع تاريخ فهرس وتطبق validate_prices وتقارن العدد المسجل قبل إرجاع الجدول والبيانات الوصفية. تعتمد التدريبات على هذا المسار؛ قد تنتشر أخطاء الملف أو parsing.

الأسطر 64–66

```
def download_all(configuration: dict) -> None:
    for asset in configuration["assets"]:
        download_asset(asset, configuration["datasets"][asset])
```

download_all تمر على الأصول المحددة في configuration وتستدعي download_asset مع إعداد كل أصل؛ لا تعيد قيمة. استخدام اللقطات الموجودة يمر بالتحقق بدلاً من الكتابة.

شرح `src/xai_finance/models/__init__.py`

بصمة المصدر SHA-256:

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الملف فارغ عند البصمة المذكورة؛ لا يحتوي أسطراً أو استيرادات أو دوال، ويحدد مجلد الحزمة عند الاستيراد.

src/xai_finance/models/baselines.py

بصمة المصدر SHA-256:
`a0804c7e3f055061729abfacaa5bcfaf63eb7fd3a0c98cb72412d7882346e4b6`.

المسؤولية: إنتاج خط أساس Persistence، حيث التنبؤ المقبل يساوي الإغلاق الحالي. يستعمله `evaluate_asset` في `pipelines/train.py` للمقارنة. لا يقرأ أو يكتب ملفات. الأسطر الفارغة 2–3 للفصل البصري.

1 `import numpy as np` — يستورد مكتبة NumPy الخارجية بالاسم `np` لتحويل المدخلات إلى مصفوفة وللتلميح بالنوع.

4 `def persistence(current_close: np.ndarray) -> np.ndarray:` — يعرّف دالة تستقبل الإغلاقات الحالية وتوثق أن الناتج مصفوفة NumPy؛ تلميح النوع ليس فحصاً إلزامياً وقت التنفيذ.

5 `return np.asarray(current_close, dtype=float).copy()` — يحوّل القيم إلى مصفوفة أعداد عشرية وينسخها ثم يعيدها. لكل عنصر $\hat{y}_i = \text{current}_i$؛ فمثلاً 100 يعطي 100. النسخ يفصل الناتج عن المدخل عند تعديل الناتج لاحقاً؛ الخط المرجعي لا يتعلم معاملات ولا يضمن اتجاه الحركة.

شرح `xai_finance/models/linear_preprocess.py`

بصمة المصدر SHA-256:

`9382546e4b3481cfe1eaa561614bb23772f02029e924f0b717d4b526600b35a2`.

المقاطع من المصدر الفعلي؛ لا تشغّل هذه الوثيقة التدريب أو تعدل الآثار. الأسطر الفارغة للتنسيق.

الأسطر 1–3

```
import numpy as np
from scipy.linalg import qr
from sklearn.base import BaseEstimator, TransformerMixin
```

NumPy للمصفوفات؛ qr من SciPy للتحليل QR؛ BaseEstimator وTransformerMixin من sklearn لتنفيذ محوّل متوافق مع Pipeline.

الأسطر 6–10

```
class IndependentColumns(TransformerMixin, BaseEstimator):
    """Training-only QR removes dependent coordinates before ordinary least squares."""

    def __init__(self, relative_tolerance: float = 1e-10):
        self.relative_tolerance = relative_tolerance
```

IndependentColumns محوّل يتعلم الأعمدة المستقلة عددياً من التدريب قبل OLS. يرث واجهة sklearn؛ relative_tolerance الافتراضي 1e-10 يحفظه المنشئ دون فحص خاص.

الأسطر 12–15

```
    def fit(self, inputs: np.ndarray, target=None):
        _, triangular, pivots = qr(inputs, mode="economic", pivoting=True)
        diagonal = np.abs(np.diag(triangular))
        self.rank_ = int((diagonal > diagonal[0] * self.relative_tolerance).sum())
```

fit تستقبل مصفوفة inputs وtarget اختياري غير مستخدم للتوافق مع sklearn. QR مع تبديل الأعمدة يعطي R ومؤشرات pivots؛ يتجاهل Q. يقدر الرتبة بعدد عناصر قطر R المطلقة التي تتجاوز أول عنصر مضروباً في التسامح. هذا تقدير عددي للرتبة وليس ضمان استقلال إحصائي.

الأسطر 16–19

```
        # A constant-only design still needs one column for sklearn's intercept-only OLS fit.
        self.retained_ = np.sort(pivots[:max(1, self.rank_)])
        self.n_features_in_ = inputs.shape[1]
        return self
```

حتى لو رتبة التقدير صفر يحتفظ بعمود واحد؛ يرتب فهارس الأعمدة المختارة ويخزن عدد السمات الأصلية ثم يعيد self طبقاً لبروتوكول fit. المصفوفة بلا أي عمود قد تفشل عند diagonal[0]؛ عدد الأعمدة المخزّن لا يفرض التحقق تلقائياً في transform.

الأسطر 21–22

```
    def transform(self, inputs: np.ndarray) -> np.ndarray:
        return np.asarray(inputs)[:, self.retained_]
```

transform تحول المدخل إلى مصفوفة وتعيد الأعمدة retained_ لجميع الصفوف. تستدعى بعد fit بواسطة Pipeline؛ لا تعيد ملاءمة الاختيار على بيانات التحقق أو الاختبار. فهرسة مدخل قبل fit أو بشكل غير متوافق قد تفشل.

شرح src/xai_finance/models/lstm_model.py

بصمة المصدر SHA-256:

.4dcbb73a31dc7f72b2360ff6d87bc1dfbb9cf379af62c1ca6723b6dd2f4c1f11

المقاطع من المصدر الفعلي؛ لا تشغّل هذه الوثيقة التدريب أو تعدل الآثار. الأسطر الفارغة للتنسيق.

الأسطر 1–13

```
from copy import deepcopy
from dataclasses import dataclass
from pathlib import Path
from time import perf_counter

import numpy as np
import torch
from torch import nn

from xai_finance.data.preprocess import TrainingScalers, fit_training_scalers
from xai_finance.data.windows import WindowDataset
from xai_finance.seed import select_device
from xai_finance.storage import write_json
```

deepcopy وdataclass وPath وperf_counter قياسية؛ NumPy وPyTorch خارجيان؛ TrainingScalers وWindowDataset وselect_device وwrite_json من المشروع للتقييس والبيانات والجهاز وحفظ السجل.

الأسطر 16–21

```
class PriceLSTM(nn.Module):
    def __init__(self, architecture: dict):
        super().__init__()
        self.lstm = nn.LSTM(architecture["features"], architecture["hidden_size"], batch_first=True)
        self.dropout = nn.Dropout(architecture["dropout"])
        self.output = nn.Linear(architecture["hidden_size"], 1)
```

PriceLSTM يرث nn.Module؛ المنشئ يستدعي منشئ الأب ويكوّن nn.LSTM بمعامل عدد السمات والحجم المخفي وbatch_first، ثم Dropout وLinear إلى عدد واحد. لا يحدد num_layers فتستخدم طبقة LSTM واحدة؛ dropout هنا بعد LSTM لا بين طبقاتها.

الأسطر 23–25

```
    def forward(self, inputs: torch.Tensor) -> torch.Tensor:
        sequence, _ = self.lstm(inputs)
        return self.output(self.dropout(sequence[:, -1, :])).squeeze(-1)
```

forward يستقبل Tensor بالشكل batch×time×features، ويأخذ سلسلة خرج LSTM ويتجاهل زوج الحالات النهائية؛ يختار آخر خطوة ويطبق dropout والرأس الخطي، ثم squeeze(-1) فيعيد قيمة واحدة لكل عينة. حالات البداية الافتراضية يديرها PyTorch عندما لا تمرر.

الأسطر 28–34

```
@dataclass
class LSTMPredictor:
    network: PriceLSTM
    scalers: TrainingScalers
    lookback: int
    feature_count: int
    device: torch.device
```

LSTMPredictor حاوية dataclass تضم الشبكة ومعياري التدريب وطول النافذة وعدد السمات والجهاز؛ منشئها مولّد.

الأسطر 36–40

```
    def predict(self, flattened: np.ndarray) -> np.ndarray:
        windows = np.asarray(flattened).reshape(-1, self.lookback, self.feature_count)
        scaled = self.scalers.transform_windows(windows)
        self.network.eval()
        outputs = []
```

predict تأخذ مصفوفة مسطحة وتعيد تشكيلها إلى نوافذ وتطبق معيار المدخلات المحفوظ، وتحول الشبكة إلى eval وتعِد قائمة النتائج. شكل خاطئ قد يفشل أو يعيد تشكيل غير مقصود إن كان العدد قابلاً للقسمة.

الأسطر 41–45

```
        with torch.no_grad():
            for start in range(0, len(scaled), 256):
                batch = torch.as_tensor(scaled[start:start + 256], device=self.device)
                outputs.append(self.network(batch).float().cpu().numpy())
        return self.scalers.inverse_target(np.concatenate(outputs))
```

بلا حساب تدرجات تقسم المدخلات دفعات حتى 256، تنقلها للجهاز وتجمع تنبؤات float على CPU ثم تعكس معيار الهدف. np.concatenate يفترض وجود عينة واحدة على الأقل؛ لا تحدث أوزان.

الأسطر 48–53

```
def probe_precision(network: PriceLSTM, fixture: tuple, requested: bool) -> tuple[torch.dtype | None, dict]:
    inputs, target = fixture
    if not requested or inputs.device.type != "cuda":
        return None, {"effective_precision": "fp32", "reason": "CPU or AMP disabled"}
    network.eval()
    precision = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
```

probe_precision تأخذ الشبكة وعينة تدريب وطلب AMP؛ تعيد FP32 في CPU أو عند تعطيل AMP، وإلا تدخل eval وتختار BF16 عند دعم CUDA أو FP16.

الأسطر 54–60

```
    try:
        with torch.no_grad():
            full_loss = nn.functional.mse_loss(network(inputs), target)
            with torch.autocast("cuda", dtype=precision):
                mixed = network(inputs)
            mixed_loss = nn.functional.mse_loss(mixed.float(), target)
        difference = float(abs(mixed_loss - full_loss) / full_loss.clamp_min(1e-8))
```

في try وno_grad تحسب MSE عينة التدريب بالدقة العادية وبـ autocast ثم تحول التنبؤ المختلط إلى float وتقدر فرق الخسارة النسبي بمقام أدناه 1e-8؛ الاختبار لعينة محددة وليس ضمان دقة لكل البيانات.

الأسطر 61–65

```
        if not torch.isfinite(mixed_loss) or difference > 0.05:
            return None, {"effective_precision": "fp32", "reason": "AMP training-fixture loss differs by >5% or
is nonfinite", "relative_loss_difference": difference if np.isfinite(difference) else None}
        return precision, {"effective_precision": str(precision), "relative_loss_difference": difference,
"criterion": "finite training-fixture loss within 5% of FP32"}
    except RuntimeError as failure:
        return None, {"effective_precision": "fp32", "reason": str(failure)}
```

ترفض الدقة المختلطة إذا الخسارة غير منتهية أو الفرق يزيد عن 5%، وإلا تعيد النوع ونتيجة الفحص؛ RuntimeError يعيد FP32 مع السبب. لا يفحص هذا الاختبار جودة التنبؤ العامة.

الأسطر 68–72

```
def scaled_loss(network: PriceLSTM, fixture: tuple, precision: torch.dtype | None) -> torch.Tensor:
    inputs, target = fixture
    with torch.autocast(device_type=inputs.device.type, dtype=precision, enabled=precision is not None):
        predictions = network(inputs)
    return nn.functional.mse_loss(predictions.float(), target)
```

scaled_loss تستخرج المدخل والهدف وتطبق autocast عند تحديد precision وتحسب التنبؤ ثم MSE في float مقابل الهدف المعياري. تستخدمها خطوات التدريب والتحقق.

الأسطر 75–81

```
def batch_probe(network: PriceLSTM, fixture: tuple, configuration: dict) -> tuple[int, list[dict]]:
    inputs, target = fixture
    device = inputs.device
    if device.type == "cpu":
        return min(32, len(inputs)), [{"batch_size": min(32, len(inputs)), "reason": "bounded CPU default"}]
    measurements = []
    free_bytes, _ = torch.cuda.mem_get_info()
```

batch_probe تأخذ عينة التدريب والإعداد. CPU يختار min(32, عدد الأمثلة)؛ على CUDA يجمع القياسات ويقرأ الذاكرة الحرة.

الأسطر 82–89

```
    for candidate in configuration["batch_candidates"]:
        candidate = min(candidate, len(inputs))
        try:
            measurements.append(measure_batch(network, (inputs[:candidate], target[:candidate]), free_bytes))
        except torch.cuda.OutOfMemoryError:
            network.zero_grad(set_to_none=True)
            torch.cuda.empty_cache()
            measurements.append({"batch_size": candidate, "oom": True})
```

يجرب كل حجم مرشح مقيداً بعدد العينات ويدعو measure_batch؛ في نفاد ذاكرة CUDA يصفر التدرجات ويفرغ cache ويسجل فشل الحجم ثم يتابع.

الأسطر 90–97

```
    valid = [measurement for measurement in measurements if measurement.get("accepted")]
    if not valid:
        if min(configuration["batch_candidates"]) <= 1:
            raise RuntimeError("No LSTM batch fits the available VRAM")
        smaller = {**configuration, "batch_candidates": [max(1, min(configuration["batch_candidates"]) // 2)]}
        selected, retries = batch_probe(network, fixture, smaller)
        return selected, measurements + retries
    return max(measurement["batch_size"] for measurement in valid), measurements
```

يحصر القياسات المقبولة؛ إذا لم يقبل شيء يعيد المحاولة بنصف أصغر مرشح حتى واحد ثم يرفع RuntimeError، وإلا يختار أكبر حجم مقبول لا أعلى throughput.

الأسطر 100–107

```
def measure_batch(network: PriceLSTM, fixture: tuple, free_bytes: int) -> dict:
    torch.cuda.reset_peak_memory_stats()
    network.train()
    for _ in range(2):
        network.zero_grad(set_to_none=True)
        scaled_loss(network, fixture, None).backward()
    torch.cuda.synchronize()
    started = perf_counter()
```

measure_batch يعيد إحصاءات الذروة ويضع الشبكة في train ويجري خطوتي backward للإحماء مع تصفير التدرجات، ثم يزامن CUDA ويبدأ القياس. لا يجري optimizer.step هنا.

الأسطر 108–114

```
    for _ in range(3):
        network.zero_grad(set_to_none=True)
        scaled_loss(network, fixture, None).backward()
    torch.cuda.synchronize()
```

```
    elapsed = perf_counter() - started
    peak = torch.cuda.max_memory_allocated()
    return {"batch_size": len(fixture[0]), "samples_per_second": 3 * len(fixture[0]) / elapsed,
"elapsed_seconds": elapsed, "peak_allocated_bytes": peak, "free_vram_before_probe": free_bytes, "accepted": peak
< 0.8 * free_bytes}
```

يقيس ثلاث عمليات backward إضافية متزامنة ويحسب الزمن والإنتاجية وذروة الذاكرة ويقبل الذروة إذا كانت أقل من 80% من الذاكرة الحرة المسجلة. المقياس تقريبي ولا يضمن الذاكرة المتاحة لاحقاً.

الأسطر 117–123

```
def train_epoch(network: PriceLSTM, tensors: tuple, optimization: dict) -> float:
    inputs, target = tensors
    network.train()
    losses = []
    for start in range(0, len(inputs), optimization["batch_size"]):
        stop = start + optimization["batch_size"]
        optimization["optimizer"].zero_grad(set_to_none=True)
```

train_epoch يحدد train ويقسم الموترات بالتتابع إلى دفعات ويصفر تدرجات Adam قبل كل دفعة. لا يظهر خلط للعينات داخل هذه الحلقة.

الأسطر 124–129

```
        loss = scaled_loss(network, (inputs[start:stop], target[start:stop]), optimization["precision"])
        if not torch.isfinite(loss):
            raise FloatingPointError("Nonfinite LSTM training loss; run aborted")
        optimization["grad_scaler"].scale(loss).backward()
        optimization["grad_scaler"].unscale_(optimization["optimizer"])
        nn.utils.clip_grad_norm_(network.parameters(), 1.0, error_if_nonfinite=True)
```

يحسب الخسارة ويفشل عند قيمة غير منتهية، ثم scale(loss).backward يحسب التدرجات عبر autograd وBPTT، وunscale يفك مقياسها قبل قص norm عند 1.0 مع رفض القيم غير المنتهية.

الأسطر 130–133

```
        optimization["grad_scaler"].step(optimization["optimizer"])
        optimization["grad_scaler"].update()
        losses.append((float(loss.detach()), len(inputs[start:stop])))
    return sum(loss * count for loss, count in losses) / len(inputs)
```

يحدث المعاملات عبر GradScaler والـ optimizer ثم يحدث scaler ويسجل الخسارة وحجم كل دفعة؛ يعيد متوسط خسارة موزوناً بحجم الدفعة، لا متوسط متوسطات غير موزون.

الأسطر 136–137

```
def checkpoint_state(network: PriceLSTM, optimization: dict, progress: dict) -> dict:
    return {"state_dict": network.state_dict(), "optimizer_state": optimization["optimizer"].state_dict(),
```

```
"amp_scaler_state": optimization["grad_scaler"].state_dict(), "torch_rng_state": torch.get_rng_state(),
"cuda_rng_states": torch.cuda.get_rng_state_all() if torch.cuda.is_available() else [], **progress}
```

checkpoint_state يجمع أوزان الشبكة وحالة optimizer وAMP وحالة مولدات Torch وCUDA عند توفره مع معلومات التقدم. يعيد قاموساً؛ الحفظ يجري في optimize_epochs.

الأسطر 140–148

```
def optimize_epochs(network: PriceLSTM, tensors: dict, settings: dict) -> tuple[list[dict], dict]:
    optimization, configuration, folder = settings["optimization"], settings["configuration"],
settings["folder"]
    best_loss, stale, history, best_state = float("inf"), 0, [], None
    started = perf_counter()
    for epoch in range(configuration["epochs"]):
        training_loss = train_epoch(network, tensors["train"], optimization)
        network.eval()
        with torch.no_grad():
            validation_loss = float(scaled_loss(network, tensors["validation"], None))
```

optimize_epochs تستخرج الإعدادات وتبدأ بأفضل خسارة لا نهائية وتاريخ فارغ، وتكرر epochs: تدريب، ثم eval وno_grad وخسارة تحقق FP32.

الأسطر 149–156

```
        if not np.isfinite(validation_loss):
            raise FloatingPointError("Nonfinite validation loss")
        history.append({"epoch": epoch + 1, "train_loss": training_loss, "validation_loss": validation_loss,
"elapsed_seconds": perf_counter() - started})
        progress = {"epoch": epoch + 1, "architecture": settings["architecture"], "configuration":
configuration}
        torch.save(checkpoint_state(network, optimization, progress), folder / "last.pt")
        if validation_loss < best_loss:
            best_loss, stale, best_state = validation_loss, 0, deepcopy(network.state_dict())
            torch.save(checkpoint_state(network, optimization, progress), folder / "best.pt")
```

ترفض خسارة تحقق غير منتهية وتسجل التاريخ والإعدادات وتحفظ last.pt كل epoch؛ إذا تحسنت الخسارة تنسخ state_dict وتحفظ best.pt وتعيد عداد الصبر إلى صفر.

الأسطر 157–164

```
        else:
            stale += 1
        write_json(folder / "history.json", history)
        print(f"LSTM epoch {epoch + 1}: train={training_loss:.6g}, val={validation_loss:.6g}", flush=True)
        if stale >= configuration["patience"]:
            break
    network.load_state_dict(best_state)
    return history, {"best_validation_scaled_mse": best_loss, "best_epoch": min(history, key=lambda row:
row["validation_loss"])["epoch"], "epochs_executed": len(history)}
```

عند عدم التحسن تزيد stale، وتكتب history.json وتطبع المعلومات وتتوقف عندما تبلغ patience، ثم تعيد أوزان أفضل تحقق وتعيد السجل ومعلومات الاختيار. يفترض وجود epoch واحد على الأقل وتحسن

صالح.

الأسطر 167–174

```
def fit_lstm(windows: WindowDataset, configuration: dict, folder: Path) -> tuple[LSTMPredictor, dict]:
    started = perf_counter()
    folder.mkdir(parents=True, exist_ok=True)
    device, reason = select_device(configuration["device"])
    scalers = fit_training_scalers(windows)
    tensors = training_tensors(windows, scalers, device)
    architecture = {"features": windows.x.shape[-1], "hidden_size": configuration["hidden_size"], "dropout":
configuration["dropout"]}
    network = PriceLSTM(architecture).to(device)
```

fit_lstm تنشئ المجلد وتختار الجهاز ومعايير التدريب والموترات ثم تبني architecture من البيانات والإعدادات وPriceLSTM على الجهاز. تستدعيها train_asset؛ لها آثار كتابة عبر التدريب.

الأسطر 175–181

```
    fixture = tuple(tensor[:max(configuration["batch_candidates"])] for tensor in tensors["train"])
    cpu_rng, cuda_rng = torch.get_rng_state(), torch.cuda.get_rng_state_all() if device.type == "cuda" else []
    precision, precision_audit = probe_precision(network, fixture, configuration["amp"])
    batch_size, probes = batch_probe(network, fixture, configuration)
    torch.set_rng_state(cpu_rng)
    if cuda_rng:
        torch.cuda.set_rng_state_all(cuda_rng)
```

تأخذ fixture من بداية التدريب للاختبارات، وتحفظ حالات RNG ثم تفحص الدقة وحجم الدفعة وتعيد حالات RNG للحد من أثر الفحوص على العشوائية اللاحقة؛ أوزان الشبكة لا يحدثها optimizer في الفحوص.

الأسطر 182–186

```
    optimization = {"batch_size": batch_size, "optimizer": torch.optim.Adam(network.parameters(),
lr=configuration["learning_rate"]), "precision": precision, "grad_scaler": torch.amp.GradScaler(device.type,
enabled=precision == torch.float16)}
    _, selection = optimize_epochs(network, tensors, {"optimization": optimization, "configuration":
configuration, "folder": folder, "architecture": architecture})
    predictor = LSTMPredictor(network, scalers, windows.x.shape[1], windows.x.shape[2], device)
    metadata = {"device": str(device), "device_reason": reason, "precision": precision_audit, "batch_size":
batch_size, "batch_probes": probes, "selection": selection, "training_seconds": perf_counter() - started,
"scaler_origin_cutoff": scalers.fitted_origin_cutoff, "scaler_target_cutoff": scalers.fitted_target_cutoff}
    return predictor, metadata
```

تنشئ Adam بمعدل الإعداد وGradScaler مفعلاً لـ FP16 فقط، وتستدعي optimize_epochs وتبني LSTMPredictor وتعيده مع معلومات الجهاز والدقة وحجم الدفعة والاختيار والتوقيت وحدود التقييس.

الأسطر 189–196

```
def training_tensors(windows: WindowDataset, scalers: TrainingScalers, device: torch.device) -> dict:
    tensors = {}
    for partition in ["train", "validation"]:
        indices = windows.partitions[partition]
```

```
        inputs = scalers.transform_windows(windows.x[indices])
        target = scalers.target.transform(windows.y[indices, None]).ravel().astype(np.float32)
        tensors[partition] = (torch.as_tensor(inputs, device=device), torch.as_tensor(target, device=device))
    return tensors
```

training_tensors تنشئ قسمي train وvalidation فقط؛ تحول نوافذهما وتقيّس y بعمود هدف ثم float32 وتنقلهما للجهاز وتعيد قاموس أزواج. لا يستعمل الاختبار في الملاءمة أو الاختيار.

الأسطر 199–205

```
def load_lstm(folder: Path, scalers: TrainingScalers, requested: str = "auto") -> LSTMPredictor:
    device, _ = select_device(requested)
    checkpoint = torch.load(folder / "best.pt", map_location=device, weights_only=True)
    network = PriceLSTM(checkpoint["architecture"]).to(device)
    network.load_state_dict(checkpoint["state_dict"])
    network.eval()
    return LSTMPredictor(network, scalers, checkpoint["configuration"]["effective_lookback"],
checkpoint["architecture"]["features"], device)
```

load_lstm يختار الجهاز ويحمل best.pt مع map_location وweights_only=True ثم يعيد بناء الشبكة وأوزانها واeval ويعيد Predictor مع المقاييس الممررة وطول النافذة المحفوظ. فشل الملف أو عدم توافقه ينتشر.

شرح ·c/xai_finance/models/sklearn_models.py

بصمة المصدر SHA-256:

.8c362c55823c423b21172dbc532f21155f3d3715895f191b55a601704b2eade7

المقاطع من المصدر الفعلي؛ لا تشغّل هذه الوثيقة التدريب أو تعدل الآثار. الأسطر الفارغة للتنسيق.

الأسطر 1–11

```
from time import perf_counter

import numpy as np
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from xai_finance.data.windows import WindowDataset
from xai_finance.models.linear_preprocess import IndependentColumns
```

perf_counter مؤقت معياري؛ NumPy للحسابات؛ sklearn توفر نموذجي الأشجار وLinearRegression وMSE وPipeline وStandardScaler. WindowDataset وIndependentColumns وحدتان داخليتان.

الأسطر 14–17

```
def candidates(configuration: dict) -> dict[str, list]:
    seed, trees = configuration["seed"], configuration["trees"]
    forests = [RandomForestRegressor(n_estimators=trees, max_depth=5, min_samples_leaf=3, random_state=seed,
n_jobs=4)]
    boosters = [GradientBoostingRegressor(n_estimators=trees, max_depth=2, learning_rate=0.03,
random_state=seed)]
```

candidates تستقبل configuration وتقرأ seed وtrees؛ تنشئ مرشح غابة بعمق 5 وأقل ورقة 3 وبأربعة عمال، ومرشح تعزيز بعمق 2 ومعدل تعلم 0.03. عدد الأشجار والبذرة من الإعداد لا ثوابت عامة.

الأسطر 18–21

```
    if configuration["search"]:
        forests.append(RandomForestRegressor(n_estimators=trees, max_depth=10, min_samples_leaf=3,
random_state=seed, n_jobs=4))
        boosters.append(GradientBoostingRegressor(n_estimators=trees, max_depth=3, learning_rate=0.1,
random_state=seed))
    return {"LinearRegression": [make_pipeline(StandardScaler(), IndependentColumns(), LinearRegression())],
"RandomForest": forests, "GradientBoosting": boosters}
```

عند search تضيف غابة عمق 10 وتعزيزاً بعمق 3 ومعدل 0.1، ثم تعيد قاموس قوائم المرشحين. الانحدار يتكون من StandardScaler ثم IndependentColumns ثم LinearRegression؛ ملاءمة Pipeline تطبق الخطوتين على التدريب فقط.

الأسطر 24–29

```
def fit_candidates(estimators: list, windows: WindowDataset) -> tuple[object, dict]:
    training, validation = windows.partitions["train"], windows.partitions["validation"]
    trials = []
    fitted = []
    for estimator in estimators:
        started = perf_counter()
```

fit_candidates تأخذ قائمة المقدرات والنوافذ وتستخرج فهارس التدريب والتحقق وتنشئ سجلات ومرشحين ملائمين؛ لكل مرشح تبدأ ساعة القياس. لا يقرأ الاختبار في هذه الدالة.

الأسطر 30–33

```
        estimator.fit(windows.flat[training], windows.y[training])
        rmse = float(np.sqrt(mean_squared_error(windows.y[validation],
estimator.predict(windows.flat[validation]))))
        trials.append({"parameters": str(estimator), "validation_rmse": rmse, "elapsed_seconds": perf_counter()
- started})
        fitted.append(estimator)
```

يلائم المرشح على النوافذ المسطحة وأهداف التدريب ثم يحسب جذر متوسط مربع خطأ التحقق ويسجل المعاملات والزمن ويحتفظ بالكائن المُلائم. التحقق يُستخدم للاختيار وليس تقديراً نهائياً غير متحيز للأداء.

الأسطر 34–38

```
    selected = int(np.argmin([trial["validation_rmse"] for trial in trials]))
    selected_model = fitted[selected]
    if isinstance(selected_model, RandomForestRegressor):
        selected_model.set_params(n_jobs=1)
    return selected_model, {"selection": "validation RMSE only; no test access; no refit", "selected_trial":
selected, "trials": trials, "training_seconds": sum(trial["elapsed_seconds"] for trial in trials), "inference":
"Forest predictions sum trees serially for deterministic floating-point accumulation"}
```

argmin يختار أدنى RMSE ويعيد النموذج المُلائم بلا إعادة تدريب مع قاموس التفاصيل؛ للغابة يضبط n_jobs=1 لتجميع أشجار متسلسل في الاستدلال. trials فارغة تؤدي إلى فشل argmin؛ يضمن المسار المعتاد وجود مرشحين.

شرح `src/xai_finance/pipelines/__init__.py`

بصمة المصدر SHA-256:

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الملف فارغ عند البصمة المذكورة؛ لا يحتوي أسطراً أو استيرادات أو دوال، ويحدد مجلد الحزمة عند الاستيراد.

شرح src/xai_finance/pipelines/explain.py

بصمة المصدر SHA-256:

.62a6194d317aeac61fb9d69d7f99c26754c1a6bfb68f36a9855763bc07253ab1

شرح مسار توليد التفسير؛ لم يُنفذ في هذه الجلسة.

الأسطر 1–7

```
from pathlib import Path
from time import perf_counter

import numpy as np
import pandas as pd
import torch
from threadpoolctl import threadpool_limits
```

Path وperf_counter معياريتان؛ NumPy وpandas وtorch وthreadpoolctl خارجية للمصفوفات والجداول وCUDA وحد الخيوط.

الأسطر 9–15

```
from xai_finance.explain.lime_explain import explain_lime
from xai_finance.explain.quality import attribution_stability, lime_weight_vector
from xai_finance.explain.shap_explain import explain_shap, shap_reconstruction
from xai_finance.pipelines.train import load_models, load_windows
from xai_finance.reports.plots import explanation_charts, save_figure
from xai_finance.seed import seed_experiment
from xai_finance.storage import read_json, write_json
```

استيرادات المشروع تربط مفسري LIME وSHAP واختبارات الجودة وتحميل النماذج والنوافذ والرسوم والبذور والتخزين.

الأسطر 18–19

```
def fixed_subset(indices: np.ndarray, count: int) -> np.ndarray:
    return indices[np.linspace(0, len(indices) - 1, min(count, len(indices)), dtype=int)]
```

fixed_subset تأخذ فهارس وعدداً وتستعمل linspace بفهرسة صحيحة لاختيار حتى count مواقع متباعدة من بداية إلى نهاية الفهارس؛ تعيد فهارس أصلية لا عينات عشوائية. عدد صفر أو فهارس فارغة يقود إلى مجموعة فارغة قد لا تحتملها المراحل اللاحقة.

الأسطر 22–32

```
def explain_experiment(folder: Path) -> None:
    metadata = read_json(folder / "run.json")
    configuration = metadata["configuration"]
    seed_experiment(configuration["seed"])
    for asset in configuration["assets"]:
        with threadpool_limits(limits=4):
            explain_asset(folder / asset, configuration)
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
    metadata["status"] = "TRAINED_AND_EXPLAINED"
    write_json(folder / "run.json", metadata)
```

explain_experiment تقرأ run.json وإعداداته وتضبط البذرة، ثم لكل أصل تحد الخيوط وتستدعي explain_asset وتفرغ cache عند توفر CUDA. بعد انتهاء جميع الأصول تحدث status وتكتب run.json؛ فشل مبكر يترك آثاراً جزئية.

الأسطر 35–40

```
def explain_asset(folder: Path, configuration: dict) -> None:
    windows = load_windows(folder)
    models = load_models(folder, configuration["device"])
    training = windows.flat[windows.partitions["train"]]
    background_indices = fixed_subset(windows.partitions["train"], configuration["background_samples"])
    background = windows.flat[background_indices]
```

explain_asset تحمل النوافذ والنماذج، وتحدد تدريباً مسطحاً كاملاً لمرجع LIME وخلفية SHAP من مجموعة ثابتة متباعدة داخل التدريب فقط. تقييد الخلفية يحمي من استخدام الاختبار كمرجع لكنه لا يضمن تمثيل التوزيع.

الأسطر 41–47

```
    for name, model in models.items():
        count = configuration["lstm_explain_samples"] if name == "LSTM" else configuration["explain_samples"]
        indices = fixed_subset(windows.partitions["test"], count)
        request = {"observations": windows.flat[indices], "names": windows.flat_names, "seed":
configuration["seed"], "cycles": configuration["permutation_cycles"]}
        context = {"folder": folder, "name": name, "model": model, "windows": windows, "indices": indices,
"background_indices": background_indices, "background": background, "training": training, "configuration":
configuration}
        print(f"{folder.name}: explaining {name}, {len(indices)} fixed test observations", flush=True)
        explain_model(context, request)
```

تمر على النماذج الأربعة وتختار عدد تفسيرات LSTM أو غيره وتختار فهارس اختبار ثابتة، وتبني request وcontext بما في ذلك أسماء السمات والبذرة والميزانية، ثم تستدعي explain_model.

الأسطر 50–60

```
def explain_model(context: dict, request: dict) -> None:
    model, indices, windows = context["model"], context["indices"], context["windows"]
```

```
    folder = context["folder"] / context["name"] / "explanations"
    folder.mkdir(parents=True, exist_ok=True)
    started = perf_counter()
    before = model.predict(request["observations"])
    inference_seconds = perf_counter() - started
    started = perf_counter()
    explanation = explain_shap(context["name"], model, context["background"], request)
    shap_seconds = perf_counter() - started
    reconstruction = shap_reconstruction(explanation, before)
```

explain_model تنشئ مجلد النموذج وتحسب وتقيس تنبؤ نفس المشاهدات قبل التفسير، وتولد SHAP وتقيس زمنه ثم تتحقق من إعادة بناء التنبؤ؛ فشل assert يوقف المسار.

الأسطر 61–66

```
    np.savez_compressed(folder / "shap.npz", values=explanation.values, base_values=explanation.base_values,
observations=request["observations"], indices=indices, feature_names=np.asarray(windows.flat_names))
    lime_audits, lime_explanations = explain_local_dates(context, folder)
    repeat_request = {**request, "seed": request["seed"] + 1, "observations": request["observations"][:1]}
    repeated = explain_shap(context["name"], model, context["background"], repeat_request)
    after = model.predict(request["observations"])
    np.testing.assert_array_equal(before, after)
```

تحفظ قيم SHAP والمرجع والمشاهدات والفهارس والأسماء في NPZ، وتولد LIME للتواريخ وتعيد SHAP لأول مشاهدة ببذرة أخرى؛ ثم تعيد التنبؤ وتطلب تطابقاً تاماً قبل وبعد. هذا الفحص لا يثبت ثبات كل الحالات.

الأسطر 67–73

```
    quality = {"shap_reconstruction": reconstruction, "prediction_unchanged_max_abs": float(np.max(np.abs(before
- after))), "shap_elapsed_seconds": shap_seconds, "shap_samples": len(indices), "test_indices":
indices.tolist(), "target_dates": [str(windows.targets[index].date()) for index in indices], "feature_cutoffs":
[str(windows.origins[index].date()) for index in indices], "background_indices":
context["background_indices"].tolist(), "background_max_target_date":
str(windows.targets[context["background_indices"][-1]].date()), "shap_seed_stability":
attribution_stability(explanation.values[0], repeated.values[0]), "lime_local": lime_audits,
"lime_seed_stability": attribution_stability(lime_weight_vector(lime_explanations[0], len(windows.flat_names)),
lime_weight_vector(lime_explanations[1], len(windows.flat_names))), "limitations": "Marginal masking and LIME
neighborhoods can violate correlated technical-feature/time/OHLC constraints. Attributions are associations, not
causality; global means cover only the declared subset."}
    quality["inference_elapsed_seconds_same_subset"] = inference_seconds
    quality["shap_overhead_ratio_same_subset"] = shap_seconds / max(inference_seconds, 1e-12)
    write_json(folder / "quality.json", quality)
    units = read_json(context["folder"] / "data_audit.json")["snapshot"]["units"]
    explanation_charts(explanation, folder, {"title": f"{context['folder'].name} | {context['name']} | {units}",
"date": str(windows.targets[indices[0]].date()), "lookback": windows.x.shape[1], "features":
windows.feature_names})
    save_regime_drivers(explanation.values, context, folder)
```

تجمع مقاييس إعادة البناء وأزمنة التفسير وتواريخ الهدف والأصل والخلفية واستقرار البذرة وقيود الترابط، وتضيف زمن inference ونسبة overhead مع مقام محمي من الصفر، وتكتب quality.json وترسم الآثار وتحسب محركات الحالات. مقارنة الزمن تعتمد نفس العينة وقياساً واحداً.

الأسطر 76–83

```
def explain_local_dates(context: dict, folder: Path) -> tuple[list[dict], list]:
    windows, configuration = context["windows"], context["configuration"]
    request = {"names": windows.flat_names, "seed": configuration["seed"], "samples":
configuration["lime_samples"]}
    audits, first_explanation = [], None
    for index in context["indices"]:
        started = perf_counter()
        explanation, audit = explain_lime(context["training"], context["model"].predict, windows.flat[index],
request)
        audit.update({"elapsed_seconds": perf_counter() - started, "target_date":
str(windows.targets[index].date()), "feature_cutoff": str(windows.origins[index].date())})
```

explain_local_dates تنشئ طلب LIME بأسماء وبذرة وعدد عينات، وتكرر على فهارس الاختبار: تستدعي explain_lime على تدريب مسطح والتنبؤ والعينة، وتضيف الزمن وتاريخي الهدف والأصل.

الأسطر 84–92

```
        write_json(folder / f"lime_{index}.json", audit)
        explanation.save_to_file(str(folder / f"lime_{index}.html"))
        audits.append(audit)
        if first_explanation is None:
            first_explanation = explanation
            explanation.save_to_file(str(folder / "lime_local.html"))
            explanation.as_pyplot_figure()
            units = read_json(context["folder"] / "data_audit.json")["snapshot"]["units"]
            save_figure(folder / "lime_local", f"{context['folder'].name} | {context['name']} | {units} | LIME |
{audit['target_date']}")
```

لكل عينة تكتب JSON وHTML وتضيف التدقيق للقائمة؛ لأول تفسير تحفظ أيضاً HTML وصورة من matplotlib بوحدة الأصل. هذه آثار كتابة ولا تنشأ عند مجرد قراءة الوثيقة.

الأسطر 93–95

```
    repeat, repeat_audit = explain_lime(context["training"], context["model"].predict,
windows.flat[context["indices"][0]], {**request, "seed": request["seed"] + 1})
    write_json(folder / "lime.json", {"primary": audits[0], "repeat_seed": repeat_audit, "all_dates": audits})
    return audits, [first_explanation, repeat]
```

تعيد LIME لأول فهرس ببذرة مختلفة، وتكتب lime.json شاملاً التدقيق الأساسي والمكرر وجميع التواريخ، ثم تعيد قائمة التدقيق وتفسيرَي أول عينة. تفترض وجود فهرس اختبار واحد على الأقل.

الأسطر 98–103

```
def save_regime_drivers(attributions: np.ndarray, context: dict, folder: Path) -> None:
    from xai_finance.evaluate.regimes import regime_labels

    windows = context["windows"]
    regimes = regime_labels(windows.trailing_returns[context["indices"]], context["configuration"]
["regime_threshold"])
    importance = np.abs(attributions.reshape(len(attributions), windows.x.shape[1], -1)).sum(axis=1)
```

save_regime_drivers تستورد regime_labels داخلياً، وتحسب حالات العينات المختارة من عوائدها الماضية وعتبة الإعداد، وتشكل قيم SHAP إلى عينة×زمن×سمة ثم تجمع القيم المطلقة عبر التأخيرات.

الأسطر 104–108

```
    rankings = []
    for regime in np.unique(regimes):
        for name, score in zip(windows.feature_names, importance[regimes == regime].mean(axis=0)):
            rankings.append({"regime": regime, "feature": name, "mean_sum_abs_shap_over_lags": float(score),
"explained_observations": int((regimes == regime).sum())})
    pd.DataFrame(rankings).to_csv(folder / "regime_drivers.csv", index=False)
```

لكل حالة موجودة ولكل سمة تحسب متوسط مجموع القيم المطلقة وعدد المشاهدات المفسرة، وتكتب regime_drivers.csv. هو تلخيص لعينة التفسير المحدودة لا ترتيب سببي لكل السوق.

شرح src/xai_finance/pipelines/report.py

بصمة المصدر SHA-256:

.30234656c31f50f42c6b899100295d15c9afb7ddf343ad2bf8e4480b1e19d305

الاقتباسات من المصدر؛ لم تنفذ أعمال التقرير أو تعدل البيانات.

الأسطر 1–14

```
from datetime import datetime, timezone
from pathlib import Path
import shutil

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.express as px

from xai_finance.config import ROOT
from xai_finance.evaluate.metrics import forecast_metrics
from xai_finance.pipelines.train import load_models, load_windows
from xai_finance.reports.plots import forecast_charts, loss_chart, metric_charts, research_flow, save_figure
from xai_finance.storage import read_json, sha256, write_json
```

استيرادات قياسية للوقت والمسار والنسخ، وخارجية للرسوم والمصفوفات والجداول وPlotly، وداخلية للمقاييس والنماذج والرسوم والتخزين.

الأسطر 17–30

```
def generate_report(folder: Path) -> None:
    metadata = read_json(folder / "run.json")
    research_flow(folder / "plots")
    asset_summaries = []
    for asset in metadata["assets_completed"]:
        asset_summaries.append(asset_report(folder / asset))
    report = {"run_id": metadata["run_id"], "provenance": "NEWLY_COMPUTED", "generated_utc":
datetime.now(timezone.utc).isoformat(), "assets": asset_summaries, "limitations": research_limitations()}
    write_json(folder / "summary.json", report)
    write_research_markdown(folder, report)
    metadata["status"] = "REPORT_GENERATED"
    metadata["reproducibility"] = {"requirements_lock_sha256": sha256(ROOT / "requirements-lock.txt"),
"code_sha256": {str(path.relative_to(ROOT)).replace("\\", "/"): sha256(path) for path in sorted((ROOT /
"src").rglob("*.py"))}}
    write_json(folder / "run.json", metadata)
    write_inventory(folder)
    print(f"Generated report: {folder / 'RESULTS.md'}", flush=True)
```

generate_report تقرأ run.json وتولد رسم التدفق وتقارير الأصول، وتكتب summary والنتائج وتغير status وتسجل بصمات القفل والشفرة ثم تكتب الفهرس. تشغيلها يغير ملفات التجربة ولم يجر هنا.

الأسطر 33–49

```
def asset_report(folder: Path) -> dict:
    audit = read_json(folder / "data_audit.json")
    metrics = read_json(folder / "metrics.json")
    predictions = pd.read_csv(folder / "predictions.csv")
    verify_saved_metrics(predictions, metrics)
    plots = folder / "plots"
    title = f"{folder.name} | {audit['snapshot']['units']}"
    forecast_charts(predictions, plots, title)
    metric_charts(metrics, plots, title)
    loss_chart(read_json(folder / "LSTM/history.json"), plots)
    extra_charts(folder, plots, predictions, audit)
    export_coefficients(folder)
    figure = px.line(predictions, x="target_date", y="predicted", color="model", title=title + " | TEST
predictions")
    actual = predictions.drop_duplicates("target_date")
    figure.add_scatter(x=actual.target_date, y=actual.actual, name="Actual", line={"color": "black"})
    figure.write_html(plots / "forecasts.html", include_plotlyjs=True, full_html=True)
    return {"asset": folder.name, "units": audit["snapshot"]["units"], "frequency": audit["snapshot"]
["frequency"], "snapshot_sha256": audit["snapshot"]["sha256"], "raw_observations": audit["snapshot"]["rows"],
"split": audit["split"], "metrics": metrics, "training": read_json(folder / "training.json"),
"explanations_available": all((folder / name / "explanations/quality.json").exists() for name in
["LinearRegression", "RandomForest", "GradientBoosting", "LSTM"])}
```

asset_report تقرأ التدقيق والمقاييس والتنبؤات وتتحقق من المقاييس وتولد رسوم التنبؤ والخطأ والخسارة والإضافات والمعاملات وHTML مستقل Plotly، وتعيد ملخص الأصل مع توفر التفسيرات. قد تفشل إذا غاب أثر مطلوب.

الأسطر 52–58

```
def verify_saved_metrics(predictions: pd.DataFrame, saved: list[dict]) -> None:
    for metric_row in saved:
        subset = predictions[predictions.model == metric_row["model"]]
        recomputed = forecast_metrics(subset.actual, subset.predicted, subset.current_close)
        for metric in ["mae", "rmse", "mape_percent", "r2", "direction_accuracy"]:
            if recomputed[metric] is not None:
                np.testing.assert_allclose(recomputed[metric], metric_row[metric], rtol=1e-8, atol=1e-8)
```

verify_saved_metrics تعيد حساب المقاييس الخمسة من التنبؤات حسب النموذج وتقارن القيم غير None بالمحفوظة بتسامح 1e-8؛ هذه مقارنة عددية وليست فحصاً شاملاً للمصدر أو لكل الحقول.

الأسطر 61–73

```
def extra_charts(folder: Path, plots: Path, predictions: pd.DataFrame, audit: dict) -> None:
    regimes = pd.DataFrame(read_json(folder / "regimes.json"))
    pivot = regimes.pivot(index="regime", columns="model", values="mae")
    pivot.plot.bar(figsize=(10, 5))
    plt.ylabel("MAE (original target units)")
    save_figure(plots / "regime_errors", folder.name + " | predeclared trailing-return regimes")
    windows = load_windows(folder)
    plt.figure(figsize=(11, 4))
    for partition, indices in windows.partitions.items():
        plt.plot(windows.targets[indices], windows.y[indices], label=f"{partition}: {len(indices)} labels")
    plt.legend()
    plt.ylabel(audit["snapshot"]["units"])
    save_figure(plots / "temporal_split", folder.name + " | chronological 80/10/10 labels")
```

extra_charts ترسم MAE بحسب الحالة من regimes.json، ثم تقرأ النوافذ وترسم أهداف التقسيم مع الوحدات وتحفظ الرسوم.

الأسطر 74–84

```
    for row in read_json(folder / "metrics.json"):
        plt.figure(figsize=(4, 4))
        confusion = np.asarray(row["up_down_confusion"])
        plt.imshow(confusion, cmap="Blues")
        for (i, j), count in np.ndenumerate(confusion):
            plt.text(j, i, str(count), ha="center", va="center")
        plt.xticks([0, 1], ["Not up", "Up"])
        plt.yticks([0, 1], ["Not up", "Up"])
        plt.xlabel("Predicted change")
        plt.ylabel("Actual change")
        save_figure(plots / f"confusion_{row['model']}", folder.name + " | " + row["model"])
```

لكل نموذج ترسم مصفوفة الالتباس الثنائية المحفوظة بخلايا وأسماء not up وup ومحاور التنبؤ والواقع ثم تحفظ الشكل؛ التعادل يندرج في not up.

الأسطر 87–96

```
def export_coefficients(folder: Path) -> None:
    models = load_models(folder, "cpu")
    linear = models["LinearRegression"]
    names = load_windows(folder).flat_names
    scaler, selector, regressor = [step[1] for step in linear.steps]
    coefficients = np.zeros(len(names))
    coefficients[selector.retained_] = regressor.coef_ / scaler.scale_[selector.retained_]
    pd.DataFrame({"feature": names, "coefficient_original_input_units": coefficients}).to_csv(folder /
"LinearRegression/coefficients.csv", index=False)
    write_json(folder / "LinearRegression/intercept.json", {"original_input_intercept":
float(regressor.intercept_ - np.dot(coefficients, scaler.mean_))})
    write_json(folder / "LinearRegression/rank_audit.json", {"relative_qr_tolerance":
selector.relative_tolerance, "training_rank": selector.rank_, "retained_columns": selector.retained_.tolist(),
"original_columns": len(names), "attribution_caution": "Dependent columns dropped by a training-only numerical
basis choice; zero attributed contribution does not mean economically uninformative."})
```

export_coefficients يحمل النماذج ويفك Pipeline، ويعيد معاملات OLS إلى وحدات مدخلات أصلية بتقسيمها على scale ويضع صفراً للأعمدة المستبعدة؛ الثابت الجديد intercept−Σ(coef_original×mean). يكتب المعاملات والثابت وتدقيق الرتبة، والصفر الناتج من اختيار الأساس لا يثبت انعدام أهمية اقتصادية.

الأسطر 99–100

```
def research_limitations() -> list[str]:
    return ["Reconstruction, not exact replication: thesis omits instrument/date/sample/units and narrative
hyperparameters; appendix implements different models and split.", "Retrospectively adjusted/revised provider
snapshots are not point-in-time historical vintages.", "House-price experiment is monthly, retrospective next-
observation regression, not publication-time forecasting or daily trading.", "All four models use identical full
windows; flattening increases collinearity and dimensionality for tabular models.", "High price-level R2 does
not establish trading skill. Compare with persistence and directional accuracy; no transaction-cost backtest or
investment-return claim.", "Bounded SHAP subset and permutation budgets are approximate; correlated-feature
marginal masking and LIME neighborhoods may be off-manifold. Attributions are not causal.", "Explanation seed
stability is a bounded numerical check, not proof of market robustness or human trust.", "Historical survey
claims have no participant count/raw responses; no new human study was conducted.", "Seeded deterministic
settings improve repeatability; cross-version/hardware bitwise equality is not guaranteed. Runtime batch probing
```

```
may choose a different batch on another machine.", "Held-out results are descriptive for one fixed chronological split; no claim of universal model superiority or statistical significance."]
```

research_limitations تعيد قائمة نصوص تقيد الاستنساخ والبيانات المعدلة والإسكان وR^2 والتفسير والعشوائية والتقسيم؛ لا تجري اختباراً.

الأسطر 103–114

```
def write_research_markdown(folder: Path, summary: dict) -> None:
    lines = [f"# Newly computed results: {summary['run_id']}", "", "Provenance: NEWLY_COMPUTED. Historical Table 4 is separately stored as THESIS_REPORTED_NOT_REPRODUCED. No survey responses were generated.", ""]
    for asset in summary["assets"]:
        test = asset["split"]["test"]
        lines.extend([f"## {asset['asset']}", "", f"Units: {asset['units']}; frequency: {asset['frequency']}; test labels: {test['target_start']} to {test['target_end']} (n={test['n']}).", "", "| Model | MAE | RMSE | MAPE % | R2 | Direction accuracy |", "| --- | ---: | ---: | ---: | ---: | ---: |"])
        for row in asset["metrics"]:
            fields = [f"{row[key]:.6g}" if row[key] is not None else "undefined" for key in ["mae", "rmse", "mape_percent", "r2", "direction_accuracy"]]
            lines.append("| " + row["model"] + " | " + " | ".join(fields) + " |")
        lines.append("")
    lines.extend(["## Scientific limits", ""] + ["- " + limitation for limitation in summary["limitations"]])
    (folder / "RESULTS.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
    shutil.copy2(folder / "RESULTS.md", ROOT / "docs/RECONSTRUCTED_RESULTS.md")
```

write_research_markdown تكتب لكل أصل جدولا للمقاييس بوحداته وتواريخه وقيوده ثم تحفظ RESULTS.md وتنسخها إلى docs/RECONSTRUCTED_RESULTS.md؛ النسخ قد يستبدل المستند السابق عند تشغيل الدالة.

الأسطر 117–119

```
def write_inventory(folder: Path) -> None:
    entries = {str(path.relative_to(folder)).replace("\\", "/"): sha256(path) for path in sorted(folder.rglob("*")) if path.is_file() and path.name not in {"inventory.json", "verification.json"}}
    write_json(folder / "inventory.json", entries)
```

write_inventory تحسب بصمات جميع ملفات المجلد باستثناء inventory.json وverification.json وتكتب JSON؛ هذا جرد سلامة نسبي لا يتضمن الملفين المستبعدين.

شرح src/xai_finance/pipelines/train.py

بصمة المصدر SHA-256:

.a744962021fcb13f7f87caaf69f5f9520f8b66ce2f71522520b193c7bd660f76

المقاطع من المصدر الفعلي؛ لا تشغّل هذه الوثيقة التدريب أو تعدل الآثار. الأسطر الفارغة للتنسيق.

الأسطر 1–13

```
from datetime import datetime, timezone
import importlib.metadata
import os
from pathlib import Path
import platform
import shutil
import subprocess
import sys

import joblib
import numpy as np
import pandas as pd
from threadpoolctl import threadpool_limits
```

مكتبات قياسية للوقت والبيئة والمسارات والعمليات، وjoblib وNumPy وpandas وthreadpoolctl للحفظ والحساب والجداول وحد الخيوط.

الأسطر 15–25

```
from xai_finance.config import ROOT
from xai_finance.data.features import features
from xai_finance.data.windows import WindowDataset, build_windows
from xai_finance.evaluate.metrics import forecast_metrics
from xai_finance.evaluate.regimes import regime_labels, regime_metrics
from xai_finance.io.data_sources import load_snapshot
from xai_finance.models.baselines import persistence
from xai_finance.models.lstm_model import fit_lstm, load_lstm
from xai_finance.models.sklearn_models import candidates, fit_candidates
from xai_finance.seed import seed_experiment
from xai_finance.storage import read_json, sha256, write_json
```

يربط ROOT وبناء السمات والنوافذ والمقاييس والحالات واللقطات وPersistence وLSTM ونماذج sklearn والبذرة والتخزين؛ هذه تبعيات داخلية لمسار التجربة.

الأسطر 28–31

```
def software_environment() -> dict:
    packages = {name: importlib.metadata.version(name) for name in ["numpy", "pandas", "scikit-learn", "torch",
"shap", "lime", "yfinance", "streamlit"]}
    gpu = subprocess.run(["nvidia-smi", "--query-gpu=name,memory.total,driver_version", "--
format=csv,noheader"], capture_output=True, text=True) if shutil.which("nvidia-smi") else None
    return {"python": sys.version, "executable": sys.executable, "platform": platform.platform(), "packages":
```

```
packages, "gpu": gpu.stdout.strip() if gpu else "nvidia-smi unavailable", "pythonhashseed":
os.environ.get("PYTHONHASHSEED", "not set at startup; numerical seeds still fixed")}
```

software_environment تجمع إصدارات الحزم ونسخة Python والمنصة ومعلومات nvidia-smi إن كان موجوداً ومتغير PYTHONHASHSEED؛ تعيد قاموساً وقد تفشل إذا تعذر استعلام إصدار حزمة.

الأسطر 34–39

```
def train_experiment(configuration: dict) -> Path:
    run_id = configuration["name"] + "_" + datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S%fZ")
    folder = ROOT / "artifacts/runs" / run_id
    folder.mkdir(parents=True, exist_ok=False)
    metadata = {"run_id": run_id, "status": "TRAINING", "provenance": "NEWLY_COMPUTED", "created_utc":
datetime.now(timezone.utc).isoformat(), "configuration": configuration, "environment": software_environment(),
"source_sha256": sha256(ROOT / "docs/original_thesis.docx"), "assets_completed": []}
    write_json(folder / "run.json", metadata)
```

train_experiment تبني run_id من اسم الإعداد ووقت UTC، وتنشئ مجلد تشغيل جديداً مع exist_ok=False، وتكتب run.json الأولي مع البيئة وبصمة ملف الرسالة وحالة TRAINING.

الأسطر 40–49

```
    for asset in configuration["assets"]:
        with threadpool_limits(limits=4):
            train_asset(asset, configuration, folder / asset)
        metadata["assets_completed"].append(asset)
        write_json(folder / "run.json", metadata)
    metadata["status"] = "TRAINED_ALL_FOUR_MODELS"
    write_json(folder / "run.json", metadata)
    write_json(ROOT / f"artifacts/latest_{configuration['name']}.json", {"run_id": run_id})
    print(f"Completed training run: {run_id}", flush=True)
    return folder
```

لكل أصل تحد الخيوط بأربعة وتستدعي train_asset ثم تحدث قائمة الإنجاز والملف. بعد اكتمال الكل تحدث الحالة وتكتب مؤشر latest وتعيد مسار التشغيل؛ فشل أثناء الحلقة قد يترك تشغيلًا غير مكتمل دون تحديث المؤشر النهائي.

الأسطر 52–54

```
def save_windows(folder: Path, windows: WindowDataset) -> None:
    folder.mkdir(parents=True, exist_ok=True)
    np.savez_compressed(folder / "windows.npz", x=windows.x, y=windows.y, origins=windows.origins.asi8,
targets=windows.targets.asi8, current=windows.current_close, trailing=windows.trailing_returns,
names=np.asarray(windows.feature_names), **windows.partitions)
```

save_windows تنشئ مجلد الأصل وتحفظ مضغوطاً x وy والتواريخ بوحدة asi8 والقيم والأسماء والتقسيمات في windows.npz؛ هذه كتابة أثر عند التدريب.

الأسطر 57–59

```
def load_windows(folder: Path) -> WindowDataset:
    with np.load(folder / "windows.npz", allow_pickle=False) as saved:
        return WindowDataset(saved["x"], saved["y"], pd.DatetimeIndex(saved["origins"]),
pd.DatetimeIndex(saved["targets"]), saved["current"], saved["trailing"], saved["names"].tolist(), {partition:
saved[partition] for partition in ["train", "validation", "test"]})
```

load_windows تقرأ NPZ دون pickle وتعيد WindowDataset مع تحويل التواريخ وحقول التقسيم؛ تعتمد أسماء وأشكال الحقول المحفوظة.

الأسطر 62–63

```
def split_manifest(windows: WindowDataset) -> dict:
    return {partition: {"n": len(indices), "origin_start": str(windows.origins[indices[0]].date()),
"origin_end": str(windows.origins[indices[-1]].date()), "target_start": str(windows.targets[indices[0]].date()),
"target_end": str(windows.targets[indices[-1]].date())} for partition, indices in windows.partitions.items()}
```

split_manifest تعيد لكل قسم العدد وحدود تاريخ الأصل والهدف من أول فهرس وآخره؛ تفترض الأقسام غير فارغة ومرتبة.

الأسطر 66–74

```
def train_asset(asset: str, configuration: dict, folder: Path) -> None:
    seed_experiment(configuration["seed"])
    prices, manifest = load_snapshot(asset)
    engineered = features(prices, manifest["frequency"])
    lookback = configuration["monthly_lookback"] if manifest["frequency"] == "monthly" else
configuration["lookback"]
    windows = build_windows(engineered, lookback)
    save_windows(folder, windows)
    engineered.to_csv(folder / "features.csv", index_label="feature_cutoff")
    write_json(folder / "data_audit.json", {"snapshot": manifest, "split": split_manifest(windows), "lookback":
lookback, "features": windows.feature_names, "information_set": "All four models use identical flattened or
recurrent full historical windows", "feature_rows_dropped_warmup": len(prices) - len(engineered), "target":
"next observed adjusted close" if manifest["frequency"] == "daily" else "next monthly observed index;
retrospective/revised series, not publication-time forecast"})
```

train_asset تثبت البذرة، تتحقق من لقطة الأصل وتحسب السمات وتختار lookback حسب التردد وتبني وتحفظ النوافذ والسمات وتقرير المصدر والتقسيم والهدف. هدف اليوم التالي هو الرصد التالي وليس بالضرورة اليوم التقويمي التالي.

الأسطر 75–83

```
    selected_models, training_audits = {}, {}
    for model_name, estimators in candidates(configuration).items():
        print(f"{asset}: fitting {model_name} on training only", flush=True)
        selected_models[model_name], training_audits[model_name] = fit_candidates(estimators, windows)
        joblib.dump(selected_models[model_name], folder / f"{model_name}.joblib")
    print(f"{asset}: baseline training complete; starting LSTM", flush=True)
    selected_models["LSTM"], training_audits["LSTM"] = fit_lstm(windows, {**configuration, "effective_lookback":
lookback}, folder / "LSTM")
```

```
    joblib.dump(selected_models["LSTM"].scalers, folder / "LSTM/scalers.joblib")
    write_json(folder / "training.json", training_audits)
```

تجهز المرشحين وتلائمهم وتختار على التحقق وتحفظ كل نموذج joblib، ثم تدرب LSTM وتحفظ scaler وسجل التدريب؛ كل ذلك يحدث فقط عند تشغيل التدريب لا عند قراءة التوثيق.

الأسطر 84–86

```
    evaluate_asset(asset, windows, selected_models, folder, configuration)
    verify_reload(folder, selected_models, windows)
    save_next_forecasts(selected_models, engineered, folder, lookback)
```

تستدعي التقييم، وفحص تحميل CPU، ثم حفظ توقع الرصد التالي للّقطة. ترتيبها يعني أن فشل فحص التحميل يمنع إنشاء توقع لاحق في ذلك التنفيذ.

الأسطر 89–93

```
def evaluate_asset(asset: str, windows: WindowDataset, models: dict, folder: Path, configuration: dict) -> None:
    test = windows.partitions["test"]
    rows, metric_rows = [], []
    predictions = {name: model.predict(windows.flat[test]) for name, model in models.items()}
    predictions["Persistence"] = persistence(windows.current_close[test])
```

evaluate_asset تحدد test وتتنبأ جميع النماذج بالنوافذ المسطحة؛ يعيد LSTMPredictor تشكيلها داخلياً. تضيف Persistence كقيمة الإغلاق الحالي، دون تدريب مرجعي.

الأسطر 94–102

```
    for model_name, predicted in predictions.items():
        model_folder = folder / model_name
        model_folder.mkdir(exist_ok=True)
        table = pd.DataFrame({"asset": asset, "model": model_name, "feature_cutoff": windows.origins[test],
"target_date": windows.targets[test], "actual": windows.y[test], "predicted": predicted, "current_close":
windows.current_close[test], "regime": regime_labels(windows.trailing_returns[test],
configuration["regime_threshold"])})
        table.to_csv(model_folder / "predictions.csv", index=False)
        metrics = {"asset": asset, "model": model_name, "provenance": "NEWLY_COMPUTED",
**forecast_metrics(windows.y[test], predicted, windows.current_close[test])}
        write_json(model_folder / "metrics.json", metrics)
        metric_rows.append(metrics)
        rows.append(table)
```

لكل نموذج تنشئ مجلداً وجدول تواريخ وقيم وحالات، وتكتب تنبؤات الاختبار وتحسب forecast_metrics وتكتب JSON، ثم تضيف البيانات إلى القوائم. الإعداد يحدد عتبة الحالة.

الأسطر 103–107

```
    combined = pd.concat(rows, ignore_index=True)
    combined.to_csv(folder / "predictions.csv", index=False)
    write_json(folder / "metrics.json", metric_rows)
```

```
    write_json(folder / "regimes.json", regime_metrics(combined))
    print(pd.DataFrame(metric_rows)[["asset", "model", "mae", "rmse", "r2"]].to_string(index=False), flush=True)
```

تجمع الجداول وتكتب predictions.csv وmetrics.json وregimes.json ثم تطبع مختصر الأخطاء. هذه النتائج بوحدات الهدف الأصلية بعد عكس معيار LSTM.

الأسطر 110–114

```
def load_models(folder: Path, device: str = "auto") -> dict:
    models = {name: joblib.load(folder / f"{name}.joblib") for name in ["LinearRegression", "RandomForest",
"GradientBoosting"]}
    models["RandomForest"].set_params(n_jobs=1)
    models["LSTM"] = load_lstm(folder / "LSTM", joblib.load(folder / "LSTM/scalers.joblib"), device)
    return models
```

load_models تقرأ نماذج joblib الثلاثة، تجعل الغابة متسلسلة، وتحمل LSTM ومعاييره للجهاز المطلوب؛ تعيد قاموساً. تحميل pickle/joblib يتطلب آثاراً موثوقة.

الأسطر 117–123

```
def verify_reload(folder: Path, trained: dict, windows: WindowDataset) -> None:
    fixture = windows.flat[windows.partitions["test"][:5]]
    reloaded = load_models(folder, "cpu")
    errors = {name: float(np.max(np.abs(model.predict(fixture) - reloaded[name].predict(fixture)))) for name,
model in trained.items()}
    for name, difference in errors.items():
        np.testing.assert_allclose(trained[name].predict(fixture), reloaded[name].predict(fixture), rtol=1e-5,
atol=1e-4)
    write_json(folder / "reload_check.json", {"cpu_reload_max_abs_error": errors, "rtol": 1e-5, "atol": 1e-4})
```

verify_reload تقارن تنبؤات أول خمس نوافذ اختبار بين النموذج الموجود وإعادة تحميله على CPU، وتسجل أقصى فرق وتفرض allclose بـ rtol=1e-5 وatol=1e-4. الفحص لهذه العينة وليس إثبات تطابق جميع البيانات.

الأسطر 126–129

```
def save_next_forecasts(models: dict, engineered: pd.DataFrame, folder: Path, lookback: int) -> None:
    window = engineered.iloc[-lookback:].to_numpy().reshape(1, -1)
    forecasts = {name: float(model.predict(window)[0]) for name, model in models.items()}
    write_json(folder / "next_forecast.json", {"feature_cutoff": str(engineered.index[-1].date()), "status":
"UNVERIFIED_NEXT_OBSERVATION_AFTER_SNAPSHOT_NOT_A_CURRENT_LIVE_FORECAST", "predictions": forecasts})
```

save_next_forecasts تسطح أحدث lookback صفوف سمات وتطلب توقع كل نموذج ثم تكتب التاريخ والحالة التي تنفي التحقق أو الآنية. لا يوجد هدف مرصود هنا للمقارنة.

الأسطر 132–134

```
def resolve_run(configuration: dict) -> Path:
    pointer = read_json(ROOT / f"artifacts/latest_{configuration['name']}.json")
```

```
    return ROOT / "artifacts/runs" / pointer["run_id"]
```

resolve_run تقرأ مؤشر latest للتكوين وتعيد مجلد التشغيل المشار إليه؛ لا تتحقق هنا أن المجلد أو كل ملفاته موجودة.

شرح src/xai_finance/reports/__init__.py

بصمة المصدر SHA-256:

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`.

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الملف فارغ عند البصمة المذكورة؛ لا يحتوي أسطراً أو استيرادات أو دوال، ويحدد مجلد الحزمة عند الاستيراد.

شرح src/xai_finance/reports/plots.py

بصمة المصدر SHA-256:

.fa28fbe115d210004bf0c43ee0e2e09d8accea28c1917e8a976ab4c5d61386fe

الاقتباسات من الملف؛ لم تُشغّل السكربتات أو تجاربها.

الأسطر 1–13

```
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import shap

PALETTE = ["#22577a", "#b15b24", "#52856e", "#8661a5", "#89939b"]
MODEL_COLORS = dict(zip(["LinearRegression", "RandomForest", "GradientBoosting", "LSTM", "Persistence"],
PALETTE))
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "font.size": 10, "figure.dpi": 130,
"axes.prop_cycle": plt.cycler(color=PALETTE)})
```

Path قياسية؛ matplotlib وNumPy وpandas وSHAP خارجية. يختار Agg قبل pyplot للعرض دون نافذة، ويحدد الألوان ونمط الرسوم العام.

الأسطر 16–21

```
def save_figure(path: Path, title: str) -> None:
    plt.gcf().suptitle(title, fontsize=10)
    plt.gcf().tight_layout()
    for extension in ["png", "svg"]:
        plt.gcf().savefig(path.with_suffix("." + extension), bbox_inches="tight")
    plt.close("all")
```

save_figure تضيف عنواناً للرسم الحالي وتضبط المساحة وتحفظ PNG وSVG ثم تغلق جميع الأشكال؛ تكتب ملفات وتعدل حالة matplotlib العامة.

الأسطر 24–36

```
def forecast_charts(predictions: pd.DataFrame, folder: Path, title: str) -> None:
    folder.mkdir(exist_ok=True)
    figure, axes = plt.subplots(2, 1, figsize=(11, 7), sharex=True)
    actual = predictions.drop_duplicates("target_date")
    axes[0].plot(pd.to_datetime(actual.target_date), actual.actual, color="#243b53", label="Actual",
linewidth=2)
    for model, subset in predictions.groupby("model"):
        axes[0].plot(pd.to_datetime(subset.target_date), subset.predicted, label=model, alpha=0.8,
color=MODEL_COLORS[model])
```

```
        axes[1].plot(pd.to_datetime(subset.target_date), subset.predicted - subset.actual, label=model,
alpha=0.7, color=MODEL_COLORS[model])
    axes[0].legend(ncol=3, fontsize=8)
    axes[0].set_ylabel("Original target units")
    axes[1].set_ylabel("Prediction error")
    axes[1].axhline(0, color="#89939b", linewidth=0.7)
    save_figure(folder / "forecasts", title + " | held-out TEST only")
```

forecast_charts تنشئ مجلد الرسم ومحورين: القيم الفعلية بعد إزالة تواريخ مكررة وتوقع كل نموذج، ثم خطأ predicted−actual لكل نموذج. تضع الوحدات وتعرض الصفر وتحفظ؛ تفترض تطابق الفعلي بين النماذج وتسميات ألوان معروفة.

الأسطر 39–46

```
def metric_charts(metrics: list[dict], folder: Path, title: str) -> None:
    table = pd.DataFrame(metrics)
    figure, axes = plt.subplots(1, 3, figsize=(12, 4))
    for axis, metric in zip(axes, ["mae", "rmse", "r2"]):
        axis.bar(table.model, table[metric], color=[MODEL_COLORS[name] for name in table.model])
        axis.set_title(metric.upper())
        axis.tick_params(axis="x", rotation=65)
    save_figure(folder / "metrics", title)
```

metric_charts تحول قائمة المقاييس لجدول وترسم أعمدة MAE وRMSE وR^2 في محاور منفصلة وتحفظ، كي لا تخلط مقاييس ذات وحدات مختلفة في محور واحد.

الأسطر 49–57

```
def loss_chart(history: list[dict], folder: Path) -> None:
    history = pd.DataFrame(history)
    plt.figure(figsize=(8, 4))
    plt.plot(history.epoch, history.train_loss, label="Training")
    plt.plot(history.epoch, history.validation_loss, label="Validation")
    plt.legend()
    plt.xlabel("Epoch")
    plt.ylabel("MSE in training-standardized target units")
    save_figure(folder / "loss", "LSTM training and validation history")
```

loss_chart ترسم خسارة تدريب وتحقق LSTM مقابل epoch بوحدات MSE للهدف المعياري؛ لا يعادل محورها RMSE بالدولار.

الأسطر 60–72

```
def research_flow(folder: Path) -> None:
    folder.mkdir(exist_ok=True)
    figure, axis = plt.subplots(figsize=(12, 5))
    axis.set_xlim(0, 12)
    axis.set_ylim(0, 5)
    axis.axis("off")
    nodes = [(1, 3, "Authentic history\nfixed snapshot + hash"), (3.4, 3, "Trailing features\nidentical
historical windows"), (5.8, 3, "80/10/10 labels\ntrain-only transforms"), (8.2, 3, "Four forecasting
models\nvalidation-only selection"), (10.6, 3, "Frozen forecasts\nheld-out TEST dates"), (8.2, 1, "SHAP +
LIME\ntraining-only background"), (10.6, 1, "Original-unit metrics\n+ persistence baseline")]
    for x, y, label in nodes:
        axis.text(x, y, label, ha="center", va="center", fontsize=9, bbox={"boxstyle": "round,pad=.6",
```

```
"facecolor": "#e9eff5", "edgecolor": "#22577a"})
    for start, end in [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (4, 6)]:
        axis.annotate("", xy=(nodes[end][0], nodes[end][1] + (0.5 if end > 4 else 0)), xytext=(nodes[start][0] +
(0.9 if end < 5 else 0), nodes[start][1]), arrowprops={"arrowstyle": "->", "color": "#22577a"})
    axis.text(6, 0.2, "Post-hoc explanations preserve forecasts. Revised monthly housing is a separate
retrospective experiment.", ha="center", fontsize=9)
    save_figure(folder / "research_flow", "Reconstructed research protocol")
```

research_flow تنشئ مجلدًا وشكلاً ثابت العقد والأسهم لوصف اللقطة والسمات والتقسيم والتدريب والاختبار والتفسير والمقاييس، وتضع تنبيه الإسكان وتحفظ. الرسم سردي وليس فحصاً آلياً للبيانات.

الأسطر 75–80

```
def explanation_charts(explanation: shap.Explanation, folder: Path, context: dict) -> None:
    title = context["title"]
    shap.plots.beeswarm(explanation, max_display=12, show=False)
    save_figure(folder / "shap_summary", title + " | fixed declared TEST subset")
    shap.plots.waterfall(explanation[0], max_display=12, show=False)
    save_figure(folder / "shap_waterfall", title + " | local prediction " + context["date"])
```

explanation_charts تستقبل Explanation ومجلداً وسياقاً، وترسم beeswarm للمجموعة المحددة وwaterfall لأول تفسير وتحفظهما دون إظهار نافذة.

الأسطر 81–89

```
    top = int(np.argmax(np.abs(explanation.values).mean(axis=0)))
    plt.figure(figsize=(7, 4))
    plt.scatter(explanation.data[:, top], explanation.values[:, top], color=PALETTE[0])
    plt.xlabel(explanation.feature_names[top])
    plt.ylabel("SHAP contribution (original target units)")
    save_figure(folder / "shap_dependence", title + " | sparse fixed-subset dependence")
    force = shap.force_plot(float(np.asarray(explanation.base_values).ravel()[0]), explanation.values[0],
explanation.data[0], feature_names=explanation.feature_names)
    shap.save_html(str(folder / "shap_force.html"), force)
    aggregate_lag_charts(explanation, folder, context)
```

تختار العمود ذي أكبر متوسط قيمة SHAP مطلقة ضمن المجموعة، وترسم الاعتماد على قيمته ثم تنشئ force plot لأول عينة وHTML، وتستدعي تجميع التأخيرات. المجموعة محدودة لا تمثل بالضرورة جميع الاختبار.

الأسطر 92–101

```
def aggregate_lag_charts(explanation: shap.Explanation, folder: Path, context: dict) -> None:
    values = explanation.values.reshape(len(explanation.values), context["lookback"], len(context["features"]))
    importance = np.abs(values).sum(axis=1).mean(axis=0)
    ranking = pd.DataFrame({"feature": context["features"], "mean_sum_abs_shap_over_lags":
importance}).sort_values("mean_sum_abs_shap_over_lags", ascending=False)
    ranking.to_csv(folder / "global_importance.csv", index=False)
    plt.figure(figsize=(8, 5))
    top = ranking.head(10).iloc[::-1]
    plt.barh(top.feature, top.mean_sum_abs_shap_over_lags, color=PALETTE[0])
    plt.xlabel("Mean sum of absolute SHAP contributions over lags")
    save_figure(folder / "global_importance", context["title"])
```

aggregate_lag_charts تعيد تشكيل المساهمات إلى عينات×تأخيرات×سمات، وتجمع القيمة المطلقة عبر التأخيرات ثم تأخذ متوسط العينات وتصنف السمات وتكتب CSV ورسم أهم عشر. الجمع المطلق يفقد اتجاه التأثير ويمنع إلغاء الإشارات.

الأسطر 102–108

```
    plt.figure(figsize=(10, 6))
    maximum = max(float(np.abs(values[0]).max()), 1e-12)
    plt.imshow(values[0].T, aspect="auto", cmap="RdBu_r", vmin=-maximum, vmax=maximum)
    plt.yticks(range(len(context["features"])), context["features"])
    plt.xlabel("Window position: oldest to newest")
    plt.colorbar(label="Signed SHAP contribution in original target units")
    save_figure(folder / "feature_time_heatmap", context["title"] + " | local " + context["date"])
```

ترسم خريطة قيم SHAP الموقعة لأول عينة، على محور السمات ومواضع النافذة، بمقياس متناظر حول الصفر وحد أدنى 1e-12؛ تحفظ الشكل. لا تستنتج سببية.

شرح src/xai_finance/seed.py

بصمة المصدر SHA-256:

.6cc9044fc7eae48d3cfb6580a0cc677faf6143a26fcc4b1c8053df684059710b

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الأسطر 1–7

```
import os
import random

os.environ.setdefault("CUBLAS_WORKSPACE_CONFIG", ":4096:8")

import numpy as np
import torch
```

os وrandom قياسيتان؛ يضبط setdefault إعداد CUBLAS قبل تحميل NumPy وtorch الخارجيتين ولا يغير قيمة سابقة.

الأسطر 10–19

```
def seed_experiment(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(True)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.allow_tf32 = False
    torch.backends.cuda.matmul.allow_tf32 = False
    torch.set_num_threads(4)
```

seed_experiment يثبت بذور random وNumPy وTorch وCUDA، ويفرض خوارزميات حتمية، ويعطل benchmark وTF32 ويحدد أربعة خيوط. يغير الحالة العامة للعملية وقد يرفع خطأ لخوارزمية غير حتمية؛ لا يضمن تطابقاً بين كل الإصدارات والأجهزة.

الأسطر 22–30

```
def select_device(requested: str) -> tuple[torch.device, str]:
    if requested == "cpu":
        return torch.device("cpu"), "CPU explicitly requested"
    if requested not in {"auto", "cuda"}:
        raise ValueError("device must be auto, cuda or cpu")
    if torch.cuda.is_available():
        try:
            torch.ones(1, device="cuda").sum().item()
            return torch.device("cuda"), "CUDA allocation verified"
```

select_device تقبل cpu أو auto أو cuda. تعيد CPU فور طلبه، وترفض قيمة غير معروفة، وتختبر تخصيص CUDA وحساباً صغيراً إن كان متاحاً قبل إرجاع الجهاز والسبب.

الأسطر 31–37

```
        except RuntimeError as failure:
            if requested == "cuda":
                raise
            return torch.device("cpu"), f"CUDA runtime failure: {failure}"
    if requested == "cuda":
        raise RuntimeError("CUDA requested but torch.cuda.is_available() is False")
    return torch.device("cpu"), "CUDA unavailable; automatic CPU fallback"
```

عند فشل CUDA تعيد CPU في auto أو تعيد رفع الاستثناء للطلب الصريح؛ وعند غيابه ترفض cuda وتختار CPU لـ auto. الاختبار لا يضمن ذاكرة كافية للتدريب كاملاً.

شرح src/xai_finance/storage.py

بصمة المصدر SHA-256:

.8975cad5dd0e2c315d38ccdfdb293750a7f2d2e1b294d52143b28da19a44e96d

الاقتباسات من المصدر؛ لم تنفذ أعمال التقرير أو تعدل البيانات.

الأسطر 1–5

```
import hashlib
import json
from pathlib import Path

import numpy as np
```

hashlib وjson وPath قياسية وNumPy خارجية لتحويل الأنواع العددية.

الأسطر 8–10

```
def sha256(path: Path) -> str:
    with path.open("rb") as stream:
        return hashlib.file_digest(stream, "sha256").hexdigest()
```

sha256 تفتح ملفاً ثنائياً وتحسب بصمته عبر file_digest وتعيد السلسلة الست عشرية؛ تتحقق من سلامة البايتات مقارنة بمرجع موثوق لا هوية المصدر وحدها.

الأسطر 13–20

```
def json_default(content):
    if isinstance(content, (np.integer, np.floating)):
        return content.item()
    if isinstance(content, np.ndarray):
        return content.tolist()
    if isinstance(content, Path):
        return str(content)
    raise TypeError(f"Unsupported JSON type: {type(content)}")
```

json_default تحول عدد NumPy إلى قيمة Python ومصفوفته إلى قائمة وPath إلى نص؛ ترفع TypeError للنوع غير المدعوم. تستعمل عند تسلسل JSON.

الأسطر 23–25

```
def write_json(path: Path, content: dict | list) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(content, indent=2, default=json_default, allow_nan=False), encoding="utf-8")
```

write_json تنشئ مجلد الأب وتكتب JSON بتنسيق indent=2 وUTF-8 مع محول الأنواع ورفض NaN؛ الكتابة تستبدل الملف إن وجد وليست ذرية صراحة.

الأسطر 28–29

```
def read_json(path: Path):
    return json.loads(path.read_text(encoding="utf-8"))
```

read_json تقرأ UTF-8 وتفك JSON وتعيد محتواه، وتسمح لأخطاء غياب الملف أو فساد البيانات بالانتشار.

شرح src/xai_finance/validation/__init__.py

بصمة المصدر SHA-256:

.b6b8bdcbbe1b5169ac671fe743f351fda10b4d54beaeb8443a0f7af288022b80

تفسير مقابل المصدر، لم تُشغّل الأوامر الموثقة.

الأسطر 1–1

```
"""Additive scientific validation; frozen research artifacts are read-only inputs."""
```

سلسلة توثيق تصف التحقق الإضافي واستعمال الآثار المجمدة مدخلات للقراءة؛ لا تنفذ فحصاً بنفسها.

شرح src/xai_finance/validation/audit.py

بصمة المصدر SHA-256:

.326bd2c60ea2d6853a41ed016821143a263a96f9d7524a3954907297aaa5a4d2

المقاطع من المصدر؛ التفسير مراجعة يدوية دون تشغيل التجارب.

الأسطر 1–11

```
import joblib
import numpy as np
import pandas as pd

from xai_finance.data.features import features
from xai_finance.data.preprocess import fit_training_scalers
from xai_finance.data.windows import build_windows
from xai_finance.evaluate.metrics import forecast_metrics
from xai_finance.io.data_sources import load_snapshot
from xai_finance.pipelines.train import load_models, load_windows, split_manifest
from xai_finance.storage import write_json
```

مكتبات joblib وNumPy وpandas خارجية؛ يستورد المصدر بناء السمات والمعايير والنوافذ والمقاييس واللقطات وتحميل النماذج والتخزين من المشروع.

الأسطر 14–19

```
def causal_feature_check(prices: pd.DataFrame, frequency: str) -> dict:
    cutoff = int(0.8 * len(prices))
    prefix = features(prices.iloc[:cutoff], frequency)
    complete = features(prices, frequency).loc[prefix.index]
    np.testing.assert_allclose(prefix, complete, rtol=0, atol=0)
    return {"prefix_rows": len(prefix), "future_deletion_max_feature_error":
float(np.max(np.abs(prefix.to_numpy() - complete.to_numpy())))}
```

causal_feature_check تعيد حساب السمات على أول 80% من الأسعار ثم على كامل السلسلة وتقارن الصفوف المشتركة بتسامح صفري، وتعيد أكبر فرق. يحسن هذا فحص الاعتماد على مستقبل الصفوف، لكنه لا يثبت توافر اللقطة المعدلة تاريخياً أو كل مظاهر التسرب.

الأسطر 22–30

```
def alignment_check(windows, prices, manifest) -> dict:
    next_dates = prices.index[prices.index.get_indexer(windows.origins) + 1]
    assert windows.targets.equals(next_dates)
    close = "adj_close" if manifest["frequency"] == "daily" else "close"
    np.testing.assert_array_equal(windows.y, prices.loc[next_dates, close])
    np.testing.assert_array_equal(windows.current_close, prices.loc[windows.origins, close])
```

```
    for earlier, later in [("train", "validation"), ("validation", "test")]:
        assert windows.targets[windows.partitions[earlier][-1]] <= windows.origins[windows.partitions[later][0]]
    return {"all_labels_equal_next_raw_observation": True, "partition_label_availability_verified": True,
"split": split_manifest(windows)}
```

alignment_check تحدد تاريخ الرصد الخام التالي لكل أصل، وتتحقق من تطابقه مع الهدف وقيم الإغلاق الحالي واللاحق، ومن إتاحة آخر تسمية قسم سابق عند أول أصل القسم التالي؛ تعيد تقرير التقسيم. تستخدم assert وقد تُزال مع تشغيل Python المحسن.

الأسطر 33–42

```
def scaler_check(windows, folder) -> dict:
    saved = joblib.load(folder / "LSTM/scalers.joblib")
    recalculated = fit_training_scalers(windows)
    for name in ["inputs", "target"]:
        for attribute in ["mean_", "scale_", "var_"]:
            np.testing.assert_array_equal(getattr(saved, name), attribute),
getattr(getattr(recalculated, name), attribute))
    restored = saved.inverse_target(saved.target.transform(windows.y[:, None]))
    np.testing.assert_allclose(restored, windows.y, rtol=1e-12, atol=1e-12)
    test_scaled = saved.target.transform(windows.y[windows.partitions["test"], None]).ravel()
    return {"saved_scalers_match_training_only_recalculation": True, "inverse_max_abs_error":
float(np.max(np.abs(restored - windows.y))), "test_standardized_target_min": float(test_scaled.min()),
"test_standardized_target_max": float(test_scaled.max())}
```

scaler_check تحمل معايير LSTM وتعيد ملاءمتها على التدريب للتحقق من mean وscale وvar، وتفحص عكس المعيار وتحسب مدى أهداف الاختبار المعيارية. هذا فحص للمعايير وليس إعادة تدريب النموذج؛ تحميل joblib يفترض ملفاً موثوقاً.

الأسطر 45–52

```
def distribution_shift(windows, models) -> dict:
    train, test = windows.partitions["train"], windows.partitions["test"]
    minimum, maximum = windows.y[train].min(), windows.y[train].max()
    outside = (windows.y[test] < minimum) | (windows.y[test] > maximum)
    inputs = windows.flat
    train_min, train_max = inputs[train].min(axis=0), inputs[train].max(axis=0)
    clipped = np.clip(inputs[test], train_min, train_max)
    audit = {"train_price_min": float(minimum), "train_price_max": float(maximum), "test_price_min":
float(windows.y[test].min()), "test_price_max": float(windows.y[test].max()),
"test_price_outside_train_fraction": float(outside.mean()), "test_feature_cells_outside_train_fraction":
float(np.mean(inputs[test] != clipped))}
```

distribution_shift تحدد مجال أهداف التدريب ونسبة أهداف الاختبار خارجه، ثم تقص كل خلية سمة اختبار إلى المجال المشاهد في التدريب وتقيس نسبة الخلايا المتغيرة؛ القياس وصفي ولا يثبت تأثيراً سببياً.

الأسطر 53–61

```
    for name in ["RandomForest", "GradientBoosting", "LSTM"]:
        predicted = models[name].predict(inputs[test])
        audit[name] = {"prediction_min": float(predicted.min()), "prediction_max": float(predicted.max()),
"mean_price_error": float(np.mean(predicted - windows.y[test]))}
        if name != "LSTM":
            difference = float(np.max(np.abs(predicted - models[name].predict(clipped))))
```

```
            np.testing.assert_allclose(predicted, models[name].predict(clipped), atol=1e-9, rtol=1e-10)
            audit[name]["clip_to_training_feature_box_max_prediction_change"] = difference
    assert minimum - 1e-8 <= audit["RandomForest"]["prediction_min"] <= audit["RandomForest"]["prediction_max"]
<= maximum + 1e-8
    return audit
```

تجمع مدى التنبؤ وانحيازه المتوسط للغابة والتعزيز وLSTM، وتختبر ثبات تنبؤ شجرتي الغابة والتعزيز عند قص السمات، وتؤكد وقوع توقعات الغابة ضمن مجال هدف التدريب. هذه assertions تخص الآثار المدققة ولا تبرهن تفوق نموذج.

الأسطر 64–75

```
def frozen_metric_check(folder, windows, models) -> list[dict]:
    table = pd.read_csv(folder / "predictions.csv")
    checked = []
    for name, group in table.groupby("model", sort=False):
        predicted = group.predicted.to_numpy()
        actual = group.actual.to_numpy()
        metrics = forecast_metrics(actual, predicted, group.current_close.to_numpy())
        if name in models:
            reloaded = models[name].predict(windows.flat[windows.partitions["test"]])
            np.testing.assert_allclose(reloaded, predicted, rtol=1e-5, atol=1e-4)
        checked.append({"model": name, **metrics})
    return checked
```

frozen_metric_check تقرأ تنبؤات CSV المحفوظة وتحسب المقاييس لكل نموذج وتفحص تنبؤ النموذج المعاد تحميله مقابل المحفوظ عند توفره؛ Persistence لا يعاد تحميله كنموذج. تعتمد ترتيب الصفوف المتطابق.

الأسطر 78–92

```
def audit_original(original, destination) -> None:
    audits, metrics = {}, []
    for asset in ["AAPL", "SP500", "NASDAQ", "GOLD", "OIL", "HOUSING"]:
        folder = original / asset
        windows = load_windows(folder)
        prices, manifest = load_snapshot(asset)
        rebuilt = build_windows(features(prices, manifest["frequency"]), windows.x.shape[1])
        np.testing.assert_array_equal(windows.x, rebuilt.x)
        np.testing.assert_array_equal(windows.y, rebuilt.y)
        models = load_models(folder, "cpu")
        audits[asset] = {"dataset": manifest, "alignment": alignment_check(windows, prices, manifest),
"causality": causal_feature_check(prices, manifest["frequency"]), "scaling": scaler_check(windows, folder),
"shift": distribution_shift(windows, models)}
        metrics.extend({"asset": asset, **row} for row in frozen_metric_check(folder, windows, models))
        print(f"AUDIT {asset}: alignment, causal features, training scalers and reload verified", flush=True)
    write_json(destination / "original_audit.json", audits)
    pd.DataFrame(metrics).to_csv(destination / "original_metrics_recalculated.csv", index=False)
```

audit_original تمر عبر ستة أصول، وتحمل اللقطة والنوافذ وتعيد بناء x وy وتتحقق من التطابق، وتحمل النماذج على CPU وتشغل فحوص المحاذاة والسمات والمعايير والانزياح والمقاييس ثم تكتب تقرير JSON وCSV. تشغيلها يعيد حساب تدقيقات ويتطلب آثاراً موثوقة؛ لم تنفذ في جلسة التوثيق.

شرح src/xai_finance/validation/design.py

بصمة المصدر SHA-256:

.83b2b73a3bb421842adf2296cf89272dc25883daa8fb03aa24e95bdeed72c9fc

المقاطع مقتبسة من المصدر، والوصف مراجعة يدوية لا يشغل التجارب.

الأسطر 1–5

```
from dataclasses import replace

import numpy as np

from xai_finance.data.windows import WindowDataset
```

replace القياسية تنسخ dataclass مع تغيير حقل، وNumPy خارجية، وWindowDataset نوع المشروع.

الأسطر 8–15

```
def target_values(windows: WindowDataset, target: str) -> np.ndarray:
    if target == "price_change":
        return windows.y - windows.current_close
    if target == "simple_return":
        if np.any(windows.current_close <= 0) or np.any(windows.y <= 0):
            raise ValueError("Simple-return protocol requires strictly positive prices")
        return windows.y / windows.current_close - 1
    raise ValueError(f"Unknown target: {target}")
```

target_values تحول الهدف إلى فرق السعر y−current أو عائد y/current−1؛ العائد يتطلب أن تكون جميع الأسعار الحالية والمستقبلية موجبة وإلا ValueError، والأسماء الأخرى مرفوضة. فرق السعر يسمح بالسعر السالب.

الأسطر 18–23

```
def implied_prices(predicted: np.ndarray, current: np.ndarray, target: str) -> np.ndarray:
    if target == "price_change":
        return current + predicted
    if target == "simple_return":
        return current * (1 + predicted)
    raise ValueError(f"Unknown target: {target}")
```

implied_prices تعكس التحويل: current+predicted للتغير أو current×(1+predicted) للعائد؛ تعيد خطأ للاسم المجهول. لا تقيد التنبؤات بموجب أو بنطاق السعر.

الأسطر 26–27

```
def transformed_windows(windows: WindowDataset, target: str) -> WindowDataset:
    return replace(windows, y=target_values(windows, target))
```

transformed_windows تعيد نسخة WindowDataset مع y محول، دون تغيير المصفوفة الأصلية عمداً؛ الحقول الأخرى مشتركة ما لم تنسخ منفصلة.

الأسطر 30–41

```
def walk_forward_partitions(windows: WindowDataset, folds: int) -> list[dict]:
    validation_size = len(windows.partitions["validation"])
    partitions = []
    for test in np.array_split(windows.partitions["test"], folds):
        cutoff = int(test[0])
        training_end = cutoff - validation_size
        if training_end < 30:
            raise ValueError("Insufficient pre-fold training history")
        partition = {"train": np.arange(training_end), "validation": np.arange(training_end, cutoff), "test":
test}
        assert windows.targets[partition["validation"][-1]] <= windows.origins[test[0]]
        partitions.append(partition)
    return partitions
```

walk_forward_partitions تقسم اختباراً إلى folds متتالية، وتأخذ حجم التحقق الأصلي وتستخدم التاريخ السابق للكتلة: تدريب حتى cutoff−validation_size وتحقق بعده. تشترط 30 تدريباً وتؤكد أن آخر تسمية تحقق معروفة بحلول أول أصل اختبار. لاحقاً قد يستفاد من تسميات اختبار أسبق عند توافرها. زيادة folds على حجم الاختبار قد تخلق كتلة فارغة ويتعذر test[0].

الأسطر 44–54

```
def naive_predictions(windows: WindowDataset, target: str) -> dict[str, np.ndarray]:
    transformed = target_values(windows, target)
    training = transformed[windows.partitions["train"]]
    test = windows.partitions["test"]
    majority = 1 if np.sum(training > 0) >= np.sum(training < 0) else -1
    nonzero = np.abs(training[training != 0])
    magnitude = float(np.median(nonzero)) if len(nonzero) else 0.0
    if np.any(test < 1):
        raise ValueError("Last-observed baseline requires a prior known target")
    assert (windows.targets[test - 1] <= windows.origins[test]).all()
    return {"Zero": np.zeros(len(test)), "TrainMean": np.full(len(test), np.mean(training)),
"LastObservedChangeOrReturn": transformed[test - 1], "TrainMajorityDirectionMedianMagnitude": np.full(len(test),
majority * magnitude)}
```

naive_predictions تحسب هدف التدريب وتستخرج إشارة الأغلبية مع تعادل لصالح +1 والوسيط للمقادير غير الصفرية، وتفحص توافر الهدف السابق لكل اختبار. تعيد صفراً ومتوسط التدريب والتغير السابق وإشارة الأغلبية مضروبة بوسيط المقدار؛ صفر التغير يطابق Persistence بعد إعادة السعر. لا تعيد ملاءمة هذه الثوابت يومياً ضمن كتلة اختبار.

شرح
:/xai_finance/validation/diagnostics.py

بصمة المصدر SHA-256:

.8f047fe994c63b1dc661ca8f5a885b6076a0fbfce9142833527d41d7b4e575a9

المقاطع من المصدر؛ التفسير مراجعة يدوية دون تشغيل التجارب.

الأسطر 1–8

```
import joblib
import numpy as np
import pandas as pd
import torch

from xai_finance.pipelines.train import load_windows
from xai_finance.storage import read_json, write_json
from xai_finance.validation.statistics import directional_statistics, paired_bootstrap
```

joblib وNumPy وpandas وtorch خارجية؛ تحميل النوافذ والتخزين ومقاييس الاتجاه وbootstrap من المشروع.

الأسطر 11–19

```
def target_distribution(windows) -> list[dict]:
    rows = []
    for partition, indices in windows.partitions.items():
        series = {"price": windows.y[indices], "price_change": windows.y[indices] -
windows.current_close[indices]}
        if np.all(windows.current_close > 0) and np.all(windows.y > 0):
            series["simple_return"] = windows.y[indices] / windows.current_close[indices] - 1
        for target, observations in series.items():
            rows.append({"partition": partition, "target": target, "mean": float(np.mean(observations)), "std":
float(np.std(observations)), "min": float(np.min(observations)), "max": float(np.max(observations)), "n":
len(observations)})
    return rows
```

target_distribution تصف لكل قسم السعر والتغير، وتضيف العائد فقط إذا كانت جميع الأسعار الحالية والأهداف في windows موجبة؛ ترجع المتوسط والانحراف الافتراضي ddof=0 والحدين والعدد. الشرط عام للبيانات كلها لا القسم وحده.

الأسطر 22–29

```
def lstm_output_bound(folder) -> dict:
    checkpoint = torch.load(folder / "LSTM/best.pt", map_location="cpu", weights_only=True)
    weight = checkpoint["state_dict"]["output.weight"].numpy()
    bias = float(checkpoint["state_dict"]["output.bias"].item())
```

```
    radius = float(np.abs(weight).sum())
    scalers = joblib.load(folder / "LSTM/scalers.joblib")
    bounds = scalers.inverse_target(np.array([bias - radius, bias + radius]))
    return {"price_lower_bound": float(bounds[0]), "price_upper_bound": float(bounds[1]), "derivation": "At
inference dropout is disabled; each LSTM hidden coordinate is o*tanh(c) in [-1,1]; affine head is within bias
+/- L1 norm of weights; then inverse training-target scaler. Bounds need not be attainable jointly."}
```

lstm_output_bound تحمل checkpoint CPU ومعيار الهدف؛ تحسب مجموع القيم المطلقة لأوزان الرأس $r=\Sigma|w_i|$ وتستنتج المجال المعياري $[b-r,b+r]$ لأن كل h_i ضمن $[-1,1]$ أثناء الاستدلال، ثم تعكس معيار الهدف. الحدود محافظة وقد لا تتحقق جميع إحداثياتها معاً، ولا تعمم على بنية مختلفة.

الأسطر 32–43

```
def supplemental_audit(original, destination) -> None:
    distributions, original_comparisons, bounds = [], [], {}
    for asset in ["AAPL", "SP500", "NASDAQ", "GOLD", "OIL", "HOUSING"]:
        folder = original / asset
        windows = load_windows(folder)
        distributions.extend({"asset": asset, **row} for row in target_distribution(windows))
        bounds[asset] = lstm_output_bound(folder)
        table = pd.read_csv(folder / "predictions.csv")
        stored = {row["model"]: row for row in read_json(folder / "metrics.json")}
        test, train = windows.partitions["test"], windows.partitions["train"]
        outside_distance = np.maximum(windows.y[train].min() - windows.y[test], 0) + np.maximum(windows.y[test]
- windows.y[train].max(), 0)
        bounds[asset]["random_forest_unavoidable_rmse_from_training_target_bounds"] =
float(np.sqrt(np.mean(outside_distance ** 2)))
```

supplemental_audit تقرأ ستة أصول ونوافذها وتستخرج توزيعات وحدود LSTM والمقاييس المحفوظة، وتحسب المسافة خارج مجال هدف التدريب وجذر متوسط مربعاتها كحد أدنى لخطأ غابة مخرجاتها داخل ذلك المجال.

الأسطر 44–51

```
        for name, group in table.groupby("model", sort=False):
            actual, predicted, current = [group[column].to_numpy() for column in ["actual", "predicted",
"current_close"]]
            rmse = float(np.sqrt(np.mean((actual - predicted) ** 2)))
            np.testing.assert_allclose(rmse, stored[name]["rmse"], rtol=1e-12)
            original_comparisons.append({"asset": asset, "model": name, "rmse": rmse,
**directional_statistics(actual - current, predicted - current), **paired_bootstrap(actual, predicted,
current)})
    pd.DataFrame(distributions).to_csv(destination / "target_distributions.csv", index=False)
    pd.DataFrame(original_comparisons).to_csv(destination / "original_comparisons.csv", index=False)
    write_json(destination / "architecture_bounds.json", bounds)
```

تقارن RMSE المعاد حسابه لكل نموذج بالقيمة المحفوظة وتضيف إحصاءات الاتجاه وbootstrap، ثم تحفظ جداول التوزيع والمقارنة وحدود البنية. لم تنفذ الحسابات في هذا العمل.

الأسطر 54–60

```
def prediction_amplitudes(predictions: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for keys, group in predictions.groupby(["design", "target", "asset", "model"]):
```

```
        actual, predicted = group.actual_target.to_numpy(), group.predicted_target.to_numpy()
        actual_std, predicted_std = float(np.std(actual)), float(np.std(predicted))
        rows.append({**dict(zip(["design", "target", "asset", "model"], keys)), "observed_target_std":
actual_std, "predicted_target_std": predicted_std, "predicted_to_observed_std": predicted_std / actual_std,
"observed_target_mean": float(np.mean(actual)), "predicted_target_mean": float(np.mean(predicted)),
"target_correlation": float(np.corrcoef(actual, predicted)[0, 1]) if predicted_std > 1e-12 else None})
    return pd.DataFrame(rows)
```

prediction_amplitudes تجمع بحسب التصميم والهدف والأصل والنموذج وتصف انحراف ومتوسط الهدف الحقيقي والمتوقع ونسبتهما وارتباطهما إن كان انحراف المتوقع كافياً. إذا كان انحراف الفعلي صفراً فقد تكون النسبة أو الارتباط غير صالحين؛ انخفاض التباين ليس برهان مهارة.

شرح
:/xai_finance/validation/experiments.py

بصمة المصدر SHA-256:

.ceb4c3e23ea536f71e455a295c9c0f9bed84d65e74897c505d0b3895fedb01ec

المقاطع من المصدر؛ التفسير مراجعة يدوية دون تشغيل التجارب.

الأسطر 1–17

```
from dataclasses import replace
import gc
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import torch
from threadpoolctl import threadpool_limits

from xai_finance.models.lstm_model import fit_lstm
from xai_finance.models.sklearn_models import candidates, fit_candidates
from xai_finance.pipelines.train import load_windows, software_environment, split_manifest, verify_reload
from xai_finance.seed import seed_experiment
from xai_finance.storage import read_json, sha256, write_json
from xai_finance.validation.design import implied_prices, naive_predictions, transformed_windows,
walk_forward_partitions
from xai_finance.validation.statistics import evaluation_metrics
```

يستورد replace وgc وPath القياسية، ومكتبات حفظ وعدد وحساب وTorch وخيوط خارجية، ودوال نماذج وتكوين وبيانات وتقييم داخلية.

الأسطر 20–28

```
def fit_models(windows, configuration: dict, folder: Path) -> tuple[dict, dict]:
    models, audits = {}, {}
    for name, estimators in candidates(configuration).items():
        print(f"{folder.name}: fitting {name}", flush=True)
        models[name], audits[name] = fit_candidates(estimators, windows)
        joblib.dump(models[name], folder / f"{name}.joblib")
    models["LSTM"], audits["LSTM"] = fit_lstm(windows, configuration, folder / "LSTM")
    joblib.dump(models["LSTM"].scalers, folder / "LSTM/scalers.joblib")
    return models, audits
```

fit_models تلائم مرشحي sklearn على النوافذ وتعتمد تحقق الاختيار ثم تحفظهم، وتدرب LSTM وتحفظ معاييره، وتعيد النماذج وسجلات التدريب. هذه الدالة تدرب وتكتب آثاراً عند استدعائها؛ لم تُنفذ هنا.

الأسطر 31–39

```
def predictions_table(windows, specification: dict, models: dict) -> pd.DataFrame:
    test, target = windows.partitions["test"], specification["target"]
    predictions = {name: model.predict(windows.flat[test]) for name, model in models.items()}
    predictions.update(naive_predictions(windows, target))
    transformed = transformed_windows(windows, target)
    tables = []
    for name, predicted in predictions.items():
        tables.append(pd.DataFrame({**specification, "model": name, "feature_cutoff": windows.origins[test],
"target_date": windows.targets[test], "current_price": windows.current_close[test], "actual_price":
windows.y[test], "predicted_price": implied_prices(predicted, windows.current_close[test], target),
"actual_target": transformed.y[test], "predicted_target": predicted}))
    return pd.concat(tables, ignore_index=True)
```

predictions_table تحصل على تنبؤات النماذج وأربعة مراجع naive وتحوّل الهدف، وتنشئ لكل نموذج صفوف بيانات تتضمن تواريخ وأسعاراً وهدفاً حقيقياً ومتوقعاً، وتعيد جدولاً موحداً. implied_prices يحول توقع الهدف إلى سعر.

الأسطر 42–46

```
def job_configuration(protocol: dict, specification: dict) -> dict:
    configuration = {**protocol["lstm"], "seed": protocol["seed"], "trees": 100, "search": True}
    if specification["design"] == "walk_forward":
        configuration["epochs"] = protocol["walk_forward"]["lstm_epochs"]
    return configuration
```

job_configuration تدمج إعداد LSTM مع البذرة و100 شجرة والبحث، وتغير حد epochs عند walk_forward حسب البروتوكول؛ تعيد القاموس.

الأسطر 49–56

```
def experiment_job(windows, specification: dict, context: dict) -> None:
    folder = context["destination"] / "experiments" / specification["design"] / specification["target"] / f"
{specification['asset']}_fold{specification['fold']}"
    if (folder / "completion.json").exists():
        previous = read_json(folder / "completion.json")
        for name, digest in previous["files"].items():
            assert sha256(folder / name) == digest, f"Completed job changed: {folder / name}"
        print(f"Verified completed job: {folder}", flush=True)
        return
```

experiment_job تحدد مجلد التصميم والهدف والأصل والطية؛ إذا وُجد completion.json تتحقق من بصمات الملفات المذكورة وتعود. الفحص لا يتأكد وحده من غياب ملفات إضافية.

الأسطر 57–65

```
    folder.mkdir(parents=True, exist_ok=False)
    seed_experiment(context["protocol"]["seed"])
    target_windows = transformed_windows(windows, specification["target"])
    configuration = job_configuration(context["protocol"], specification)
    write_json(folder / "design.json", {**specification, "configuration": configuration, "split":
split_manifest(windows), "target_definition": context["protocol"]["targets"][specification["target"]],
```

```
"input_windows_source": str(context["original"] / specification["asset"] / "windows.npz"),
"input_windows_sha256": sha256(context["original"] / specification["asset"] / "windows.npz")})
    np.savez_compressed(folder / "target_and_partitions.npz", y=target_windows.y, origins=windows.origins.asi8,
targets=windows.targets.asi8, **windows.partitions)
    models, training = fit_models(target_windows, configuration, folder)
    write_json(folder / "training.json", training)
    verify_reload(folder, models, target_windows)
```

لعمل جديد تنشئ المجلد دون استبدال وتضبط البذرة وتحول الهدف وتكتب التصميم وبصمة نوافذ المصدر وتحفظ الأهداف والتقسيمات، ثم تدرب وتحفظ وتسجل وتتحقق من إعادة التحميل. هذه عمليات تعديل آثار وتجارب فعلية إن شغلت.

الأسطر 66–75

```
    table = predictions_table(windows, specification, models)
    table.to_csv(folder / "predictions.csv", index=False)
    metric_rows = [{**specification, "model": name, **evaluation_metrics(group)} for name, group in
table.groupby("model", sort=False)]
    write_json(folder / "metrics.json", metric_rows)
    write_json(folder / "completion.json", {"files": {str(path.relative_to(folder)): sha256(path) for path in
folder.rglob("*") if path.is_file()}, "status": "EXECUTED_AND_CPU_RELOAD_VERIFIED"})
    print(pd.DataFrame(metric_rows)[["asset", "model", "rmse", "rmse_ratio_vs_zero",
"direction_accuracy"]].to_string(index=False), flush=True)
    del models
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
```

تكتب تنبؤات ومقاييس ونهاية العمل مع بصمات الملفات ثم تطبع النتائج وتحرر النماذج وcache CUDA عند توفره. تحسب المقاييس على الأقسام المحددة في النوافذ.

الأسطر 78–89

```
def run_experiments(context: dict) -> None:
    protocol = context["protocol"]
    write_json(context["destination"] / "environment.json", {**software_environment(), "torch_cuda_available":
torch.cuda.is_available(), "torch_cuda_version": torch.version.cuda, "cudnn_version":
torch.backends.cudnn.version(), "protocol_sha256": sha256(context["destination"] / "protocol.json")})
    with threadpool_limits(limits=4):
        for target, assets in protocol["fixed_split"].items():
            for asset in assets:
                windows = load_windows(context["original"] / asset)
                experiment_job(windows, {"asset": asset, "target": target, "design": "fixed_split", "fold": 0},
context)
        for asset in protocol["walk_forward"]["assets"]:
            windows = load_windows(context["original"] / asset)
            for fold, partition in enumerate(walk_forward_partitions(windows, protocol["walk_forward"]
["folds"]), 1):
                experiment_job(replace(windows, partitions=partition), {"asset": asset, "target":
"price_change", "design": "walk_forward", "fold": fold}, context)
```

run_experiments تكتب معلومات البيئة وبصمة البروتوكول وتحد الخيوط، ثم تشغل التجارب الثابتة للأهداف والأصول المحددة وwalk-forward لأصوله مع استبدال التقسيمات لكل طية. يحدد البروتوكول النطاق؛ لا يجوز اعتبار النتائج تجارب جديدة مستقلة تماماً عن الاختبار المعروف.

الأسطر 92–102

```
def aggregate_experiments(destination: Path) -> None:
    tables = [pd.read_csv(path) for path in sorted((destination / "experiments").rglob("predictions.csv"))]
    predictions = pd.concat(tables, ignore_index=True)
    predictions.to_csv(destination / "new_predictions.csv", index=False)
    metric_rows = []
    for keys, group in predictions.groupby(["design", "target", "asset", "model"], sort=False):
        assert not group.target_date.duplicated().any()
        group = group.sort_values("target_date")
        metric_rows.append({**dict(zip(["design", "target", "asset", "model"], keys)),
**evaluation_metrics(group)})
    pd.DataFrame(metric_rows).to_csv(destination / "new_metrics.csv", index=False)
    write_json(destination / "new_metrics.json", metric_rows)
```

aggregate_experiments تقرأ كل predictions.csv في مجلد التجارب وتجمعها، وتتحقق من عدم تكرار تاريخ الهدف داخل كل تصميم وهدف وأصل ونموذج، وترتب التاريخ وتحسب مقاييس مجمعة وتكتب CSV وJSON. تفترض وجود ملفات ومحاذاة صحيحة.

شرح src/xai_finance/validation/figures.py

بصمة المصدر SHA-256:

.fbe85343ee9122d490b8e4dc89dd8184f34bdd51babcbab9e83064165517c51c

المقاطع من المصدر؛ النصوص الطويلة المقتبسة في قوالب التقرير تؤدي وظيفة توثيق النتائج المحفوظة ولا تنفذ تجارب.

الأسطر 1–10

```
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import TwoSlopeNorm
import numpy as np
import pandas as pd

from xai_finance.storage import read_json
```

Path معيارية؛ matplotlib وNumPy وpandas خارجية. يحدد Agg قبل استيراد pyplot لتوليد رسوم دون نافذة، وread_json داخلية.

الأسطر 13–15

```
LEARNED = ["LinearRegression", "RandomForest", "GradientBoosting", "LSTM"]
ASSETS = ["AAPL", "SP500", "NASDAQ", "GOLD", "OIL"]
COLORS = dict(zip(LEARNED + ["Persistence", "Zero"], ["#2c7fb8", "#e69f00", "#a6761d", "#009e73", "#666666",
"#666666"]))
```

ثوابت أسماء النماذج والأصول والألوان تربط كل فئة بلون؛ الأصول الخمسة يومية ويضاف الإسكان صراحة في الرسم المجمد.

الأسطر 18–21

```
def save_figure(figure, destination: Path, name: str) -> None:
    figure.savefig(destination / f"{name}.png", dpi=180, bbox_inches="tight")
    figure.savefig(destination / f"{name}.svg", bbox_inches="tight")
    plt.close(figure)
```

save_figure تحفظ الشكل PNG بدقة 180 وSVG ثم تغلقه؛ المدخل شكل ووجهة واسم والآثار كتابتان على القرص.

الأسطر 24–39

```
def frozen_forecasts(original: Path, destination: Path, audits: dict) -> None:
    figure, axes = plt.subplots(3, 2, figsize=(14, 12), constrained_layout=True)
    for axis, asset in zip(axes.ravel(), ASSETS + ["HOUSING"]):
        table = pd.read_csv(original / asset / "predictions.csv", parse_dates=["target_date"])
        actual = table[table.model == "Persistence"]
        axis.plot(actual.target_date, actual.actual, color="black", label="Observed", linewidth=1.5)
        for model in ["RandomForest", "GradientBoosting", "LSTM", "Persistence"]:
            subset = table[table.model == model]
            axis.plot(subset.target_date, subset.predicted, label=model, color=COLORS[model], alpha=0.85,
linewidth=1)
        axis.axhline(audits[asset]["shift"]["train_price_max"], color="red", linestyle="--", label="Training
maximum")
        axis.set_title(asset)
        axis.set_ylabel(audits[asset]["dataset"]["units"])
        axis.tick_params(axis="x", rotation=25)
    axes[0, 0].legend(fontsize=8)
    figure.suptitle("Frozen level-model forecasts: severe range shift in five series; oil remains in range")
    save_figure(figure, destination, "frozen_level_predictions")
```

frozen_forecasts تنشئ ستة محاور، وتقرأ التنبؤات الفعلية من صفوف Persistence وتعرض أربعة نماذج وحد أعلى هدف التدريب ووحدات الأصل ثم تحفظ الشكل. العنوان نص ثابت يتعلق بالنتائج المحددة، ولا يمثل اختباراً جديداً.

الأسطر 42–54

```
def rmse_comparison(original_metrics, metrics, destination: Path) -> None:
    figure, axes = plt.subplots(1, 3, figsize=(17, 5), constrained_layout=True)
    for axis, target in zip(axes, ["level", "price_change", "simple_return"]):
        if target == "level":
            subset = original_metrics[original_metrics.asset.isin(ASSETS)].copy()
            zero = subset[subset.model == "Persistence"].set_index("asset").rmse
            subset["ratio"] = subset.rmse / subset.asset.map(zero)
        else:
            subset = metrics[(metrics.design == "fixed_split") & (metrics.target == target)].copy()
            subset["ratio"] = subset.rmse_ratio_vs_zero
        assets = [asset for asset in ASSETS if asset in subset.asset.to_list()]
        matrix = subset.pivot(index="asset", columns="model", values="ratio").reindex(index=assets,
columns=LEARNED)
        axis.imshow(np.log2(matrix.to_numpy()), cmap="RdBu_r", norm=TwoSlopeNorm(vmin=-0.15, vcenter=0, vmax=5))
```

rmse_comparison تنشئ أعمدة لمستوى السعر والتغير والعائد؛ تقسم RMSE المستوى على Persistence، وباقي المقارنات تأخذ النسبة المحفوظة، ثم ترتب مصفوفة الأصول والنماذج وتعرض log2 للنسبة. الصفر اللوني يعني نسبة 1؛ قيم غير موجبة أو ناقصة تحتاج حذراً.

الأسطر 55–62

```
        for row in range(len(matrix)):
            for column in range(len(LEARNED)):
                axis.text(column, row, f"{matrix.iloc[row, column]:.3f}", ha="center", va="center", fontsize=9)
        axis.set_xticks(range(4), ["OLS", "RF", "GBM", "LSTM"])
        axis.set_yticks(range(len(assets)), assets)
        axis.set_title(target.replace("_", " "))
    figure.suptitle("Price RMSE / zero-change RMSE; lower is better, 1 = persistence (colors use log2 ratio)")
    save_figure(figure, destination, "target_representation_rmse")
```

تضع القيمة الأصلية لكل نسبة كنص، وتعين تسميات المحاور والعنوان وتحفظ الرسم؛ تستخدم ألواناً مبنية على اللوغاريتم لا القيم الخام.

الأسطر 65–79

```
def bootstrap_intervals(metrics, destination: Path) -> None:
    subset = metrics[(metrics.design == "fixed_split") & (metrics.target == "price_change") &
metrics.model.isin(LEARNED)]
    figure, axes = plt.subplots(1, 5, figsize=(16, 4), constrained_layout=True)
    for axis, asset in zip(axes, ASSETS):
        selected = subset[subset.asset == asset].set_index("model").reindex(LEARNED)
        baseline_mse = (selected.rmse / selected.rmse_ratio_vs_zero) ** 2
        central = selected.mse_difference_vs_zero / baseline_mse
        low, high = selected.mse_difference_ci_low / baseline_mse, selected.mse_difference_ci_high /
baseline_mse
        axis.errorbar(central, range(4), xerr=np.vstack([central - low, high - central]), fmt="o", capsize=3)
        axis.axvline(0, color="black", linestyle="--")
        axis.set_yticks(range(4), ["OLS", "RF", "GBM", "LSTM"])
        axis.set_title(asset)
        axis.set_xlabel("MSE difference / zero MSE")
    figure.suptitle("Price-change models: exploratory paired block-bootstrap 95% intervals; left of zero favors
model")
    save_figure(figure, destination, "paired_uncertainty")
```

bootstrap_intervals تختار تجارب التغير ثابتة التقسيم للنماذج المتعلمة، وتستنتج MSE المرجع من RMSE ونسبته ثم تقسم متوسط فرق MSE وحدود الفاصل عليه وترسم فواصل لكل أصل. السالب يفضل النموذج؛ الفواصل الاستكشافية ليست مصححة للمقارنات المتعددة.

الأسطر 82–97

```
def walk_forward_errors(predictions, destination: Path) -> None:
    figure, axes = plt.subplots(2, 1, figsize=(12, 8), constrained_layout=True)
    for axis, asset in zip(axes, ["AAPL", "SP500"]):
        selected = predictions[(predictions.design == "walk_forward") & (predictions.asset == asset)]
        for model in LEARNED:
            table = selected[selected.model == model].sort_values("target_date")
            delta = (table.predicted_price - table.actual_price) ** 2 - (table.current_price -
table.actual_price) ** 2
            axis.plot(pd.to_datetime(table.target_date), delta.cumsum(), label=model, color=COLORS[model])
        for _, fold in selected[selected.model == "Zero"].groupby("fold"):
            axis.axvline(pd.to_datetime(fold.target_date.min()), color="grey", linestyle=":", linewidth=0.8)
        axis.axhline(0, color="black", linewidth=0.8)
        axis.set_title(asset)
        axis.set_ylabel("Cumulative squared-error difference")
        axis.legend(fontsize=8)
    figure.suptitle("Walk-forward price-change forecasts versus zero change; negative favors model; dotted fold
starts")
    save_figure(figure, destination, "walk_forward_cumulative_loss")
```

walk_forward_errors ترسم لـ AAPL وSP500 المجموع التراكمي للفارق بين خطأ النموذج المربع وخطأ الثبات، بعد ترتيب التواريخ، وتحدد بدايات الطيات؛ القيم السالبة التراكمية لصالح النموذج. الفارق مربع بوحدات الأصل.

الأسطر 100–113

```
def directional_accuracy(metrics, destination: Path) -> None:
    subset = metrics[(metrics.design == "fixed_split") & (metrics.target == "price_change")]
    figure, axis = plt.subplots(figsize=(11, 5), constrained_layout=True)
    names = LEARNED + ["TrainMajorityDirectionMedianMagnitude"]
    locations = np.arange(len(ASSETS))
    for offset, name in enumerate(names):
        selected = subset[subset.model == name].set_index("asset").reindex(ASSETS)
        axis.bar(locations + (offset - 2) * 0.15, selected.direction_accuracy, width=0.15, label="Train majority
sign" if name == names[-1] else name)
    axis.set_xticks(locations, ASSETS)
    axis.set_ylim(0, 0.75)
    axis.set_ylabel("Three-way sign accuracy")
    axis.legend(fontsize=8, ncol=3)
    axis.set_title("Directional accuracy: zero-change forecast abstains (0% active coverage), omitted from
bars")
    save_figure(figure, destination, "directional_accuracy")
```

directional_accuracy تختار تجربة التغير وتعرض دقة الإشارة ثلاثية الحالات للنماذج وأغلبية التدريب، وتستثني الصفر من الأعمدة لأنه يمتنع عن تغير غير صفري. المحور مقيد إلى 0.75 وقد يخفي قيماً أعلى منه.

الأسطر 116–127

```
def generate_figures(original: Path, destination: Path) -> None:
    output = destination / "figures"
    output.mkdir(exist_ok=True)
    metrics = pd.read_csv(destination / "new_metrics.csv")
    original_metrics = pd.read_csv(destination / "original_metrics_recalculated.csv")
    predictions = pd.read_csv(destination / "new_predictions.csv")
    plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False})
    frozen_forecasts(original, output, read_json(destination / "original_audit.json"))
    rmse_comparison(original_metrics, metrics, output)
    bootstrap_intervals(metrics, output)
    walk_forward_errors(predictions, output)
    directional_accuracy(metrics, output)
```

generate_figures تنشئ مجلد figures وتقرأ جداول النتائج والتدقيق، وتضبط نمط matplotlib وتستدعي دوال الرسوم الخمس. تنفيذها يكتب ملفات صور؛ لم تشغل هنا.

شرح src/xai_finance/validation/report.py

بصمة المصدر SHA-256:

.5fdd20c75269865ed58bbf667360ec618a37bf36187624f03d0386d11264f242

المقاطع من المصدر؛ النصوص الطويلة المقتبسة في قوالب التقرير تؤدي وظيفة توثيق النتائج المحفوظة ولا تنفذ تجارب.

الأسطر 1–9

```
from pathlib import Path
import xml.etree.ElementTree as ET

import pandas as pd

from xai_finance.config import ROOT
from xai_finance.storage import read_json, sha256
from xai_finance.validation.diagnostics import prediction_amplitudes
from xai_finance.validation.figures import LEARNED, generate_figures
```

Path وElementTree من المكتبة القياسية، وpandas خارجية؛ ROOT والقراءة والبصمة وتشخيص السعات وثوابت ودوال الرسوم داخلية.

الأسطر 12–18

```
def markdown_table(table: pd.DataFrame) -> str:
    formatted = table.copy()
    for column in formatted:
        formatted[column] = formatted[column].map(lambda entry: f"{entry:.6f}" if isinstance(entry, float) else
str(entry))
    lines = ["| " + " | ".join(formatted.columns) + " |", "| " + " | ".join(["---"] * len(formatted.columns)) +
" |"]
    lines.extend("| " + " | ".join(row) + " |" for row in formatted.to_numpy().tolist())
    return "\n".join(lines)
```

markdown_table تنسخ DataFrame وتحول float إلى ست منازل وغيرها إلى نص، ثم تنشئ صف رؤوس وفاصل وصفوف Markdown وتعيد السلسلة. لا تهرب الشرطة العمودية داخل القيم تلقائياً.

الأسطر 21–25

```
def rmse_table(metrics: pd.DataFrame) -> str:
    order = LEARNED + ["Zero", "TrainMean", "LastObservedChangeOrReturn",
"TrainMajorityDirectionMedianMagnitude"]
    pivot = metrics.pivot(index="asset", columns="model", values="rmse")
    pivot = pivot.reindex(columns=[name for name in order if name in pivot.columns]).rename(columns=
{"LinearRegression": "OLS", "RandomForest": "RF", "GradientBoosting": "GBM", "TrainMean": "Train mean",
```

```
"LastObservedChangeOrReturn": "Last change/return", "TrainMajorityDirectionMedianMagnitude": "Train majority"})
    return markdown_table(pivot.reset_index())
```

rmse_table تعيد ترتيب أعمدة النماذج والمعايير الساذجة الموجودة وتعيد تسميتها ثم تنشئ جدول RMSE لكل أصل؛ pivot يفترض تفرد زوج الأصل والنموذج.

الأسطر 28–33

```
def dataset_table(audits: dict) -> pd.DataFrame:
    rows = []
    for asset, audit in audits.items():
        dataset, split = audit["dataset"], audit["alignment"]["split"]
        rows.append({"asset": asset, "symbol": dataset["symbol"], "frequency": dataset["frequency"], "units":
dataset["units"], "snapshot_start": dataset["first_date"], "snapshot_end": dataset["last_date"], "raw_n":
dataset["rows"], "train_n": split["train"]["n"], "validation_n": split["validation"]["n"], "test_n":
split["test"]["n"], "train_last_label": split["train"]["target_end"], "test_first_label": split["test"]
["target_start"], "test_last_label": split["test"]["target_end"], "snapshot_sha256": dataset["sha256"]})
    return pd.DataFrame(rows)
```

dataset_table تحول تقارير التدقيق إلى جدول بالأصل والمزود والوحدات والمدد والأعداد وتواريخ الأهداف وبصمة اللقطة وتعيده.

الأسطر 36–42

```
def execution_summary(destination: Path) -> pd.DataFrame:
    rows = []
    for path in sorted((destination / "experiments").rglob("training.json")):
        design = read_json(path.parent / "design.json")
        lstm = read_json(path)["LSTM"]
        rows.append({**{name: design[name] for name in ["asset", "target", "design", "fold"]}, "device":
lstm["device"], "precision": lstm["precision"]["effective_precision"], "batch_size": lstm["batch_size"],
"best_epoch": lstm["selection"]["best_epoch"], "epochs_executed": lstm["selection"]["epochs_executed"],
"training_seconds": lstm["training_seconds"], "training_label_cutoff": lstm["scaler_target_cutoff"]})
    return pd.DataFrame(rows)
```

execution_summary تجمع training.json لكل مهمة وتقرأ design.json وتستخرج إعداد تنفيذ LSTM والجهاز والدقة والدفعة وأفضل epoch والزمن وحد آخر هدف تدريب وتعيد DataFrame.

الأسطر 45–59

```
def evidence_tables(destination: Path, original: Path) -> dict:
    audits = read_json(destination / "original_audit.json")
    datasets = dataset_table(audits)
    datasets.to_csv(destination / "dataset_definitions.csv", index=False)
    historic = pd.DataFrame(read_json(ROOT / "docs/thesis_reported_reference.json")["models"])
    historic.to_csv(destination / "thesis_reported_separate.csv", index=False)
    metrics = pd.read_csv(destination / "new_metrics.csv")
    amplitudes = prediction_amplitudes(pd.read_csv(destination / "new_predictions.csv"))
    amplitudes.to_csv(destination / "prediction_amplitudes.csv", index=False)
    training = execution_summary(destination)
    training.to_csv(destination / "training_execution_summary.csv", index=False)
    frozen = pd.read_csv(destination / "original_metrics_recalculated.csv")
    frozen.pivot(index="asset", columns="model", values="rmse").to_csv(destination /
"frozen_level_rmse_separate.csv")
    generate_figures(original, destination)
```

```
    return {"audits": audits, "datasets": datasets, "historic": historic, "metrics": metrics, "frozen": frozen,
"amplitudes": amplitudes, "training": training}
```

evidence_tables تقرأ التدقيق والمرجع التاريخي والمقاييس والتنبؤات وتنتج جداول مجموعات البيانات والمرجع والتباين والتدريب وRMSE المجمد، ثم تولد الصور وتعيد قاموس الأدلة. هي دالة كتابة وليست قراءة فقط.

الأسطر 62–69

```
def principal_findings(evidence: dict) -> str:
    metrics = evidence["metrics"]
    learned = metrics[(metrics.design == "fixed_split") & metrics.model.isin(LEARNED)]
    point_wins = learned[learned.rmse_ratio_vs_zero < 1]
    favorable_intervals = learned[learned.mse_difference_ci_high < 0]
    walk = metrics[(metrics.design == "walk_forward") & metrics.model.isin(LEARNED)]
    walk_wins = walk[walk.rmse_ratio_vs_zero < 1]
    return f"The new fixed-split analyses produce {len(point_wins)} price-RMSE point improvements over zero
change across {len(learned)} learned-model comparisons, but {len(favorable_intervals)} have an exploratory,
unadjusted paired-bootstrap interval wholly favoring the model. Thus these results do not establish a reliable
learned-model advantage. The walk-forward analysis has {len(walk_wins)} point improvements among {len(walk)}
learned-model comparisons, reported separately below. Near-persistence performance after changing the target is
a correction of a representation failure, not evidence that the thesis's LSTM ranking was reproduced."
```

principal_findings ترشح النتائج المتعلمة وتعد نسب RMSE الأقل من 1 وفواصل bootstrap التي حدها الأعلى دون الصفر، وتفصل walk-forward؛ تعيد نصاً تفسيرياً. الفواصل استكشافية غير مصححة ولا تثبت تفوقاً عاماً.

الأسطر 72–76

```
def audit_narrative(evidence: dict, destination: Path) -> str:
    shift = pd.DataFrame([{ "asset": asset, "train_min": audit["shift"]["train_price_min"], "train_max":
audit["shift"]["train_price_max"], "test_min": audit["shift"]["test_price_min"], "test_max": audit["shift"]
["test_price_max"], "test_outside_fraction": audit["shift"]["test_price_outside_train_fraction"]} for asset,
audit in evidence["audits"].items()])
    bounds = read_json(destination / "architecture_bounds.json")
    bound_table = pd.DataFrame([{"asset": asset, "LSTM_price_ceiling": row["price_upper_bound"],
"RF_unavoidable_RMSE": row["random_forest_unavoidable_rmse_from_training_target_bounds"]} for asset, row in
bounds.items()])
    return f"""## Audit of the frozen run
```

audit_narrative تنشئ جدولي انزياح الأسعار وحدود LSTM والغابة ثم تبدأ قالب تقرير التدقيق النصي.

الأسطر 77–97

```

All six stored window matrices were rebuilt exactly from the hash-verified snapshots. Every label equals the
next raw observed price, every final feature row equals the origin observation, and train/validation label
observation dates precede or equal the next partition's first origin. The audit's label-availability boundary
flag means this observation-index comparison; it does not independently establish publication timestamps or
historical vintages. Deleting the future portion of each snapshot leaves all earlier engineered features exactly
unchanged. No future interpolation or imputation is used. Rolling windows overlap across boundaries as
legitimate historical context: for a one-observation horizon, an earlier label actually available at the later
forecast origin does not require a 30-observation embargo.

Saved LSTM input and target scaler means, variances and scales match training-only recomputation exactly. Target
```

```
scaling/inversion round trips were verified, including test targets outside the training range. Scaling does not
restrict a test observation to the training range and no clipping is applied during normal prediction. The input
scaler fits overlapping training-window rows, so observations receive repeated weights; this is documented
training-only weighting, not future access. Linear regression has its own training-only flattened-input
standardization and QR rank selection; the tree models use unscaled features. All saved models were reloaded on
CPU and compared with the frozen predictions. Stored RMSE values were independently recomputed from frozen CSVs.

The information set assumes a forecast **after the origin session closes**, when that session's high, low, close
and volume are known. This is a next-observation forecast, not a pre-close signal or an executable intraday
trading strategy. Yahoo adjusted histories are retrospective snapshots; exact corporate-action vintages were not
available. Monthly housing is revised and has release lags, so its chronology is only valid for the
retrospective observation-index task, not real-time publication-date forecasting.

{markdown_table(shift)}

The 80/10/10 protocol fits models on training only, uses validation for selection/early stopping, and does not
refit on train plus validation. Thus daily models have a roughly one-year gap between their final fitted label
and the test period. This is intentional in the frozen specification and makes level extrapolation unusually
demanding. Feature drift affects approximately half the test feature cells for the rising daily assets. Oil has
no feature cells outside the training min/max box; its negative 2020 price remains intact. Change-target
volatility also shifts: test versus training standard deviation is about 2.3 times higher for Apple and 3.2
times higher for gold, whereas oil's training dispersion is inflated by its historical shock. Exact
level/change/return distribution summaries are in `target_distributions.csv`.

## Tree extrapolation and the LSTM ceiling

For a squared-error regression forest, each leaf is a mean of training targets; the ensemble is another average.
Its predictions therefore stay within the training target range. The table gives the minimum possible RMSE of
**any** predictor constrained to that range: the RMS distance of test targets to the training interval. This
geometric diagnostic uses test outcomes for explanation only; it does not train or choose models.

{markdown_table(bound_table)}

For every actual test input, clipping each feature to its training minimum/maximum changed RF and GBM
predictions by exactly zero. This follows the fitted trees' threshold partitions and directly demonstrates their
inability to extend trends outside the feature box. GBM is an additive sum of residual trees: unlike RF, it is
**not generally bounded by the original training target range**. Its fitted response remains piecewise constant
beyond its split thresholds. The distinction matters when explaining extrapolation. See [the official decision-
tree guide](https://scikit-learn.org/1.5/modules/tree.html) for the piecewise-constant extrapolation limitation.

The frozen LSTM also has a structural ceiling: at inference each last hidden coordinate is `output_gate *
tanh(cell)` in [-1,1]; the linear head is bounded by `bias +/- sum(abs(weights))`. Inverse target scaling
converts that interval into price units. These are valid global bounds, not necessarily jointly attainable
extremes. Gold's ceiling is below most test prices; Apple, S&P 500 and Nasdaq also exceed their respective
ceilings at the upper end. This explains a limitation of these **particular fitted networks**, not a theorem
that all recurrent forecasters must fail. Optimization, target representation, validation regime, finite
training budget and feature shift also matter.
"""
```

يتضمن قالب التدقيق وصف فحص السمات والمقاييس والتقييس والافتراض بعد الإغلاق، والتعديلات الاستعادية والتقسيم وعدم إعادة الملاءمة، ويفرق قيد مجال RandomForest عن ثبات GBM خارج حدود الانقسام ويشرح حد LSTM. هي ادعاءات التقرير ويجب ربطها بنتائج التدقيق المحفوظة لا اعتبار النص اختباراً جديداً.

الأسطر 100–114

```
def experiment_narrative(evidence: dict, destination: Path) -> str:
    metrics = evidence["metrics"]
    fixed = metrics[metrics.design == "fixed_split"]
    delta = fixed[fixed.target == "price_change"]
    returns = fixed[fixed.target == "simple_return"]
    walk = metrics[metrics.design == "walk_forward"]
    walk_intervals = walk[walk.model.isin(LEARNED)][["asset", "model", "rmse_ratio_vs_zero",
"mse_difference_ci_low", "mse_difference_ci_high"]]
    intervals = fixed[fixed.model == "LSTM"][["asset", "target", "rmse_ratio_vs_zero", "mse_difference_vs_zero",
"mse_difference_ci_low", "mse_difference_ci_high"]]
    direction = delta[delta.model.isin(LEARNED + ["TrainMajorityDirectionMedianMagnitude"])][["asset", "model",
```

```
"direction_accuracy", "direction_coverage"]]
    wins = fixed[fixed.model.isin(LEARNED) & (fixed.rmse_ratio_vs_zero < 1)]
    robust = wins[wins.mse_difference_ci_high < 0]
    amplitudes = evidence["amplitudes"]
    lstm_amplitudes = amplitudes[(amplitudes.design == "fixed_split") & (amplitudes.model == "LSTM")][["asset",
"target", "observed_target_std", "predicted_target_std", "predicted_to_observed_std"]]
    training = evidence["training"]
    walk_epochs = training.loc[training.design == "walk_forward", "epochs_executed"].max()
```

experiment_narrative ترشح جداول fixed split والأهداف وwalk-forward والفواصل والاتجاه والنتائج الأفضل والنطاقات وتجد أكبر عدد epochs لتكوين جداول السرد. تفترض وجود الأعمدة والصفوف المطلوبة.

الأسطر 115–160

```
    return f"""## New experiments: design and execution

These are exploratory sensitivity analyses: the original test outcomes were already known. The timestamped
protocol was recorded before additional model training; no new test results were used to alter candidates,
features, training budgets or reported outcomes. The protocol SHA256 is `{sha256(destination /
'protocol.json')}`. Target definitions, exclusions and budgets are retained in `protocol.json`.

All models retain the exact frozen 30-by-20 feature windows and chronological dates. The primary experiment
changes only the label to `delta[t+1] = P[t+1] - P[t]`, then reconstructs `P_hat[t+1] = P[t] + delta_hat[t+1]`.
It runs on all five daily assets, retaining the negative oil observation. A separate simple-return experiment
uses `r[t+1] = P[t+1]/P[t] - 1` and reconstructs `P_hat[t+1] = P[t] * (1 + r_hat[t+1])` on the four strictly
positive daily series. Oil returns are excluded by protocol, with no filtering or clipping of its change
experiment. Features still include levels: these experiments do not claim to make the entire input process
stationary.

OLS retains training-only scaling/QR rank selection. RF tries 100 trees with depth 5 or 10 and leaf size 3. GBM
tries 100 trees with (depth 2, rate .03) or (depth 3, rate .1). Validation RMSE in the **new target units**
selects the candidate; return RMSE selection weights errors differently from price RMSE. LSTM uses the original
32 hidden units, dropout .1, Adam .001, gradient clipping 1, at most 60 epochs and patience 8, training-only
target/input scalers, no shuffling and identical fixed seed 42. AMP is enabled only if its training-fixture loss
check is finite and within 5% of FP32. CUDA/CPU choice, actual precision, batch/VRAM probes, best/last
checkpoints, epoch histories, scaler cutoffs, candidates and CPU reload checks are saved for every job. Price
predictions are not truncated to historical bounds and returns are not constrained to force plausible results.

Four explicit naive target forecasts are included: zero (exact persistence after reconstruction), training mean
(fixed drift), the most recent observed change/return, and the training-majority sign times the training median
nonzero magnitude. The latter's magnitude is an explicit forecast convention, not an optimized trading strategy.
Last-observation forecasts use the prior label, already known at the current origin; they do not use the unknown
current test label. Mean and majority baselines are fitted using training labels only. Zero/drift methods are
standard benchmarks; see [Forecasting: Principles and Practice](https://otexts.com/fpp3/simple-methods.html).

### Fixed-split price-change experiment: reconstructed-price RMSE

{rmse_table(delta)}

### Fixed-split simple-return experiment: reconstructed-price RMSE

{rmse_table(returns)}

The OLS change-target forecasts should be almost identical to the original price-target forecasts: with an
intercept and the current price in the feature span, subtracting current price from the label and adding it back
at prediction is an algebraic reparameterization. A difference in RF/LSTM behavior is therefore informative
about representation and model constraints, rather than evidence that changing a target guarantees new
information.

The following post-training diagnostic compares LSTM target-prediction dispersion with actual target dispersion.
Low reconstructed-price error can arise from forecasting near-zero changes, which is close to persistence,
without accurately predicting the day's fluctuation. It must not be presented as evidence that the network
recovered a strong return signal. Full amplitudes, means and sample correlations for all models are in
`prediction_amplitudes.csv`; these are descriptive, not new selection criteria.

{markdown_table(lstm_amplitudes)}
```

```
### Walk-forward validation

Two assets (Apple and S&P 500) receive three expanding-history refits, covering the original 271 test
observations in contiguous disjoint blocks of 91, 90 and 90. Before each block, the latest 270 available labels
are validation and all earlier labels are training. Each fold refits preprocessing, candidate selection and all
four models from scratch. The first fold uses the original training/validation boundaries; later folds can use
earlier test labels only once they are available. Forecasts within each block continue to use the actual latest
observed feature history; parameters remain frozen until the next block. This is block walk-forward one-step
prediction, not a 90-step recursive forecast.

The LSTM walk-forward cap is 30 epochs with patience 8; other settings and tabular candidate budgets are
unchanged. The largest actual walk-forward epoch count is {walk_epochs}, with every job's actual device,
precision, batch, best epoch and fitted-label cutoff exported in `training_execution_summary.csv`. Validation
continues to reserve a 270-observation tail, so these refits reduce staleness incrementally but do not implement
daily training or final train-plus-validation refitting. The caps differ from the fixed-split experiment and
should not be conflated with a pure refit-frequency causal test. Label availability is explicitly asserted under
the daily after-close assumption; general temporal validation motivation is described in [scikit-learn's
temporal-split documentation](https://scikit-
learn.org/1.5/modules/generated/sklearn.model_selection.TimeSeriesSplit.html).

{rmse_table(walk)}

{markdown_table(walk_intervals)}

### Directional accuracy and uncertainty

{markdown_table(direction)}

Directional accuracy compares three-way signs of predicted and observed price change. Nonzero coverage and
accuracy conditional on an active forecast are also saved. Zero-change forecasts have 0% active coverage; their
three-way score counts only exactly flat outcomes. They are **not a 0%-skill trading classifier**. Binary
up/not-up accuracy is provided separately with zero mapped to not-up. Training-majority direction is the
appropriate simple active comparator; 50% is not an assumed empirical baseline. Direction scores do not include
trading costs, calibration or profitability.

{markdown_table(intervals)}

Each interval is a paired circular block bootstrap of daily `model squared price error - zero squared price
error`, using block length 10, 2,000 resamples and seed 42. Negative differences favor the model. Price RMSE
ratios below 1 occur in {len(wins)} of the {len(fixed[fixed.model.isin(LEARNED)])} learned-model/target/asset
comparisons; {len(robust)} have an unadjusted interval entirely below zero. Intervals are exploratory, use one
prespecified block length, do not include training uncertainty, and have no multiple-comparison correction.
Nominal coverage depends on the blocks adequately representing serial dependence; nonstationarity and fold-
dependent models can weaken that approximation. Marginal wins or intervals overlapping zero cannot establish
reliable superiority. Cross-asset dependence and the shared test period also prevent treating these comparisons
as independent replications.

All MAE, RMSE, MAPE, R², direction/coverage, target-unit errors and bootstrap details are in
`new_metrics.csv/json`. Price MAPE uses nonzero actual prices with zero counts recorded; target MAPE is also
exported for transparency, but near-zero returns/changes make it unstable and it is not a selection criterion.
Price R² can remain very high for persistence because the level varies slowly; target R² and error relative to
zero change are more informative about incremental forecast skill. No returns, price errors or metrics are
pooled across incompatible asset units.
"""
```

قالب تجربة التحقق يدرج تصميم الأهداف والإعدادات والمعايير الساذجة وجداول الأخطاء وwalk-forward بثلاث طيات، والاتجاه، وفواصل bootstrap المزدوجة بطول 10 و2000 سحب. ينبه إلى استخدام اختبار معروف وغياب تصحيح التعدد وعدم شمول عدم يقين التدريب؛ النص ليس تنفيذ تدريب.

الأسطر 163–173

```
def write_report(destination: Path) -> None:
    protocol = read_json(destination / "protocol.json")
    original = ROOT / "artifacts/runs" / protocol["original_run"]
    evidence = evidence_tables(destination, original)
    verification = read_json(destination / "verification.json")
    replication = read_json(destination / "determinism_replication/verification.json")
    precision = pd.read_csv(destination / "reconstruction_precision_comparison.csv")
```

```
    precision = precision.loc[precision.max_price_difference > 1e-8, ["asset", "model", "max_price_difference",
"as_executed_rmse", "float64_rmse", "rmse_difference", "direction_predictions_changed"]]
    test_suite = ET.parse(destination / "tests_final.xml").getroot().find("testsuite")
    relative = destination.relative_to(ROOT).as_posix()
    frozen = evidence["frozen"].pivot(index="asset", columns="model", values="rmse").reset_index()
```

write_report تقرأ البروتوكول والأدلة والتحقق والتكرار والمقارنة العددية وXML، وتحسب المسار والجداول؛ تستدعي evidence_tables التي تكتب جداول ورسومات.

الأسطر 174–204

```
    report = f"""# Scientific validation report

Frozen run: `full_20261021T0409010846472`. Additional validation: `{protocol['validation_id']}`. All new outputs
are under `{relative}`. This review keeps the thesis's claims, frozen reconstruction and new exploratory
experiments as separate evidence categories. No winner is assumed.

## Main conclusion

Persistence winning the five daily level-forecast benchmarks is a valid empirical outcome. The audit found no
observation-index chronological leakage, label-shift error or inverse-scaling error in the frozen
implementation. Point-in-time data correctness was not established: Yahoo adjustments are retrospective, and
housing observation months are not release dates. Severe upward level shifts, fixed training cutoffs, tree
extrapolation constraints and the particular fitted LSTM's bounded output explain much of the very large
learned-model errors. Changing to change/return labels tests those constraints while preserving the information
set. It does not create predictable daily return information or entitle LSTM to win.

{principal_findings(evidence)}

## Distinct datasets and result provenance

{markdown_table(evidence['datasets'][['asset', 'symbol', 'frequency', 'raw_n', 'train_n', 'validation_n',
'test_n', 'train_last_label', 'test_first_label', 'test_last_label']])}

Exact units, source symbols, snapshot dates and SHA256 hashes are in `dataset_definitions.csv` and
`original_audit.json`. Daily sources are fixed Yahoo snapshots for 2015-01-01 inclusive through 2026-01-01
exclusive; gold/oil calendars differ from equities. Housing is the separate monthly FRED CSUSHPINSA series. New
change/return training is restricted to daily assets; housing is audited and retained in the frozen table only.

### Thesis-reported values: historical claims, not reproduced observations

{markdown_table(evidence['historic'])}

Source: Chapter 4 section 4.2 Table 4, paragraphs P1026-P1049, figure P1050; TOC page 101, physical pagination
unverified. The thesis does not supply the dataset, units, dates or saved runs needed to compare those numbers
numerically with the new asset-specific metrics. Its appendix instead shows DecisionTree/XGBoost on missing
`ether.csv`, a contemporaneous Close label and full-data mean imputation before splitting. These screenshots do
not substantiate the four-model next-period result table. No assertion is made that the original Table 4 was
produced by those screenshots, or that its underlying experiment necessarily leaked: the required evidence is
absent. See `docs/THESIS_TRACEABILITY.md` and `docs/thesis_reported_reference.json`, both preserved unchanged.

### Frozen reconstructed price-level RMSE: independently recalculated

{markdown_table(frozen)}

Daily persistence wins all five assets. Monthly housing differs: OLS beats persistence. These numbers are
retained in a separate CSV and are never replaced by the new experiments.

{audit_narrative(evidence, destination)}

{experiment_narrative(evidence, destination)}
```

ينشئ قالب التقرير بعناوين الخلاصة والبيانات والمرجع التاريخي والمقاييس المجمدة ثم يدرج سرد التدقيق والتجارب. فصل المرجع التاريخي عن إعادة البناء أساسي لتجنب مقارنة قيم دون وحدات أو بيانات معروفة.

الأسطر 205–225

```
## Verification and preservation

Executed {len(verification['jobs'])} additional training jobs, each containing four learned models and four
naive baselines. Independently reloaded every learned model on CPU and replayed all
{verification['aggregate_rows']:,} saved forecast rows; all {verification['aggregate_metric_rows']} aggregate
metric records were recomputed from serialized predictions. Each job has hashes for its models, best/last
checkpoints, histories, labels, splits and outputs. One AAPL change-target LSTM was retrained independently: all
epoch training/validation losses and all {replication['test_n']} test predictions were bitwise identical on this
machine. That is a same-environment reproducibility check, not a multi-seed robustness study.

An initial replay check used a fixed absolute tolerance across target units and failed for two near-zero Nasdaq
change predictions. The maximum CPU/CUDA difference was 0.0000076294 index points. Diagnosis then replayed every
LSTM on the original CUDA device: all 15 jobs' predictions matched their saved values **bitwise**. CPU
differences were at most 0.0000000662 training-target standard deviations, below one FP32 machine epsilon at
unit scale. CPU replay now uses an absolute bound of eight FP32 epsilons times the training target standard
deviation, with zero relative tolerance; sklearn/baseline replay uses 1e-9 absolute/relative tolerance. This
precision-aware verification change does not alter a model or empirical result. The initial failed log and all
cross-device measurements remain in `replay_and_preservation.log` and `cross_device_diagnostics.json/log`.

A second verification attempt exposed a distinct, minor arithmetic effect: the LSTM target predictions are
native FP32, so NumPy computes `1 + predicted_return` in FP32 before multiplying by the FP64 origin price. Exact
replay must preserve that native dtype; casting the saved return to FP64 first produces slightly different
implied prices. The failed attempt is retained in `replay_and_preservation_verified.log`; the successful full
check is in `replay_and_preservation_final.log`. Both as-executed new result files remain untouched. A separate
sensitivity calculation reconstructs **all** saved targets in FP64, without retraining, and saves
`float64_reconstructed_predictions.csv`, `float64_reconstructed_metrics.csv` and provenance. Material
differences are confined to these four LSTM return jobs:

{markdown_table(precision)}

The largest price-rounding difference is about 0.001394 Nasdaq index points; the largest absolute RMSE
difference is about 0.000079 points. No directional predictions change. These are separately labeled numerical
sensitivity results, not replacements for the as-executed tables above, and they do not establish learned-model
superiority.

The complete automated suite executed {test_suite.attrib['tests']} tests, with {test_suite.attrib['failures']}
failures, {test_suite.attrib['errors']} errors and {test_suite.attrib['skipped']} skips. New boundary tests
cover return reconstruction/scaling, rejection of negative-price returns, known-label naive forecasts, disjoint
walk-forward blocks and directional abstention. Logs and JUnit XML are isolated in this validation directory.
See `verification.json`, `tests_final.xml`, `tests_final.log` and `determinism_replication/verification.json`.

All {verification['preservation']['protected_files']:,} files in the pre-review preservation inventory retained
their byte counts and SHA256 hashes. This includes the original run and checkpoints, previous
result/verification files, input snapshots, thesis, existing source/configuration and verified offline defense
package. The original defense ZIP SHA256 remains `{verification['preservation']['offline_zip_sha256']}`. No
original result or package was overwritten; this validation is an additive supplement.

## Figures and defense explanation

New publication exports (PNG and SVG) are in this run's `figures/` directory: `frozen_level_predictions`,
`target_representation_rmse`, `paired_uncertainty`, `walk_forward_cumulative_loss`, and `directional_accuracy`.
The existing figures and offline package remain untouched. The separate `offline_validation.html` embeds these
charts for an offline defense explanation.
```

يسرد القالب التحقق المسجل وإعادة التحميل والفروق بين الأجهزة والدقة العددية والاختبارات والمحافظة والصور. هذه أوصاف آثار محفوظة من تشغيل سابق وليست دليلاً على تنفيذها في جلسة التوثيق الحالية.

الأسطر 226–238

```
Suggested oral explanation: “The historical thesis ranks LSTM first but omits the experimental details needed
for exact replication. Our transparent next-observation benchmark includes persistence and reveals that level
models struggle when test prices rise beyond training support. We verified chronology and scaling, then ran
```

```
controlled change/return targets and expanding-history validation. These reduce representation-related failures,
but the evidence does not justify declaring a universal winner. We report uncertainty and all outcomes rather
than tuning to the thesis.”

## Remaining limitations and next research priority

- These analyses reuse an observed test period. A genuinely unseen subsequent period or new prespecified
historical holdout is needed for confirmatory comparison; repeatedly choosing models from these test tables
would contaminate future claims.
- One seed, one 30-session lookback, two tree candidates, small recurrent networks and fixed epoch budgets do
not establish architecture-wide rankings. Stronger regularization, dimensionless relative-price features, train-
plus-validation refitting and shorter trailing validation windows are reasonable future experiments, not
retrospectively applied fixes.
- Walk-forward covers only two equity series and three refits; validation preserves a 270-session tail. This is
a feasibility-bounded sensitivity analysis, not exhaustive daily retraining or cross-market confirmation.
- Yahoo retrospective adjustments, continuous-futures rolls and the negative oil observation limit economic
interpretation. Oil change targets remain numerically valid but its extreme training shock affects the loss
distribution. Monthly housing lacks vintage/release information and is not a real-time forecasting claim.
- MAPE around zero, level R², directional abstention, unadjusted intervals and cross-asset dependence require
the qualifications above. No transaction-cost, trading-profit, probability calibration or causal conclusions are
established.
- Existing SHAP/LIME artifacts explain the frozen **level models**. They were preserved and are not relabeled as
explanations of newly trained return/change models. Marginal perturbations of correlated financial indicators
can be off-manifold; predictive fit and explanation plausibility do not validate human trust or causality. No
survey responses were invented.

The next priority is an independently held-out, predeclared comparison of parsimonious relative-price/return
features with zero/drift benchmarks and a final-refit policy, followed by multiple seeds and cost-aware trading
evaluation if trading claims are intended. The current evidence supports a defensible audit and exploratory
comparison, not rewriting the thesis's historical numbers.
"""
```

يضم نصاً مقترحاً للدفاع وحدوداً: الفترة المعروفة، بذرة وحيدة، نطاق walk-forward، مراجعات البيانات، أخطاء التفسير وغياب دراسة إنسانية وتكاليف التداول. هذه قيود على الاستنتاج ولا تغير النتائج.

الأسطر 239–242

```
    (destination / "SCIENTIFIC_VALIDATION_REPORT.md").write_text(report, encoding="utf-8")
    canonical = ROOT / "docs/SCIENTIFIC_VALIDATION_REPORT.md"
    if not canonical.exists():
        canonical.write_text(report, encoding="utf-8")
```

يكتب التقرير إلى destination، ويكتب النسخة القياسية في docs فقط إن لم تكن موجودة. تشغيل الدالة له آثار كتابة؛ لم تنفذ هنا.

شرح
·c/xai_finance/validation/statistics.py

بصمة المصدر SHA-256:

.973bcca38ac776ed48d48ede9b73fb194f045600fe0eec80971a96b2253d9ac3

المقاطع مقتبسة من المصدر، والوصف مراجعة يدوية لا يشغل التجارب.

الأسطر 1–3

```
import numpy as np

from xai_finance.evaluate.metrics import forecast_metrics
```

NumPy خارجية وforecast_metrics داخلية لإعادة استعمال مقاييس السعر.

الأسطر 6–10

```
def directional_statistics(actual_change: np.ndarray, predicted_change: np.ndarray) -> dict:
    actual_sign, predicted_sign = np.sign(actual_change), np.sign(predicted_change)
    active = predicted_sign != 0
    hits = actual_sign == predicted_sign
    return {"direction_accuracy": float(np.mean(hits)), "direction_coverage": float(np.mean(active)),
"direction_accuracy_when_active": float(np.mean(hits[active])) if active.any() else None,
"binary_up_accuracy_zero_is_not_up": float(np.mean((actual_change > 0) == (predicted_change > 0))),
"actual_up_fraction": float(np.mean(actual_change > 0)), "actual_flat_fraction": float(np.mean(actual_change ==
0))}
```

directional_statistics تقارن sign للتغير الفعلي والمتوقع بثلاث حالات، وتحسب نسبة التنبؤ غير الصفري ودقته المشروطة ودقة الصعود الثنائية مع الصفر غير صعود ونسب الصعود والثبات. إذا لم توجد تنبؤات نشطة ترجع None للدقة المشروطة؛ الامتناع لا يساوي بالضرورة قرار هبوط.

الأسطر 13–20

```
def paired_bootstrap(actual: np.ndarray, predicted: np.ndarray, zero: np.ndarray) -> dict:
    loss_difference = (predicted - actual) ** 2 - (zero - actual) ** 2
    generator = np.random.default_rng(42)
    block_length, draws = 10, 2000
    starts = generator.integers(0, len(actual), size=(draws, int(np.ceil(len(actual) / block_length))))
    indices = ((starts[..., None] + np.arange(block_length)) % len(actual)).reshape(draws, -1)[:, :len(actual)]
    interval = np.quantile(loss_difference[indices].mean(axis=1), [0.025, 0.975])
    return {"mse_difference_vs_zero": float(np.mean(loss_difference)), "mse_difference_ci_low":
float(interval[0]), "mse_difference_ci_high": float(interval[1]), "bootstrap_block_length": block_length,
"bootstrap_draws": draws, "bootstrap_seed": 42}
```

paired_bootstrap تحسب $d_t=(\hat{y}_t-y_t)^2-(zero_t-y_t)^2$ على المشاهدات المزدوجة؛ السالب لصالح النموذج. بمولد seed=42 تختار بدايات 2000 إعادة أخذ كتل دائرية طولها 10 وتلتف modulo n، وتحسب متوسط d لكل عينة وكمّي 2.5% و97.5%. الفرق بوحدة السعر المربعة لا RMSE. الفاصل تقريبي لاستقرار الاعتماد الزمني ولا يشمل عدم يقين إعادة التدريب أو تصحيح تعدد المقارنات؛ الإدخال الفارغ غير صالح.

الأسطر 23–31

```
def evaluation_metrics(table) -> dict:
    actual, predicted, current = [table[column].to_numpy() for column in ["actual_price", "predicted_price",
"current_price"]]
    metrics = forecast_metrics(actual, predicted, current)
    metrics.update(directional_statistics(actual - current, predicted - current))
    metrics.update(paired_bootstrap(actual, predicted, current))
    metrics["rmse_ratio_vs_zero"] = metrics["rmse"] / float(np.sqrt(np.mean((actual - current) ** 2)))
    target_metrics = forecast_metrics(table.actual_target.to_numpy(), table.predicted_target.to_numpy(),
np.zeros(len(table)))
    metrics.update({f"target_{name}": target_metrics[name] for name in ["mae", "rmse", "r2", "mape_percent",
"mape_excluded_zero_targets"]})
    return metrics
```

evaluation_metrics تستخرج السعر الحقيقي والمتوقع والحالي، تجمع مقاييس السعر والإشارة وbootstrap مقابل الثبات، ثم نسبة RMSE إلى RMSE الثبات ومقاييس الهدف المحول مقابل صفر. عند مقام ثبات صفري قد تكون النسبة غير معرفة؛ MAPE للأهداف الصغيرة غير مستقر ولا يمثل معيار اختيار هنا.

شرح START_DEMO.bat

بصمة المصدر SHA-256:

.f129e483d502a52bc8a7996397051670c7bf3c33ccdfd8b96750f37fc5f8f9d6

لا ينفذ هذا الشرح ملفات التشغيل.

الأسطر 1–4

```
@echo off
setlocal
cd /d "%~dp0"
title Mohammad XAI - CPU Dashboard
```

يعزل البيئة ويحدد مجلد batch وعنوان نافذة العرض ويخفي الأوامر.

الأسطر 5–9

```
if not exist ".demo_venv\Scripts\python.exe" (
  echo Run SETUP_DEMO.bat first to prepare this computer.
  if /i not "%XAI_DEMO_NO_PAUSE%"=="1" pause
  exit /b 1
)
```

يتحقق من وجود مفسر .demo_venv، ويعرض تعليمات الإعداد ويتوقف اختيارياً ويخرج برمز 1 إن غاب.

الأسطر 10–14

```
if /i "%~1"=="stop" goto stop_server
".demo_venv\Scripts\python.exe" scripts\demo_wizard.py start %*
goto finished
:stop_server
".demo_venv\Scripts\python.exe" scripts\demo_wizard.py stop
```

إن كان الوسيط الأول stop ينتقل إلى الفرع المعني؛ وإلا ينفذ wizard start مع جميع الوسائط ثم ينتقل إلى finished. فرع stop يستدعي wizard stop دون تمرير بقية الوسائط.

الأسطر 15–18

```
:finished
set "demo_result=%errorlevel%"
if not "%demo_result%"=="0" if /i not "%XAI_DEMO_NO_PAUSE%"=="1" pause
exit /b %demo_result%
```

يحفظ رمز الخروج ويتوقف اختيارياً عند الفشل فقط ثم يعيده للبيئة المستدعية.

شرح `tests/conftest.py`

بصمة المصدر SHA-256:

`41ee4b27c2f77914a6efc6a25cb5452bd277dcd104e953578b05f7d9e9cf6b08`.

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–6

```
import numpy as np
import pandas as pd
import pytest

from xai_finance.data.features import features
from xai_finance.data.windows import build_windows
```

يستورد NumPy وpandas وpytest ودالتي السمات والنوافذ لتجهيز اختبارات، لا بيانات بحثية.

الأسطر 9–14

```
@pytest.fixture
def analytic_prices():
    """Analytic test fixture only; never used by research download/training commands."""
    index = pd.bdate_range("2020-01-01", periods=330)
    close = 100 + np.arange(len(index)) * 0.1 + np.sin(np.arange(len(index)))
    return pd.DataFrame({"open": close - 0.2, "high": close + 1, "low": close - 1, "close": close, "adj_close":
close, "volume": 1000 + np.arange(len(index))}, index=index)
```

analytic_prices fixture ينشئ 330 تاريخ عمل وسعرًا تحليلياً 100+0.1t+sin(t)، وOHLC وحجماً صناعياً؛ يعيد DataFrame للاختبارات فقط.

الأسطر 17–19

```
@pytest.fixture
def window_fixture(analytic_prices):
    return build_windows(features(analytic_prices, "daily"), 30)
```

window_fixture fixture يحول الأسعار الصناعية إلى سمات يومية ونوافذ 30 رصداً ويعيدها للاختبارات التابعة.

شرح tests/test_dashboard.py

بصمة المصدر SHA-256:

`3f0c02101dd582654eb007414d60de23a2dfc0e6d48bad229bf2d8defe5767dd`.

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–4

```
from pathlib import Path
import shutil

from streamlit.testing.v1 import AppTest
```

Path وshutil قياسيتان وAppTest أداة اختبار Streamlit.

الأسطر 7–15

```
def test_missing_artifacts_shows_actionable_setup_commands(tmp_path):
    project = Path(__file__).resolve().parents[1]
    dashboard = tmp_path / "dashboard"
    dashboard.mkdir()
    shutil.copy2(project / "dashboard/app.py", dashboard / "app.py")
    app = AppTest.from_file(str(dashboard / "app.py")).run(timeout=30)
    assert not app.exception
    assert any("No saved experiment" in message.value for message in app.info)
    assert "xai_finance.cli train" in app.code[0].value
```

ينشئ مشروعاً مؤقتاً مع نسخة الواجهة دون آثار، ويشغلها ويؤكد عدم الاستثناء وظهور رسالة غياب التشغيل وأمر التدريب؛ لا يتحقق من التبويبات عند وجود بيانات.

شرح tests/test_explain_consistency.py

بصمة المصدر SHA-256:

.1f5cfcbbdfcc19a8085cf5345073b26973612b89c41d0b379bff0cac53d48065

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–6

```
import numpy as np
import pytest

from xai_finance.explain.lime_explain import explain_lime
from xai_finance.explain.shap_explain import explain_shap, shap_reconstruction
from xai_finance.models.sklearn_models import candidates, fit_candidates
```

يستورد أدوات NumPy وpytest ودوال التفسير وملاءمة النماذج.

الأسطر 9–19

```
@pytest.mark.parametrize("model_name", ["LinearRegression", "RandomForest", "GradientBoosting"])
def test_shap_additivity_and_posthoc_explanation_preserve_forecast(window_fixture, model_name):
    model, _ = fit_candidates(candidates({"seed": 42, "trees": 5, "search": False})[model_name], window_fixture)
    observations = window_fixture.flat[window_fixture.partitions["test"][:2]]
    before = model.predict(observations)
    background = window_fixture.flat[window_fixture.partitions["train"][::50]]
    request = {"observations": observations, "names": window_fixture.flat_names, "seed": 42, "cycles": 1}
    explanation = explain_shap(model_name, model, background, request)
    audit = shap_reconstruction(explanation, before)
    np.testing.assert_array_equal(before, model.predict(observations))
    assert audit["max_absolute_error"] < 0.01
```

اختبار معلم لثلاثة نماذج جدولية: يلائم نموذجاً صغيراً على fixture ويحسب تفسير SHAP لعينتين بخلفية تدريبية، ثم يتحقق من الجمع وثبات التوقع وخطأ إعادة البناء أقل من 0.01؛ لا يثبت سببية التفسير.

الأسطر 22–30

```
def test_lime_accepts_batch_prediction_and_returns_measured_surrogate_error(window_fixture):
    model, _ = fit_candidates(candidates({"seed": 42, "trees": 5, "search": False})["GradientBoosting"],
window_fixture)
    training = window_fixture.flat[window_fixture.partitions["train"]]
    observation = window_fixture.flat[window_fixture.partitions["test"][0]]
    before = model.predict(observation[None])
    _, audit = explain_lime(training, model.predict, observation, {"names": window_fixture.flat_names, "seed":
42, "samples": 100})
    np.testing.assert_array_equal(before, model.predict(observation[None]))
    assert audit["absolute_local_error"] == pytest.approx(abs(audit["model_prediction"] -
audit["local_surrogate_prediction"]))
    assert np.isfinite(audit["weighted_neighborhood_r2"])
```

يلائم GradientBoosting صغيراً ويشرح عينة بـ100 اضطراب، ويتحقق من عدم تغيير التنبؤ ومن أن الخطأ هو القيمة المطلقة للفرق بين النموذج والبديل وأن R^2 الجوار منتهٍ؛ لا يشترط جودة البديل.

شرح tests/test_lstm_training.py

بصمة المصدر SHA-256:

.d4768f11fab72c7d225aea64631e1b32b7673b1706f89e6542d6546447720941

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–6

```
import numpy as np
import pytest
import torch

from xai_finance.models.lstm_model import PriceLSTM, fit_lstm, load_lstm, measure_batch
from xai_finance.seed import seed_experiment
```

يستورد أدوات الاختبار وTorch ومكونات LSTM والبذرة.

الأسطر 9–17

```
@pytest.mark.skipif(not torch.cuda.is_available(), reason="CUDA hardware diagnostic requires an NVIDIA device")
def test_cuda_batch_probe_supports_cudnn_backward():
    """Regression: 2026-10-21 batch probe used eval mode before cuDNN backward."""
    seed_experiment(42)
    network = PriceLSTM({"features": 3, "hidden_size": 4, "dropout": 0.1}).cuda()
    fixture = (torch.randn(8, 5, 3, device="cuda"), torch.randn(8, device="cuda"))
    measurement = measure_batch(network, fixture, torch.cuda.mem_get_info()[0])
    assert measurement["samples_per_second"] > 0
    assert all(torch.isfinite(parameter.grad).all() for parameter in network.parameters())
```

يُتخطى اختبار CUDA عند عدم توفره؛ ينشئ شبكة ومدخلات صناعية ويفحص أن قياس الدفعة يسمح بـ cuDNN backward وإنتاجية موجبة وتدرجات منتهية.

الأسطر 20–28

```
def test_cpu_lstm_training_checkpoint_and_inverse_scale_roundtrip(window_fixture, tmp_path):
    seed_experiment(42)
    configuration = {"device": "cpu", "hidden_size": 4, "dropout": 0.1, "batch_candidates": [16, 32], "amp":
False, "learning_rate": 0.001, "epochs": 2, "patience": 2, "effective_lookback": 30}
    model, audit = fit_lstm(window_fixture, configuration, tmp_path)
    reloaded = load_lstm(tmp_path, model.scalers, "cpu")
    heldout = window_fixture.flat[window_fixture.partitions["test"][:3]]
    np.testing.assert_allclose(model.predict(heldout), reloaded.predict(heldout), atol=1e-6)
    assert np.isfinite(model.predict(heldout)).all()
    assert audit["device"] == "cpu"
```

يدرب نموذج CPU صغيراً لحد epochين في مجلد مؤقت، ويحمل checkpoint ويقارن تنبؤ ثلاث نوافذ ويتحقق من النتائج المنتهية والجهاز. هذا اختبار اصطناعي لا تدريب البحث المحفوظ.

شرح tests/test_metrics_and_models.py

بصمة المصدر SHA-256:

.f7c3adb98439eb45660323b411d58d89c754c28ca89bc487061eb4e0bbfc5e01

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–8

```
import joblib
import numpy as np
import pytest

from xai_finance.evaluate.metrics import forecast_metrics
from xai_finance.evaluate.regimes import regime_labels
from xai_finance.models.baselines import persistence
from xai_finance.models.sklearn_models import candidates, fit_candidates
```

joblib وNumPy وpytest خارجية ودوال المقاييس والحالات والمرجع والمرشحين من المشروع.

الأسطر 11–17

```
def test_metrics_match_hand_computable_original_unit_errors():
    metrics = forecast_metrics(np.array([2, 4]), np.array([1, 5]), np.array([1, 5]))
    assert metrics["mae"] == 1
    assert metrics["rmse"] == 1
    assert metrics["mape_percent"] == 37.5
    assert metrics["r2"] == 0
    assert metrics["direction_accuracy"] == 0
```

يتحقق من مقاييس يدوية للقيمتين [2,4] والتنبؤ [1,5]: MAE وRMSE يساويان 1 وMAPE 37.5% وR^2 صفر والدقة الاتجاهية صفر.

الأسطر 20–24

```
def test_zero_target_mape_is_explicitly_undefined():
    metrics = forecast_metrics(np.zeros(2), np.ones(2), np.zeros(2))
    assert metrics["mape_percent"] is None
    assert metrics["mape_excluded_zero_targets"] == 2
    assert metrics["r2"] is None
```

يتحقق أن MAPE غير معرف عند كل أهداف صفرية، وأن العدد المستبعد 2 وR^2 غير معرف لهدف ثابت.

الأسطر 27–29

```
def test_persistence_uses_current_close_without_future_labels():
    current = np.array([100, 103, 101])
```

```
    np.testing.assert_array_equal(persistence(current), [100, 103, 101])
```

يؤكد أن Persistence يعيد الإغلاق الحالي كما هو دون قراءة الهدف المستقبلي.

الأسطر 32–38

```
def test_repeated_forest_inference_is_bitwise_stable(window_fixture):
    """Regression: parallel tree accumulation changed last bits during XAI checks."""
    model, _ = fit_candidates(candidates({"seed": 42, "trees": 40, "search": False})["RandomForest"],
window_fixture)
    observations = window_fixture.flat[window_fixture.partitions["test"]]
    expected = model.predict(observations)
    for _ in range(20):
        np.testing.assert_array_equal(expected, model.predict(observations))
```

يلائم غابة صناعية ويتحقق من تطابق التنبؤ بتكراره عشرين مرة؛ فحص تكرار محلي لا ضمان عبر كل الأجهزة.

الأسطر 41–42

```
def test_regime_boundary_uses_declared_trailing_return_threshold():
    np.testing.assert_array_equal(regime_labels(np.array([0.06, -0.06, 0.05, -0.05, 0]), 0.05), ["bull", "bear",
"sideways", "sideways", "sideways"])
```

يفحص bull وbear عند تجاوز ±0.05 وsideways عند المساواة أو الصفر.

الأسطر 45–54

```
@pytest.mark.parametrize("model_name", ["LinearRegression", "RandomForest", "GradientBoosting"])
def test_saved_tabular_model_preserves_predictions_without_refitting(window_fixture, tmp_path, model_name):
    estimators = candidates({"seed": 42, "trees": 5, "search": False})[model_name]
    model, audit = fit_candidates(estimators, window_fixture)
    test = window_fixture.partitions["test"]
    before = model.predict(window_fixture.flat[test])
    joblib.dump(model, tmp_path / "model.joblib")
    reloaded = joblib.load(tmp_path / "model.joblib")
    np.testing.assert_allclose(before, reloaded.predict(window_fixture.flat[test]), atol=1e-8)
    assert audit["trials"][0]["validation_rmse"] >= 0
```

اختبار معلم لثلاثة نماذج يلائم ويحفظ مؤقتاً ويعيد تحميل joblib ويقارن التنبؤ ويتحقق من RMSE تحقق غير سالب.

شرح tests/test_provider_integration.py

بصمة المصدر SHA-256:

.b53ad6c2aa4b7d61ed4a0280cad3b641e7e797970c3ad5eea9bb5573a9b7548c

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–8

```
import numpy as np
import pytest

from xai_finance.data.features import features
from xai_finance.data.windows import build_windows
from xai_finance.explain.shap_explain import explain_shap, shap_reconstruction
from xai_finance.io.data_sources import load_snapshot
from xai_finance.models.sklearn_models import candidates, fit_candidates
```

يستورد أدوات بناء البيانات والنماذج وSHAP وتحميل اللقطة للتحقق التكاملي.

الأسطر 11–22

```
@pytest.mark.integration
def test_real_provider_ols_explanation_reconstructs_stably():
    """Regression: provider-rounded dependent features caused catastrophic OLS cancellation."""
    prices, _ = load_snapshot("AAPL")
    windows = build_windows(features(prices, "daily"), 30)
    model, _ = fit_candidates(candidates({"seed": 42, "trees": 5, "search": False})["LinearRegression"],
windows)
    indices = windows.partitions["test"][np.linspace(0, len(windows.partitions["test"]) - 1, 10, dtype=int)]
    background = windows.flat[windows.partitions["train"][np.linspace(0, len(windows.partitions["train"]) - 1,
8, dtype=int)]]
    request = {"observations": windows.flat[indices], "names": windows.flat_names, "seed": 42, "cycles": 1}
    explanation = explain_shap("LinearRegression", model, background, request)
    audit = shap_reconstruction(explanation, model.predict(windows.flat[indices]))
    assert audit["max_absolute_error"] < 1e-3
```

اختبار integration يحمل لقطة AAPL المحلية، ويبني النوافذ ويلائم OLS على التدريب، ويختار 10 اختبارات و8 خلفيات ويفحص إعادة بناء SHAP بأقل من 1e-3. يتطلب لقطة محفوظة ويدرّب نموذج اختبار؛ لا يثبت إعادة إنتاج الرسالة.

شرح tests/test_scientific_validation.py

بصمة المصدر SHA-256:

.f4be0f78d9bcfde0e8d6a2af62da9621e9eeb5fcce6020f6ded0c11b21fde65a

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–8

```
from dataclasses import replace

import numpy as np
import pytest

from xai_finance.data.preprocess import fit_training_scalers
from xai_finance.validation.design import implied_prices, naive_predictions, target_values, transformed_windows,
walk_forward_partitions
from xai_finance.validation.statistics import directional_statistics
```

يستورد replace وNumPy وpytest ودوال المعايير وتصميم التحقق والاتجاه.

الأسطر 11–19

```
@pytest.mark.parametrize("target", ["price_change", "simple_return"])
def test_target_reconstruction_and_scaling_restore_next_price(window_fixture, target):
    transformed = transformed_windows(window_fixture, target)
    scalers = fit_training_scalers(transformed)
    standardized = scalers.target.transform(transformed.y[:, None])
    restored = implied_prices(scalers.inverse_target(standardized), window_fixture.current_close, target)
    np.testing.assert_allclose(restored, window_fixture.y, atol=1e-12)
    assert transformed.x is window_fixture.x
    assert not np.shares_memory(transformed.y, window_fixture.y)
```

لكل هدف تغير أو عائد يختبر التحويل والتقييس والعكس إلى السعر التالي، وأن x مشترك وy الجديد لا يشارك الذاكرة.

الأسطر 22–28

```
def test_return_protocol_rejects_negative_price_but_changes_preserve_it(window_fixture):
    prices = window_fixture.current_close.copy()
    prices[0] = -1
    altered = replace(window_fixture, current_close=prices)
    with pytest.raises(ValueError, match="strictly positive"):
        target_values(altered, "simple_return")
    np.testing.assert_allclose(implied_prices(target_values(altered, "price_change"), prices, "price_change"),
altered.y)
```

يضع سعراً سالباً صناعياً ويتوقع رفض العائد، مع إمكان إعادة تركيب السعر من التغير.

الأسطر 31–38

```
def test_walk_forward_blocks_are_disjoint_and_fit_only_available_labels(window_fixture):
    partitions = walk_forward_partitions(window_fixture, 3)
    np.testing.assert_array_equal(np.concatenate([partition["test"] for partition in partitions]),
window_fixture.partitions["test"])
    for partition in partitions:
        assert np.intersect1d(partition["train"], partition["validation"]).size == 0
        assert window_fixture.targets[partition["train"][-1]] <= window_fixture.origins[partition["validation"]
[0]]
        assert window_fixture.targets[partition["validation"][-1]] <= window_fixture.origins[partition["test"]
[0]]
    assert len(partitions[-1]["train"]) > len(partitions[0]["train"])
```

يتحقق من تغطية طيات الاختبار المتصلة دون تداخل وتوافر تسميات التدريب والتحقق وزيادة طول التدريب مع الطيات؛ لا يختبر أزمنة نشر البيانات.

الأسطر 41–52

```
@pytest.mark.parametrize("target", ["price_change", "simple_return"])
def test_naive_predictions_do_not_use_unknown_test_labels(window_fixture, target):
    before = naive_predictions(window_fixture, target)
    test = window_fixture.partitions["test"]
    changed_y = window_fixture.y.copy()
    changed_y[test] *= 2
    altered = replace(window_fixture, y=changed_y)
    after = naive_predictions(altered, target)
    for name in ["Zero", "TrainMean", "TrainMajorityDirectionMedianMagnitude"]:
        np.testing.assert_array_equal(before[name], after[name])
    assert before["LastObservedChangeOrReturn"][0] == after["LastObservedChangeOrReturn"][0]
    np.testing.assert_array_equal(before["LastObservedChangeOrReturn"], target_values(window_fixture, target)
[test - 1])
```

يغير أهداف الاختبار ويقارن المراجع الثلاثة الثابتة والهدف السابق لأول عينة؛ يثبت أن LastObserved لكل أصل يستخدم الهدف السابق، الذي قد يكون هدف اختبار سابقاً أصبح معروفاً.

الأسطر 55–60

```
def test_zero_prediction_is_directional_abstention():
    metrics = directional_statistics(np.array([1.0, -1.0, 0.0]), np.zeros(3))
    assert metrics["direction_accuracy"] == pytest.approx(1 / 3)
    assert metrics["direction_coverage"] == 0
    assert metrics["direction_accuracy_when_active"] is None
    assert metrics["binary_up_accuracy_zero_is_not_up"] == pytest.approx(2 / 3)
```

يتحقق من دقة الإشارة ثلاثية الحالات 1/3 عندما يتنبأ الصفر للثلاثة، وتغطية صفر ودقة نشطة غير معرفة ودقة ثنائية 2/3.

شرح tests/test_temporal_integrity.py

بصمة المصدر SHA-256:

.956f8288c073dc7938f6cc1202e9165bab37409a23827cab7fa7b0cf9eea7142

وصف الاختبارات لا يعني أنها شُغلت في جلسة التوثيق.

الأسطر 1–9

```
import numpy as np
import pandas as pd
import pytest

from xai_finance.data.features import features
from xai_finance.data.preprocess import fit_training_scalers
from xai_finance.data.split import chronological_indices
from xai_finance.data.validate import validate_prices
from xai_finance.data.windows import build_windows
```

يستورد مكتبات الاختبار ودوال السمات والتقييس والتقسيم والتحقق وبناء النوافذ.

الأسطر 12–20

```
@pytest.mark.parametrize("frequency", ["daily", "monthly"])
def test_future_price_mutation_preserves_past_features(analytic_prices, frequency):
    prices = analytic_prices if frequency == "daily" else analytic_prices[["close"]]
    cutoff = prices.index[190]
    changed = prices.copy()
    changed.loc[changed.index > cutoff] *= 20
    original = features(prices, frequency)
    mutated = features(changed, frequency)
    pd.testing.assert_frame_equal(original.loc[:cutoff], mutated.loc[:cutoff])
```

يغير الأسعار بعد حد زمني لعينة يومية وشهرية ويتحقق من ثبات السمات السابقة؛ لا يثبت سلامة الإصدارات التاريخية للمزود.

الأسطر 23–31

```
def test_targets_are_exactly_next_observed_close_and_windows_end_at_origin(analytic_prices):
    engineered = features(analytic_prices, "daily")
    windows = build_windows(engineered, 30)
    next_dates = analytic_prices.index[analytic_prices.index.get_indexer(windows.origins) + 1]
    assert windows.targets.equals(next_dates)
    np.testing.assert_array_equal(windows.y, analytic_prices.loc[next_dates, "adj_close"])
    np.testing.assert_array_equal(windows.x[:, -1, 0], analytic_prices.loc[windows.origins, "adj_close"])
    np.testing.assert_array_equal(windows.x[0], engineered.iloc[:30])
    assert (windows.origins < windows.targets).all()
```

يتحقق أن الهدف هو الإغلاق المعدل للرصد الخام التالي وأن نهاية كل نافذة عند الأصل، وأن أول نافذة تتطابق مع أول ثلاثين صف سمة.

الأسطر 34–42

```
def test_split_targets_never_cross_partition_boundary(window_fixture):
    partitions = window_fixture.partitions
    count = len(window_fixture.y)
    assert len(partitions["train"]) == int(0.8 * count)
    assert len(partitions["validation"]) == int(0.1 * count)
    assert sum(len(indices) for indices in partitions.values()) == count
    for earlier, later in [("train", "validation"), ("validation", "test")]:
        assert window_fixture.targets[partitions[earlier][-1]] < window_fixture.targets[partitions[later][0]]
        assert window_fixture.targets[partitions[earlier][-1]] <= window_fixture.origins[partitions[later][0]]
```

يتحقق من حجم التدريب والتحقق ومجموع الأقسام وترتيب تواريخ الأهداف وإتاحة التسمية السابقة عند أصل القسم التالي.

الأسطر 45–53

```
def test_heldout_mutation_cannot_change_fitted_scalers(window_fixture):
    before = fit_training_scalers(window_fixture)
    validation_and_test = np.r_[window_fixture.partitions["validation"], window_fixture.partitions["test"]]
    window_fixture.x[validation_and_test] *= 1000
    window_fixture.y[validation_and_test] *= 1000
    after = fit_training_scalers(window_fixture)
    np.testing.assert_array_equal(before.inputs.mean_, after.inputs.mean_)
    np.testing.assert_array_equal(before.target.mean_, after.target.mean_)
    assert before.fitted_target_cutoff < str(window_fixture.targets[validation_and_test[0]].date())
```

يضرب نوافذ وأهداف التحقق والاختبار في 1000 ثم يعيد ملاءمة المعايير، ويتأكد من ثبات متوسطي التدريب وسبق آخر هدف تدريب؛ يعدل fixture مؤقتاً.

الأسطر 56–59

```
def test_target_inverse_transform_restores_original_units(window_fixture):
    scalers = fit_training_scalers(window_fixture)
    normalized = scalers.target.transform(window_fixture.y[:, None])
    np.testing.assert_allclose(scalers.inverse_target(normalized), window_fixture.y, rtol=1e-12)
```

يتحقق من أن معيار الهدف وعكسه يسترجعان الوحدات الأصلية ضمن rtol محدد.

الأسطر 62–73

```
@pytest.mark.parametrize("fault", ["duplicate", "reversed", "bad_high", "nan"])
def test_invalid_provider_snapshot_is_rejected(analytic_prices, fault):
    if fault == "duplicate":
        analytic_prices = pd.concat([analytic_prices, analytic_prices.iloc[[-1]]])
    elif fault == "reversed":
        analytic_prices = analytic_prices.iloc[::-1]
    elif fault == "bad_high":
        analytic_prices.iloc[0, analytic_prices.columns.get_loc("high")] = 0
```

```
    else:
        analytic_prices.iloc[0, 0] = np.nan
    with pytest.raises(ValueError):
        validate_prices(analytic_prices, "daily")
```

يحقن تكرار تاريخ أو ترتيباً معكوساً أو high خاطئاً أو NaN ويتوقع ValueError من تحقق الأسعار.

الأسطر 76–78

```
def test_insufficient_history_fails_instead_of_empty_test_set():
    with pytest.raises(ValueError, match="Too few"):
        chronological_indices(5)
```

يتوقع رفض تقسيم خمس عينات بدلاً من إنتاج اختبار فارغ.